

SPCA 109N

**MASTER OF
COMPUTER APPLICATIONS**

FIRST YEAR - SECOND SEMESTER

CORE PAPER - VII

**OBJECT ORIENTED
ANALYSIS AND DESIGN**



**INSTITUTE OF DISTANCE EDUCATION
UNIVERSITY OF MADRAS**

WELCOME

Warm Greetings.

It is with a great pleasure to welcome you as a student of Institute of Distance Education, University of Madras. It is a proud moment for the Institute of Distance education as you are entering into a cafeteria system of learning process as envisaged by the University Grants Commission. Yes, we have framed and introduced as per AICTE Choice Based Credit System(CBCS) in Semester pattern from the academic year 2020-21. You are free to choose courses, as per the Regulations, to attain the target of total number of credits set for each course and also each degree programme. What is a credit? To earn one credit in a semester you have to spend 30 hours of learning process. Each course has a weightage in terms of credits. Credits are assigned by taking into account of its level of subject content. For instance, if one particular course or paper has 4 credits then you have to spend 120 hours of self-learning in a semester. You are advised to plan the strategy to devote hours of self-study in the learning process. You will be assessed periodically by means of tests, assignments and quizzes either in class room or laboratory or field work. In the case of PG (UG), Continuous Internal Assessment for 20(25) percentage and End Semester University Examination for 80 (75) percentage of the maximum score for a course / paper. The theory paper in the end semester examination will bring out your various skills: namely basic knowledge about subject, memory recall, application, analysis, comprehension and descriptive writing. We will always have in mind while training you in conducting experiments, analyzing the performance during laboratory work, and observing the outcomes to bring out the truth from the experiment, and we measure these skills in the end semester examination. You will be guided by well experienced faculty.

I invite you to join the CBCS in Semester System to gain rich knowledge leisurely at your will and wish. Choose the right courses at right times so as to erect your flag of success. We always encourage and enlighten to excel and empower. We are the cross bearers to make you a torch bearer to have a bright future.

With best wishes from mind and heart,

DIRECTOR

**MCA
FIRST YEAR - SECOND SEMESTER**

**CORE PAPER - VII
OBJECT ORIENTED ANALYSIS
AND DESIGN**

EDITING AND CO-ORDINATION

Dr. S. Sasikala

Associate Professor in Computer Science

Institute of Distance Education

University of Madras

Chepauk, Chennai - 600 005

MASTER OF COMPUTER APPLICATION
FIRST YEAR - SECOND SEMESTER
Paper - VII
OBJECT ORIENTED ANALYSIS AND DESIGN
SYLLABUS

Objectives of the Course : To understand the Object-based view of systems. To develop robust object-based models for Systems and to inculcate necessary skills to handle complexity in software design.

Course Outcomes: After successful completion of this course, the students should be able to
Ability to analyze and model software specifications. Ability to abstract object-based views of generic software systems, Ability to deliver robust software components.

Unit - I : System Development - Object Basics - Development Life Cycle - Methodologies - Patterns - Frameworks - Unified Approach - UML.

Unit - II : Use-Case Models - Object Analysis - Object relations - Attributes - Methods - Class and Object responsibilities - Case Studies.

Unit - III : Design Processes - Design Axioms - Class Design - Object Storage - Object Interoperability - Case Studies.

Unit - IV : User Interface Design - View layer Classes - Micro-level Processes - View Layer Interface - Case Studies.

Unit - V : Quality Assurance Tests - Testing Strategies - Object orientation on testing - Test Cases - Test Plans - Continuous testing - Debugging Principles - System Usability - Measuring User Satisfaction - Case Studies.

Recommended Texts

- 1) Ali Bahrami, Reprint 2009, Object Oriented Systems Development, Tata McGraw Hill International Edition.

Reference Books

- 1) G. Booch, 1999. Object Oriented Analysis and Design, 2nd Edition, Addison Wesley, Boston.
- 2) R.S. Pressman, 2010, Software Engineering A Practitioner's approach, Seventh Edition, Tata McGraw Hill, New Delhi.
- 3) Rumbaugh, Blaha, Premerlani, Eddy, Lorensen, 2003, Object Oriented Modeling And Design, Pearson education, Delhi.

MASTER OF COMPUTER APPLICATION
FIRST YEAR - SECOND SEMESTER
Paper - VII
OBJECT ORIENTED ANALYSIS AND DESIGN
SCHEME OF LESSONS

Sl.No.	Title	Page
1.	System Development	1
2.	System Development	18
3.	Methodology and Frame Work	35
4.	Unified Modeling Language	55
5.	Use Case Driven	76
6.	Object Analysis	96
7.	Object Relationship Attributes and Methods	117
8.	Object Oriented and Design Axioms	138
9.	Designing Classes	158
10.	Object Storages and Object Interoperability	178
11.	Object Relational Systems and Access Layer	200
12.	Views Layer Designing Interface Objects	223
13.	A View Layer Interface	241
14.	Software Quality Assurance	266
15.	System usability and measuring user satisfaction	283
16.	Testing Strategies	299

LESSON - 1

SYSTEM DEVELOPMENT

Objectives

After studying this lesson you should be able to

- The object-oriented philosophy and why we need to study it.
- The unified approach.
- Describe system development methodology and object orientation.
- Why we need to study object oriented concepts
- Objects and classes and their differences
- Class attributes and methods
- The concept of messages
- Class hierarchy inheritance and multiple inheritance.
- Object relationships and associations.
- Encapsulation and information hiding
- Polymorphism
- Advantage of the object oriented approach
- Aggregations
- Static and dynamic binding
- Object persistence.
- Meta classes.

1.1 Introduction

Software development is dynamic and always undergoing major change. The methods we will use in the future no doubt will differ significantly from those currently in practice. We can anticipate which methods and tools are going to succeed, but we cannot predict the future. Factors other than just technical superiority will likely determine which concepts prevail.

System development refers to all activities that go into producing an information systems solution. System development activities consist of systems analysis, modeling, design, implementation, testing, and maintenance.

A software development methodology is a series of processes that, if followed, can lead to the development of an application. The software processes describe how the work is to be carried out to achieve the original goal based on the system requirements. Furthermore, each process consists of a number of steps and rules that should be performed during development system is in operation. Furthermore, we study the unified approach, which is the methodology used in this lesson for learning about object-oriented systems development.

If there is a single motivating factor behind object oriented system development, it is the desire to make software development easier and more natural by raising the level of abstraction to the point where application can be implemented in the same terms in which they are described by user.

In an object model we call the former properties or attributes and the latter procedures or methods. Properties describe the state of an object. methods define its behavior.

1.1 Two orthogonal views of the software

Object-oriented systems development methods differ from traditional development techniques in that the traditional techniques view software as a collections of programs and isolated data.

What is a program?

"A software system is a set of mechanisms for performing certain action on certain data".

(Or) Algorithms + Data Structures = Programs

This means that there are two different, yet complementary ways to view software construction:

we can focus primarily on the functions or primarily on the data. The heart of the distinction between traditional system development methodologies and newer object-oriented methodologies lies in their primary focus, where the traditional approach focuses on the functions

of the system - what is it doing? Object-oriented system development centers on the object, which combines data and functionality. As we will see, this seemingly simple shift in focus radically changes the process of software development.

1.2 Object-oriented system development methodology

Object-oriented development offers a different model from the traditional software development approach, which is based on functions and procedures. In simplified terms, object-oriented system development is a way to develop software by building self contained modules or objects that can be easily replaced, modified, and reused.

In an object oriented environment, software is a collection of discrete object that encapsulate their data as well as the functionality to model real world “objects”. An object orientation yields important benefits to the practice of software construction. Each object has attributes and methods. Objects are grouped into classes; in object oriented terms, we discover and describe the classes involved in the domain.

In an object-oriented system, everything is an object and each object is responsible for itself. For example every windows application needs windows objects that can open themselves on screen and either display something or accept input. A windows object is responsible for things like opening, sizing, and closing itself.

A chart object is responsible for things like maintaining in data and labels and even for drawing itself. The object oriented environment emphasizes its cooperative philosophy by allocating tasks among the objects of the applications. In other words rather than writing a lot of code to do all the things that have to be done, you tend to create a lot of helpers that take on an active role, a spirit and that form a community whose interactions become the application,

1.3 Why an object orientation?

Object oriented methods enable us to create sets of objects that work together synergistically to produce software that better model their problem domains than similar system produced by traditional techniques. The systems are easier to adapt to changing requirement easier to maintain more robust and promote greater design and code reuse. Object oriented development allows us to create modules of functionality. Once objects are defined, it can be taken for granted that they will perform their desired functions and you can seal them off in your mind like black boxes.

- Higher level of abstractions: The top-down approach supports abstraction at the function level. The object oriented approach supports abstraction at the object level. Since object encapsulate both data and functions they work at a higher level of abstraction. The development can proceed at the object level and ignore the rest of system for as long as necessary. This makes designing, coding, testing, and maintaining the system much simpler.
- Seamless transition among different phases of software development. The traditional approach to software development require different styles and methodologies for each step of the process. Moving from one phase to another requires a complex transition of perspective between models that almost can be in different worlds. This transition not only can slow the development process but also increases the size of the project and the chance for errors introduced in moving from one language to another.
- Encouragement of good programming techniques. A class in an object oriented system carefully delineates between its interface and the implementation of the interface. The routines and attributes within a class has are held together tightly. In a properly designed system the classes will be grouped into subsystem but remain independent therefore, changing one class has no impact on other classes and so the impact is minimized. however the object-oriented approach is not a panacea; nothing is magical here that will promote perfect design or perfect code. The object oriented method tends to promote clearer designs, which are easier to implement and provides for better overall communication. Using object oriented language is not strictly necessary to achieve the benefits of an object orientation.
- Promotion of reusability. Object are reusable because they are modeled directly out of a real-world problem domain. Each object stands by itself or within a small circle of peers. Within this framework the class does not concern itself with the rest of the system or how it is going to be used within a particular system. This means that classes are designed generically, with reuse as a constant background goal.

1.4 Overview of the unified approach

The unified approach (UA) is a methodology for software development that is proposed by the author and used in this lesson. The UA based on methodologies by booch, Rumbaugh

and Jacobson tries to combine the best practices processors and guidelines along with the object management group's unified modeling language

The unified modeling language (UML) is a set of notations and conventions used to describe and model an application. However the UML does not specify a methodology or what steps to follow to development an application, that would be the task of the UA

The heart of the UA is Jacobson's case. The use case represents a typical interaction between a user and a computer system to capture the user's goals and needs. In its simplest usage , you capture a use case by talking to typical users and discussing the various ways they might want to use the system. The use cases are entered into all other activities of the UA.

The main advantage of an object oriented system is that the class tree is dynamic and can grow. Your function as a developer in an object oriented environment is to foster the growth of the class tree by defining new, more specialized classes to perform the tasks your applications require.

Layered architecture is an approach to software development that allows us to create object that represent tangible elements of the business independent of how they are represented to the user through an interface or physically stored in a database.

The layered approach consists of view or user interface, business and access layers. This approach reduces the interdependence of the user interface database access and business control therefore it allows for a more robust and flexible system.

1.5 Organization of this lesson

An object oriented approach allows the base concepts of the language to be extended to include ideas closer to those of its application you can define a new data type in terms of an existing data type until it appears that the language directly support the primitives of the application. The real advantage of using an object oriented approach is that you can build on what you already have.

The object-oriented system development life cycle (SDLC), the essence of the software process is the transformation of users needs in the application domain into a software solution that is executed in the implementation domain. The concepts of use case or set of scenarios can be a valuable tool for understanding the users needs.

1.6 An object-oriented philosophy

Most programming language provide programmers with a way of describing processes. Although most programming languages are computationally equivalent the ease of description. It has been said that “one should speak english for business, French for seduction, german for engineering and Persian for poetry”. A similar quip could be made about programming language.

A fundamental characteristic of object oriented programming is that it allows the base concepts of the language to be extended to include ideas and terms closer to those of its application. The fundamental difference between the object oriented system and their traditional counterparts is the way in which you approach problems.

Most traditional development methodologies are either algorithm centric or data centric. In an algorithm-centric methodology, you think of an algorithm that can accomplish the task then build data structure for that algorithm to use. In a data centric methodology you can think how to structure the data then build the algorithm around that structure.

1.7 Objects

The term object means a combination of data and logic that represents some real world entity. For example, consider a Saab automobile. The Saab can be represents in a computer program as an object. The “data” part of this object would be the car’s name, color, number of doors, price and so forth. The “logic” part of the object could be a collection of the programs. For example mileage, change mileage, stop, go etc.

1.8 Object are grouped in classes

Classes are used to distinguish one type of object from another. In the context object-oriented systems, a class is a set of object that share a common structure and common behavior, a single object is simply an instance of a class. A class is a specification of structure, behavior and inheritance for objects.

Classes are an important mechanism for classifying objects. The chief role of a class is to define the properties and procedures and applicability of its instances. There may be many different classes. Think of a class as an object template (see fig 1.1). Every object of a given class has the same data format and responds to the same instructions.

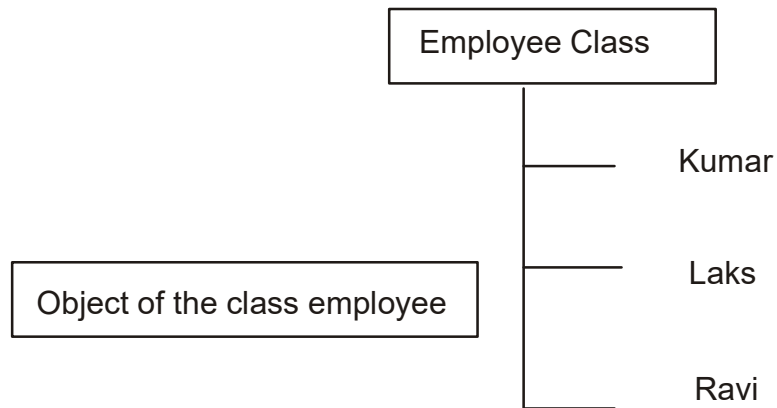


Figure 1.1

For example, employees, such as kumar, ravi, laks all are instances of the class employee. You can create unlimited instances of a given class. The instructions responded to by each of those instances of employee are managed by class.

1.9 Attributes: Object state and properties

Properties represent the state of an object. Often we want to refer to the description of these properties rather than how they are represented in a particular programming language. In our example, the properties of a employee, such a emp.name,s.s.no, emp. add, pincode (see figure 1.2).

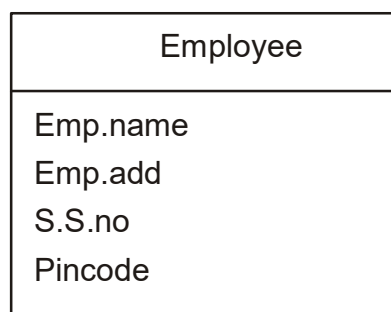


Figure 1.2

1.10 Object behavior and method

When we talk about a car, we usually can describe the set of things we normally do with it or that it can do on its own. We can drive a car. Each of these statement is a description of the object's behavior. In the object model object behavior is described in methods or procedures. A method implements the behavior of an object. Behavior denotes the collection of methods that abstractly describes what an object is capable of doing.

1.11 Objects respond to messages

Methods conceptually are equivalent to the function definitions used in procedural languages. For example a draw method would tell a chart how to draw itself. Objects perform operations in response to messages.

Messages essentially are nonspecific function calls: we would send a draw message to a chart when we want the chart to draw itself. A message is different from a subroutine call, since different objects can respond to the same message in different ways. In the top example, depicted in figure 1.3 we send a break message to the car object. In the middle example, we send a multiplication message to 5 object followed by the number by which we want to multiply 5. In the bottom example a compute payroll message is sent to the employee object.

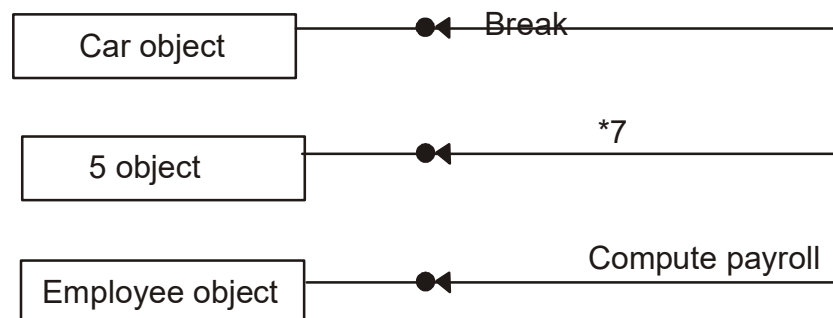


Figure 1.3

Note that the message makes no assumptions about the class of the receiver or the arguments; they are simply objects. It is the receiver's responsibility to a message in an appropriate manner. This gives you a great deal of flexibility, since different objects can respond to the same message in different ways. This is known as polymorphism meaning "many shapes". Polymorphism is the main difference between a message and a subroutine call.

1.12 Encapsulation and information hiding

Information hiding is the principle of concealing the internal data and procedures of an object and providing an interface to each object in such a way as to reveal as little as possible about its inner workings. For example, Simula provides no protection, or information hiding for objects meaning that an object's data or instance variables, may be accessed wherever visible.

C++ has a very general encapsulation protection mechanism with public, private, and protected members. Public members may be accessed from be public. Private members are accessible only from within a class. An object data representation, such as a list or an array usually will be private. Protected members can be accessed only from subclasses. Another issue is per-object or per-class protection. In per class protection, the most common form class methods can access any object of that class and not just the receiver. In per object protection methods can access only the receiver. An important factor in achieving encapsulation is the design of different classes of objects that operate using a common protocol, or object's user interface.

1.13 Class hierarchy

An object oriented system organizes classes into a subclass Superclass hierarchy. Different properties and behaviors are used as the basis for making distinction between classes and subclasses. At the top of the class hierarchy are the most general classes and at the bottom are the most specific. The family car figure 1.4 is a subclass of car. A subclass inherit all of the properties and methods defined in its superclass. In this cases we can drive a family car just as we can drive any car or indeed, almost any motor vehicle. Subclasses generally add new methods and properties specific to that class.

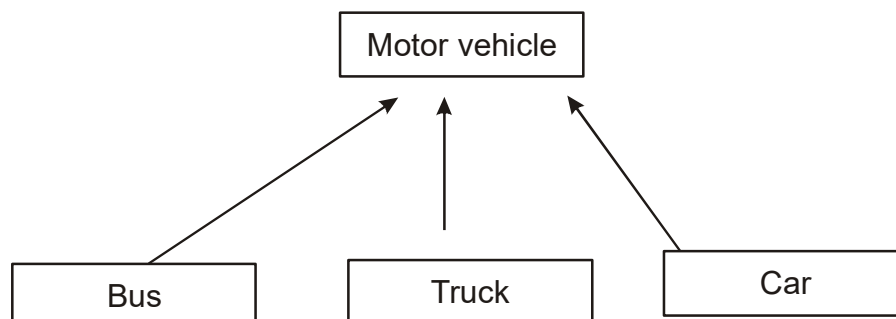


Figure 1.4

A class may simultaneously be the subclass to some class and a super class to another classes. Truck is a subclass of motor vehicle and a superclass of both 18 wheeler and pickup. For example, ford is a class that define ford car objects (see figure 1.5).

The car class is a formal class, also called an abstract class. Formal or abstract classes have no instances but define the common behaviors that can be inherited by more specific classes. In some object oriented languages the term superclass and subclass are used instead of base and derived.

1.13.1 Inheritance

Inheritance is the property of object-oriented system that allow objects to be built from other objects. Inheritance allows explicitly taking advantages of the

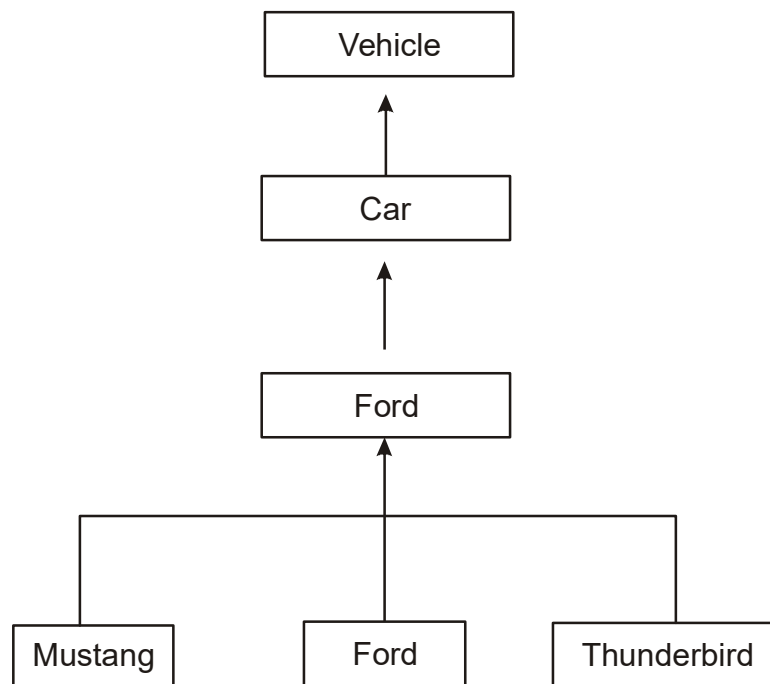


Figure 1.5

commonality of objects when constructing new classes. Inheritance is a relationship between classes where one class is parent class of another class. The parent class also is known as the base class or superclass. The ford class inherit the general behavior from the car class and adds behavior specific to fords.

However the stop method is not defined in the mustang class, so the hierarchy is searched until a stop method is found. The stop method is found in the ford class, a superclass of the mustang class and it is invoked (see figure 1.6).

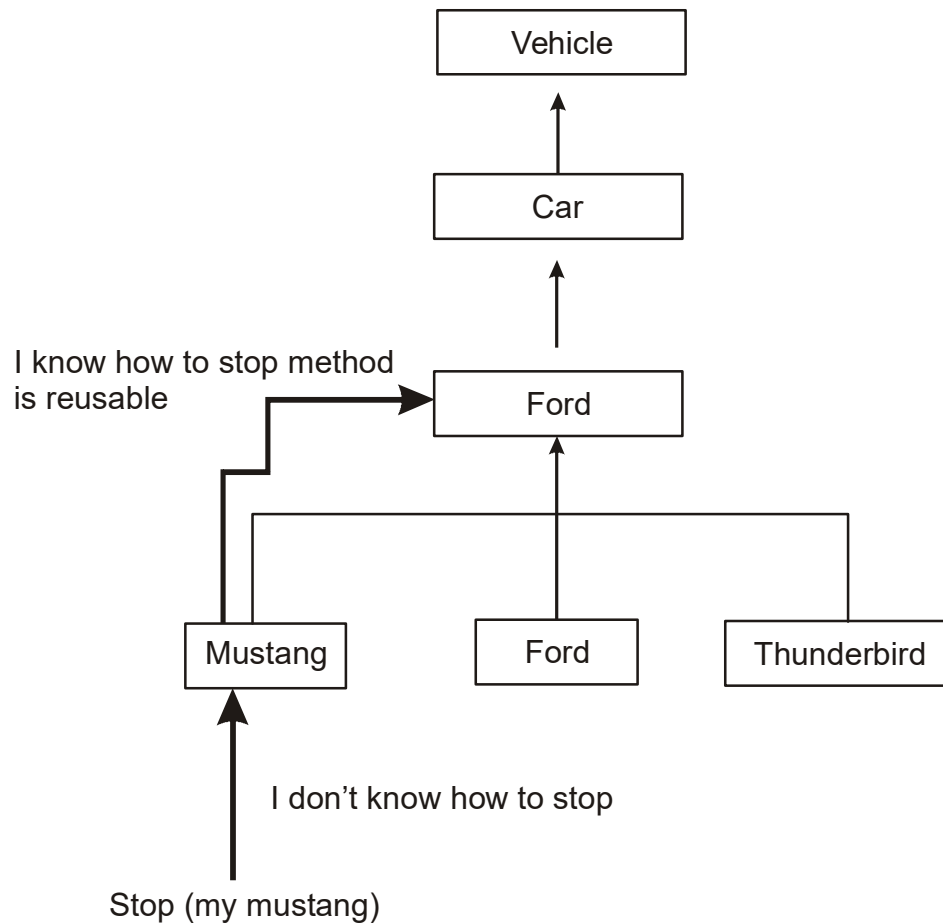


Figure 1.6

Dynamic inheritance

Allows objects to change and evolve over time. Since base classes provide properties and attributes for objects, changing base classes changes the properties and attributes of a class. A previous example was a windows object changing into an icon and then back again, which involves changing a base class between a windows class and an icon class. More specifically dynamic inheritance refers to the ability to add, delete or change parents from objects at a run time.

1.13.2 Multiple inheritance

Some object-oriented systems permit a class to inherit its state and behaviors from more than one superclass. This kind of inheritance is referred to as multiple inheritance. For example, a utility vehicle inherits attributes from both the car and track classes. (see figure 1.7)

Multiple inheritance can pose some difficulties. For example several distinct parent classes can declare a member within a multiple inheritance hierarchy. This then can become an issue of choice particularly when several superclasses define the same method. It also is more difficult to understand programs written in multiple inheritance systems one way of achieving the benefits of multiple inheritance in a language with single inheritance is to inherit from the most appropriate class and then add an object of another class as an attribute.

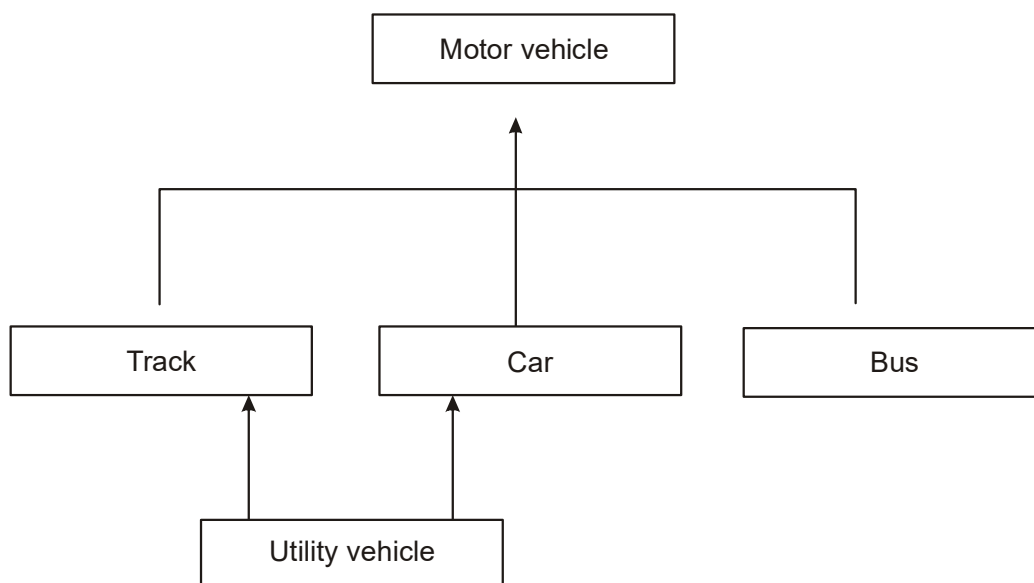


Figure 1.7

1.14 Polymorphism

Poly means “many” and morph means “form”. In the context of object oriented systems, it means objects that can take on or assume many different forms. Polymorphism means that the same operation may behave differently on different classes.

1.15 Object relationships and associations

Association represents the relationship between objects and classes. For example in the statement “a pilot can fly planes” (see figure 1.8) the italicized term is an association. Association are bi-directional, that means they can be traversed in both directions, perhaps with different connotations. For example can fly connects a pilot to certain airplanes. The inverse of can fly could be called is flown by



Figure 8

An important issue in association is cardinality. Cardinality constrains the number of related objects and often is described as being “one” or “many”.

1.15.1 Consumer-produce Association

A special form of association is a consumer producer relationship, also known as a client-server association or a use relationship. The consumer producer relationship can be viewed as one-way interaction. One object requests the service of another object.

The object that makes the request is the consumer or client and the object that receives the request and provides the service is the provider or server. For example, we have a print object that prints the consumer object. The print producer provides the ability to print other objects. Figure 1.9 depicts the consumer producer association.

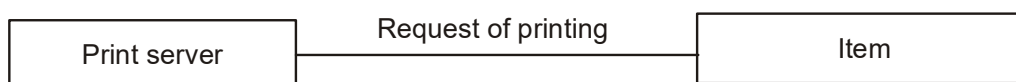


Figure 1.9

1.16 Aggregation and object containment

All objects except the most basic ones, are composed of and may contain other objects. For example a spreadsheet is an object composed of cells and cells are objects that may contain text, mathematical formulas, video, and so forth. This is possible because an object's attributes need not be simple data fields; attributes can reference other objects. Since each object has an identity one object can refer to other objects. This is known as aggregation where an attribute can be an object itself.

1.17 Advanced Topics

1.17.1 Object and Identity

A special feature of object oriented system is that every object has its own unique and immutable identity. An object's identity comes into being when the object is created and continues to represent that object from then on. This identity never is confused with another object. In an object system object identity often is implemented through some kind of object identifier (OID) or unique identifier (UID). An OID is dispensed by a part of the object oriented programming system that is responsible for guaranteeing the uniqueness of every identifier. OIDs are never reused.

As another example (see figure 1.10) cars may have an "owner" property of class person. A particular instance a car a black Saab 900s, is owned by person instance named kumar. In an object system, the relationship between the car and kumar can be implemented and maintained by a reference.

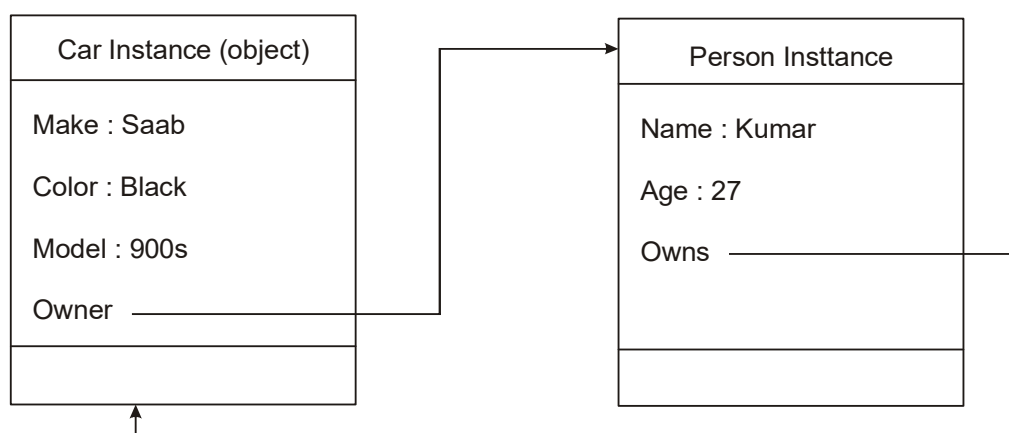


Figure 1.10

1.17.2 Static and Dynamic Binding

The process of determining at run time which function to invoke is termed dynamic binding. Making this determination earlier, at compile time, is called static binding. Static binding optimizes the calls; dynamic binding occurs when polymorphic calls are issued. Not all function invocations require dynamic binding.

Dynamic binding allows some method invocation decisions to be deferred until the information is known. A run time selection of methods often is desired, and even required, in many applications including databases and user interaction.

For example a cut operation in an Edit submenu may pass the cut operation to any object on the Desktop, each of which handles the message in its own way. If an object can cut many kinds of objects to be cut are available in the receiving object; the particular method being selected is based on the actual type of object being cut.

1.17.3 Object Persistence

Objects have a lifetime. They are explicitly created and can exist for a period of time that traditionally has been the duration of the process in which they were created. A file or a database can provide support for objects having a longer lifetime longer than the duration of the process for which they were created. From a language perspective, this characteristic is called object persistence.

An object can persist beyond application session boundaries, during which the object is stored in a file or a database, in some file or database form. The object can be retrieved in another application session. And will have the same state and relationship to other objects as at the time it was saved. The lifetime of an object can be explicitly terminated. After an object is deleted its state is inaccessible and its persistent storage is reclaimed.

1.17.4 Meta - Classes

Earlier we said that in an object oriented system everything is an object numbers, arrays, fields, forms, and so forth. How about a class? Is a class an object? yes a class is an object, so if it is an object, it must belong to a class. Indeed, such a class belongs to a class called a meta class or a class of classes.

For example classes must be implemented in some way perhaps with dictionaries for methods, instances and parents and methods to perform all the work of being a class. This can be declared in a class named metaclass. the meta class also can provide services to application program, such as returning a set of all methods, instances or parents for review.

Summary

- In an object oriented environment software is a collection of discrete that encapsulate their data and the functionality to model real world “objects”
- An object orientation produces systems that are easier to evolve more flexible, more robust, and more reusable than a top down structure approach.
- Allows working at a higher level of abstraction.
- Encourages good development practices.
- Promotes reusability.
- The unified approach is the methodology for software development proposed and used in this lesson.
- UA consists of the following concepts
- Use case driven development
- The layered approach
- Incremental development and prototyping
- Continuous testing
- Object-oriented analysis
- Utilizing the unified modeling language for modeling
- Repositories of reusable class and maximum reuse.
- Object oriented design
- Object oriented software development is a significant departure from the traditional structured approach.
- Poly means “many” and morph means “form”.

- The main advantage of the object oriented approach is the ability to reuse code and develop more maintainable systems in a shorter amount of time.
- Each object is an instance of a class, classes are organized hierarchically in a class tree and subclasses inherit the behavior of their superclasses.
- A file or a database can provide support for objects having a longer lifeline longer than the duration of the process for which they were created.

Questions:

1. What is system development methodology?
2. What are orthogonal views of software?
3. What are the advantages of object oriented development?
4. What is an object?
5. What is polymorphism?
6. What is inheritance?
7. What is data abstraction?
8. What is a protocol?
9. What is difference between an object's method and object's attributes?
10. How does object oriented development eliminate duplication?
11. Why is encapsulation important?
12. What is an instance?
13. What is a formal class?
14. What is a consumer producer relationship?
15. What is association?
16. Why is polymorphism useful?
17. What is difference between a method and message?
18. How does object oriented development eliminate duplication?
19. How are objects identified in an object oriented system?

LESSON - 2

SYSTEM DEVELOPMENT LIFE CYCLE

Objectives:

After studying this lesson you should be able to

- The software development process
- Building high quality software
- Object oriented systems development
- Use case driven systems development.
- Prototyping.
- Component based development.
- Rapid application development.

Introduction

The essence of the software development process that consists of analysis, design, implementation, testing and refinement is to transform user's needs into a software solution that satisfies those needs. However some people view the software development process as interesting but feel it has little importance in developing software.

It is tempting to ignore the process and plunge into the implementation and programming phases of software development, much like the builder who would bypass the architect. Some programmers have been able to ignore the counsel of the system development in building a system, but in general the dynamics of software development provide little room for such shortcuts, and bypasses have been less than successful.

The prototype also can give users a chance to comment on the usability and usefulness of the design and let you assess the fit between the software tools selected the functional specification and users needs.

This lesson introduces you to the system development life cycle in general and more specifically to an object oriented approach to software development. The main point of this lesson is the idea of building software by placing emphasis on the analysis and design aspects of the software life cycle.

2.1 The Software development process

System development can be viewed as a process. Furthermore, the development itself in essence is a process of change, refinement, transformation or addition to the existing product. Within the process, it is possible to replace one sub-process with a new one. As long as the new sub-process has the same interface as the old one to allow it to fit into the process as a whole.

For example the object oriented approach provides us a set rules for describing inheritance and specialization in a consists way when a sub-process changes the behavior the behavior of its parent process. The process can be divided into small interacting phasesub process.

The sub processes must be defined in such a way that they are clearly spelled out, to allow each activity to be performed as independently of other sub-processes as possible.

Each sub-process must have the following.

- A description in terms of how it works
- Specification of the input required for the process
- Specification of the output to be produced

The software development process also can be divided into smaller interacting sub-processes. Generally the software development process can be viewed as a series of transformations, where the output of one transformations becomes the input of the subsequent transformation (see figure 2.1).

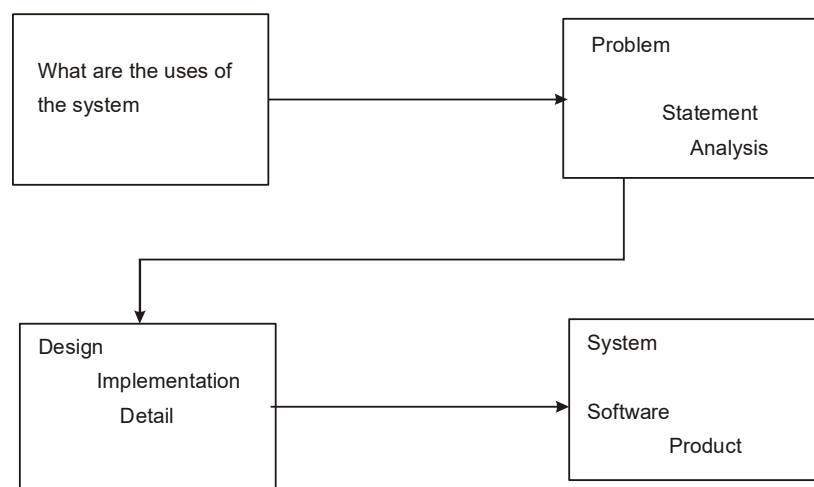


Figure 2.1

- Transformation 1 (analysis) translates the user's needs into system requirements and responsibilities. The way they use the system can provide insight into the users requirements.
- Transformation 2 (design) begins with a problem statement and ends with a detailed design that can be transformed into an operational system. This transformation includes the bulk of the software development activity, including the definition of how to build the software, its development, and its testing. It also include the design descriptions, the program and the testing materials
- Transformation 3 (implementation) refines the detailed design into the system deployment that will satisfy the users needs. This takes into accounting the equipment, procedures, people and the like. It represents embedding the sys product within its operational environment.

An example of the software development process is the waterfall approach. Which starts with deciding what is to do done. Once the requirements have been determined, we next must decide how to accomplish them.

This is followed by a step in which we do it, whatever "it" has required us to do. We then must test the result to see if we have satisfied the users requirements. Finally we use what we have done (see figure 2.2).

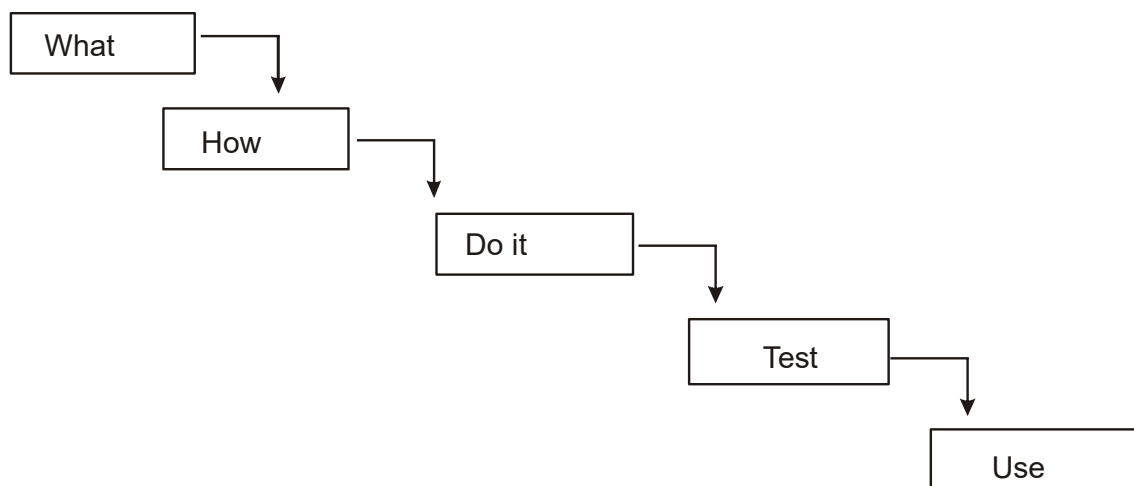


Figure 2.2

For example if a company has experience in building accounting systems, then building another such product based on the exiting design is best managed with the waterfall model, as it has been described. Where there is uncertainly regarding what is required or how it can built, the waterfall model fails. This model assumes that the requirements are known before the design begins, but one may need experience with the product before the requirements can be fully understood.

The waterfall model is the best way to manage a project with a well understood product, especially very large projects. The system is installed in the real world the environment frequently changes altering the accuracy of the original problem statement and, consequently generating revised software requirements. This can complicate the software development process even more.

For example a new class of employee or another shift of workers may be added or the standard workweek or the piece rate changed. By definition, any such changes also change the environment requiring changes in the programs. As each such request is processed, system and programming changes make the process increasingly complex since each request must be considered in regard to the original statement of need as modified by other requests.

2.2 Building high quality software

The software process transforms the users need via the application domain to a software solution that satisfies those needs. Once the system exits we must test it to see if it is free of bugs. High quality products must meet users needs and expectations. To achieve high quality in software we need to be able to answer the following questions:

1. How do we determine when the system is ready for delivery?
2. Is it now an operational system that satisfies user's needs?
3. Is it correct and operating as we thought it should?
4. Does it pass an evaluation process?

There are two basic approaches to system testing. We can test a system according to how it has been built. Blum[3] describes a means of a system evaluation in terms of four quality measures:

- Correspondence
- Correctness
- Verification
- Validation

Correspondence

It is measures how well the delivered system matches the needs of the operational environment as described in the original requirement statement.

Correctness

It is measures the consistency of the product requirements with respect to the design specification.

Verification

It is the exercise of determining correctness. However correctness always is objective.

Validation

It is the task of predicting correspondence. True correspondence cannot be determined until the system is in place see figure 2.3.

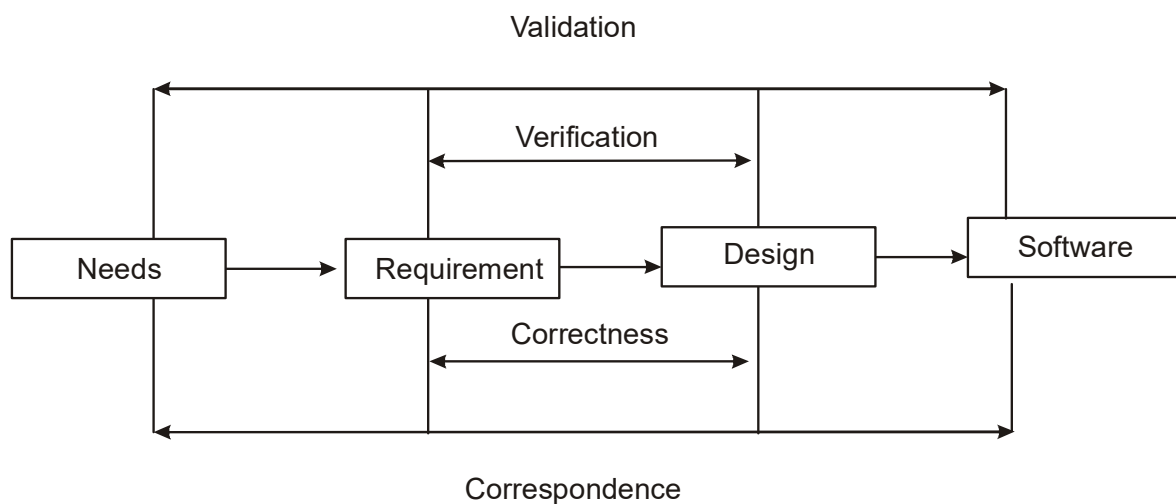


Figure 2.3

Validation, however is always subjective and it address a different issue the appropriateness of the specification. This may be considered 20/20 hindsight: dif we uncover the true users needs and therefore establish the proper design?

Validation begins as soon as the project starts, but verification can begin only after a specification has been accepted. Verification and validation are independent of each other

It is possible to have a product that corresponds to the specification but if the specification proves to be incorrect, we do not have the right product.

For example say a necessary report is missing from the delivered product, since it was not included in the original specification. A product also may be correct but not correspond to the users needs.

2.3 Object oriented system development: A use case driven approach

The object oriented software development life cycle (SDLC) consists of three macro process:

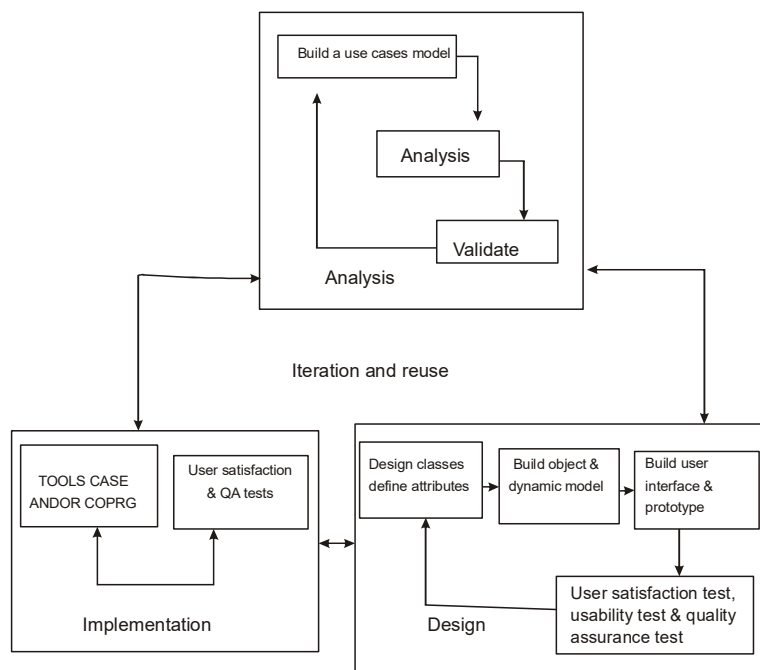


Figure 2.4

- Object oriented analysis.
- Object oriented design.
- Object oriented implementation (see fig. 2.4)

The use case mode: can be employed throughout most activities of software development. Further more by following the life cycle model Jacobson, one can produce design that are traceable across requirements, analysis, design, implementation and testing(as show on in fig 2.5). the main advantage is that all decisions can be traced back directly to user requirements.

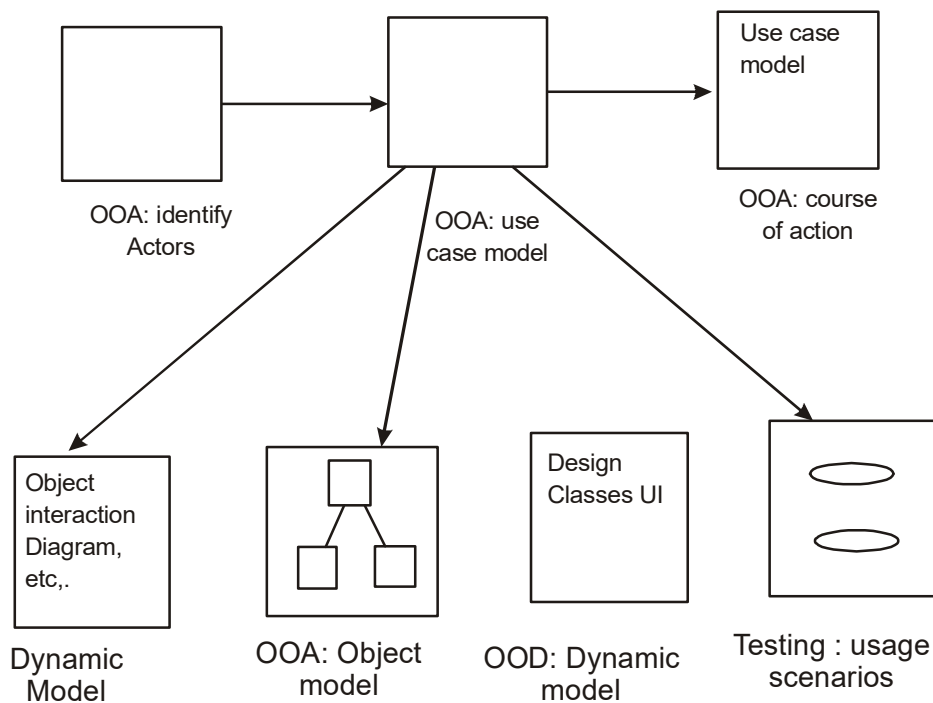


Figure 2.5

Object orient system development includes these activities.

- Object oriented analysis use case driven.
- Object oriented design.
- Prototyping.
- Component based development.
- Incremental testing.

2.3.1 Object oriented analysis use case driven

The object orient analysis phase of software development is concerned with determining the system requirements and identifying classes and their relationship to other classes in the problem domain. To understand the system requirements we need to identify the user or the actors. Who are the actors and the how do they use the system? In object oriented as well as trade additional development. Jacobson [10] came up with the concepts of the use case.

The concepts worked so well that it became a primary element in system development, the object oriented programming community has adopted use cases to a remarkable degree. Scenarios are a great way of examining who does what in the interactions among objects and what role they play that is their interrelationships.

The intersection among objects role to achieve a given goal is called collaboration. The scenarios represent only one possible example of the collaboration. Expressing these high level processes and interactions with customers in a scenario and analyzing it is referred to as use case modeling.

The use case model represents the users view of the system or users needs. For example consider a word process where a user may want to be able to replace a word with its synonym or create a hyperlink.

These are some uses of the system or a system responsibility, this process of developing uses case like other object oriented activities is iterative once your use case model is better understood and developed you should start to identify classes and create their relationships.

For example the objects in the incentive payroll system might include the following examples.

- The employee, worker, supervisor, office administrator
- The paycheck
- The product being made
- The process used to make the product.

In this case it can be useful to pose the problem in terms of analogous physical objects, kind of a mental simulation.

2.3.2 Object oriented Design

The goal of object oriented design (OOD) is to design the classes identified during the analysis phase and the user interface. During this phase we identify and define additional objects and classes that support implementation of the requirements. For example during the design phase you might need to add objects for the user interface to the system

Object-oriented design and object oriented analysis are distinct disciplines, but they can be intertwined. Object oriented development is highly incremental, in other words, you do some more of each again and again gradually refining and completing models of the system. The activities and focus of object oriented analysis and object oriented design are intertwined grown not built. (see figure 2.4)

First build model based on object and their relationships then iterate and refine the model:

- Design and refine classes.
- Design and refine attributes.
- Design and refine methods.
- Design and refine structure.
- Design and refine associations.

Here are a few guidelines to use in your object oriented design:

- Reuse rather than build a new class. Know the existing classes.
- Design a large number of simple classes rather than a small number of complex classes.
- Design methods.
- Critique what you have proposed. If possible go back and refine the classes.

2.3.3 Prototyping

Although the object oriented analysis and design describe the system features, it is important to construct a prototype of some of the key system components shortly after the products are selected. It has been said “a picture may be worth a thousand words, but a prototype is worth a thousand pictures”. Not only is this true, it is an understatement of the value of

software prototyping. Essentially a prototype is a version of a software product developed in the early stages of the products life cycle for specific, experimental purpose.

A prototype enable you to fully understand how easy or difficult it will be to implement some of the features of the system. It also can give users a chance to comment on the usability and usefulness of the user interface design and lets you assess the fit between the software tools selected the functional specification, and the user needs.

Traditionally prototyping was used as a “quick and dirty” way to test the design user interface and so forth something to be thrown away when the “industrial strength” version was developed. However the new trend such as using rapid application development, is to refine the prototype into the final product. Prototyping provides the developer a means to test and refine the user interface and increase the usability of the system.

As the underlying prototype design begins to become more consistent with the application requirements, more details can be added to the application, again with further testing, evaluation, and rebuilding.

Prototype have been categorized in various ways. The following categories are some of the commonly accepted prototypes and represent very distinct ways of viewing a prototype, each having its own strengths.

- A horizontal prototype is a simulation of the interface but contains no functionality. This has the advantages of being very quick to implement, providing a good overall feel of the system and allowing users to evaluate the interface on the basis of their normal expected perception of the system.
- A vertical prototype is a subset of the system features with complete functionality. The principal advantage of this method is that the few implemented functions can be tested in great depth. In practice prototypes are a hybrid between horizontal and vertical.
- An analysis prototype is an aid for exploring the problem domain. This class of prototype is used to inform the user and demonstrate the proof of a concept. It is not used as the basis of de, however and is discarded when it has served its purpose. The final product will use the concepts exposed by the prototype not its code.

- A domain prototype is an aid for the incremental development of the ultimate software solution. It often is used as a tool for the staged delivery of subsystems to the users or other members of the development team.

Prototyping should involve representation from all user groups that will be affected by the project, especially the end users and management members to ascertain that the general structure of the prototype meets the requirements established for the overall design. The purpose of this review is threefold:

1. To demonstrate that the prototype has been developed according to the specification and that the final specification is appropriate.
2. To collect information about errors or other problems in the system, such as user interface problems that need to be addressed in the intermediate prototype stage.
3. to give management and everyone connected with the project the first glimpse of what the technology can provide.

The evaluation can be performed easily if the necessary supporting data is readily available. Testing considerations must be incorporated into the design and subsequent implementation of the system.

2.3.4 Implementation: Component based Development

Manufacturers long ago learned the benefits of moving from custom development to assembly from prefabricated components. Component based manufacturing makes many products available to the marketplace that otherwise would be prohibitively expensive. If products from automobiles to plumbing fittings to PCs were custom designed and built for each customer, the way business applications are then large market for each customer, the way business applications are then large markets for these product would not exist.

For example, computer aided software engineering (CASE) tools allow their users to rapidly develop information systems. The main goal of CASE technology is the entire information system's development life cycle process using a set of integrated software tools, such as modeling, methodology, and automatic code generation. However most often the code generated by CASE tools is only the skeleton of an application and a lot needs to be filled in by programming by hand. A new generation of CASE tools is beginning to support components based development.

Component based development (CBD)

It is an industrialized approach to the software development process. Application development moves from custom development to assembly of rebuilt, retested, reusable software components that operate with each other. Two basis ideas underlie component based development. First the application development can be improved significantly if application can be assembled quickly from prefabricated software components could be made available to developers in both general and specialist catalogs.

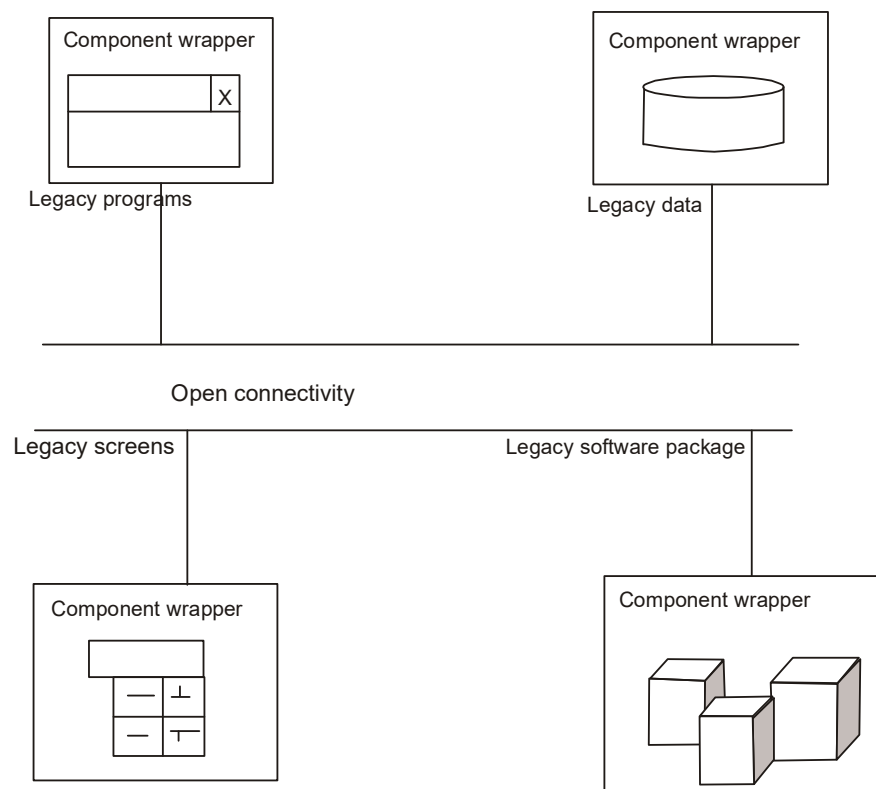


Figure 2.6

A CBD developer can assemble components to construct a complete software system. Components themselves may be constructed from other components and so on down to the level of rebuilt components or old fashioned code written in language such as C assembler, or COBOL. Existing applications support critical services within an organization and therefore cannot be thrown away:

The CBD approach to legacy integration involves application wrapper surrounds a complete system, both code and data. This wrapper then provides an interface that can interact with both the legacy and the new software system (see figure 2.6). off the shelf application wrappers are not widely available. At present, most application wrappers are homegrown within organizations.

The software components are the functional units of a program building blocks offering a collection of reusable services. A software component can request a service from another component or deliver its own services on request. The delivery of services is independent, which means that components work together without interfering with each other. Each component is unaware of the context without interfering with each other.

Rapid application development (RAD) is a set of tools and techniques that can be used to build an application faster than typically possible with traditional methods. The term often is used in conjunction with software prototyping. It is widely held that, to achieve RAD, the developer sacrifices the quality of the product for a quicker delivery. This is not necessarily the case. RAD is concerned primarily with reducing the “time to market” not exclusively the software development time. In fact one successful RAD application achieved a substantial reduction in time to market but realized no significant reduction in the individual so cycles.

RAD does not replace the system development life cycle but complements it since it focuses more on process description and can be combined perfectly with the object oriented approach.

The task of RAD is to build the application quickly and incrementally implement the design and user requirements, through tools such as VB or PB. RAD involves a number of iterations. Through each iteration we might understand the problem a little better and make an improvement. RAD encourages the incremental development approach of “grow, do not build” software.

2.3.5 Incremental testing

If you wait after development to test an application for bugs and performance, you could be wasting thousand of dollars and hours of time. That’s what happened at bankers Trust in 1992. “our testing was very complete and good but it was costing a lot of money and would add months onto a project”, says Glenn Shimamoto, vice president of technology and strategic planning at the new York bank. The problem was that developer would turn over applications to a quality assurance(QA) group for testing only after development was completed. Since the QA group wasn’t included in the initial plan, it had no clear picture of the system characteristics until it came time to test.

2.4 Reusability

A major benefit of object oriented system development is reusability, and this is the most difficult promise to deliver on. For an object to be really reusable, much more effort must be spent designing it. The potential benefits of reuse are clear increased reliability, reduced time and cost for development and improved consistency. You must effectively evaluate existing software components for reuse by asking the following questions as they apply to the intended applications.

- Has my problem already been solved?
- Has my problem been partially solved?
- What has been before to solve a problem similar to this one?

To answer these questions we need detailed summary information about existing software components. In addition to the availability of the information, we need some kind of search mechanism that allows us to define the candidate object simply and then generate broadly or narrowly defined queries. Thus the ideal system for reuse would function like a skilled reference librarian.

The reuse strategy can be based on the following:

- Information hiding
- Conformance to naming standards.
- Creation and administration of an object repository.
- Encouragement by strategic management of reuse as opposed to constant redevelopment.
- Establishing targets for a percentage of the object in the project to be reused.

2.5 Patterns

An emerging idea systems development is that the process can be improved significantly if a system can be analyzed, designed, and built from prefabricated and predefined system components. One of the first things that any science or engineering discipline must have is a vocabulary for expressing its concepts and a language for relating them to each other.

A **Pattern** is instructive information that captures the essential structure and insight of a successful family of proven solutions to a recurring problem that arises within a certain context and system of forces.

It is immensely popular to use the word pattern to garner an audience. However not every solution algorithm, best practice, maxim, or heuristic constitutes a pattern more key pattern ingredients may be absent. Even if something appears to have all requisite pattern components it should not be considered a pattern until it has been verified to be recurring phenomenon.

A “pattern in waiting” which is not yet known to recur sometime is called a **proto pattern**. many also feel it is inappropriate to decisively call something a pattern until it has undergone some degree of peer scrutiny or review[5]. Explains that a good pattern will do the following:

- It solves a problem. Pattern capture solutions not just abstract principles or strategies.
- It is proven concept. Pattern capture solution with a track record not theories or speculation.
- The solution is not obvious.
- It describes a relationship. Pattern do not just describe modules but describe deeper system structure and mechanisms.
- The pattern has a significant human component.

2.5.1 Generative and Non generative pattern

Generative pattern are patterns that not only describe a recurring problem, they can tell us how to generate something and can be observed in the resulting system architectures they helped shape. We should strive to document generative pattern because they not only show us the characteristics of good system they teach us how to build them.

2.5.2 Pattern Template

Every pattern must be expressed “in the form of rule which establishes a relationship between a context, a system of forces which arises in that context and a configuration, which allows these force to resolve themselves in that context”.

The following essential components should be clearly recognizable on reading a pattern.

- **Name.** A meaningful name. This allows us to use a single word or short phrase to refer to the pattern and the knowledge and structure it describes.
- **Problem.** A statement of the problem that describes its intent. The goals and objectives it wants to reach within the given context and forces.
- **Context.** The preconditions under which the problem and its solution seem to recur and for which the solution is desirable.
- **Forces.** A description of the relevant forces and constraints and how they interact or conflict with one another and with the goal we wish to achieve. Forces reveal the presence of the tension or dissonance they create.
- **Solution** Static relationship and dynamic rules describing how to realize the desired outcome. This often is equivalent to giving instructions that describe how to construct the necessary products. The solution should describe not only the static structure but also dynamic behavior.
- **Examples.** One or more sample application of the pattern that illustrate a specific initial context: an example may be supplemented by a sample implementation to show one way the solution might be realized. Easy to comprehend examples from known system usually are preferred.
- **Resulting context.** The state or configuration of the system after the pattern has been applied including the consequences of applying the pattern. It describes the post conditions and side effects of the pattern. This is sometimes called a resolution of forces.
- **Rationale.** A justifying explanation of steps or rules in the pattern and also of the pattern as a whole in terms of how and why it resolves its forces in a particular way to be in alignment with desired goal, principles, and philosophies. Related pattern. The static and dynamic relationship between this pattern and others within the same pattern language or system related pattern often share common forces.
- **Known uses.** The known occurrences of the pattern and its application within existing systems. This helps validate a pattern by verifying that it indeed is a proven solution to a recurring problem. Known uses of the pattern often can serve as instructional example.

Summary

- High quality provides users with an application that meets their needs and expectations.
- Four quality measures have been described
 1. correspondence measures how well the delivered system corresponds to the needs of the problem
 2. correctness determines whether or not the system correctly computes the results based on the rules created during the system analysis and design, measuring the consistency of product requirements with respect to the design specification.
 3. verification is the task of determining correctness.
 4. validation is the task of predicting correspondence.
- Component based development (CBD) is an industrialized approach to software development.
- The rapid application development approach to system development rapidly develops so to quickly and incrementally implement the design by using tools such as CASE.

Questions:

1. What is the waterfall SDLC?
2. What is software correspondence?
3. What is so correctness?
4. What is software validation?
5. What is software verification?
6. How is software verification different from validation?
7. What is prototyping and why is it useful?
8. What is use case modeling?
9. What is object modeling?
10. What is RAD?
11. Why is CBD important?
12. What are the object oriented analysis process?
13. What are some of the advantages and disadvantages of prototyping?
14. Describe the pattern?

LESSON - 3

METHODOLOGY AND FRAMEWORK

Objectives:

After studying this lesson you should be able to

- Object oriented methodologies
- The Rumbaugh et al OMT.
- The Booch methodologies.
- Jacobson's methodologies.
- Frameworks.
- Unified approach(UA).

Introduction

In the 1980s many methodologists were wondering how analysis and design methods and process would fit into an object oriented world. Object oriented methods suddenly had become very popular, and it was apparent that the techniques to help people execute good analysis and design were just as important as the object oriented concept itself. 1980s to 1990s in between period the number of method are developed.

These methodologies and many other forms of notational language provided system designers and architects many choices but created a very split, competitive, and confusing environment.

The trend in object oriented methodologies sometimes called second generation object oriented methods, has been toward combining the best aspects of the most popular methods instead of coming out with new methodologies which was the tendency in first generation object oriented methods.

3.1 Survey of some of the object oriented methodologies

Many methodologies are available to choose from for system development. Each methodology is based on modeling the business problem and implementing the application in an object oriented fashion; the differences lie primarily in the documentation of information and

modeling notation and language. An application can be implemented in many ways to meet the same requirements and provide the same functionality.

In the following sections we look at the methodologies and their modeling notations developed by Rumbaugh et al., and Jacobson which are origins of the Unified Modeling Language (UML). Each method has its strengths.

- The Rumbaugh et al. method is well suited for describing the object model or the static structure of the system.
- The Jacobson et al. method is good for producing user driven analysis models.
- The Booch method produces detailed object oriented design models.

3.2 Rumbaugh et al.'s object modeling technique

The object modeling technique(OMT) presented by Jim Rumbaugh and his coworkers describes a method for the analysis, design and implementation of a system using an object oriented technique. OMT is fast intuitive approach for identifying and modeling all the objects making up a system.

A process description and consumer producer relationship can be expressed using OMT's functional model. OMT consists of four phases, which can be performed iteratively.

- Analysis. The results are objects and dynamic and functional models.
- System design. The results are a structure of the basic architecture of the system along with high level strategy decisions.
- Object design. This phase produces a design document, consisting of detailed object static, dynamic, and functional models.
- Implementation. This activity produces reusable, extendible, and robust code.

OMT separates modeling into three different parts.

- An object model, presented by the object model and the data dictionary.
- A dynamic model, presented by the state diagrams and event flow diagrams.
- A functional model, presented by data flow and constraints.

3.2.1 The object model

The object model describes the structure of objects in a system their identity relationships to other objects, attributes and operations. The object model is represented graphically with an object diagram (see figure 3.1). the object diagram contains classes interconnected by association lines. Each class represents a set of individual objects. The association lines establish relationship among the classes. Each association line represents a set of links from the objects of one class to the objects of another class.

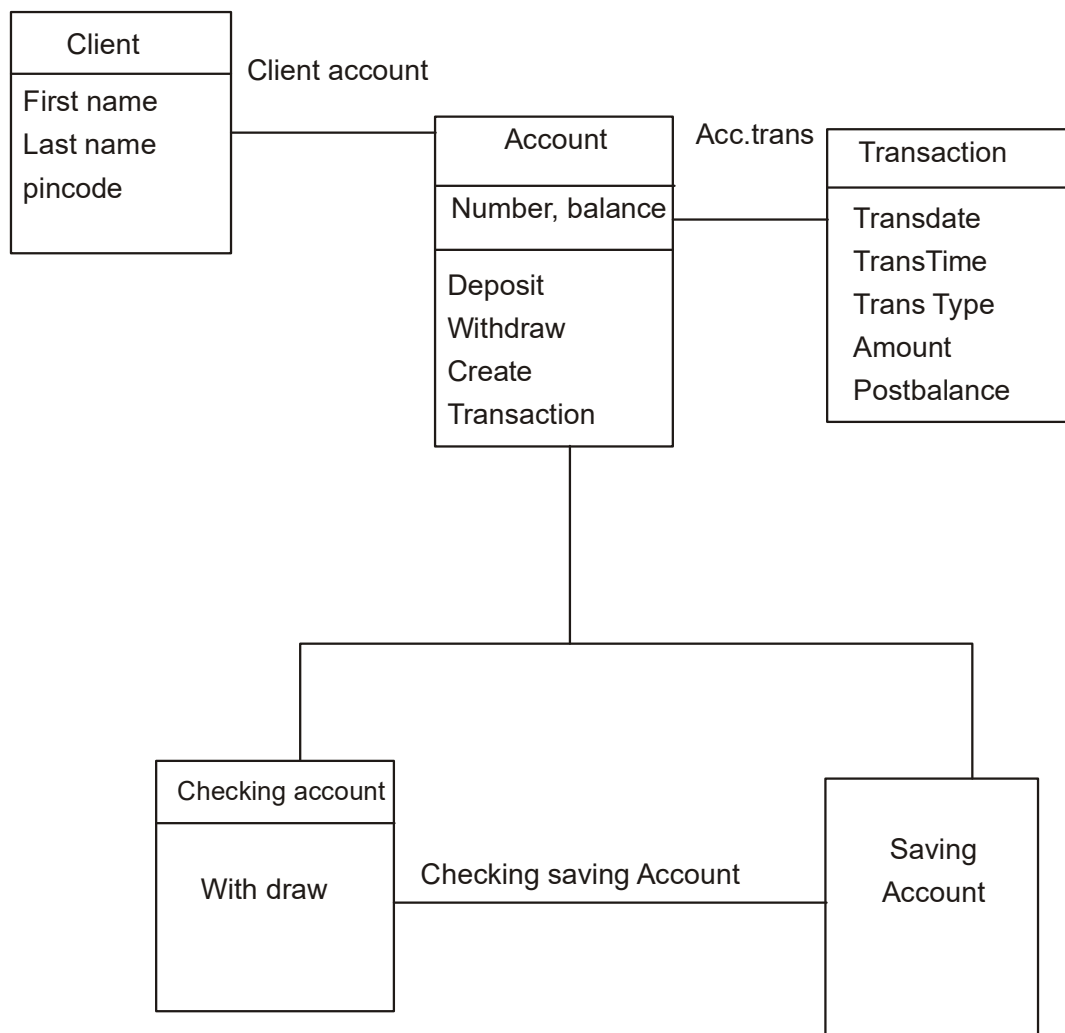


Figure 3.1

3.2.2 The OMT Dynamic Model

OMT provides a detailed and comprehensive dynamic model in addition to letting you depict states, transitions, events and actions. The OMT state transition diagram is a network of states and event(see fig 3.2). Each state receives one or more events, at which time it makes the transition to the next state. The next state depends on the current state as well as the event.

3.2.3 The OMT functional model

The OMT data flow diagram (DFD) shows the flow of data between different processes in a business. An OMT DFD provides a simple and intuitive method for describing business processes without focusing on the details of computer system.

Data flow diagrams use four primary symbols:

1. process

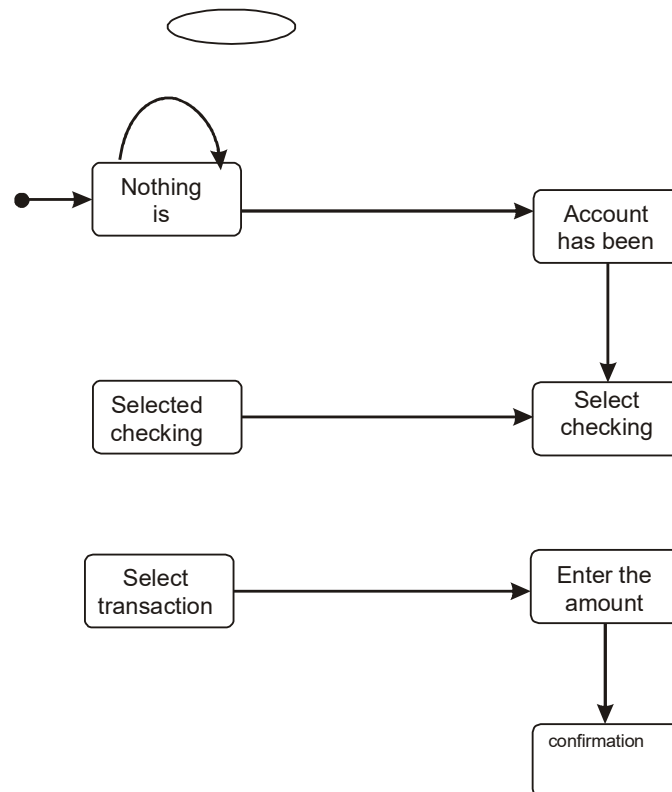


Figure 3.2

The process is any function being performed. for example, verify password or pin in the ATM system (see figure 3.3).

2. Data flow



The data flow shows the direction of the data element movement. For example, Pin code.

3. Data store

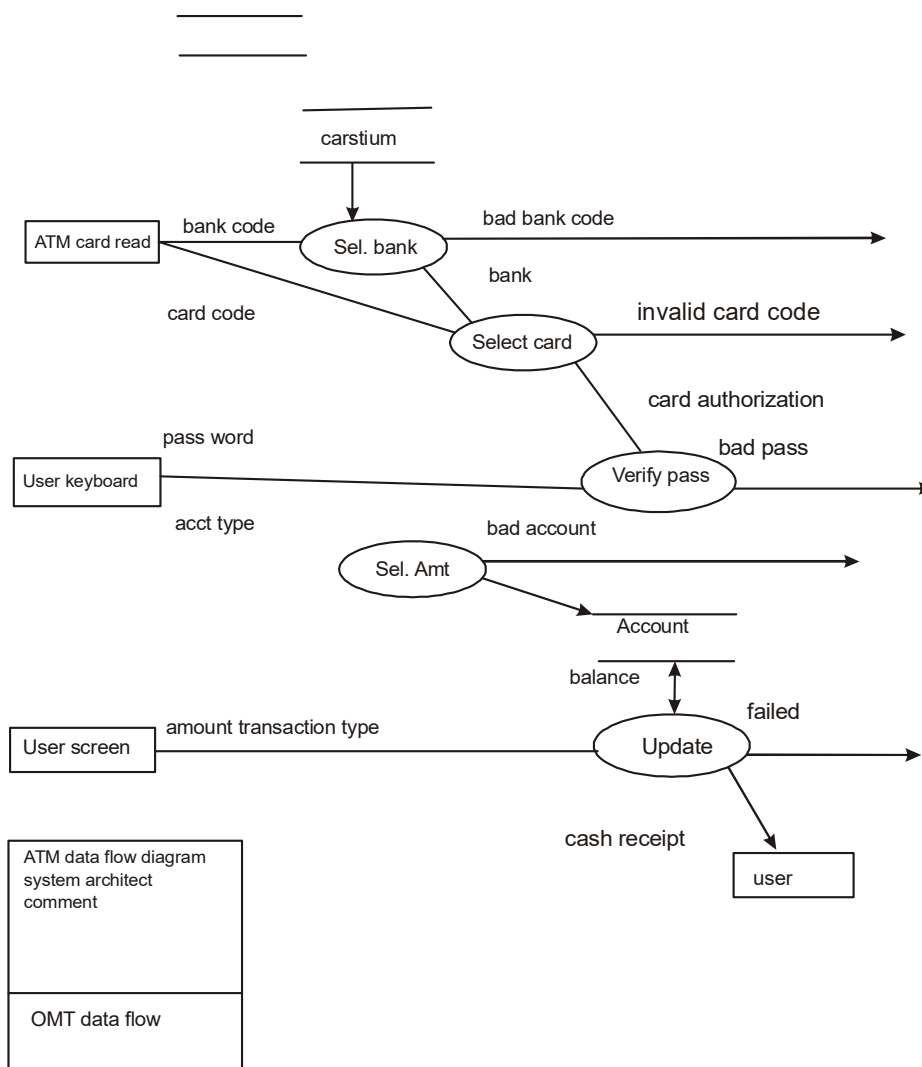


Figure 3.3

The data store is a location where data are stored. For example, account is a data store in the ATM example.

4. External entity



An external entity is a source or destination of a data element. For example, the ATM card reader.

3.3 The Booch Methodologies

The booch methodologies is a widely used object oriented methods that help you design your system using the object paradigm. It covers the analysis and design phases of an object oriented system. Even though booch defines a lot of symbols to document almost every design decision, if you work with his method, you will notice that you never use all these symbol and diagrams. You start with class and object diagrams (see fig 3:4 and 3.5) in the analysis phase and refine these diagrams in various steps. The Booch method consists of the following diagrams:

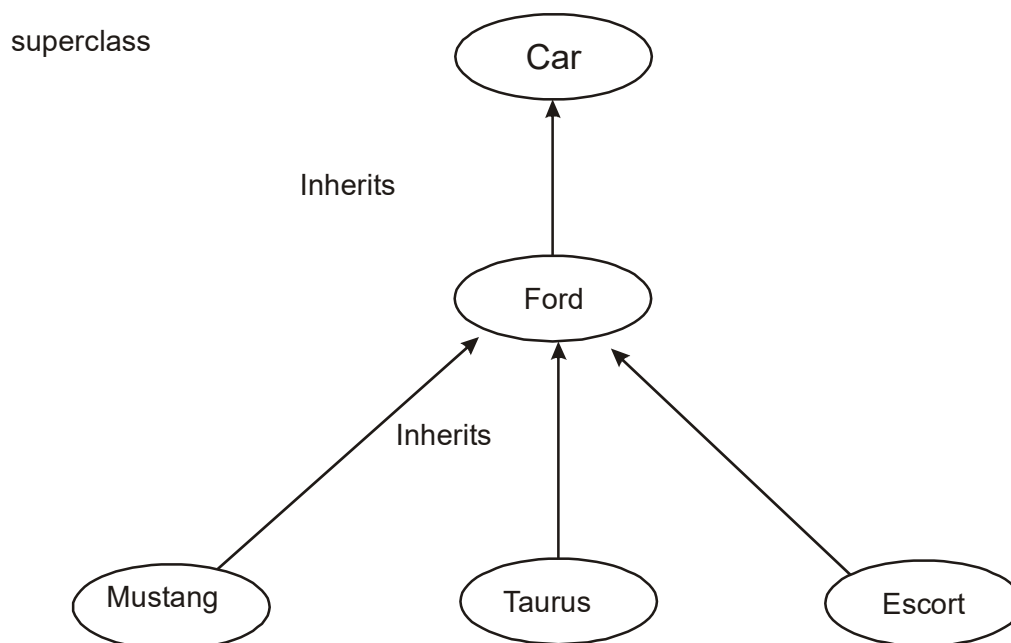


Figure 3.4

- Class diagrams
- Object diagrams
- State transition diagrams
- Module diagrams
- Process diagrams
- Interaction diagrams

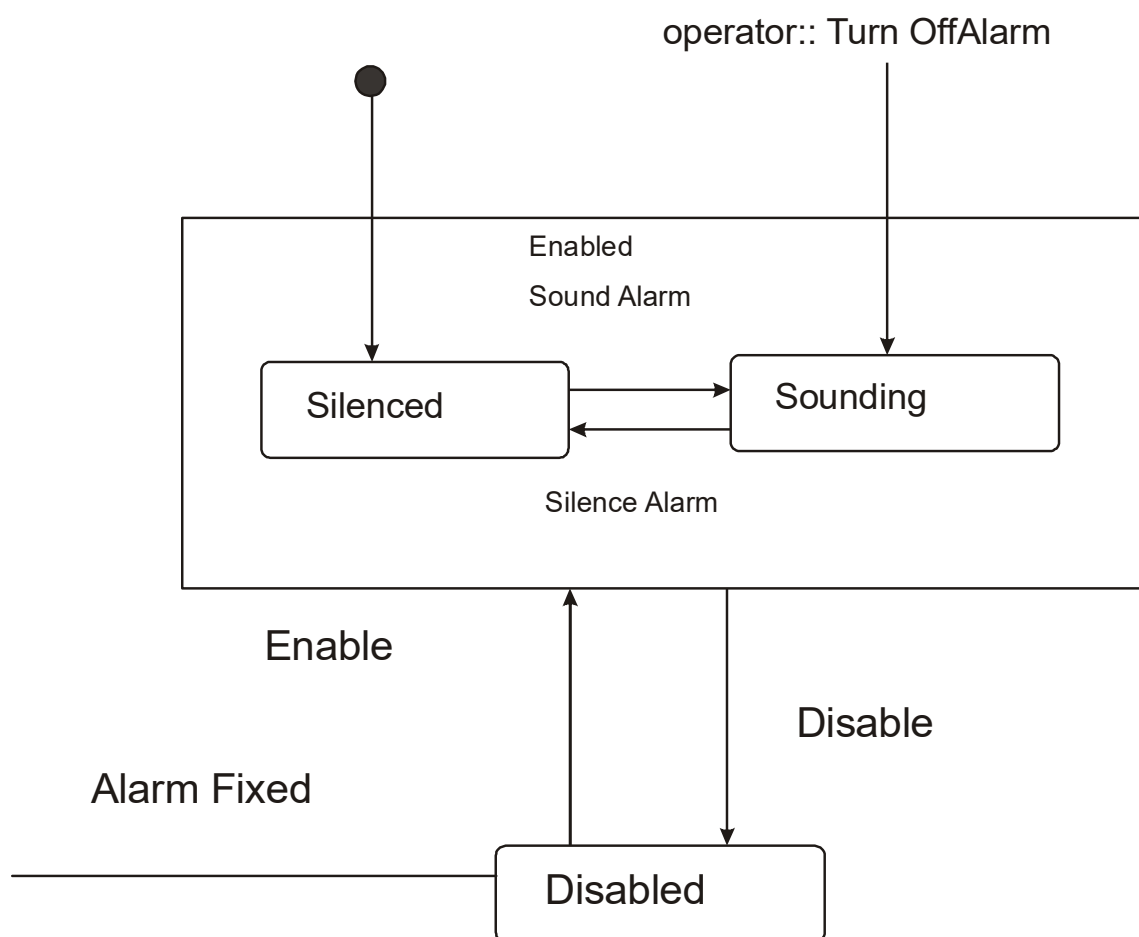


Figure 3.5

3.3.1 The macro development process

The macro process serves as a controlling framework for the micro process and can take week or even months. The primary concern of the macro is technical management of the system. Such management is interested less in the actual object oriented design than in how well the project corresponds to the requirements set for it and whether it is produced on time.

The macro development process consists of the following steps:

- Conceptualization. During Conceptualization you establish the core requirements of the system. You establish a set of goals and develop a prototype to prove the concept.
- Analysis and development of the model. In this step you use the class diagram to describe the role and responsibilities objects are to carry out in performing the desired behavior of the system.
- Design or create the system architecture. In the design phase you use the class diagram to decide what classes exists and how they relate to each other. Next, you can the object diagram to decide what mechanisms are used to regulate how object collaborate.
- Evolution or implementation. Successively refine the system through many iterations. Produce a stream of software implementations.
- Maintenance. Make localized changes to the system to add new requirements and eliminate bugs.

3.3.2 The macro development process

Each macro development process has its own micro development processes. The micro process is a description of the day to day activities by a single or small group of software developers, which could look blurry to an outside viewer, since the analysis and design phases are not clearly defined.

The micro development process consists of the following steps:

1. Identify classes and object.
2. Identify class and object semantics.
3. Identify class and object relationships.
4. Identify class and object interfaces and implementation.

3.4 The Jacobson et al. methodologies

The Jacobson et al. methodologies

e.g.,

- Object oriented business Engineering(OOBE).
- Object oriented software Engineering(OOSE).
- Objectors

Cover the entire life cycle and stress trace ability between the different phases, both forward and backward. This trace ability enables reuse of analysis and design work, possibly much bigger factors in the reduction of development time than reuse of code.

3.4.1 Use Case

Use cases are scenarios for understanding system requirements. A use case is an interaction between users and a system. The use case model captures the goal of the user and the responsibility of the system to its users(see figure 3.6). in the requirements analysis the use cases are described as one of the following.

- Nonformal text with no clear flow of events.
- Text, easy to read but with a clear flow of events to follow.
- Formal style using pseudo code.

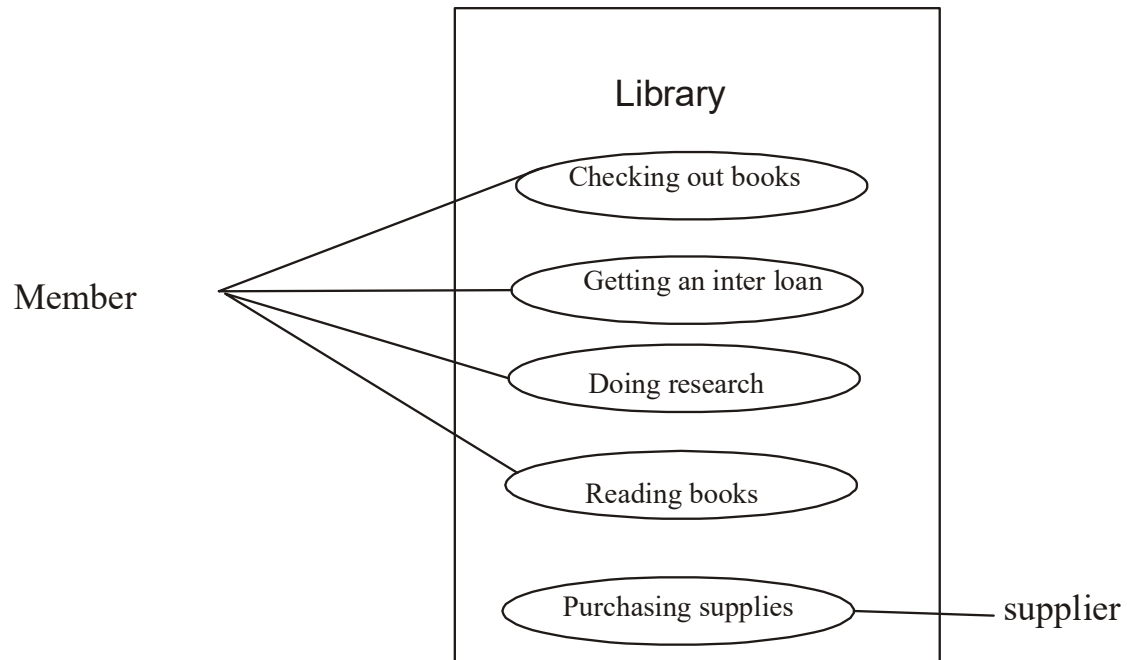


Figure 3.6

The use when the use case begins and ends.

- How and when the use case begins and ends.
- The interaction between the use case and its actors, including when the interaction occurs and what is exchanged.
- How and when the use case will need data stored in the system or will store data in the system.
- Exceptions to the flow of events
- How and when concepts of the problem domain are handled

The uses relationship reuses common behavior in different use cases. Use cases could be viewed as concrete or abstract. An abstract use case is not complete and has no actors that initiate it but is used by another use case.

3.4.2 Object oriented software engineering :objectory

Object oriented software engineering(OOSE), also called objectory is a method of object oriented development with the specific aim to fit the development of large real time system. The development process called use case driven development, stresses that use cases are involved in several phases of the development (see fig 3.7). including analysis, design, validation and testing

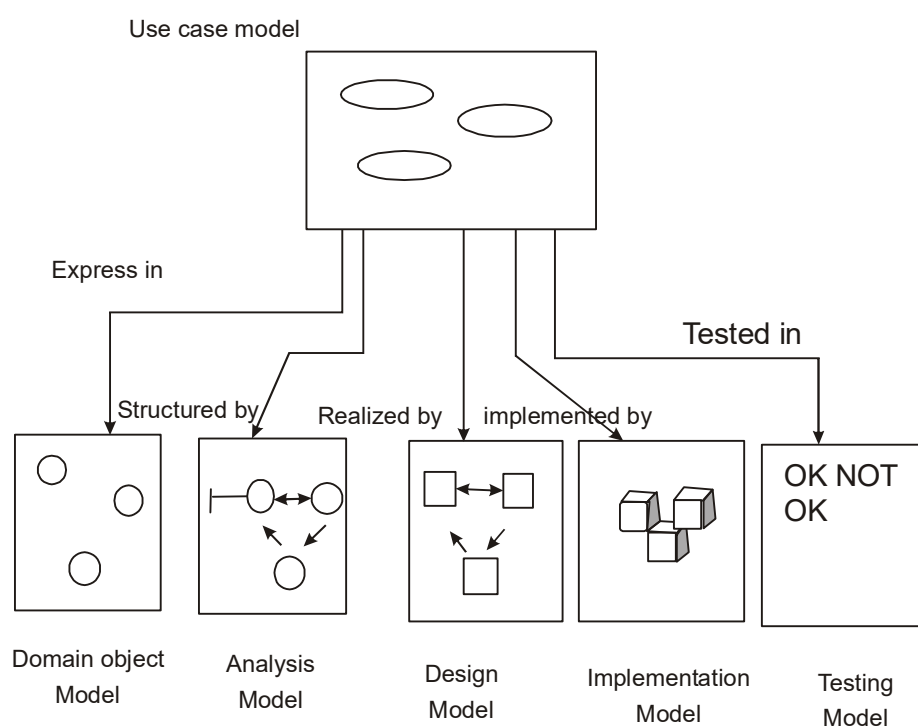


Figure 3.7

Jacobson et al.'s objectory has been developed and applied to numerous areas and embodied in the CASE tool systems.

Objectory is built around several different models:

- Use case model. The use case model defines the outside and inside of the system's behavior.
- Domain object model. The objects of the real world are mapped into the domain object model.

- Analysis object model. The analysis object model presents how the source code should be carried out and written.
- Implementation model. The implementation model represents the implementation of the system.
- Test model. The test model constitutes the test plans, specification and reports.

3.4.4 Object oriented business engineering

Object oriented business engineering (OOBE) is object modeling at the enterprise level. Use case again are the central vehicle for modeling, providing trace ability throughout the software engineering process.

- Analysis phase. The analysis phase defines the system to be built in terms of the problem domain object model, the requirements model and the analysis model. The analysis process should not take into account the actual implementation environment.
- Design and implementation phase. The implementation environment must be identified for the design model. This includes factors such as database management system (DBMS) distribution of process constraints due to the programming language available components libraries and incorporation of graphical user interface tools. It may be possible to identify the implementation environment concurrently with analysis.
- Testing phase. Finally Jacobson describes several testing level band techniques. The level include unit testing, integration testing and system testing.

3.5 Framework

Framework are a way of delivering application development patterns to support best practice sharing during application development not just within one company. This is not an entirely new idea. Consider the following.

- An experienced programmer almost never code a new program from scratch she'll use macros copy libraries, and template like code fragments from earlier programs to make a start on a new one.
- A seasoned data modeling consultant who has worked on many consulting projects performing data modeling almost never builds as new data model from scratch he'll

have selection of model fragments that have been developed over time to help new modeling projects hit the ground running.

A framework is a way of presenting a generic solution to a problem that can be applied to all levels in a development. A definition of an object oriented software framework is given by Gamma et al.[15].

“A framework is a set of cooperating classes that make up a reusable design for a specific class of software, a framework provides architectural guidance by partitioning the design into abstract classes and defining their responsibilities and collaborations.”

Gamma et al. describe the major difference between design pattern and frameworks as follows.

- Design pattern are more abstract than frameworks. Frameworks can be embodied in code but only examples of pattern can be embodied in code. a strength of frameworks is that they can be written down in programming language and not only studied but executed and reused directly. In contrast design pattern have to be implemented each time they are used. Design pattern also explain the intent, trade offs and consequences of a design.
- Design patterns are smaller architectural elements than framework. A typical framework contains several design pattern but the reverse is never true.
- Design patterns are less specialized than frameworks. Frameworks always have a particular application domain. In contrast, design pattern can be used in nearly any kind of application.

3.6 The unified approach

The approach promoted in this lesson is based on the best practices that have proven successful in the system development and more specifically, the work done by Booch and Jacobson. The main motivation here is to combine the best practices, processes, methodologies object oriented concepts and system development.

The unified approach to software development revolve around the following processes and concepts (see figure 3.8) the processes are:

- Use case driven development.
- Object oriented analysis
- Object oriented design
- Incremental development and prototyping
- Continuous testing

The method and technology employed include

Unified modeling language used for modeling

Layered approach.

Repository for object oriented system development pattern and frameworks.

Component based development.

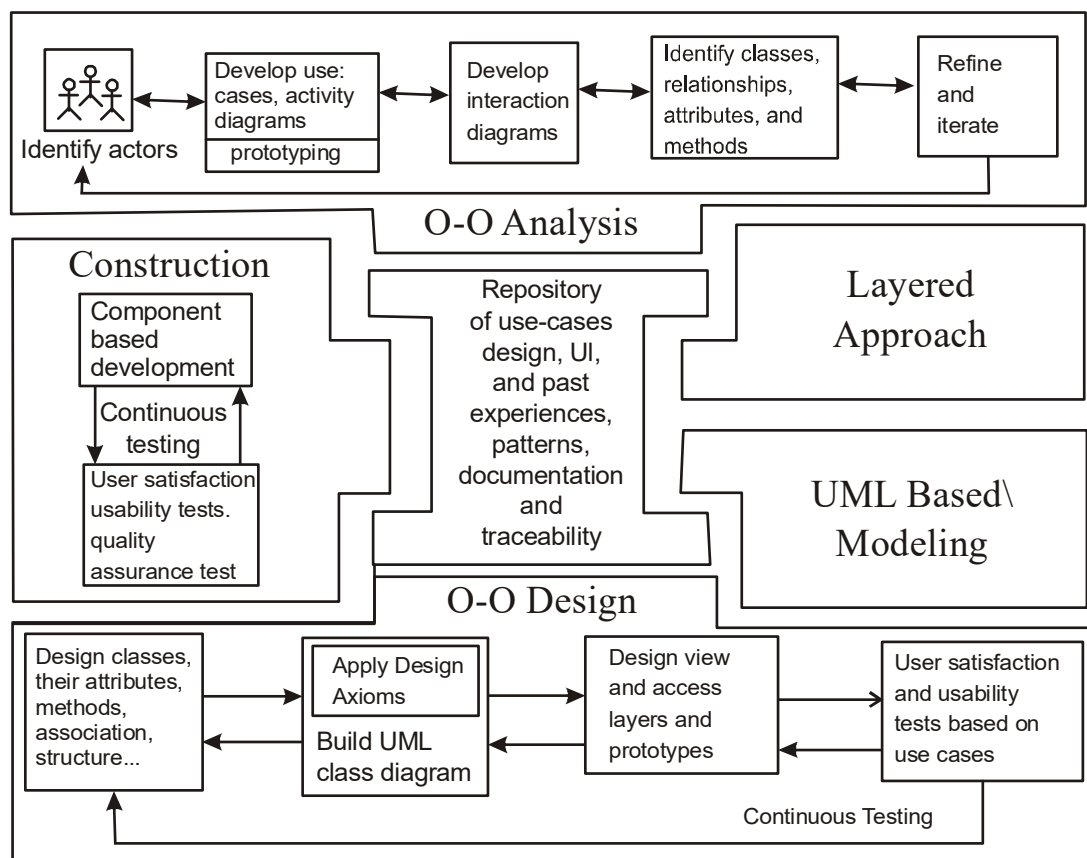


Figure 3.8

3.6.1 object oriented analysis

analysis is the process of extracting the needs of a system and what the system must do to satisfy the user's requirements. The goal of object oriented analysis is to first understand the domain of the problem and the system's responsibilities by understanding how the users use or will use the system. These models concentrate on describing what the system does rather than how it does it. Separating the behavior of a system from the way it is implemented viewing the system from the user's perspective rather than that of the machine. OOA process consists of the following steps.

- Identify the actors
- Develop a simple business process model using UML. Activity diagram.
- Develop the use case
- Develop interaction diagrams
- Identify classes.

3.6.2 object oriented design

The most comprehensive object oriented design method. Ironically, since it is so comprehensive, the method can be some what imposing to learn and especially tricky to figure out where to start. Booch's object diagrams, and Rumbaugh et al.'s domain models. Furthermore by following Jacobson et al.'s life cycle model, we can produce design that are traceable across requirements, analysis, design, coding, and testing. OOD process consists of:

- Designing classes, their attributes, methods, associations, structures and protocols, apply design axioms.
- Design the access layer
- Design and prototype user interface
- User satisfaction and usability test based on the usage/use cases
- Iterate and refine the design

3.6.3 Iterative development and continuous testing

You must iterate and reiterate until, eventually, you are satisfied with the system. Since testing often uncovers design weaknesses or at least provides additional information you will

want to use. During this iterate process, your prototypes will be incrementally transformed into the actual application. The UA encourages the integration of testing plans from day 1 of the project. golaveC

3.6.4 Modeling Based in the unified Modeling language

The unified modeling language was developed by joint effort of the leading object technologists Grady Booch, Ivar Jacobson and Rumbaugh with contributions from many others. The UML merges the best of the notation used by the three most popular analysis methodologies.

- Booch's methodologies
- Jacobson et al.'s use case
- Rumbaugh et al.'s object modeling technique.

UML is becoming the universal language for modeling systems it is intended to be used to express models of many different kinds and purposes.

3.6.5 The UA proposed repository

In modern businesses best practice sharing is way to ensure that solutions to process and organization problems in one part of the business are communicated to other parts where similar problems occur. Best practice sharing eliminates duplication of the problem solving. The idea promoted here is to create a repository that allow the maximum reuse of previous experience and previously objects, b pattern, framework, and user interfaces in an easily accessible manner with a completely available and easily utilized format.

The advantage of repositories from those projects might be useful. You can select any piece from a repository from the definitions to one data element to a diagram all its symbols and all their dependent definitions to entries for reuse. The same arguments can be made about patterns and frameworks. Specification of the software components, describing the behavior of the component and how it should be used are registered in the repository for future reuse by teams of developers.

The repository should be accessible to many people. Furthermore it should be relatively easy to search the repository for classes based on their attribute, methods, or other characteristics.

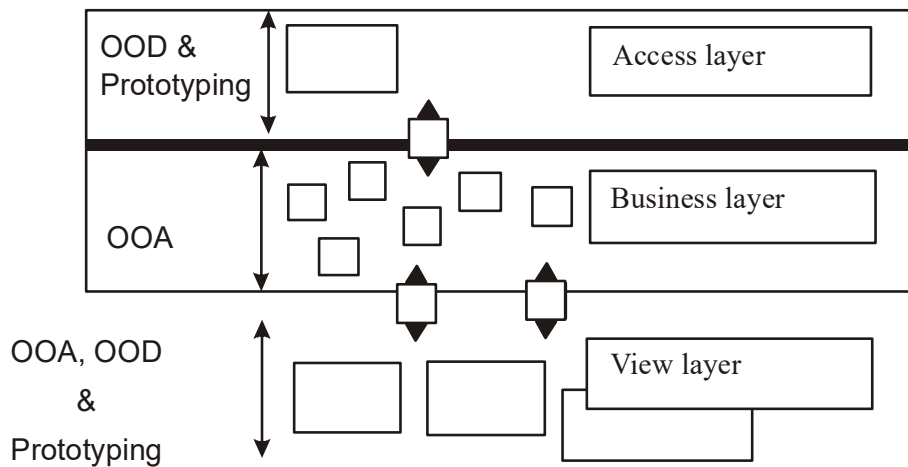


Figure 3.11

3.6.6.1 The business layer

The business layer contains all the objects that represent the business. this is where the real objects such as order, customer, line item, inventory and invoice exist. The responsibilities of the business layer are very straightforward: model the objects of the business and how they interact to accomplish the business processes. These objects should not be responsible for the following:

- Displaying details. Business objects should have no special knowledge of how they are being displayed and by whom. They are designed to be independent of any particular interface.
- Data access details. Business also should have no special knowledge of “where they come from”. It does not matter to the business model whether the data are stored and retrieved via SQL or file I/O.

A business model captures the static and dynamic relationships among a collection of business objects. Static relationships include object associations and aggregations. Dynamic relationships show how the business objects interact to perform tasks.

3.6.6.2 The user interface layer

The user interface layer consists of objects with which the user interacts as well as the objects needed to manage or control the interface. The user interface layer also is called the view layer.

This layer typically is responsible for two major aspects of the application.

- Responding to user interaction. The user interface layer objects must be designed to translate actions by the user, such as clicking on a button or selecting from menu, into an appropriate response. That response may be to open or close another interface or to send a message down into the business layer to start some business process.
- Displaying business objects. This layer must paint the best possible picture of the business object for the user. In one interface this may mean entry fields and list boxes to display an order and its items.

3.6.6.3 The Access layer

The Access layer consists objects that know how to communicate with the place where the data actually reside, whether it be a relational database, mainframe, internet, or file. The access layer has two major responsibilities.

- Translate request. The access layer must be able to translate any data related requests from the business layer into the appropriate protocol for data access.
- Translate results. The access layer also must be able to translate the data retrieved back into the appropriate business objects and pass those back up into the business layer.

Summary

- Jacobson et al. have a strong method for producing user driven requirements and object oriented analysis models
- Booch has a strong method for producing detailed object oriented design model. naved some bar odn
- Rumbaugh et al.'s OMT has strong method for modeling the
- problem domain.
- The main idea behind a pattern is the documentation to help categorize, communication about and locate solutions to recurring problems. STMO od To randy art su

- The UA is attempt to combine the best practices, processes and guidelines along with UML notation and diagrams for better understanding object oriented concepts and object oriented system develop. n the UA consists of the following processes.
- Use case driven development
- Object oriented analysis
- Object oriented design
- Incremental development and prototyping
- Continuous testing

Questions:

1. What is a method?
2. What is a methodologies?
3. What is process?
4. What is a use case?
5. What is objectory?
6. Describe the difference between patterns and framework?
7. What is the strengh of the Jacobson et al. methodology?
8. Name five Booch diagrams.
9. What is strenth of OMT?
10. What are the phases of the OMT? Briefly describe each phase.

LESSON - 4

UNIFIED MODELING LANGUAGE

Objectives:

- Modeling and its benefit
- Different type of models
- Basic of unified modeling language (UML) and its modeling diagrams
- UML class diagram
- UML use case diagram
- UML sequence diagram
- UML collaboration diagram
- UML state chart diagram
- UML activity diagram
- UML component diagram
- UML deployment diagram

Introduction

A model is an abstract representation of a system, constructed to understand the system prior to building or modifying it. The term system is used here in a broad sense to include any process or structure. For eg. The organizational structure of cooperation, health service, computer software, instruction of any sort the national economic and forth all would be termed system.

A model is simplified because reality is too complex or large and much of the complexity actually is irrelevant to the problem we are trying to describe or solve. A model provides a means for conceptualization and communication of ideas in a precise manner. The characteristic of simplification and representation are difficult to achieve in the real world, since they frequently contradict each other. Also, to open

Most modeling techniques used for analysis and design involve graphic language. These graphic languages are sets of symbols the symbols are used according to certain rules of the methodologies for communicating the complex relationship of information more clearly than

descriptive text. The main goal of most CASE tools is to aid us in using these graphic language along with their associated methodologies.

For example, objectory is built around several different models. MU

- Use case model. The use case model defines the outside and inside of the system's behavior.
- Domain object model. The objects of the real world are mapped into the domain object model.^{sb 30}
- Analysis object model. The analysis object model presents how the source code should be carried out and written.
- Implementation model. The implementation model represents the implementation of the system.
- Test model. The test model constitutes the test plans, specification and reports.

4.1 static and dynamic models

Models can represent static or dynamic situation. Each representation has different implication for how the knowledge about the model might be organized and represented.

4.1.1 Static model

A static model can be viewed as a snapshot of a system's parameters at rest or at a specific point in time. Static models are needed to represent the structural or static aspects of a system. For example, a customer could have more than one account or an order could be aggregated from one or more line items.

4.1.2 Dynamic model

A dynamic model in contrast to a static model, can be viewed as a collection procedure or behavior that taken together reflect the behavior of a system over time. Dynamic relationship show how the business objects interact to perform tasks. A system can be described by first developing its static model, which is the structure of its objects and their relationship to each other frozen in time, a base lin. Then we can examine changes to the objects and their relationship over time. The UML interaction diagram and activity models are examples of UML dynamic models

4.2 Why modeling?

Building a model for a software system prior to its construction is as essential as having a blueprint for building a large building. Many other factors add to a project's success, but having a rigorous modeling language is essential. A modeling language must include

- Model elements fundamental modeling concepts and semantics.
- Notation-visual rendering of model elements.
- Guidelines -expression of usage within the trade.

The use of visual notation to represent or model a problem can provide us several benefits relating to clarity, familiarity, maintenance, and simplification.

- **Clarity.** We much better to picking out error and omissions from a graphical or visual representation than from listing of code or tables of numbers.
- **Familiarity.** The representations form for the model may turn out to be similar to the way in which the information actually is represented and used by the employees currently working in the problem domain.
- **Maintenance.** Visual notation can improve the maintainability of a system. The visual identification of location to be changed and the visual confirmation of those changes will reduce error.
- **Simplification.** Use of higher level representation generally result in the use of fewer but more general constructs, contributing to simplicity and conceptual understanding.

To summarize, here are few key ideas regarding modeling.

- A model is rarely correct on the first try.
- Always seek the advice and criticism of others. You can improve a model by reconciling different perspectives.
- Avoid excess model revisions as they can distort the essence of your model. Let simplicity and elegance guide you through the process.

4.3 Introduction to the unified modeling language

The unified modeling language is language for specifying, constructing, visualizing, and documenting the software system and its components. The UML is a graphical language with sets of rules and semantics. In a form known as object constraint language (OCL). OCL is specification language that uses simple logic for specifying the properties of a system. To goals of the unification effort were to keep it simple to cast away elements of exiting Booch, OMT and OOSE methods that did not work in practice to add elements from other methods that were more effective and to invent new methods only when an existing solution was unavailable.

For example activity diagrams accomplish much of what people want from DFD and then some activity diagram also are useful for modeling work flow. The authors of the UML clearly are promoting the UML diagrams over all others for object oriented projects but do not condemn all other diagram.

The primary goals in the design of the UML were as follows

1. Provide user a ready to use expressive visual modeling language so they can develop and exchange meaningful models.
2. Provide extensibility and specification mechanisms to extend the core concepts.
3. Be independent of particular programmer language and development processes.
4. Provide a formal basis for understanding the modeling language.
5. encourage the growth of the OO tools market
6. Support higher level development concepts.
7. Integrate best practices and methodologies

4.4 UML diagrams

Every complex system is best approached through a small set of nearly independent vies of a model; no single view is sufficient. Every model may be expressed at different level of fidelity. The best models are connected to reality. The UML define nine graphical diagrams.

1. class diagrams
2. use case diagram
3. behavior diagram (dynamic)

- 3.1 Interaction diagram
 - 3.1.1 sequence diagram
 - 3.1.2 Collaboration diagram
- 3.2 Statement diagram
- 3.3 Activity diagram
- 4 Implementation diagram
 - 4.1 component diagram
 - 4.2 deployment diagram.

4.5 UML class diagram

The UML class diagram also referred to as object modeling is the main static analysis diagram. these diagram show the static structure of the model. A class diagram is a collection of static modeling elements, such as classes and their relationship, connected as graph to each other and to their contents. Object modeling is the process by which the logical objects in the real world are represented by the actual objects in the program. This visual representation of the objects, their relationship and their structures is for ease of understanding.

The main task of object modeling is to graphically show what each will do in the problem domain, describe the structure and the relationship among objects by visual notation and determine what behavior fall within and outside the problem domain.

4.5.1 Class Notation: Static structure

A class is drawn as a rectangle with three components separated by horizontal lines. The top name compartment holds the class name, other general properties of the class, such as attributes, are in the middle components and the bottom component holds a list of operation see figure 4.1.either or both the attribute and operation components may be suppressed.

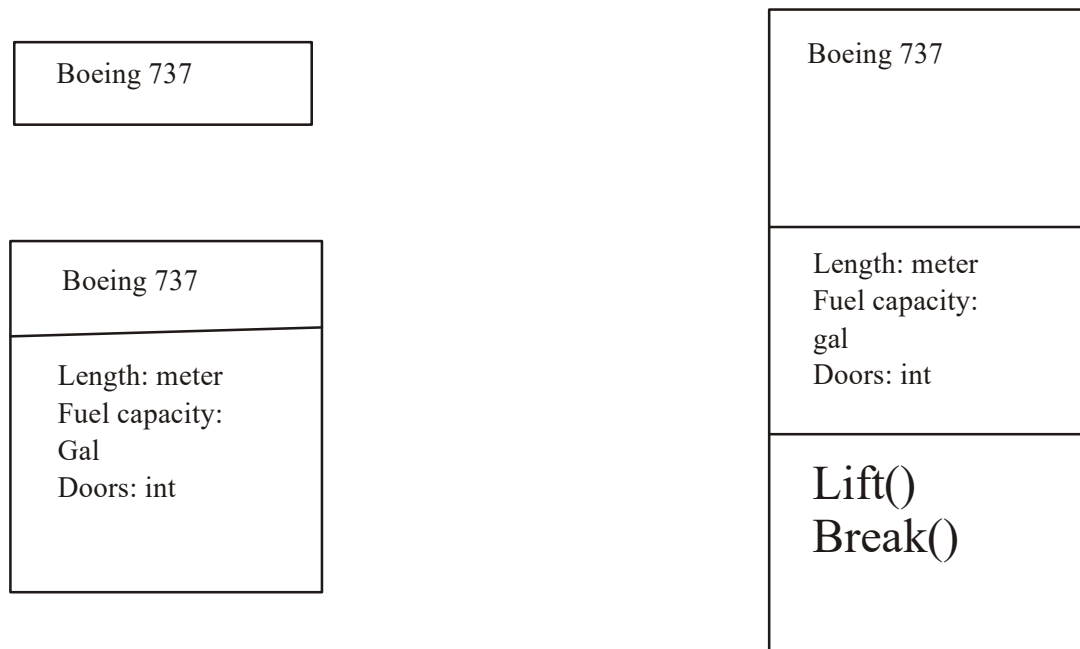


Figure 4.1

4.5.2 Object diagram

A static diagram is an instance of a class diagram. It shows a snapshot of the detailed state of the system at a point in time. Notation is the same for an object diagram and class diagram. class diagram can contain objects so a class diagram with objects and no classes is an object diagram.

4.4.3 Class interface notation

Class interface notation is used to describe the externally visible behavior of a classes. For example an operation with public visibility. Identifying class interfaces is a design activity of object oriented system development. The UML notation for an interface is a small circle with the name of the interface connected to the class. For example a person object may need to interact with the bank account object to get the balance, this relationship is depicted in figure 4.2 with UML class interface notation.



Figure 4.2

4.5.4 Binary association notation

A binary association notation is drawn as a solid path connecting two classes, or both ends may be connected to the same class. An association may have an association name. Furthermore, the association name may have an optional black triangle in it, the point of the triangle indicating the direction in which to read the name. The end of the association where it connects to a class, is called the association role see figure 4.3.

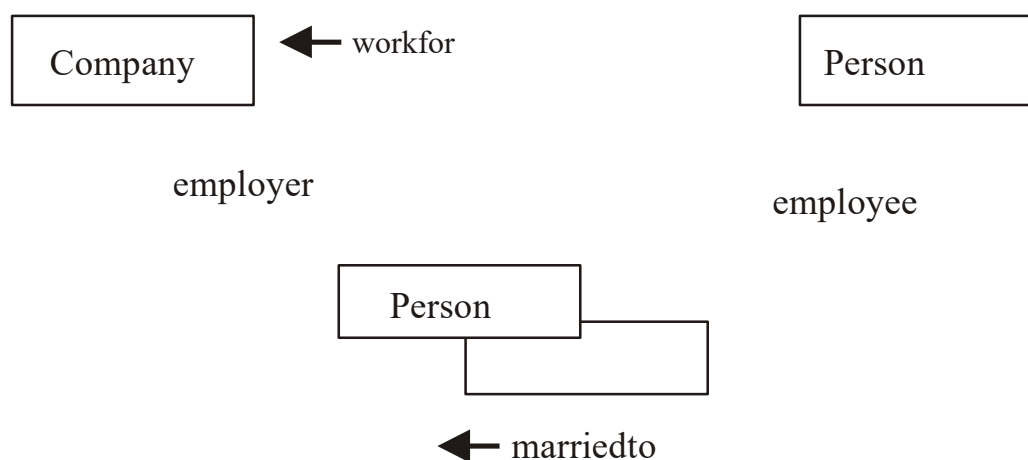


Figure 4.3

4.5.5 Association Role

A simple association the technical term for it is binary association is drawn as a solid line connecting two class symbols. The end of an association where it connects to a class, shown the association role. The role is part of the association, not part of the class. The UML uses the term association navigation or navigability to specify a role affiliated with each end of an association relationship. In the UML association is represented by an open arrow as representation in figure 4.4. In figure 4.4 the association is navigable in only one direction from the bank accounts to person but not the reverse. This might indicate a design decision, but it also might indicate an analysis decision, that the person class is frozen and cannot be extended to know about the bank account class, but the bank account class can know about the person class.



Figure 4.4

4.5.6 Qualifier

A qualifier is an association attribute. For example a person object may be association to a bank object. An attribute of this association is the account#. The account# is the qualifier of this association see figure 4.5.

A qualifier is shown as a small rectangle attached to the end of an association path between the final path segment and the symbol of the class to which it connects,

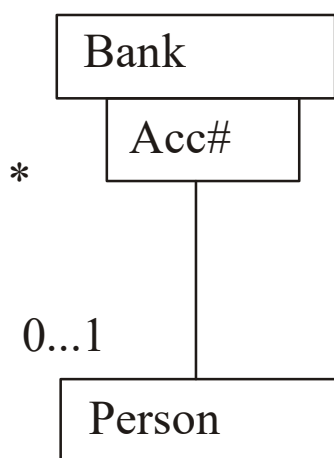


Figure 4.5

4.5.7 Multiplicity

Multiplicity specifies the range of allowable associated classes. It is given for role within association parts within compositions, repetitions, and other purposes. A Multiplicity sep is shown as a text string comprising a period separated sequence of integer intervals, where an intervals represents a range of integers in this format see figure 4.5.

lower bound.. upper bound.

The terms lower bound and upper bound are integer values. The star character (*) may be used for the upper bound denoting an unlimited upperbound. If a single integer value is specified then the integer range contains the single values. for example,

0..1

0..*

1..3,7..10,15, 19..*

4.5.8 OR Association

An or association indicates a situation in which only one of several potential association may be instantiated at one time for any single object. This shown as a dashed line connecting two or more association, all of which must have a class in common, with the constraint string or labeling the dashed line see figure 4.6.

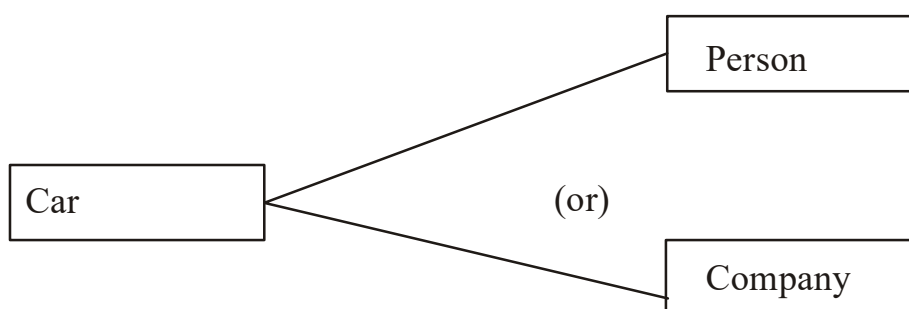


Figure 4.6

4.5.9 Association class

An Association class is an association that also has class properties. An association class is shown as a class symbol attached by a dashed line to an association path. The name in the class symbol and the name string attached to the association path are same see figure 4.7.

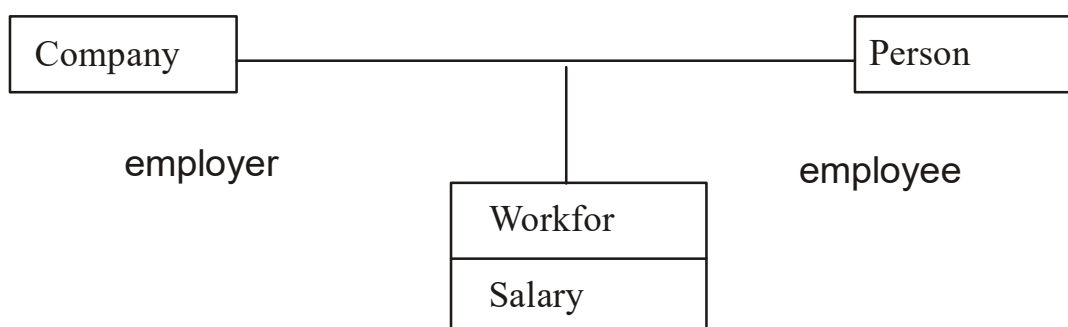


Figure 4.7

if an association class has attributes but no operations or other associations. Then the name may be displayed on the association path and omitted from the association class to emphasize its “association nature”.

If it has operational and attributes then the name may be omitted from the path and placed in the class rectangle to emphasize its “class nature”.

4.5.10 N-Ary Association

An **N-Ary Association** is an association among more than two classes. Since N Ary Association is more difficult to understand, it is better to convert an NArY Association to binary association. However here for the sake of completeness we cover the notation of NArY Association.

An N-Ary Association is shown as large diamond with path from the diamond to each participant class. An association class symbol may be attached to the diamond by a dashed line, indic an N-Ary Association that has attributes, operation or association. The example depicted in figure 4.8 shows the grade book of a class in each semester.

4.5.22 Aggregation and composition (a-part-of)

Aggregation is a form of association. A hollow diamond is attached to the end of the path to indicate aggregation. However the diamond may not be attached to both ends of a line, and it need not be presented at all see figure 4.9.

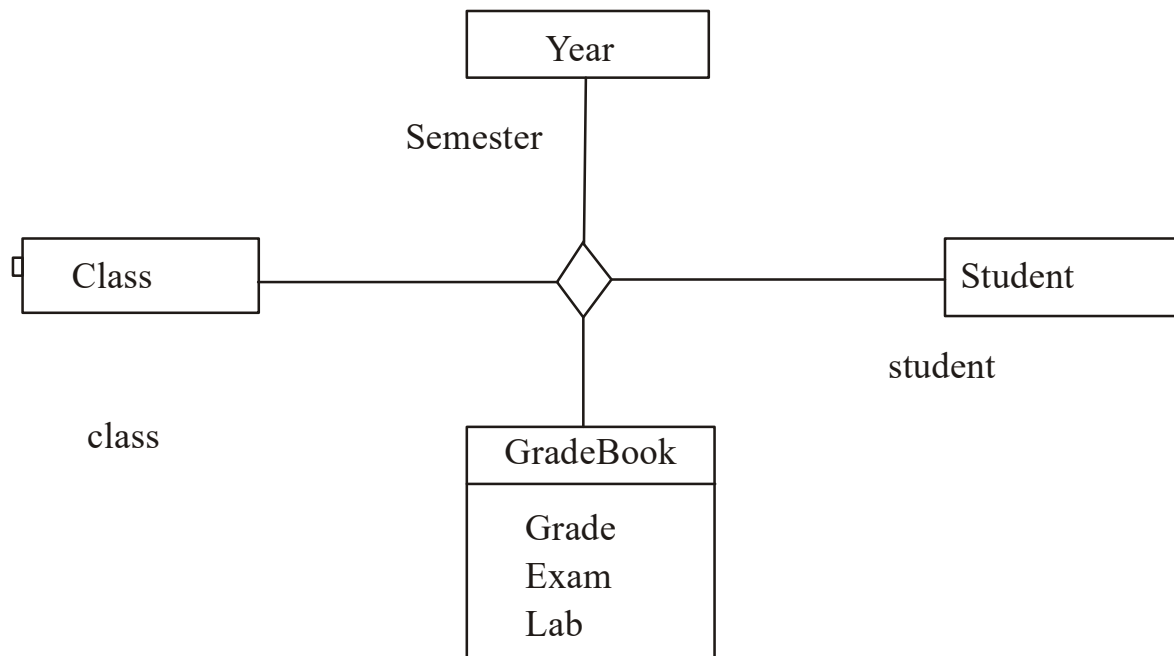


Figure 4.8

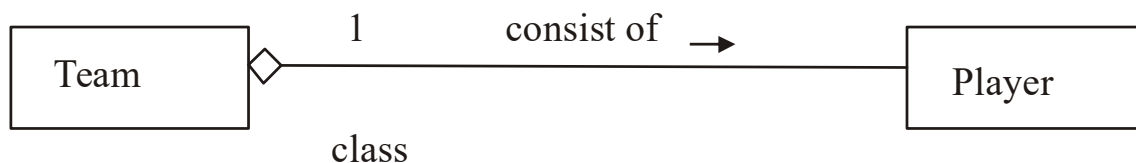


Figure 4.9

composition also known as the a-part-of is a form of aggregation with strong ownerships to represent the component of a complex object. Composition also is referred to as a partwhole relationship. The UML provides a graphically nested form that in many cases is more convenient for showing composition see figure 4.10.

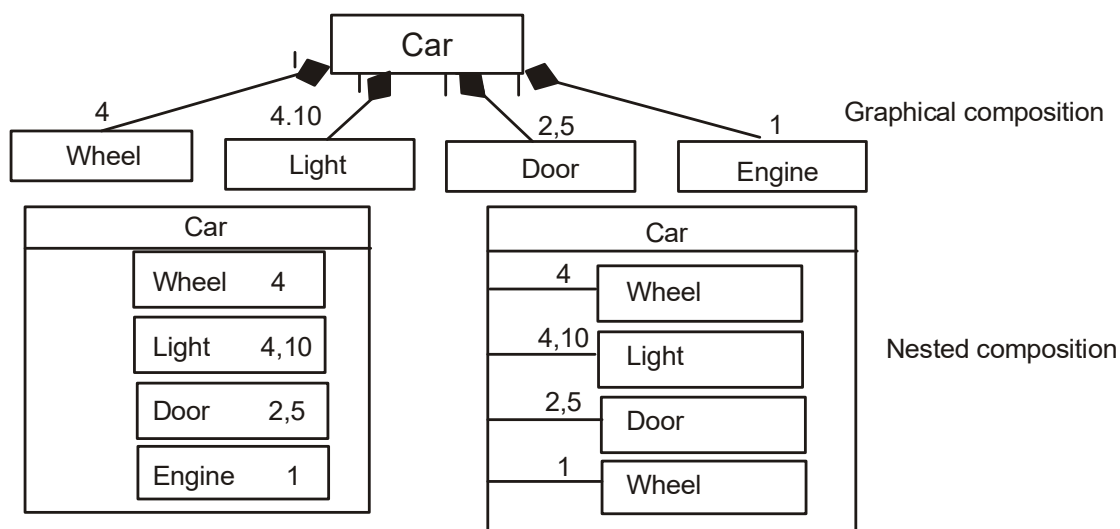


Figure 4.10

4.5.12 Generalization

Generalization is the relationship between a more general class and a more specific class. Generalization is displayed as a directed line with a closed hollow arrowhead at the super class end see figure 4.11. Separate target style

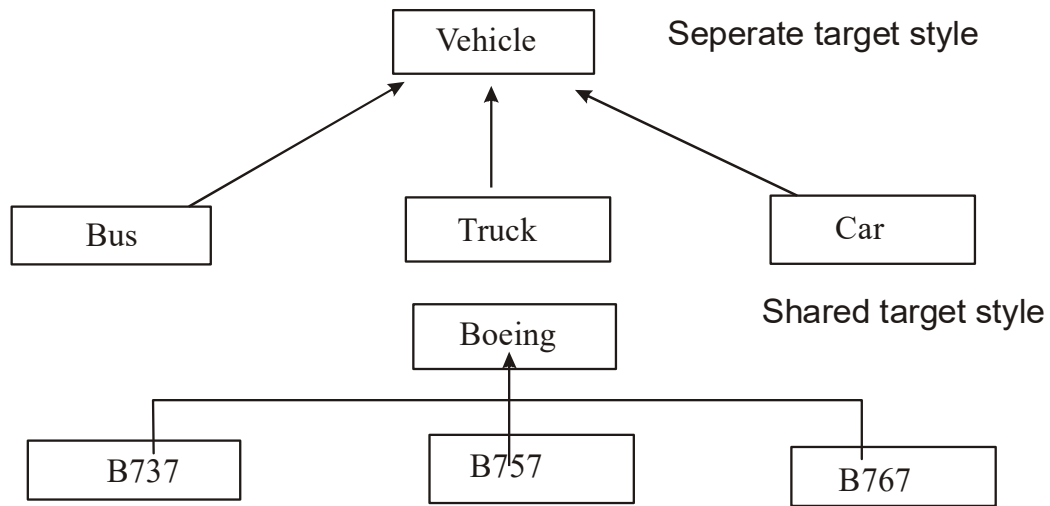


Figure 4.11

The UML allow a discriminator label to be attached to a Generalization of the super class. Ellipses(...) indicate that the Generalization is incomplete and more subclasses exists that are not shown (see figure 4.12). the constructor complete indicates that the Generalization is complete and no more subclasses are needed.

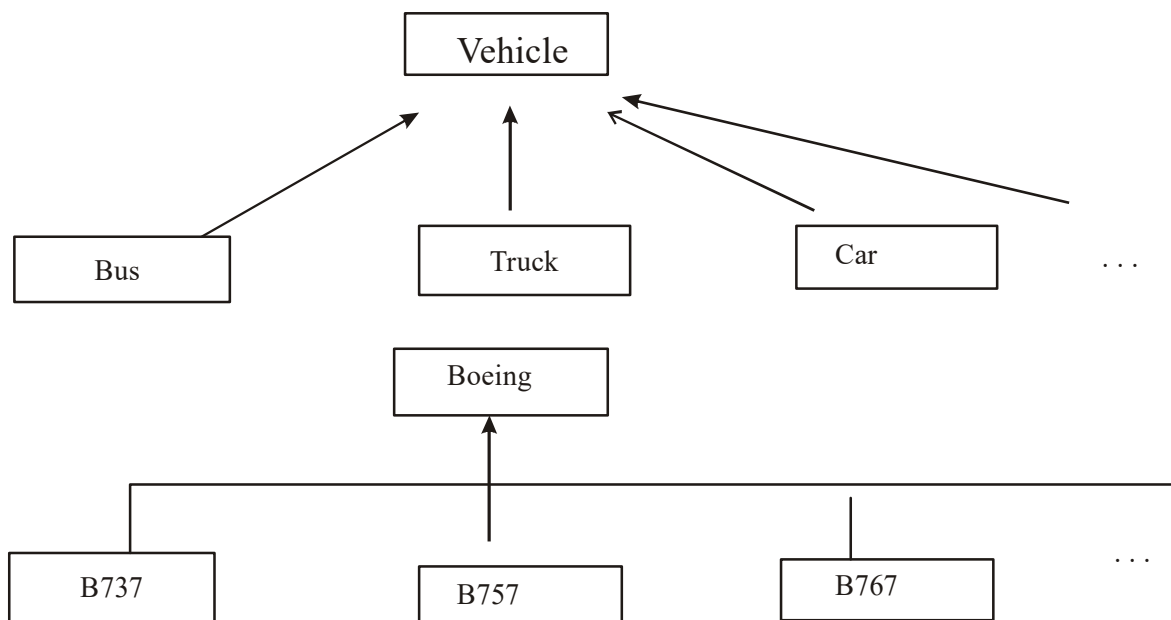


Figure 4.12

4.6 Use case, diagram

the use case concepts was introduced by Jacobson in the object oriented software engineering (OOSE) method. The functionality of a system is described in a number of different use cases. The description of use case define what happens in the system when the use case is performed. In essence, the use case model defines the outside and inside of the system's behavior. Use cases represent specific flows of events in the system.

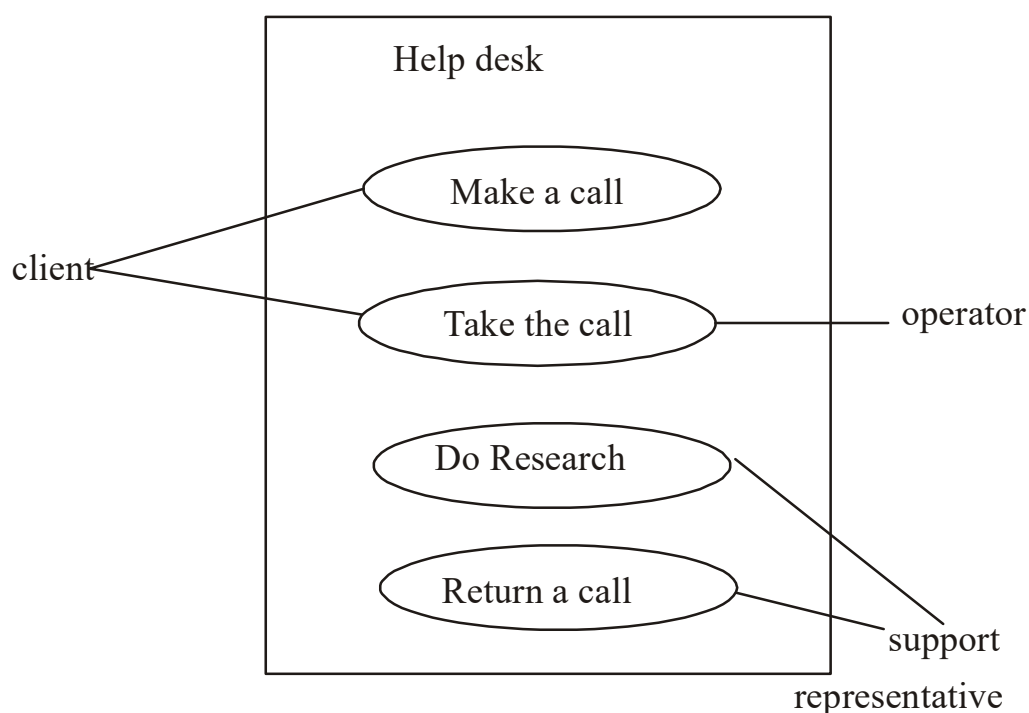


Figure 4.13

A use case diagram is a graph of actor a set of use cases enclosed by a system boundary, communication association between the actors and the use cases and generalization among the use cases. Figure 4.13 diagram use cases for a help desk. A use cases diagram shows the relationship among the actors and use case within a system. Some calls can be answered immediately, other calls require research and return call.

An actor is shown as a class rectangle with the label <<actor>> or the label and a stick figure or just the stick figure with the name of the actor below the figure see figure 4.14.



Figure 4.14

These relationship are shown in use case diagram.

1. Communication. The communication relationship of an actor in use case is shown by connecting the actor symbol to use case symbol with a solid path.
2. Uses. A uses relationship between use cases is shown by a generalization arrow from the use case.
3. Extends. The extends relationship is used when you have been one use case that is similar to another use case but does a bit more.

4.7 UML dynamic modeling

Booch explain that describing a systematic event in a static medium such as on a sheet of paper is difficult, but the problem confronts almost every discipline. In object oriented development, you can express the denamic semantics of a problem with the following diagrams.

Behavior diagram (dynamic)

- Interaction diagram
- Sequence diagram
- Collaboration diagram
- State chart diagram
- Activity diagram

Each class may have an associated activity diagram that indicates the behavior of the class's instance.

4.7.1 UML interaction diagram

Interaction diagram are diagram that describe how groups of objects collaborate to get the job done. Interaction diagrams capture the behavior of a single use case. Showing the pattern of interaction among objects. There are two kinds of interaction models:

1. sequence diagrams
2. collaboration diagrams.

4.7.1.1 UML sequence diagram sequence diagrams are an easy and intuitive way of describing the behavior of a system by viewing the interaction between system and its environment. A sequence diagram has two dimensions:

- **vertical dimension**
- **horizontal dimension**

the vertical dimension represents time, the horizontal dimension represents the different object. Vertical line is called the object's lifeline. The lifeline represents object's existence during the interaction. An object is shown as a box at the top of a dashed line see figure 4.15. A role is a slot for an object within a collaboration that describes the type of object that may play the role and its relationship to other role.

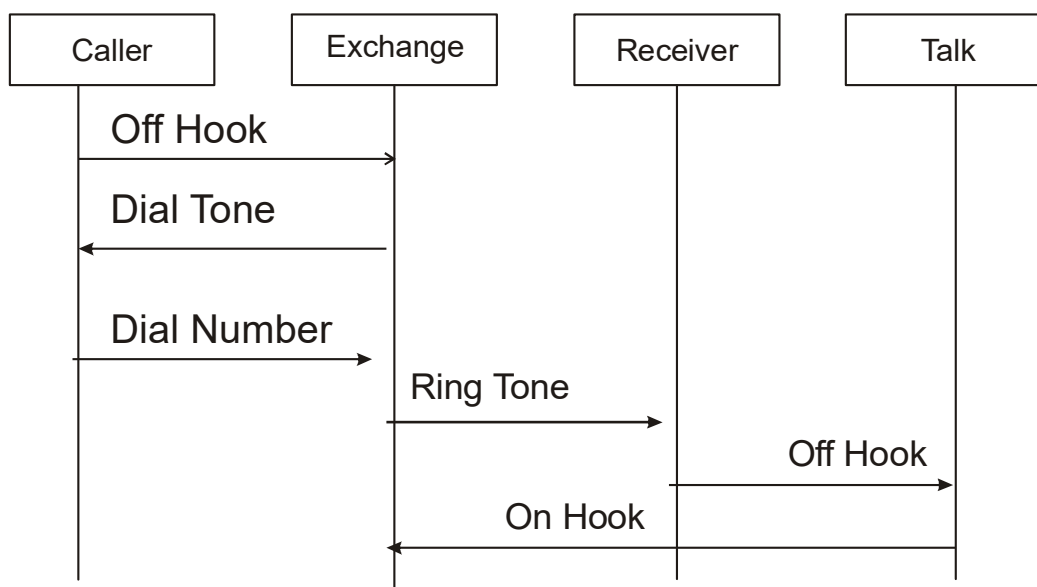


Figure 4.15

4.7.1.2 UML collaboration Diagram

another type of interaction diagram is the collaboration diagram. A collaboration diagram represents a collaboration which is set of object related in a particular context and interaction which is set of message exchanged among the objects within the collaboration to achieve a desired outcome. For example how the object are linked together and the layout can be overlaid with package or other information.

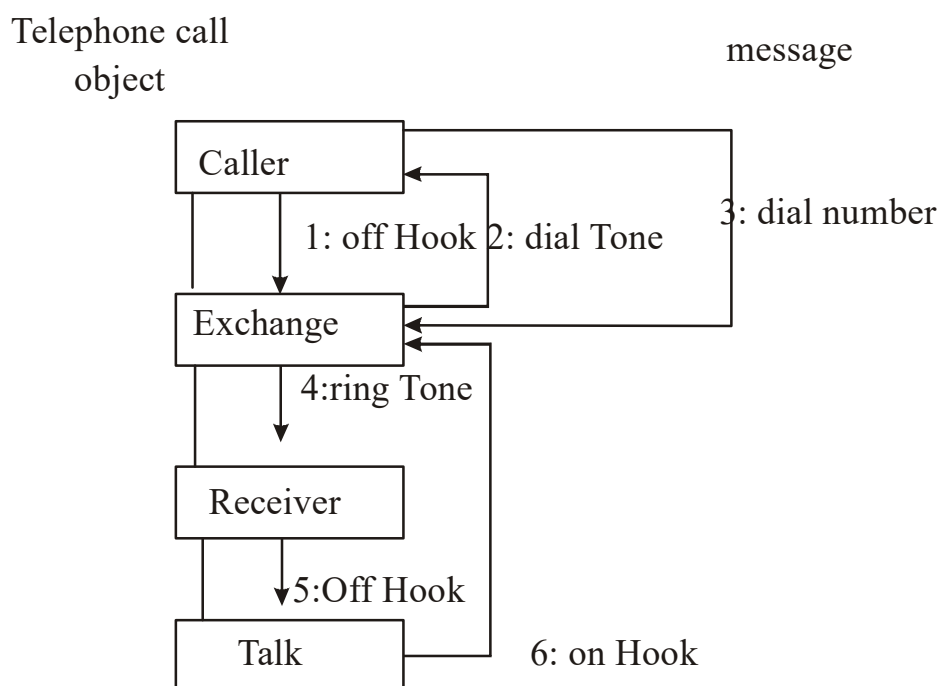


Figure 4.16

A collaboration diagram provides several numbering schemes. The simplest is illustrated in figure 4.16. you can also use a decimal numbering scheme (see figure 4.17) where 1.2 dial number means that the caller(1) is calling the exchange(2), hence the number 1.2. The UML uses the decimal scheme because it makes it clear which operation is calling which other operation.

4.7.2 UML State Chart diagram

A State Chart diagram shows the sequence of states that an object through during its life in response to outside stimuli and messages.

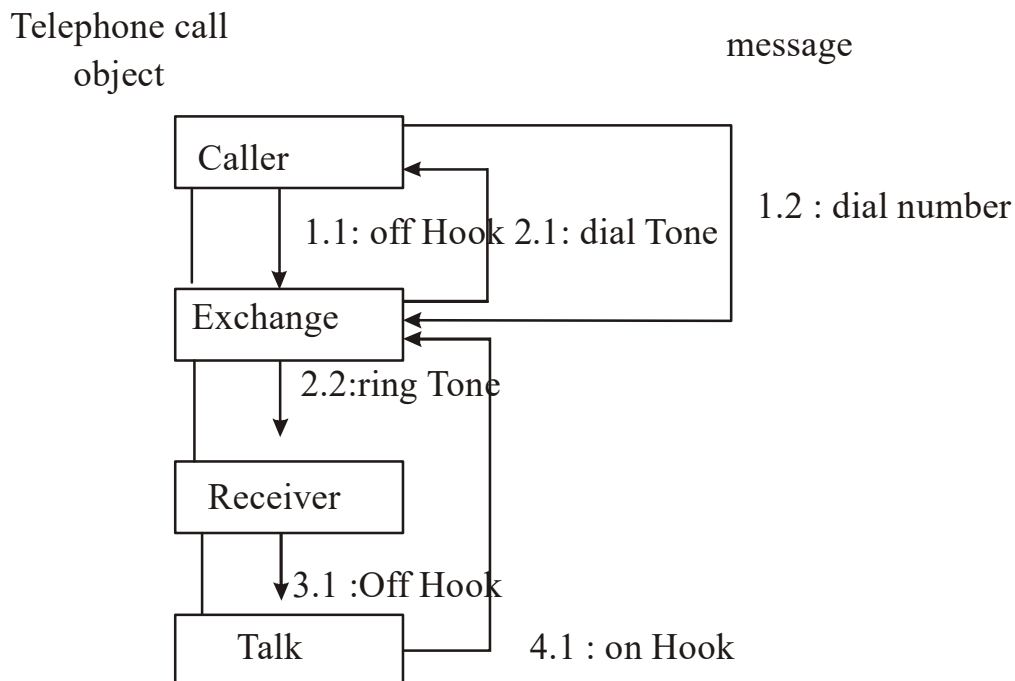


Figure 4.17

In this case the object would invoke an operation from its class that would modify the value associated with the attribute Employee, changing it from the old address to the new address, herefore the state of the employee object has been changed.

A state is represented as a rounded box, which may contain One or more compartments. The compartments are all optional. The mame compartment and the internal transition compartment are two Such compartments:

- The name compartment holds the optional name of the state. States without names are “anonymous” and all are distinct. Do not show the same named state twice in the diagram.
- The internal transition compartment holds a list of internal actions or activities performed in responses to event received while the object is in the state with out changing states.

The state chart support nested state machines to activate a sub state machine use the keyword do: do/machine-name. If this state is entered after the entry action is completed, the

nested state machine will be executed with its initial state. A final state is shown as a circle surrounding a small dot, a bull's eye. This represents the completion of activity in the enclosing state and triggers a transition on the enclosed state labeled by the implicit activity completion event usually displayed as an unlabeled transition see figure 4.18.

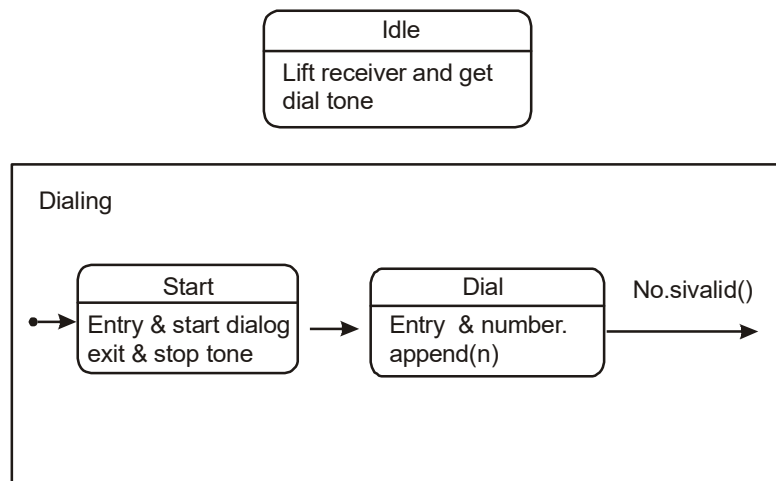


Figure 4.18

A complex transition “fires” all its destination states. A complex transition is shown as a short heavy bar. The bar may have one or more solid arrows from the bar to state. A transition string may be shown near the bar. Individual arrows do not have their own transition string see figure 4.19.

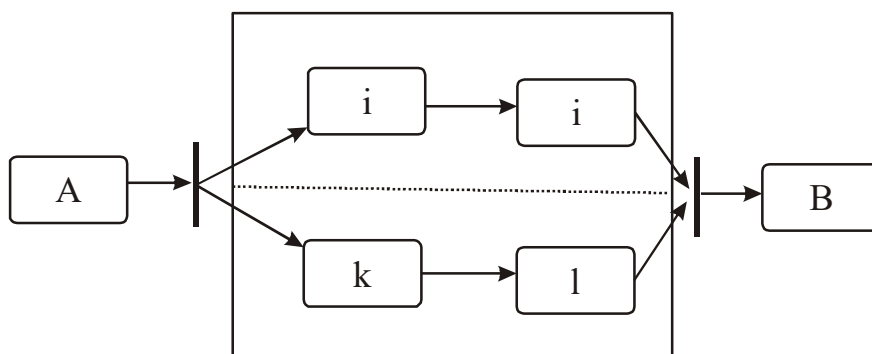


Figure 4.19

4.8 UML Extensibility

In this section we look at general purpose mechanisms which may be applied to any modeling element, and at the extensibility of the UML.

4.8.1 Model constraints and comments

Constraints are assumptions or relationship among model element specifying condition and propositions that must be maintained as true. Otherwise the system described by the model would be invalid. Constraints are shown as text in braces {} see figure 4.20. the UML also provide language for writing constraint in the OCL. However the constraint may be written in a natural language. A constraint may be a “comment” in which case it is written in text. Work for

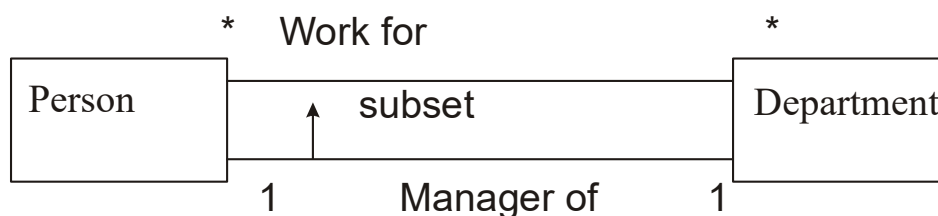


Figure 4.20

The example depicted in figure 4.20 shows two classes and two association. The constraint is shown as a dashed arrow from one element to the other, labeled by the constraints string in braces. The direction of the arrow is relevant information within the constraint.

4.8.2 Note

A note is a graphic symbol containing textual information, it is also could contain embedded images. It is attached to the diagram rather than to a model element. A note is shown as a rectangle with a “bent corner” in the upper right corner. It can contains any length text see figure 4.21. JMU LI

4.8.3 Stereotype

Stereotype represent a built in extensibility mechanisms of the UML. User defined extensions of the UML are enabled through the use of stereotype and constraints. The general presentation of a stereotype is to use a figure for the base element but place a keyword string above the name of the element. The figure can be used in one of two ways:

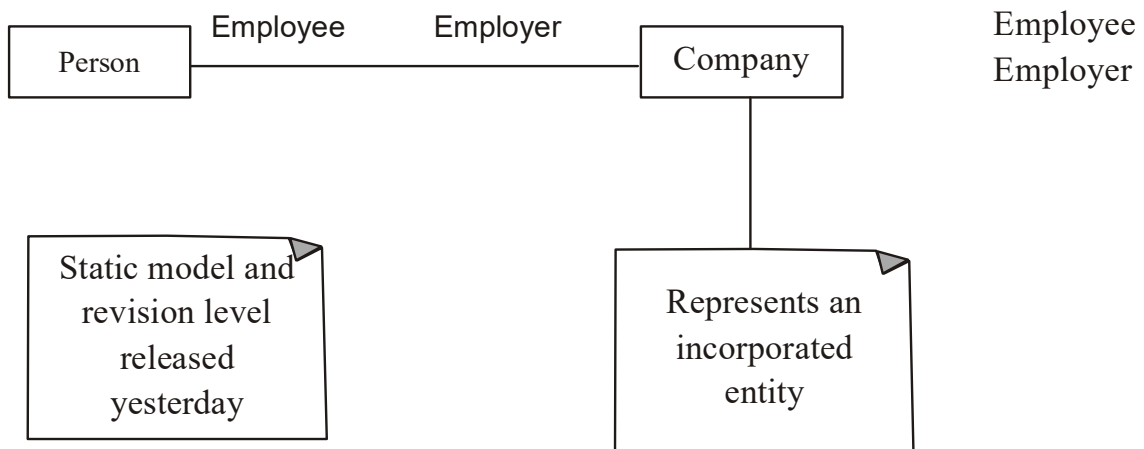


Figure 4.21

1. Instead of or in addition to the stereotype keyword string as part of the symbol for the base model element
2. As the entire base model element see figure 4.22. other information contained by the base model element symbol is suppressed.

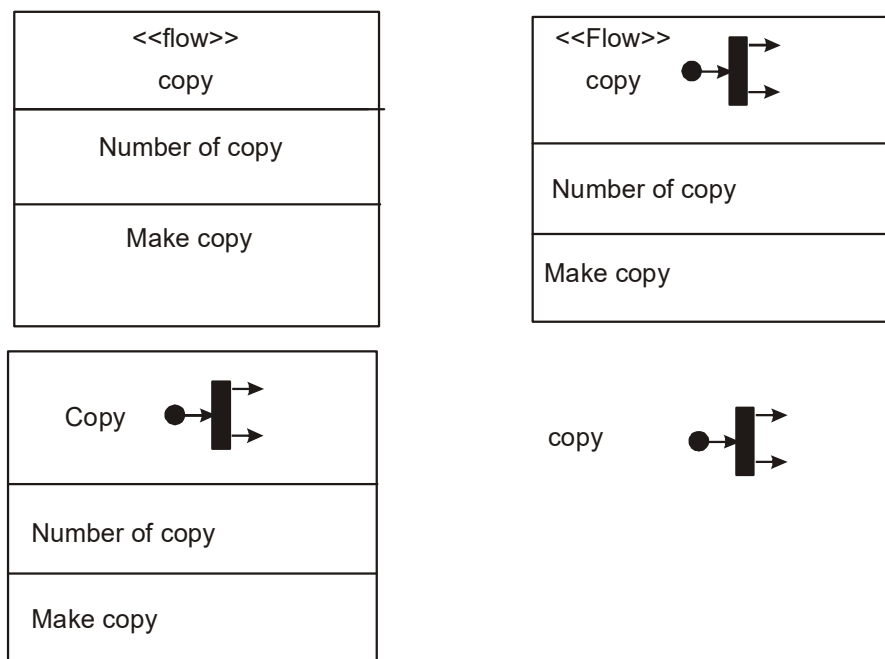


Figure 4.22

Summary

- The unified modeling language was developed by Booch, Jacobson and Bum Baugh. The UML encompasses the unification of their modeling notations.
- The UML class diagram is the main static structure with analysis diagram for the system. It represents the class structure of a system with relationship between classes and inheritance structure.
- The use case diagram captures information on how the system or business works or how wish it to work.
- In UML collaboration diagram is an alternative view of the sequence diagram showing in a scenario how objects interrelate with one another.
- Implementation diagram show the implementation phase of system development such as the source code and run time implementation structures. The two types of implementation diagram are component diagram.
- UML graphical notations can be used not only to describe the system's components but also to describe a model itself; this is known as a meta model.

Questions

1. What is model?
2. Why do we need to model a problem?
3. What is data modeling?
4. Describe the class diagram.
5. What is an association role?
6. What is a qualifier?
7. What are some of the forms of association? draw their UML representations.
8. When would you use interaction diagram?
9. What is the purpose of an activity model?
10. What are describe the relationship in use case diagram.

LESSON - 5

USE CASE DRIVEN

Objectives:

The object oriented analysis process.

- The use case modeling and analysis
- Identifying actors
- Identifying use cases.
- Developing effective documentation.

Introduction

The first step in finding an appropriate solution to given problem is to understand the problem and its domain. The main objective of the analysis is to capture a complete, unambiguous, and consistent picture of the requirements of the system and what the system must do to satisfy the user's requirements and needs. This is accomplished by constructing several models of the system that concentrate on describing what the system does rather than how it does it.

Analysis is the process of transforming a problem definition from a fuzzy set of facts and myths into a coherent statement of a system's requirements. We looked at the software development process as three basic transformations the objective of this lesson is to describe transformation 1, which is the transformation of the users needs into set of problem statement and requirements, analysis involves a great deal of interaction with the people who will be affected by the system, including the actual users and anyone else on whom its creation will have an impact. The analyst has four major tools at his or her disposal for extracting information about a system

- Examination of existing system documentation.
- Interviews
- Questionnaire
- Observation

In addition there are minor methods such as literature review. However these activities, design and implementation. The analyst is concerned with uses of the system identifying the

objects and inheritance and thinks about the events that change the state of object. the designer adds detail to this model perhaps designing screens, user interaction, and database access. The thought process flows so naturally from analyst to designer that it may be difficult to tell where analysis ends and design begins.

5.1 Why Analysis Is Difficult Activity

analysis is a creative activity that involves understanding the problem, its associated constraints and method of overcoming those constraints. This is an iterative process that goes on until the problem is well understood. Norman explains the three most common sources of requirements difficulties.

1. Fuzzy descriptions
2. Incomplete requirements
3. Unnecessary features

A common problem that leads to requirements ambiguity is a fuzzy and ambiguous description, such as “fast response time” or “very easy and very secure updating mechanisms”. Incomplete requirements mean that certain requirements necessary for successful system development are not included for a variety of reasons. These reasons could include the user forgetting to identify them, high cost, politics within the business or oversight by the system developer. However because of the iterative nature of object oriented analysis and the unified approach, most of the incomplete requirements can be identified in subsequent tries.

When addressing features of the system keep in mind that every additional feature could affect the performance, complexity, stability, maintenance and support costs of an application. A number of other factors also may affect the design of an application. For example deadlines may require delivering a product to market with a minimal design process, or comparative evolutions may force considering additional features. Remember that additional features and shortcuts can affect the product. There is no simple equation to determine when a design trade off is appropriate.

Analysis is a difficult activity. You must understand the problem in some application domain and then define a solution that can be implemented with software. Experience often is the best teacher. If the first try reflects the errors of an incomplete understanding of the problems, refine the application and try another run.

5.2 Business Object Analysis: Understanding The Business layer

Business object analysis is a process of understanding the system's requirements and establishing goals of an application. The main intent of this activity is to understand user's requirements. The outcome of the business object analysis is to identify classes that make up the business layer and the relationship that play a role in achieving system goal. To understand the users requirements we need to find out how they "use" the system. This can be accomplished by developing use cases. Use cases are scenarios for understanding system requirements.

In addition to developing use cases, which will be described in the next section, the uses and the objectives of the application must be discussed with those who are going to use it or be affected by the system. Usually domain users or experts are the best authorities. Try to understand the expected inputs and desired responses.

Defer unimportant details until later. State what must be done, not how it should be done. This of course is easier said than done. Yet another tool that can be very useful for understanding users requirements is preparing a prototype of the user interface. Preparation of a prototype usually can help you better understand how the system will be used, and therefore it is a valuable tool during business object analysis.

Having established what users want by developing then documenting and modeling the application, we can proceed to the design and implementation. The unified approach(UA) steps can overlap each other. The process is iterative and you may have to backtrack to previously completed steps for another try. Separating the what from the how is no simple process. Fully understanding a problem and defining how to implement it may require several tries or iterations.

5.3 Use case driven object oriented analysis: The unified approach

The object oriented analysis (OOA) phase of the unified approach uses actors and use case to describe the system from the users perspective. The actors are external factors that interact with the system; use cases are scenarios that describe how actors use the system. The use cases identified here will be involved throughout the development process.

The OOA process consists of the following steps see figure 5.1.

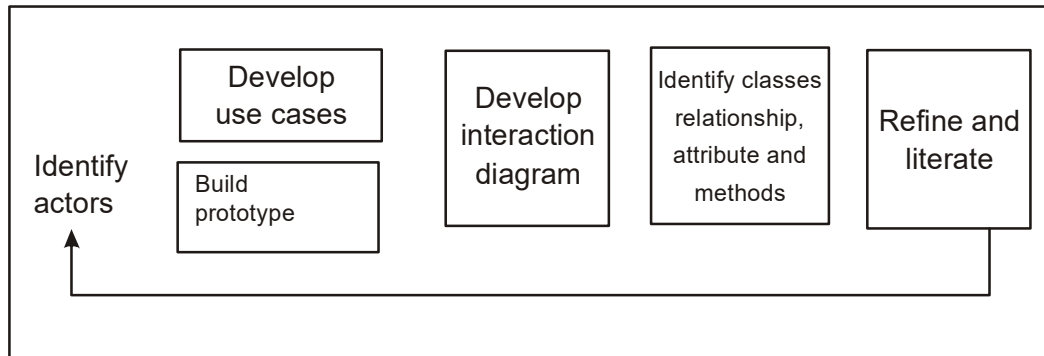


Figure 5.1

1. identify the actors:
 - who is using the system?
 - Or, in the case of a new system who will be using the system?
2. Develop a simple business process model using UML activity diagram?
3. Develop the use case:
 - What are the users doing the system?
 - Or, in case of the new system what will users be doing with the system?
 - Use cases provide us with comprehensive documentation of the system under study.
4. Prepare interaction diagram
 - Determine the sequence
 - Develop collaboration diagram
5. Classification develop a static UML class diagram
 - Identify classes.
 - Identify relationship
 - Identify attributes.
 - Identify methods
 - Iterate and refine: if needed repeat the preceding steps.
6. This lesson focuses on steps 1 to 3.

5.4 Business process modeling

This is not necessarily the start of every project but when required business processes and user requirements may be modeled and recorded to any level of detail. This may include modeling as is processes and the application that support them and any number of phased would be models of reengineered processes or implementation of the system.

These activities would be enhanced and supported by using an activity diagram. Business process modeling can be very time consuming so the main idea should be to get a basic model without spending too much time on the process. The advantage of developing a business process model is that it make you more familiar with the system and developing the user requirements and also aids in developing use case.

For example let us define the steps or activities involved in using your school library. These activities can be represented with an activity diagram see figure 5.2. Developing an activity diagram of the business process can give us a better understanding.

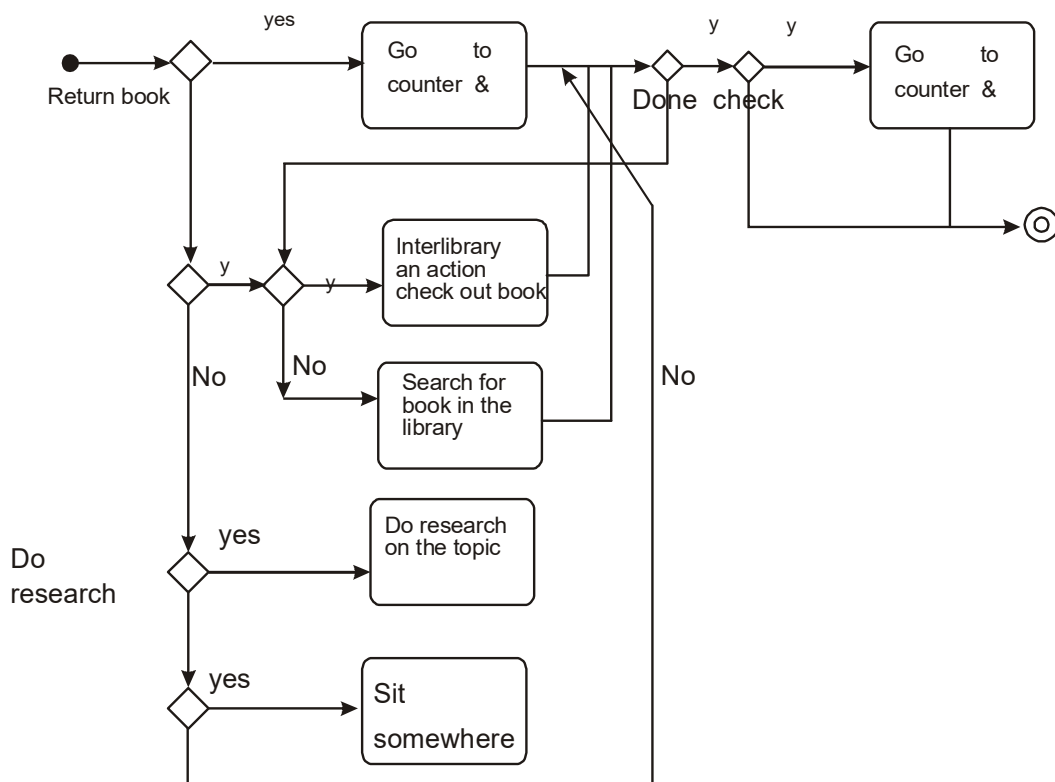


Figure 5.2

5.5 Use case Model

Use cases are scenarios for understanding system requirements. A use case model can be instrumental in project development, planning, and documentation of system requirements. A use case is an interaction between users and a system it captures the goal of the users and the responsibility of the system to its users.

For example take a car typical uses of a car include “take you different places” or “haul your stuff” or a user may want to use it “off the road”. The use case model describes the uses of the system and shows the courses of events that can be performed. In other words it shows a system in term of its users and how it is being used from a user point of view.

Furthermore, it define what happens in the system when the use case is performed. In essence, the use case model tries to systematically identify uses of the system and therefore the system’s responsibilities. A use case model also can discover classes and the relationship among subsystems of the systems.

Since the use case model provides an external view of a system or application it is directed primarily toward the user or the “actors” of the system not its implementers see figure 5.3. the use case model expresses what the bus or application will do and not how, that is the responsibility of the UML class diagram.

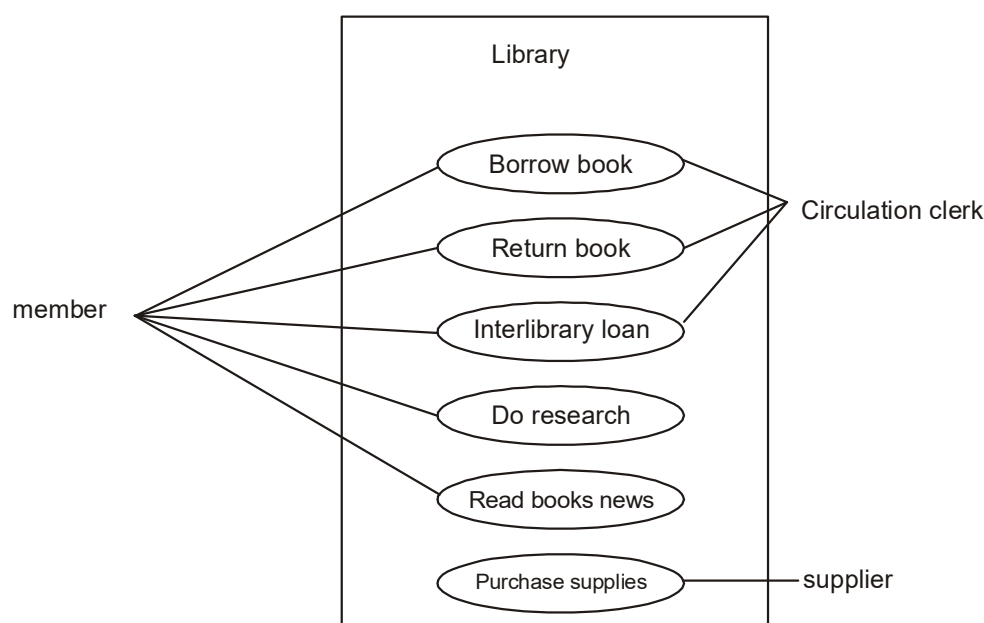


Figure 5.3

The UML class diagram also called an object model represents the static relationship between objects, inheritance, association and the like. The object model represents an internal view of the system as opposed to the use case model which represented the external view of the system.

5.5.1 Use cases under the microscope

An important issue that can assist us in building correct use cases is the differentiations between user goal and system interaction. Use cases represent the things that the user is doing with the system which can be different from the users goals. However by focusing on users goals first we can come up with use cases to satisfy them. Let us take a closer look at the definition of use case by Jacobson et al. “a use case is a sequence of transactions in a system whose task is to yield results of measurable value to an individual actor of the system”.

Now let us take at the key words of this definitions:

- Use case. Use case is a special flow of events through the system. By definition, many courses of events are possible and many of these are very similar. It is suggested that, to make a use case model meaningful, we must group the courses of events and call each group a use case class. For example how you would borrow a book from the library depends on whether the book is located in the library, whether you are the member of the library, and so on.
- Actors. An actor is a user playing a role with respect to the system. When dealing with actor, it is important to think about roles rather than just people and their job title. For instance a first class passenger may play the role of business class passenger. The actor is the key to finding the correct use cases. Actors carry out the use cases. A single actor may perform many use cases. Furthermore a use case may have several actor performing it.
- In a system. This simply means that the actor: communicate with the system's use case.
- A measurable value. A use case must help the actor to perform a task that has some identifiable value. For example the performance is a use case in term of price or cost. For example borrowing books is something of value for a member of the library.

- Transaction. A transaction is an atomic set of activities that are performed either fully or not at all. A transaction is triggered by a stimulus from an actor to the system or by a point in time being reached in the system.

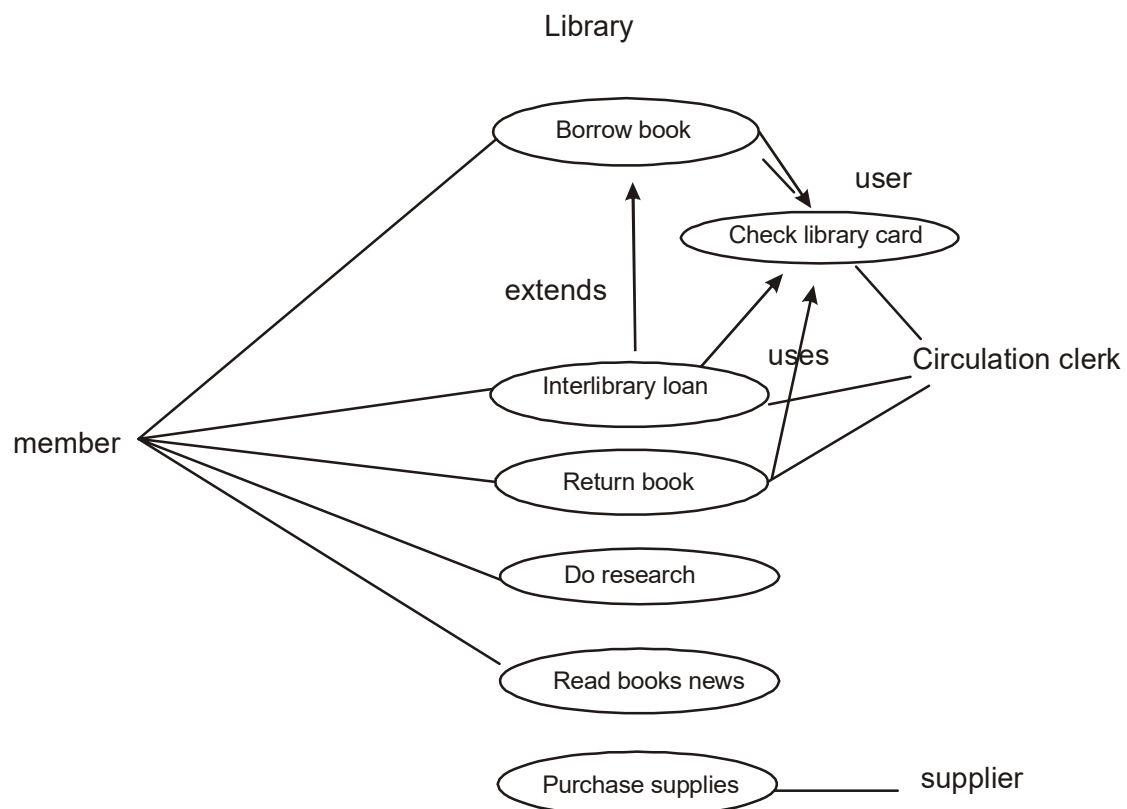


Figure 5.4

The following are some example of use cases for library see figure 5.4. three actors appear in figure 5.4, a member, a circulation clerk, and a supplier.

- **Use case name:** Borrow books. A member takes books from the library to read at home, registered them at the checkout desk so the library can keep track of its books. Depending on the member's record different courses of event will follow.
- **Use case name:** Get an interlibrary loan. A member requests a book that the library does not have. The book is located at another library and ordered through an interlibrary loan.

- **Use case name:** Return books. A member brings borrowed books back to the library
- **Use case name:** check library card. A member submits his or her library card to the clerk, who checks the borrower's record
- **Use case name:** Do research. A member comes to the library to do research. The member can search in a variety of ways to find information on the subjects of that research
- **Use case name :** read books newspaper. A member comes to the library for a quiet place to study or read a newspaper, journal, or book.
- **Use case name :** Purchased supplies. The supplier provides the books, journal, and newspapers purchased by the library.

5.5.2 uses and extends Association

A use case description can be difficult to understand if it contains too many alternatives or exceptional flows of events that are performed only if certain conditions are met as the use case instance is carried out. The extends and uses association often are sources of confusion, so let us take a look at these relationships.

The extends association is used when you have one use case that is similar to another use case but does a bit more or is more specialized., in essence, it is like a subclass. In our example, checking out a book is the basic use case. This is the case that will represent what happens when all goes smoothly. To remedy this problem we can use the extends association. Here you put the base or normal behavior in one use case and the unusual behavior somewhere else but instead of cutting and pasting the shared behavior between the base and more specialized use cases the use association occurs when you are describing your use case and notice that some of them have sub flows in common. To avoid describing a sub flow more than once in several use cases, you can extract the common sub flow and make it a use case of its own. This new use case and this new extracted use case is called a uses association. The uses association help us avoid redundancy by allowing a use case to be shared. For example, checking a library card is common among the borrow books, return books. The similarity between extends and uses association is that both can be viewed as kind of inheritance. When you want to share common sequences in several use cases, utilize the uses association by extracting common

sequences into a new shared use case. The extends association is found when you add a bit more specialized, new use case that extends some of the use cases that you have.

Use cases could be viewed as concrete or abstract. An abstract use case is not complete and has no initiation actors but is used by a concrete use case, which does interact with actors. Fowler and Scott provide us excellent guidelines for addressing variations in use case modeling.

1. capture the simple and normal use case first.
2. for every step in that use case.
3. extract common sequences into a new shared use case with the uses association.
If you are adding more specialized or exceptional use cases take advantage of use case you already have with the extends association.

5.5.3 identifying the actors

Identifying the actor is as important as identifying class, structure, -association, attributes and behavior. The term actor represents the role a user plays with respect to the system. When dealing with actors, it is important to think about roles rather than people or job title. However an actor should represent a single user in the library example, the member can perform tasks some of which can be done by other and others that are unique. However try to isolate the roles that the users can play see figure 5.5. Gause and Weinberg explain what is known as the railroad paradox. Gause and Weinberg concluded that the railroad paradox appears everywhere there are products and goes like this.

- The product is not satisfying the users.
- Since the product is not satisfactory, potential users will not use it.
- Potential users ask for a better product.
- Because the potential users do not use the product, the request is denied.

User can play the role of Actor perform USE CASE

User can play the role of Actor perform USE CASE

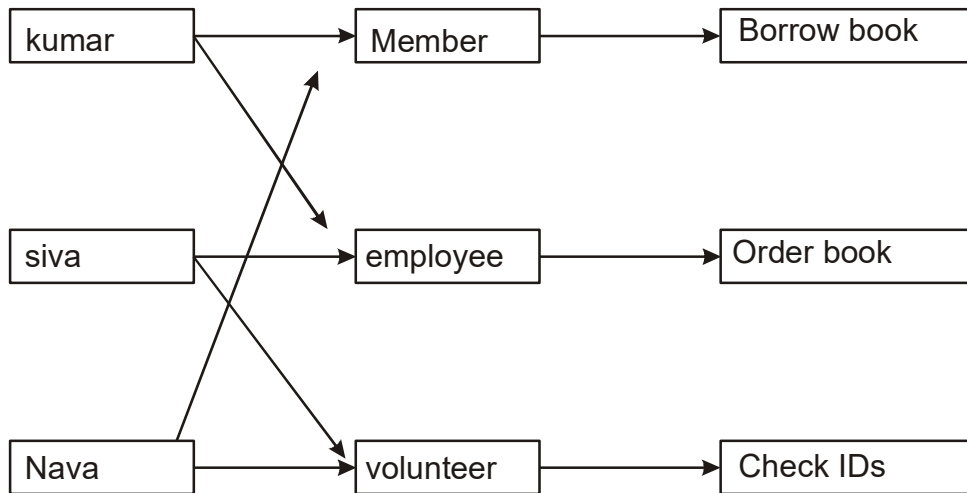


Figure 5.5

They are not consulted and the product stays bad. The rail- road paradox suggest that a new product actually can create users where none existed before. When requirements for new applica- tion are modeled and designed by a group that excludes the tagged users, not only will the application not meet the users needs but potential users will feel no involvement in the process and not be committed to giving the application a good try. Always remember veblen's principle: "there's no change no matter how awful, that won't benefit some people; and no change no matter how good that won't hurt some".

Jacobson et al. provide us with what I call the two tree rule for identifying actors: start with naming at least two, preferably can be identified in the subsequent iterations.

For example assume we are modeling a company that sep in marketing jewelry. The first actor that comes to mind is the final customer actually tree different regular customer would buy the product. Another type of actor is the jewelry buyers for exclusive stores, they know all about quality and nothing else a third type is customer is boutique owners.

Guidelines for finding use cases when you have defined a set of actors, it is time to describe the way they interact with system. This should be carried out sequentially, but an iterated approach may be necessary. Here are the steps for finding use cases.

1. For each actor find the tasks and functions that the actor should be able to perform or that the system needs the actor to perform. The use case should represent a course of events that leads to a clear goal.
2. name the use cases
3. Describe the use cases briefly by applying terms with which the user is familiar. This makes the description less ambiguous

It is important to separate actor from users. The actor each represent a role that one or several users can play. Therefore it is not necessary to model different actor that can perform the same use case in the same way. The approach should allow different users to be different actor and play one role when performing a particular actor's use case. Thus each use case has only one main actor. To achieve this you have to.

- Isolate users from actors
- Isolate actor from other actors
- Isolate use case that have different initiating actor and slightly different behavior.

When specifying use cases you might discover that some of them are variants of each other. If so try to see how you can reuse the use case through extends or uses associations.

5.5.5 How Detailed must a use case be? When to stop decomposing and when to continue

A use case as already explained describes the courses of event that will be carried out by the system. Use case are an essential tool in capturing requirements and planning and controlling any software development project. Capturing use case is primary task of the analysis phase. Although most use cases are captured at the beginning of the project.

5.5.6 dividing use cases into Packages

each use case represents a particular scenario in the system. You may model either how the system currently works or how you want it to work. Typically a design is broken down into packages. You must narrow the focus of the scenarios in your system. For example in a library

system the various scenarios involve a supplier providing books or member doing research or borrowing books. Many application may be associated with the library system and one or more database used to store the information see figure 5.6.

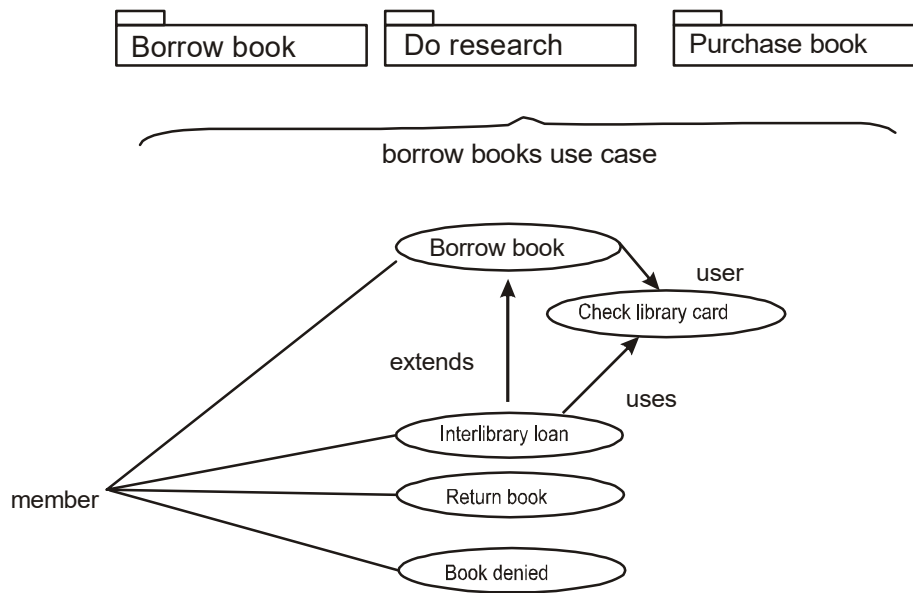


Figure 5.6

5.6 Development effective documentation

Documentation your project not only provide a valuable reference point and form of communication but often helps reveal issues and gaps in the analysis and design. A document can serve as a communication vehicle among the project's team members. Application software is expected to provide a solution to a problem. It is very difficult, if not impossible to document a poorly understood problem. The main issue in documentation during the analysis phase is to determined what the system must do.

5.6.1 organization conventions for documentation

The documentation depends on the organization rules and regulations. Most organization have established standards or conventions for developing documentation. However in many organization the standard broader on the nonexistent. In other cases the standards may be excessive. Each organization determines what is best for it and you must respond to that definition and refinement. However this template which is based on the unified approach life cycle see

figure 1.1 assists you in organizing and composing your models into analysis effective documentation.

5.6.2 Guidelines for developing effective documentation

Bell and evens provide us the following guideline for making documents fit the need and expectations of your audience.

- **Common cover.** All document should share a common cover sheet that identifies the document the current version, and the individual responsible for the content. As the document proceeds through the life cycle phases, the responsible individual may change. That change must be reflected in the cover sheet[2] figure5.7 depicts a cover sheet template.

(Document name) for (product) (version number) Responsible Individual Name: Title :
--

Figure 5.7

- **80-20 rule.** As for many application the 80 20 rule generally applies for documentation 80 percent of work can be done with 20 percent of the documentation.
- **familiar vocabulary.** The formality of a document will depended on how it is used and who will read it. When developing a documentation use a vocabulary that your reader understand and are comfortable with.
- **Make the document as short as possible.** Assume that you are developing a manual. The key in developing analysis effective manual is to eliminate all repetition.
- **Organize the document.** Use the rules of good organization within each section. Appendix a provides a template for developing documentation for a project. Most CASE tools provide documentation capability by providing customizable report. The purpose of these guidelines is to assist you in creating analysis effective documentation.

5.7 CASE study: Analyzing The Vianet bank ATM-the use case Driven process

5.7.1 Background

Much of the work that must be done in the early stages of the system development process involves gathering requirements and other related information. These activities focus on gaining a better understanding of the business problem to be solved and the requirements and restrictions related to the application being developed

Using a CASE tool such as the popkin object arcutech or similar tool, enables you to systematically capture requirements identify classes, design and finally implement the application.

Let us review object oriented analysis which is divided into the following activities.

1. Identify the actor
2. Develop a business process model using a UML activity diagram.
3. Develop the use case
4. Develop interaction diagram
5. Develop a static UML class diagram
6. If needed, repeat the preceding steps.

5.7.2 Identifying actors and use cases for the ViaNet bank ATM system

The bank application will be used by one category of users. Bank clients. Notice that identifying the actor of the system is analysis iterative process and can be modified as you learn more about the system in real life application these use cases are created by system requirements, examination of existing system documentation, interviews, questionnaire, observation, etc.

- Use case name bank ATM transaction. The bank client interact with the bank system by going through the approval process. Here are the steps in the ATM transaction use case.
 1. Insert ATM card
 2. perform the approval process
 3. ask type of transaction

4. enter type of transaction
5. perform transaction.
6. eject card
7. request take card
8. take card.

These steps are shown in the figure 5.8.

- **Use case name:** bank ATM transaction. The bank clients interact with the bank system by going through the approval process.
- **Use case name:** Approval process the client enters a pin code that consists of four digits. If the pin code is valid, the client's account becomes available. See figure 5.9.
- **Use case invalid pincode.** If the Pincode is not valid, an appropriate message is displayed to the client.
- **Use case name:** Deposit amount the bank clients interact with the bank system after the approval process by requesting to deposit money to analysis account the client selects the account for which a deposit is going

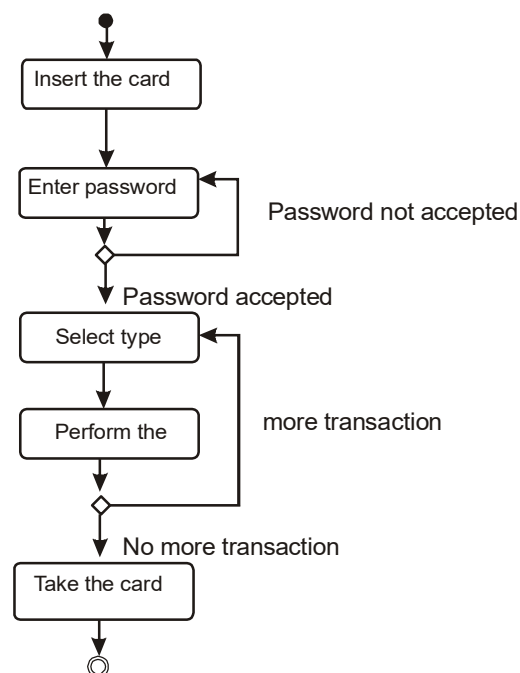


Figure 5.8

- to be made and enter any amount in dollar currency. A system creates a record of the transaction see fig., :5.10 This use case extend the bank ATM transaction use case here are the steps
 1. Request account type
 2. Request deposit amount
 3. Enter deposit amount
 4. Put the cheque or cash in the envelope and insert in to ATM
- Use case Name Deposit savings the client select the savings account for which the deposit is going to be made. All other steps are similar to deposit amount use case. The system creates a record of the transaction this use case extend the deposit amount see fig. 5.11

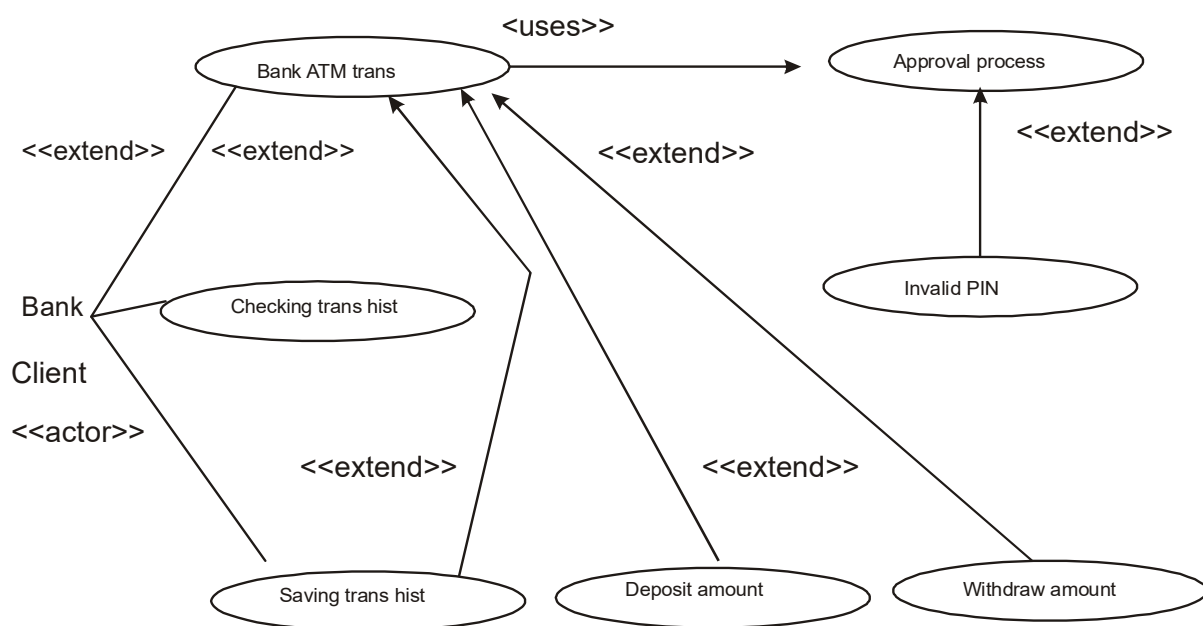


Figure 5.9

- Use Case Name Withdraw checking: The client tries to withdraw analysis amount from his/her checking account the amount is less than or equal to the checking account balance, and the transaction is performed the system creates the record of the transaction. This use case extend withdraw amount use case. See fig., 5.10

- Use case name-Saving Transaction history: The bank client request a history of transaction for a savings account a system display the transaction history for the savings account. This use case extend bank transaction use case see fig. 5.9.

5.7.3 The Via Net Bank package system:

Each use case represents a particular scenario in the system. As explained earlier it is better to break down the use case in to packages. Narrow the focus of the scenario in the system. In the bank system, with the various scenarios involves checking account, saving account and general bank transaction see fig. 5.12 remember use case is method for capturing a wave for the system or business work. Use case are used to model the scenarios. The Scenarios are described textually or through a sequence of steps.

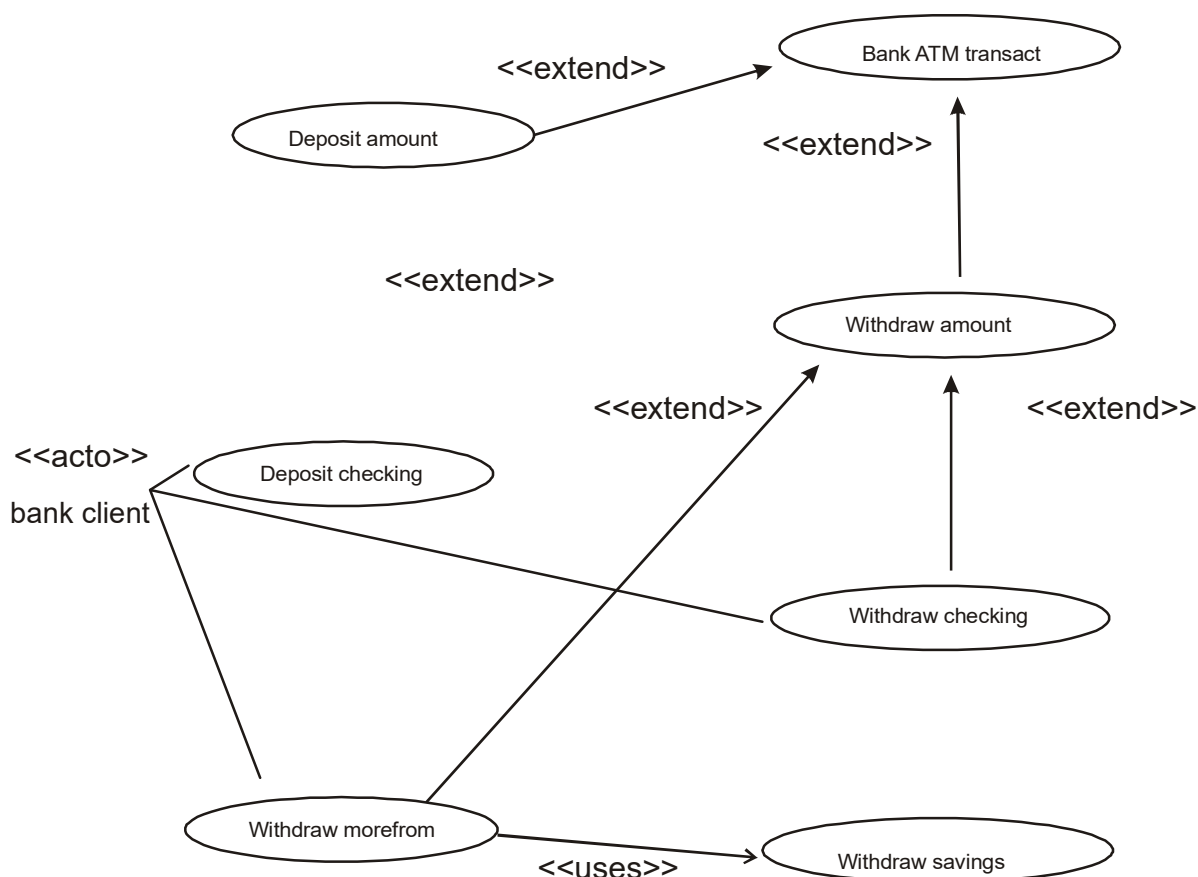


Figure 5.10

Summary:

- The main objective of the analysis is to capture a complete, and consistent picture of the requirement of the system
- This modules concentrate on the describing what the system does rather than how system does it. Separating the behavior of the system from the wave it is implemented requires viewing the system.
- We saw that use case are analysis essential tool in capturing requirements. Capturing use cases is one of the first thing to in coming up with requirements.

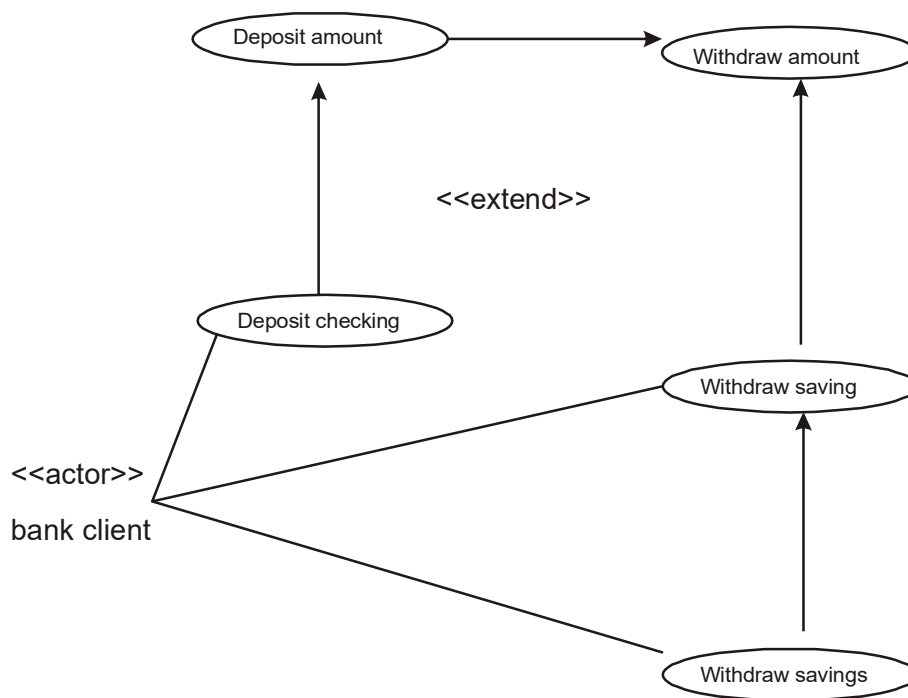


Figure 5.11

- Requirements must be traceable across analysis, design, coding and testing.
- The key in developing effective documentation is to eliminate all repetition, present summaries, reviews, organization chapters in less than three pages and make chapter heading task oriented so that the table of content.
- Use the 80-20 rule: 80 percent of work can be done with 20 percent of the documentation.

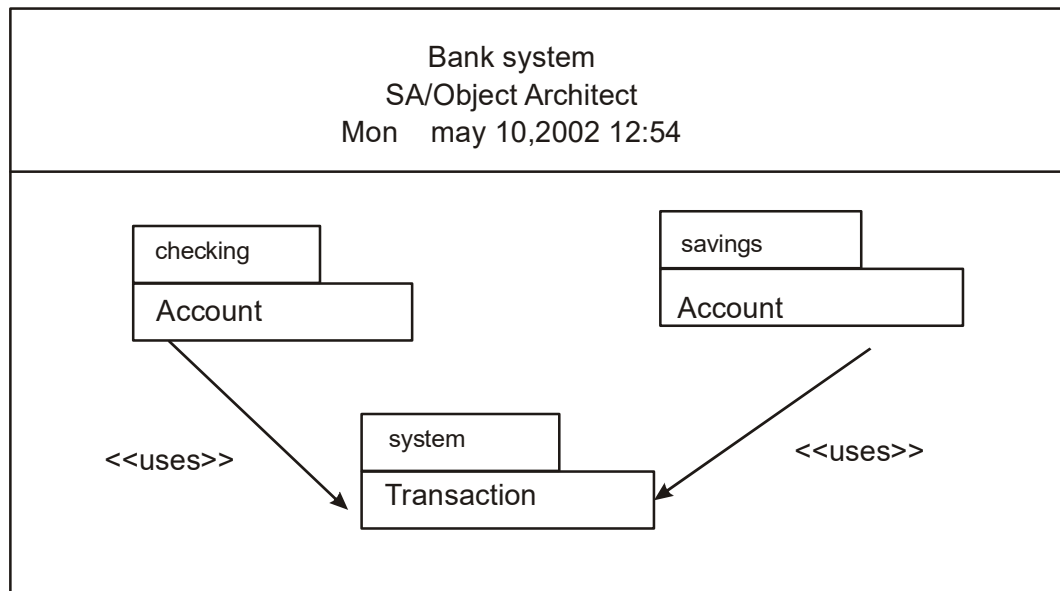


Figure 5.12

Questions:

1. what is the purpose of analysis? why do we need analysis?
2. why is analysis a difficult task?
3. what is the what model?
4. what is use case model?
5. what is involved in the analysis process? where should we start?
6. why is use case modeling useful in analysis?
7. who are the actors?
8. why are uses and extends associations useful in use case modeling?
9. how would you identify actors?
10. what is the 80-20 rule?
11. why is documentation a an important part of analysis?
12. what is the difference between users and actors?

LESSON - 6

OBJECT ANALYSIS

Objective

- The concept of classification
- How to identify classes with the noun phrase approach
- How to identify classes with the common pattern approach How to identify classes and object behavior analyzed by sequence/collaboration modeling
- How to identify classes with the classes, responsibilities and collaborator(CRC) approach.

Introduction

Object oriented analysis is a process by which we can identify classes that play a role in achieving system goals and requirements. Booch argues that “There is no such a thing as the perfect class structure, nor the right set of objects. As in any engineering discipline, or design choices compromisingly shaped by many computing factors.

In this lesson we look at 4 approaches for identifying classes and their behaviors in the problem domain. Discovering classes is one of the hardest activities in the object oriented analysis. However in the process is incremental and iterative and furthermore, as you gain experience it will become easier to identify the classes.

6.1 Classification theory

Classification, the process of checking to see if an object belongs to category or a class, is regarded as a basic attribute of a human nature. Human beings classify information every instant of their waking lives. We recognize the objects around us, and we move and act in relation to them. A human being is a very sophisticated information system, partly because he or she possesses a superior classification capability (11).

Clearly you have some general idea of what cars look like, sound like do and are good for you have an notion of car kind or in object oriented terms in the class car. Classes are an important mechanism for classifying objects. The chief role of a class is to define the attributes, methods, and applicability of its instances.

The Problem of classification may be regarded as one of discriminating things, not between the individual objects but between classes, via the search for the features or invariant attributes or behaviors among members of a class., Classification can be defined as the categorization of input data into identifiable classes via the extraction of significant features of attributes of the data from a background of irrelevant detail.

6.2 Approaches for Identifying classes

In the following sections, we look at four alternative approaches for identifying classes, the noun

- The noun process approach; the common class patterns approach; the use case driven, sequence/collaboration modeling approach; and the classes, responsibilities, and collaborators (CRC) approach.

The First two approaches have been included to increase your understanding of the subject, the unified approach uses the use case driven approach for identifying classes and understanding the behavior of objects. However, you always can combine these proaches to identify classes for a given problem. ap-

Another approach that can be used of identifying classes is classes, responsibilities, and collaborators(CRC) developed by Cunningham, Wilkerson, and beck classes, responsibilities, and collaborators, more technique than method, is used for identifying classes responsibilities and therefore their attributes and method.

6.3 Noun Phrase Approach

The noun phrase approach was proposed by Rebecca wrifs Brock, Brain Wilkerson. In this method you read through the requirements or use cases looking for noun phrases. Nouns in the textual description are considered to be classes and verbs to be methods of the classes. All plurals are changed to singular, the nouns are listed, and the list divided into three categories see Fig 6.1.

- Relevant classes.
- fuzzy classes
- irrelevant classes.

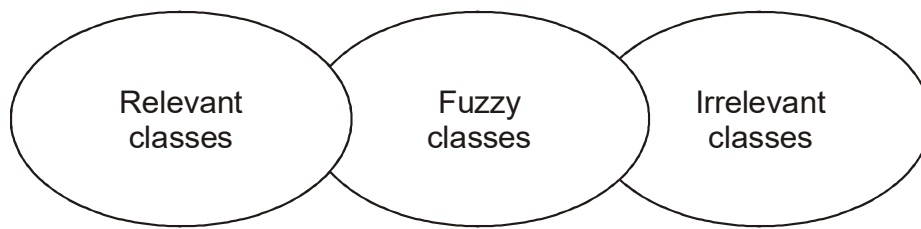


Figure 6.1

It is safe to scrap the irrelevant classes, which either have no purpose or will be unnecessary. Candidate classes then are selected from the other two categories. Keep in mind that identifying classes and developing a UML class diagram just like other activities is an iterative process. Depending on whether such object is for the analysis or design phase of development, some classes may need to be added or removed from the model and, remember, flexibility is a virtue. You must be able to formulate a statement of purpose for each candidate class, if not simply eliminate it.

6.3.1 Identifying Tentative Classes

The following are guidelines for selecting classes in an application:

- Look for nouns and noun phrases in the use cases.
- Since Classes are implicit or taken from general knowledge.
- All Classes must make sense in the application domain, avoid computer implementation classes-defer them to the design stage.
- Carefully choose and define class names.

The more practice you have the better you get at identifying classes. Finding classes is incremental and iterative process. Booch explains elegantly “Intelligent classification is Intellectually hard work, and best comes about through an incremental and iterative process. This incremental and iterative nature is evident in the development of such diverse software technologies as graphical user interfaces, database standards, and even fourth generation languages”

6.3.2 Selecting Classes from the Relevant and Fuzzy categories.:

The following guideline help in selecting candidate classes from the relevant and fuzzy categories of classes in the problem domain.

- **Redundant classes.** Do not keep two classes that express the same information. If more than one word is being used to describe the same idea, select the one that is the most meaningful in these as a whole. Choose your vocabulary carefully, use the word that is being used the user of the system.
- **Adjectives classes.** Wirfs Brock, Wilkerson and Wiener warn us about adjectives: “Be wary of the use of adjectives. Adjectives can be used in many ways. An adjective can suggest a different kind of object, different use of the same object, or it could be utterly irrelevant. Does the object represented by the noun behave differently when the adjective is applied to it? If the use of the adjective signals that the behavior of the object is different, then make a new class”
- **Attribute classes.** Tentative objects that are used only as values should be defined or restated as attributes and not as a class. For example, client status and demographic of client are not classes but attributes of the client class.
- **Irrelevant classes.** Each class must have a purpose and every class should be clearly defined and necessary. You must formulate a statement of purpose for each candidate class. If you cannot come up with a statement of purpose, simply eliminate the candidate class.

Remember that this is an incremental process. Some classes will be missing others will be eliminated or refined later. Unless you are starting with a lot of domain knowledge, you probably are missing more classes than you will eliminate. Although some classes ultimately may become super classes at this stage simply identify them as individual, specific classes. You will have adequate opportunity to revise it.

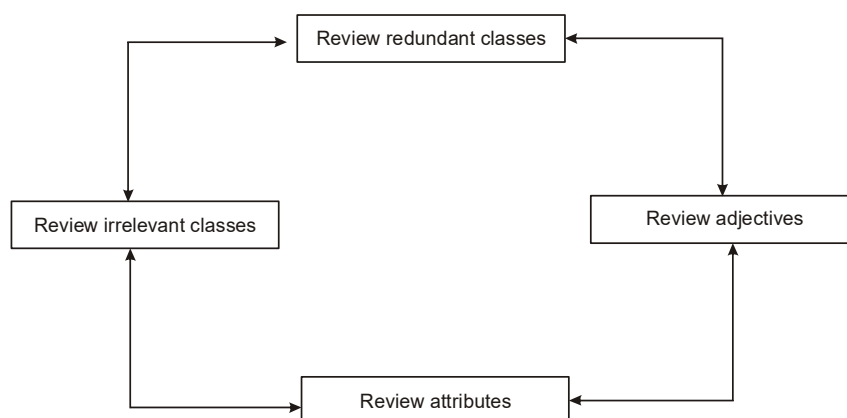


Figure 6.2

Like any other activity of software development the process of identifying relevant classes and eliminate irrelevant classes is an incremental process. Each iteration often uncovers some classes that have been overlooked. the repetition of the entire process, combined with what you already have learned and the reworking of your candidate classes will enable you to gain a better understanding of the system and the classes that make up you're a application.

Classification is the essence of good object oriented analysis and design. You must continue this refining cycle through the development process until you are satisfied with the results. Remember that this process is not sequential. You can move back and forth among these steps as often as you like see figure 6.2.

6.3.3 *The Via Net bank ATM System: Identifying Classes by using Noun Phrase Approach*

To better understanding the noun phrase method, we will go through a case and apply the noun phrase strategy for identifying the classes. We must start by reading the use cases and applying the principles discussed in this lesson for identifying classes.

6.3.4 *Initial List of Noun Phrases: Candidate Classes*

The initial of the use cases of the bank system produces the following noun phrases

Account

Account Balance

Amount

Approval Process

ATM Card

ATM Machine

Bank

Bank Client

Card

Cash

Check

Checking

Checking Account
Client
Client's Account
Currency
Dollar
Envelope
Four Digits
Fund
Invalid PIN
Message
Money
Password
PIN
PIN Code
Record
Savings
Savings Account
Step
System
Transaction
Transaction History

It is safe to eliminate the irrelevant classes. The candidate classes must be selected from relevant and fuzzy classes. The following irrelevant classes can be eliminated because they do not belong to the problem statement: envelope, four digits and step Strikeouts include eliminated classes.

phrases

Account

Account Balance

Amount

Approval Process

ATM Card

ATM Machine

Bank

Bank Client

Card

Cash

Check

Checking

Checking Account

Client

Client's Account

Currency

Dollar

Envelope

Four Digits

Fund

Invalid PIN

Message

Money

Password

PIN

PIN Code

Record

Savings
 Savings Account
 Step
 System
 Transaction
 Transaction History

6.3.5 Reviewing the Redundant Classes and Building a Common Vocabulary

We need to review the candidate list to see which classes are redundant. If different words are being used to describe the same idea, we must select the one that is the most meaningful in the context of the system and eliminate the others.

The following are the different class names that are being used to refer to the same concept:

Client, Bank client	=	Bank client (the term chosen)
Account, Client's Account	=	Account
PIN PIN Code	=	PIN
Checking, Checking Account	=	Checking account
Saving, Saving Account	=	Saving Account
Fund, Money	=	Fund
ATM Card, Card	=	ATM Card

Here is the revised list of candidate classes:

Account
~~Account Balance~~
~~Amount~~
 Approval Process
 ATM Card
 ATM Machine

Bank
Bank Client
~~Card~~
Cash
Check
~~Checking~~
Checking Account
~~Client~~
~~Client's Account~~
Currency
Dollar
~~Envelope~~
~~Four Digits~~
Fund
Invalid PIN
Message
~~Money~~
~~Password~~
~~PIN~~
~~PIN Code~~
Record
~~Savings~~
Savings Account
~~Step~~
System
Transaction
~~Transaction History~~

6.3.6 Reviewing the classes containing Adjectives

We again review the remaining list, now with an eye on classes with adjective. The main question is this: does the object represented by the noun behave differently, when the adjective is applied to it? If an adjective suggests a different kind of class or the class represented by the noun behaves differently when the adjective is applied to it, then we need to make a new class.

6.3.7 reviewing the Possible Attributes

the next review focuses on identifying the noun phrases that are attribute not classes. The noun phrases used only as value should be restated as attributes. This process also will help us identify the attribute is the classes in the system.

Amount: A value, not a class.

Account balance: an attribute of the Account class.

Invalid PIN: It is only a value, not a class.

Password: An attribute, possibly of the bank client class.

Transaction History: An attribute, possibly of the Transaction class.

Pin: An attribute, possibly of the Bank Client class

Here is revised list of candidate classes. Notice that the eliminated classes are strikeouts.

Account

Account Balance

Amount

Approval Process

ATM Card

ATM Machine

Bank

Bank Client

Card

Cash

Check

Checking
Checking Account
Client
Chent's Account
Currency
Dollar
Envelope
Four Digits
Fund
Invalid PIN
Message
Money
Password
PIN
PIN Code
Record
Savings
Savings Account
Step
System
Transaction
Transaction History

6.3.8 *Reviewing the class purpose*

Identifying the classes that play a role in achieving system goal and requirements is a major activity of object oriented analysis. Each class must have a purpose. Every class should be clearly and necessary in the context of achieving the system's goals. If you cannot formulate

a statement of purpose for a class, simply eliminate it the classes that add no purpose to the system have been deleted from the list.

6.4 Common class Pattern Approach.

The second method for identifying classes is using common class pattern, which is based on a knowledge base of the common classes that have been proposed by various researchers such as Sahlaer and mellor and Coad and Yourdon. They have compiled and listed the following pattern for finding the candidate class and object.

- **Context. Concept class**

Context. A concept is a particular idea or understanding that we have of our world. The concept class encompasses principles that are not tangible but used to organize or keep track of business activities or communication.

Example. Performance is an example of concept class object.

- **Name. events class**

Context. Event classes are points in time that must be recorded. Things happen, usually to something else at a given date and time or as step in an ordered sequence. Associated with thing remembered are attributes such as who, what, when, where, how, or why. Example landing, interrupt, request and order are possible events.

- **Name organization class**

Context. An organization class is a collection of people, resources, facilities, or group to which the users belong, their capabilities have a defined mission.

Example. An accounting department might be considered a potential class.

- **Name. People class**

Context. The people class represents the different role users play in interacting with application. People carry out some function.

Example employee, client, teacher, and manager are example of people.

- **Name place class**

Context. Places are physical location that the system must keep information about.

Example. Building, stores, sites, and offices are example of place.

- **Name. *Tangible things and device class***

Context, this class includes physical object or groups of objects that are tangible and devices with the application interacts.

6.5 Use case driven approach: Identifying classes and their behaviors through sequence / collaboration modeling

The use case driven approach is the third approach that we examine in this lesson and the one that is recommended. From the previous lesson. We learned that use cases are employed to model the scenarios in the system and specify what external actors interact with the scenarios. The scenarios are described in text or through a sequence of steps.

6.5.1 Implementation of scenarios

The UML specification recommends that at least one scenario be prepared for each significantly different use case instance. Each scenario shows a different sequences of interaction between actors and the system, with all decisions definite. When you have arrived at the lowest use case level you may create a child sequences diagram or accompanying collaboration diagram for the use case.

Like use case diagram sequences diagram are used to model scenarios in the system. Whereas use cases and the steps or textual descriptions that define them offer a high level view of a system the sequences diagram enables you to model a more specific analysis and also assists in the design enables you to model a more specific analysis and also assists in the design of the system by modeling the interaction between objects in the system.

6.5.2 The Via Net bank ATM system: Decomposing a use case scenario with a sequences Diagram: Object behavior Analysis

A sequences diagram represents the sequences and interactions of a given use case or scenario. Sequences diagram are among the most popular UML diagrams and if used with an object model or class diagram can capture most of the information about a system. Most object to object interactions and operations are considered events include signals, input decisions, interrupts, transitions and action to or from users or external devices.

The event line represent a message sent performed by the “to” object. The “to” object performs the operation using a method that its class contains. Developing sequences or

collaboration diagram requires us to think about objects that generate these events and therefore will help us in identifying classes. Sequences and collaboration diagram represent the order in which things occur and how the objects in the system send messages to one another. These diagrams provide a macro level analysis of a dynamics of a system.

You can draw sequences diagram to model each scenario that exists when bank client withdraw, deposits or needs information on an account. By walking through the steps, you can determine what objects are necessary for those steps to take place. Therefore, the process of creating sequences or collaboration diagram can assist you in identifying classes or objects of the system. We identified the use cases for the bank system. The following are the low level use cases.

Deposit checking

Deposit Savings

Invalid PIN

Withdraw Checking

Withdraw More from Checking

Withdraw Savings

Withdraw Saving Denied

Let us create a sequences/collaboration diagram for the following use cases.

- Invalid pin use case
- Withdraw checking use case
- Withdraw more checking use case

Consider how we would prepare a sequences diagram for the invalid pin use case. Here we need to think about the sequences of activities that the actor bank client performs.

- Insert ATM card
- Enter pin number
- Remove the ATM card

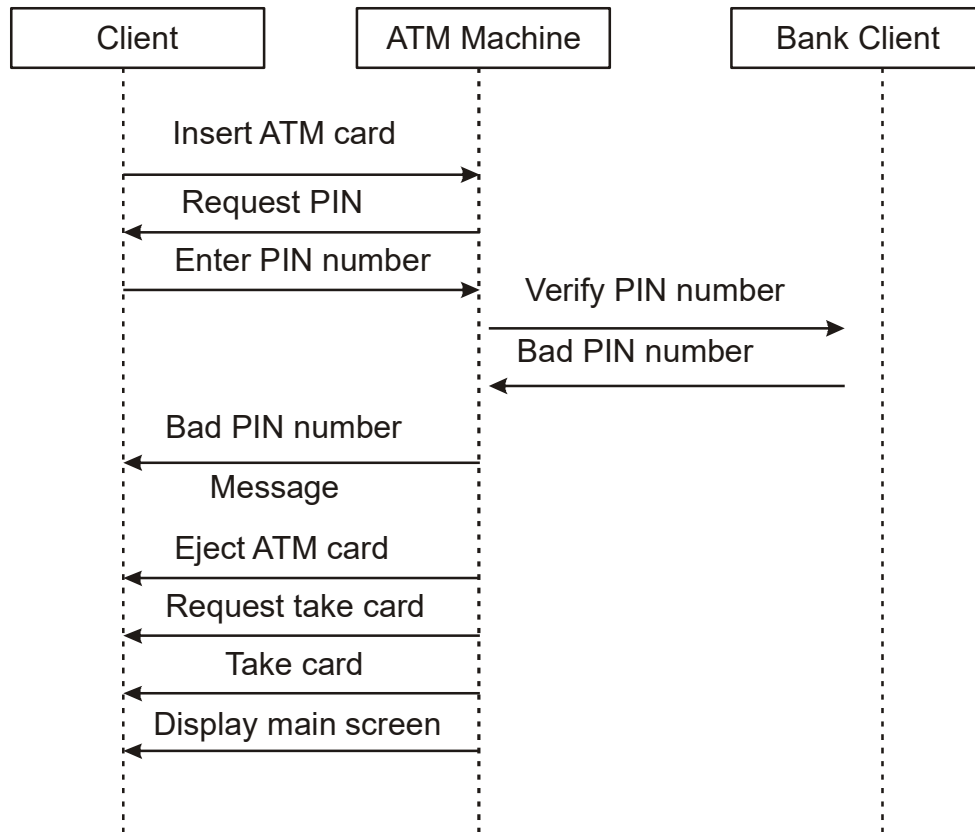


Figure 6.4

We are interacting with an ATM machine and Bank client. -Now that we have identified the objects involved are in the use case we need to list them in a line along the top of a page and drop dotted lines beneath each object see figure 6.4. the client in this case is whoever tries to access an account through the ATM, and may or may not have an account. The bank client on the other hand has an account

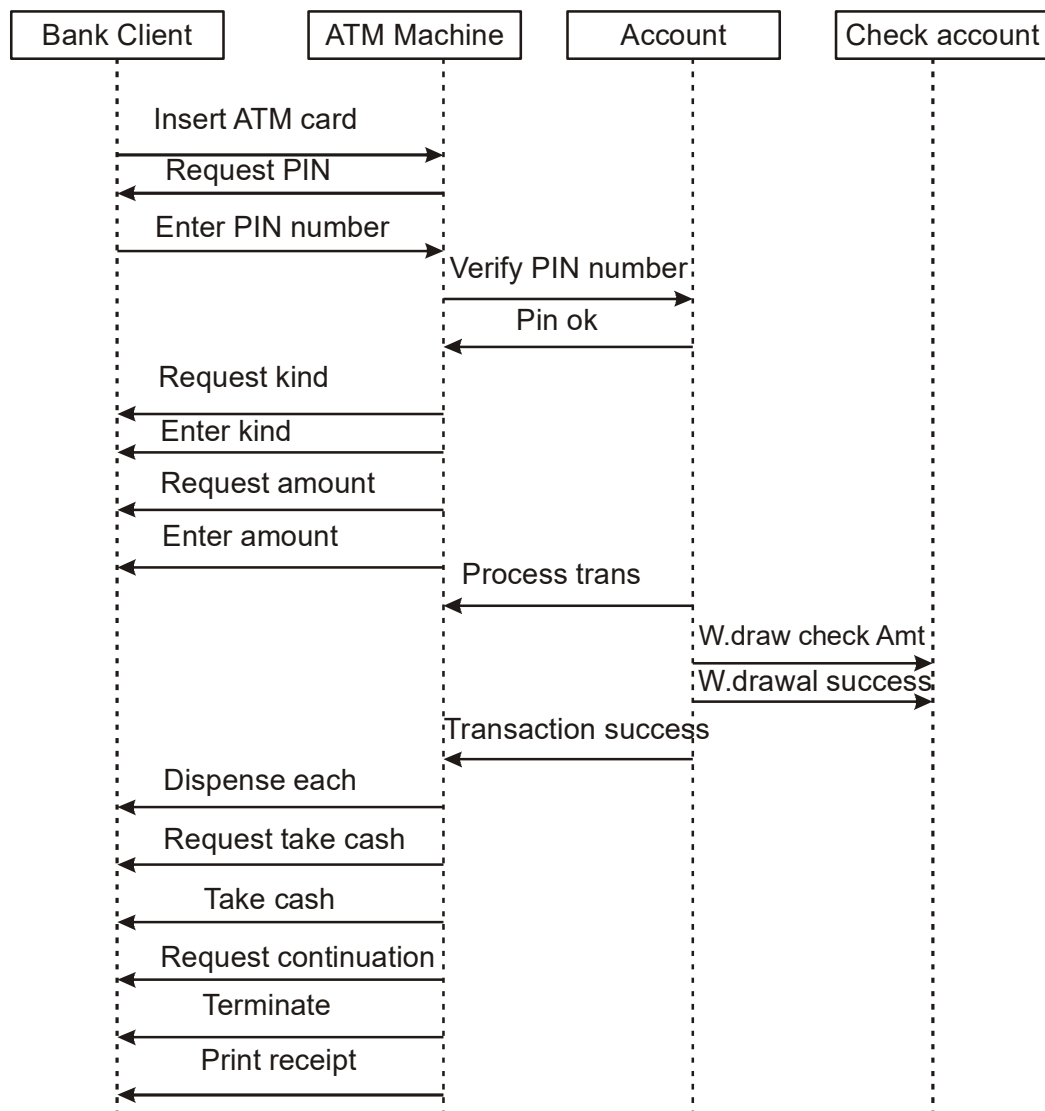


Figure 6.5

In effect the event arrow suggests that a message is moving between those two objects. In other cases an object becomes active when it receives a message and then becomes inactive as soon as it responds. Similarly we can develop sequences diagrams for there use cases as in figure 6.5 and 6.6.

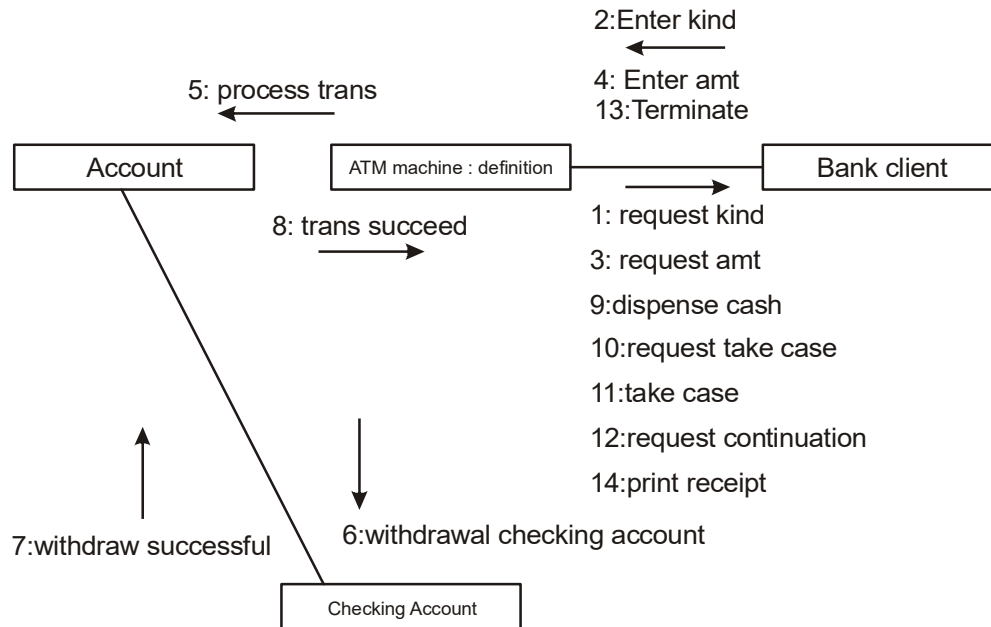


Figure 6.6

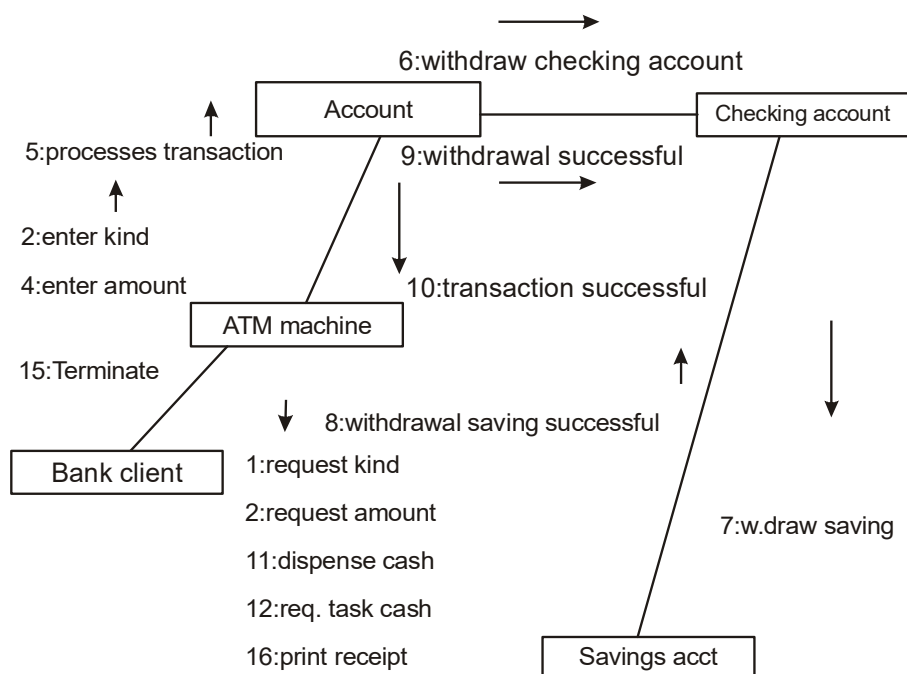


Figure 6.8

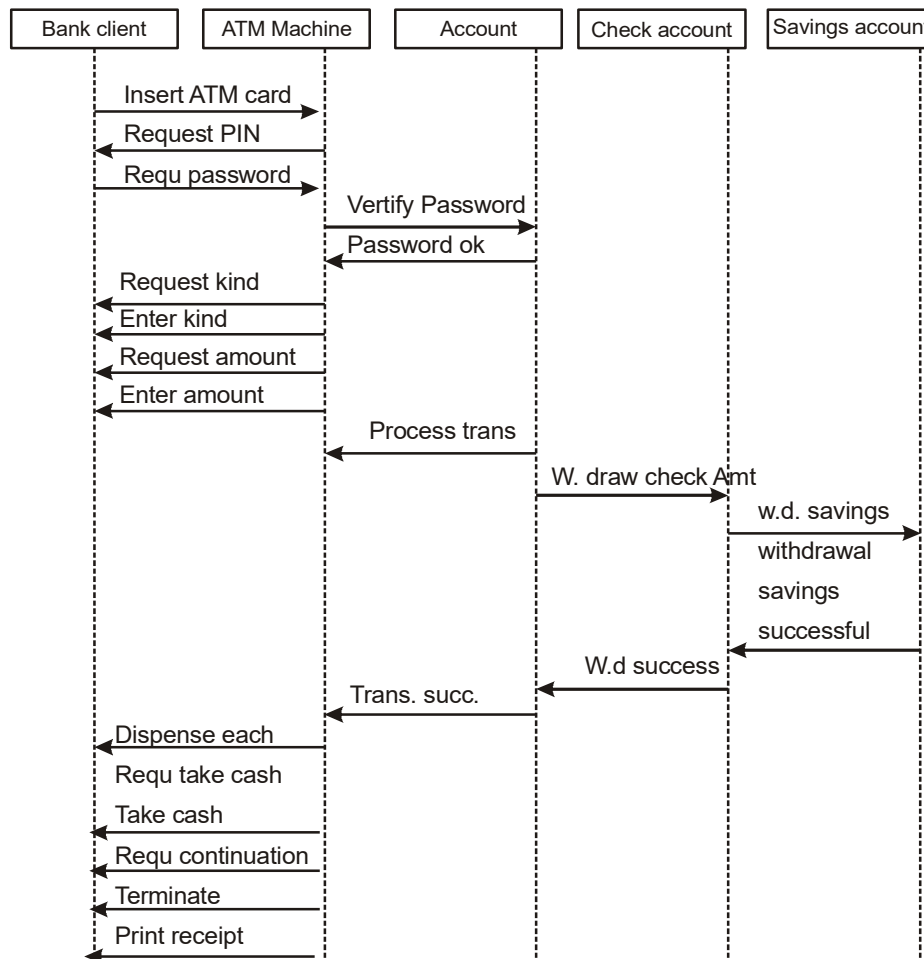


Figure 6.7

Collaboration diagram are just another view of the sequences diagram and therefore can be created automatically, most UML modeling tools automatically create them see figure 6.6 and 6.7. the following classes have been identified by modeling the UML sequences collaboration diagram: bank, bank client, ATM machine, account, checking account and saving account.

6.6 class, responsibilities and collaborators

Class, responsibilities and collaborators developed by Cunningham, Wilkerson and beck, was first presented as a way of teaching the basic concepts of object oriented development,

class, responsibilities and collaborators is a technique used for identifying classes responsibilities and therefore their attributes and methods Furthermore class, responsibilities and collaborators can help us identify classes. class, responsibilities and collaborators is more a teaching technique than a method for identifying classes. class, responsibilities and collaborators is based on the idea that an object either can accomplish a certain responsibility itself or it may require the assistance of other objects.

All the information for an object is written on a card which is cheap, portable, readily available and familiar. Figure 6.9 shows an idealized card. The class name should be appear in the upper left hand corner, a bulleted list of collaborators should appear in the right third

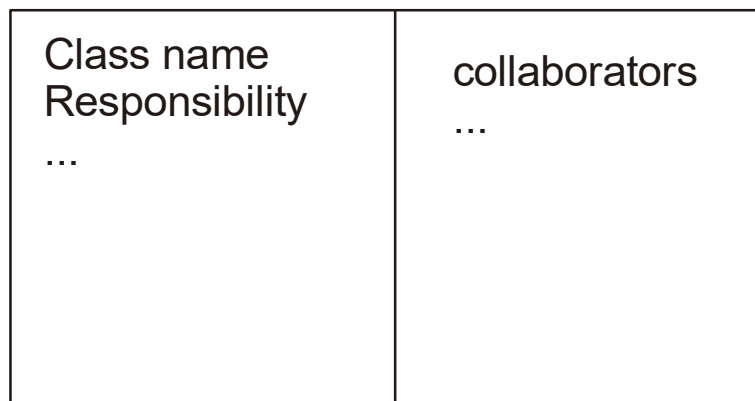


Figure 6.9

6.6.1 classes, responsibilities. And collaborators process

The classes, responsibilities. And collaborators process consists of three steps see figure 6.10.

1. Identify classes responsibilities
2. Assign responsibilities.
3. Identify collaborators.

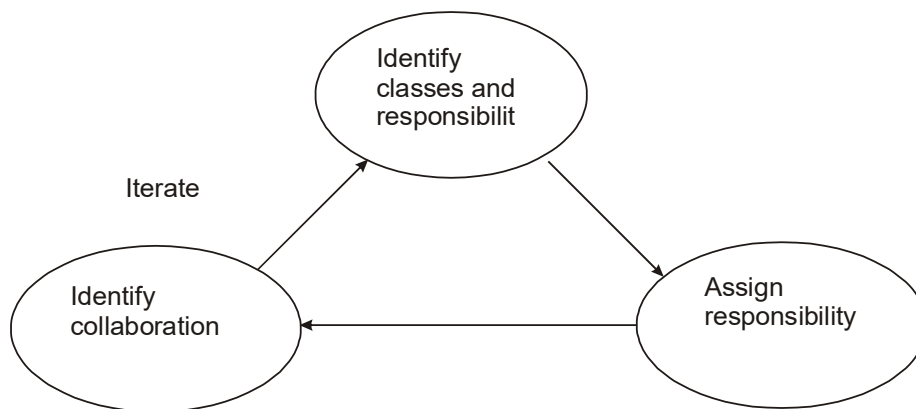


Figure 6.10

Classes are identified and grouped by common attributes which also provides candidates for super classes. The class names then are written onto classes, responsibilities. And collaborators cards. The card also notes sub and super classes to show the class structure. The idea in locating collaborators is to identify how classes interact. Classes that have a close collaboration are grouped together physically.

6.7 Naming classes

Naming a class is an important activity. Here are guidelines for naming classes.

- The class name should be singular. The class should describe a single object so it should be the singular form of noun.
- One general rule for naming classes is that you should use names with which the users or client are comfortable.
- The name of the class should reflect its intrinsic nature. Stick with the general terminology of the subject matter.
- Use readable names. Capitalize class names. By convention the class name must begin with an upper case letter. For compound words capitalize the first letter of each word

Summary

- The problem of classification may be regarded as one of discriminating things, not between the individual objects.

- There is no such thing as the perfect class structure or the right set of objects.
- In this lesson we studied four approaches for identifying classes:
 1. The noun phrase
 2. Common class pattern
 3. Use case driven
 4. Classes, responsibilities and collaborators.
- To identify classes using the noun phrase approach read through use cases looking for noun phrases.
- The second method for identifying classes is the common class pattern approach based on the knowledge base of the common classes proposed by various researchers.
- The third method we studied was use case driven

Questions:

1. where do object come from?
2. what are the events classes?
3. what are the concepts classes?
4. how would you name classes?
5. what is the common class patterns strategy?
6. what criteria would you use to eliminate a class?
7. explain the places class source?
8. why class, responsibilities, and collaborators useful?

LESSON - 7

OBJECT RELATIONSHIP, ATTRIBUTES AND METHODS

Objectives

- Analyzing relationship among classes.
- Identifying association.
- Association patterns.
- Identifying super and subclass hierarchies.
- Identifying aggregation or a part of compositions.
- Class responsibilities.
- Identifying attributes and methods by analyzing use cases and other UML diagrams.

Introduction

In an object oriented environment objects task on an active role in a system. Of course objects do not exist in isolation but interact with each other. Indeed these interactions and relationship are the application. All object stand in relationship to others on whom they rely for services and control. The relationship among objects is based on the assumptions each makes about the other objects including what operations can be performed and what behavior results. Threetype relationships among objects are

1. Association. How are objects associated? This information will guide use cases in designing classes.
2. Super sub structure. How are objects organized into super classes and sub classes? This information provides use cases the direction of inheritance.
3. Aggregation and a part of structure. What is the composition of complex classes?. This information guides use cases in defining mechanisms that properly manage object within object.

Generally speaking the relationship among objects are known as associations

For example, a customer places an order for soup. The order is the association between the customer and soup objects. The hierarchical or super sub relation allows the sharing of

properties or inheritance. A part of structure is a familiar means of organizing components of a bigger object a building.

In the lesson we look at guideline for identifying association super sub and a part of relationship in the problem domain. We then proceed to identify attributes and methods. To do this we must first determine the responsibilities of the system. We saw that the system responsibilities can be identified by analyzing use cases and their sequences and collaboration diagram. Once you have identifying the system responsibilities and what information the system need to remember you can assign each responsibility to the class to which it logically belongs. This also aids in determining the purpose and role each class plays in the application.

7.1 Associations

Association represents a physical or conceptual connection between two or more objects. For example if an object has the responsibility for telling another object that a credit card number is valid or not valid the two classes have an association. We learned that the binary association are shown as line connecting two class symbols. Ternary and higher order association shown as diamonds connecting to a class symbol by lines and the association name is written above or below the line. The association name can be omitted if the relationship is obvious. In some cases you will want to provide names for the roles played by the individual classes making up the relationship. The role name on the side closest to each describes the role that class plays relative to the class at the other end of the line and vice versa see figure 7.1.

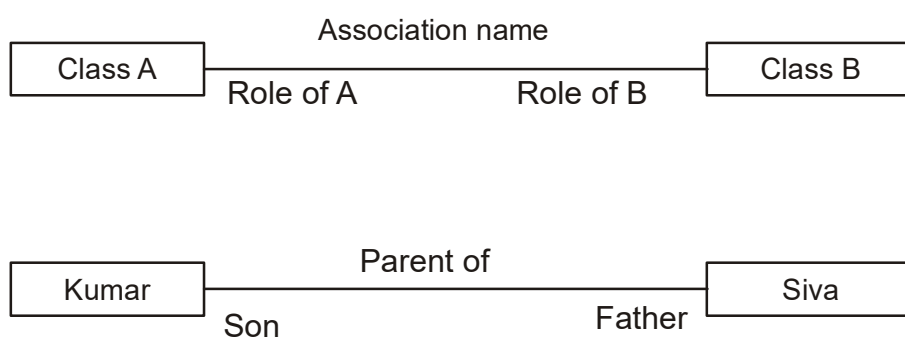


Figure 7.1

7.1.1 Identifying Association

Identifying associations begins by analyzing the interactions between classes. After all any dependency relationship between two or more classes is an association. You must examine

the responsibilities to determining dependencies. In other words if an object is responsible for specific task and lack all the necessary knowledge needed to perform the task, then and lacks all the necessary knowledge needed to perform the task, then the object must delegate the task to another object the processes such knowledge. Wirfs-Brock, Wilkerson, and Wiener provide the following questions that can help use cases to identifying association.

- Is the class capable of fulfilling the required task by itself?
- If not what does it need?
- From what other class can it acquire what it need?

Answering these questions helps use cases identify association. The approach you should task to identify association is flexibility. First extract all candidates association from the problem statement and get them down on paper. You can refine them later. Notice that a part of structures and association are very similar. So how do you distinguish one from the other? It depends on the problem domain. After all a-part-of structure is a special case of association. Simply pick the one most natural for the problem domain. If you can represent the problem more easily with association then select it otherwise use a part of structure which is described later in this lesson.

7.1.2 Guideline for identifying association

The following are general guideline for identifying the tentative association.

- A dependency between two or more classes may be an association. Association often corresponds to a verb or prepositional phrase, such as part of next to works for or contained in.
- A reference from one class to another is an association. Some associations are implicit or taken from general knowledge.

7.1.3 Common Association pattern

The common association pattern are based on some of the common association defined by researchers and practioners: Rumbaugh et analyzing. Coad and yourdon and other. These include

- Location association next to part of contained in for example consider a soup object cheddars cheese is a part is soup. The a part of relation is a special type of association discussed I more detail later in the lesson.

- Communication association-talk to order to. For example, customer places an order with an operator person see figure 7.2.

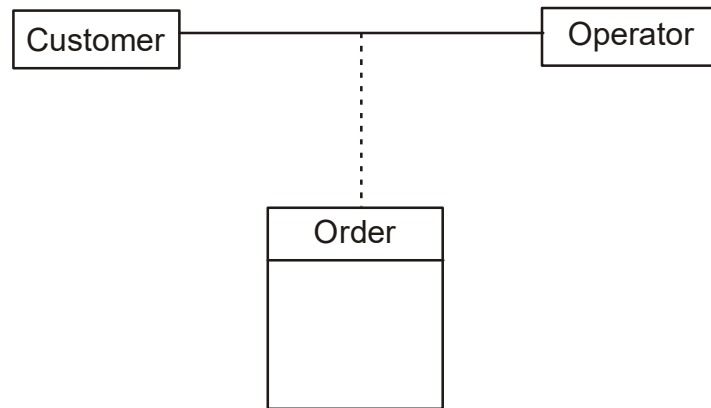


Figure 7.2

These association pattern and similar ones can be stored in the repository and added to as more pattern are discovered. However currently this capability of the unified approach's repository is more conceptual than real but it is my hope that CASE tool vendors in the near future will provide this capability.

7.1.4 Eliminate Unnecessary Associations

- Implementation association. Defer implementation specific association to the design phase. Implementation association are concerned with the implementation or design of the class within certain programming or development environment and not relationship among business objects.
- Ternary associations. Ternary or n-ary association is an association among more than two class. ternary association complicate the representation. When possible restate ternary association as binary associations.
- Directed action association. Directed actions associations can be defined in terms of other associations. Since they are redundant avoid these types of association. For example Grandparent of can be defined in terms of the parent of association see figure 7.3.

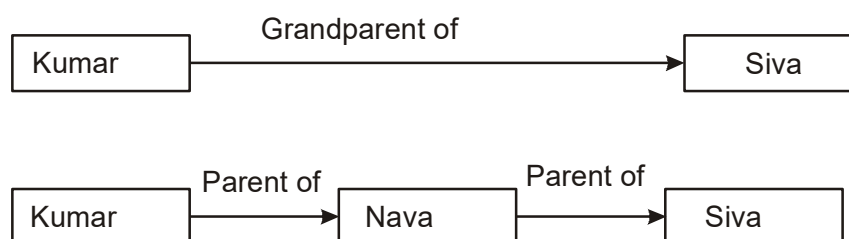


Figure 7.3

Choose association names carefully. Do not say how or why a situation came about, say what it is. Add role names where appropriate, especially to distinguish multiple associations. These often are discovered by testing access paths to do objects.

7.2 Super-sub class Relationship

The other aspect of classification is identification of super sub relationship among classes. For most part a class is part of a hierarchy of classes where the top class is the most general one from it descent all other, more specialized classes.

The super sub relationship represent the inheritance relationship between related classes, and the class hierarchy determines the lines of inheritance between classes, and the class hierarchy determines the lines of inheritance between classes. Class inheritance is useful for number of reasons. For example in some cases you want to create a number of classes that are developed a class that you can use but you need to modify that class. Subclasses are more specified versions of their super classes. The classes are not ordered this way for convenience's sake.

Super classes sub class relationship, also known as generalizations hierarchy, allow objects to be built from other objects, such relationship allow use cases to explicitly take advantage of the commonality of objects when constructing new classes. The super sub class hierarchy is a relationship between classes, where one class is the parent class of another class.

Recall from that the parent class also is known as the base or super class or ancestor. The super sub class hierarchy is based on reinvention. The real advantage of using this technique is that we can build on what we already have and more important reuse what we ready have. Inheritance allows classes instance is defined in that class's methods, a class also inherits the behavior and attributes of all of its super classes. Now let use cases take a look at guideline for identifying classes.

7.2.1 Guidelines for identifying Super sub Relationship a generalization

The following are guidelines for identifying super sub relationship in the application:

- Top down. Look for noun phrases composed of various adjectives in a class name. Often you can discover additional special cases. Avoid excessive refinement. Specialize only when the subclasses have significant behavior. For example a phone operator employee can be represented as a cook as well as a clerk or manger because they all have similar behavior.
- Bottom up. Look for classes with similar attributes or methods. In most cases you can group them by moving the common attributes and methods to an abstract class. You may have to alter the definitions a bit this is acceptable as long as generalization truly applies. However do not force classes to fit a preconceived generalization structure.
- Reusability. Move attributes and behavior as high as possible in the hierarchy. At the same time do not create very specialized classes at the top of the hierarchy. this is easier said than done. The balancing act can be achieved through several iterations. This process ensures that you design objects that can be reused in another application.
- Multiple inheritance Avoid excessive use of multiple inheritance. Multiple inheritance brings with it complications such as how to determine which behavior to get from which class particularly when several ancestors define the same method
- It also is more difficult to understanding programs written in a multiple inheritance system. one way of achieving the benefit of multiple inheritance is inherit from the most appropriate class and add an object of another class as an attribute see figure 7.4.

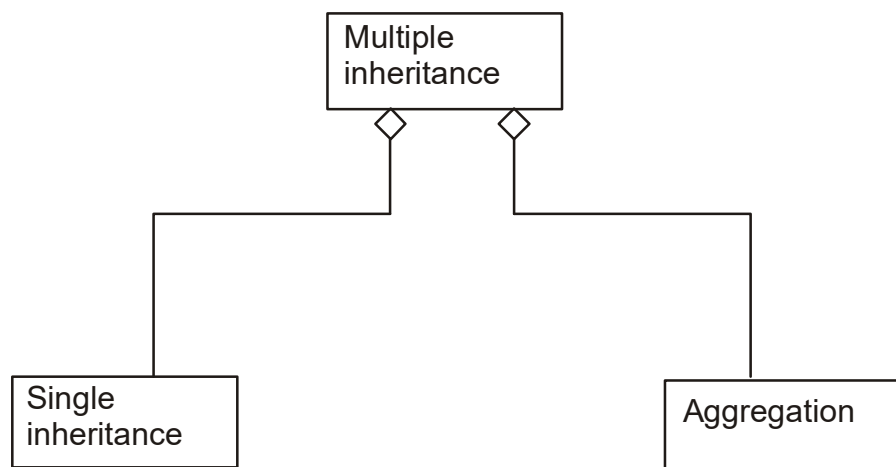


Figure 7.4

7.3 A-part of Relationship Aggregation

A-part of Relationship is called **Aggregation**, represents the situation where a class consists of several components classes A class that is composed of other classes does not behave like its parts, actually it behaves very differently. For example a car consists of many other classes, one of which is a radio but a car does not behave like a radio see figure 7.5.

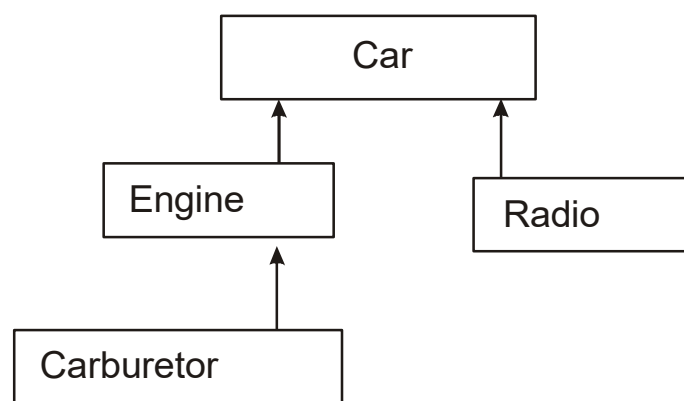


Figure 7.5

Two major properties of a part of relationship are transitivity and anti symmetry.

- **Transitivity.** The property where if A is part of B and B is part of Class then A is part of C. for example a carburetor is part of an engine and an engine is part of a car

therefore carburetor is part of a car. Figure 5.3 shows a part of structure.

- Anti symmetry. The property of a part of relation where if A is Part of B then B is not part of A. For example an engine is part of car, but car is not part of an engine.
- A clear distinction between the part and the whole can help use cases determine where responsibilities for certain behavior must reside. This is done mainly by asking the following questions.
- Does the part class belong to a problem domain?
- Is the part class within the system's responsibilities?
- Does the part class capture more than a single value?
- Does it provide a useful abstraction in dealing with the problem domain?

We saw that the UML uses hollow or filled diamonds to represents aggregation. A filled diamond signifies the strong form of aggregation which is composition.

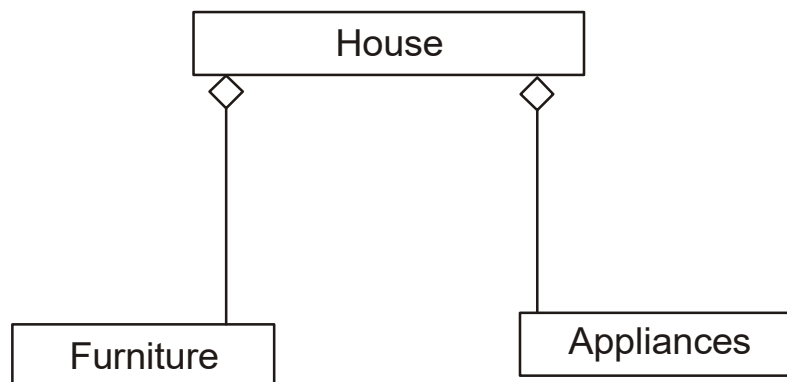


Figure 7.6

For example one might represent aggregation such as container and collection as hollow diamonds see figure 7.6 and 7.7 and use a solid diamond to represent composition which is a strong form of aggregation see figure 7.5.

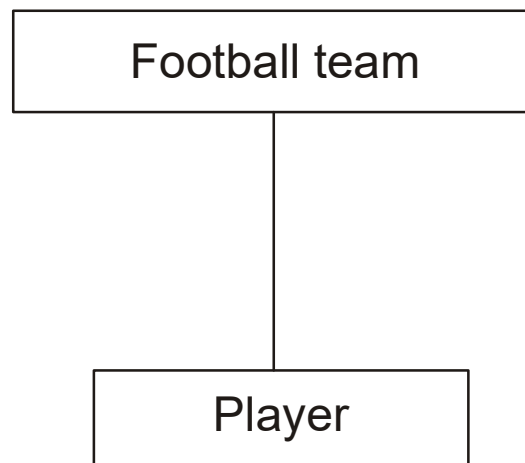


Figure 7.7

7.3.1 A part of Relationship pattern

To identify a part of structures Coad and our on provide the following guidelines:

- **Assembly.** An assembly is constructed from its parts assembly part situation physically exists; for example a French onion soup is an assembly of onion, butter, flour, wine, cheese and so on.
- **Container.** A physical whole encompasses but is not constructed from physical parts; for example, a house can be considered as a container for furniture and appliances see figure 7.6.
- **Collection member.** A conceptual whole encompasses parts that may be physical or conceptual; for example a football team is collection of players see figure 7.7.

7.4 Case study: Relationship analysis for the via net bank ATM system

To better gain experience in object relationship analysis we use the farailiar bank system case and apply the concepts in this lesson for identifying association super sub relationship and a part of relationship for the classes identifying. As explained before we must start by reading the requirements specification which is presented here. Furthermore object oriented analysis and design are performed in an iterative process using class diagrams.

Analysis is performed on piece of the system, design details are added to this partial analysis model and then the design is implemented. Changes can be made to the implementation and brought back into the analysis model to continue the cycle. This iterative process is unlike the traditional waterfall technique, in which all analysis is completed before design begins.

7.4.1 Identifying Classes Relationship

One of the strengths of object oriented analysis is the ability to model objects as they exist in the real world. To accurately do this, you must be able to model more than just an objects internal workings. You also must be able to model how objects relate to each other. Several different relationship exist in the via net bank ATM system so we need to define them.

7.4.2 developing a UML class diagram based on the use cases analysis the UML class diagram is the main static analysis and design diagram of a system. The analysis generally consists of the following class diagrams

- One class diagram for the system which shows the identity and definition of classes in the system their interrelationship and various package containing groupings of classes.
- Multiple class diagrams that represents various pieces, or views, of the system class diagram.
- Multiple class diagrams that show the specific static relationship between various classes.

First we need to create the classes that have been identified in the previous lesson. We will add relationship later see figure 7.8.

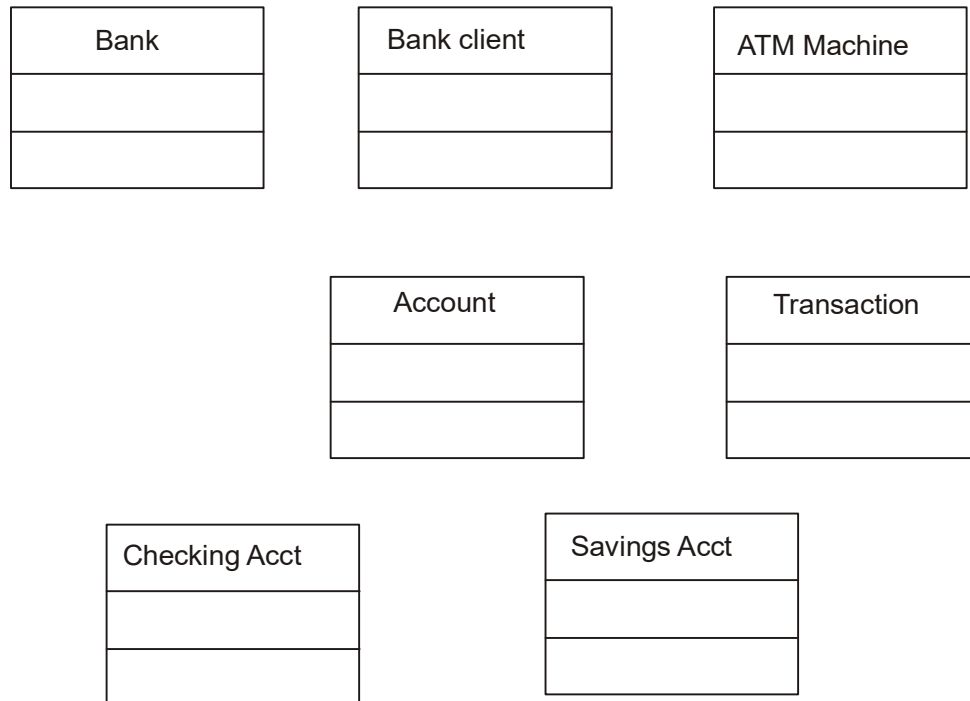


Figure 7.8

7.4.3 defining Association Relationship

Identifying association begins by analysis the interaction of each class. Remember that any dependency between two or more classes is an association. The following are general guidelines for identifying the tentative association as explained in this lesson.

- Association often corresponding to verb or prepositional phrases such as part of next to, works for, or contained in
- A reference from one class to another is an association. Some association are implicit or taken from general knowledge

Some common pattern of association are these. 0

- Location association. for example, next to, part of, contained in.
- Directed action association.
- Communication association. for example, talk to, order from

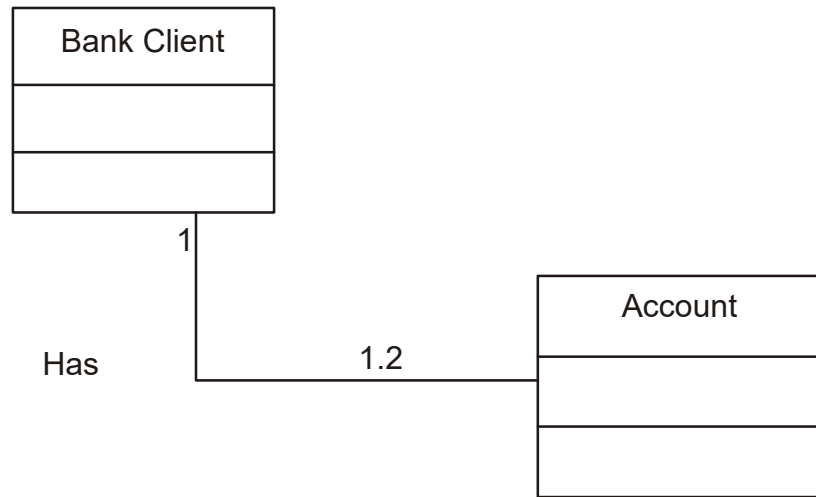


Figure 7.9

The first obvious relation is that each account belongs to bank client since each bank client has an account. Therefore there is an association between the bank client and account classes. We need to establish cardinality among these classes. By default in most case tools such as SA/Object architect all association are considered one to one. However since each Bank client can have one or two accounts, we need to change the cardinality of the association see figure 7.9, other association and their cardinalities are defined in figure 7.10.

7.4.4 Defining super sub Relationships

Let use cases review the guidelines for identifying super sub relationship

- Top-down. Look for noun composed of various adjective in the class name.
- Bottom-up. Look for classes with similar attributes or methods. In most cases you can group them by moving the common attributes and methods to an abstract class.
- Reusability. Move attributes and behavior as high as possible in the hierarchy.
- Multiple inheritance. Avoid excessive use of multiple inheritance.

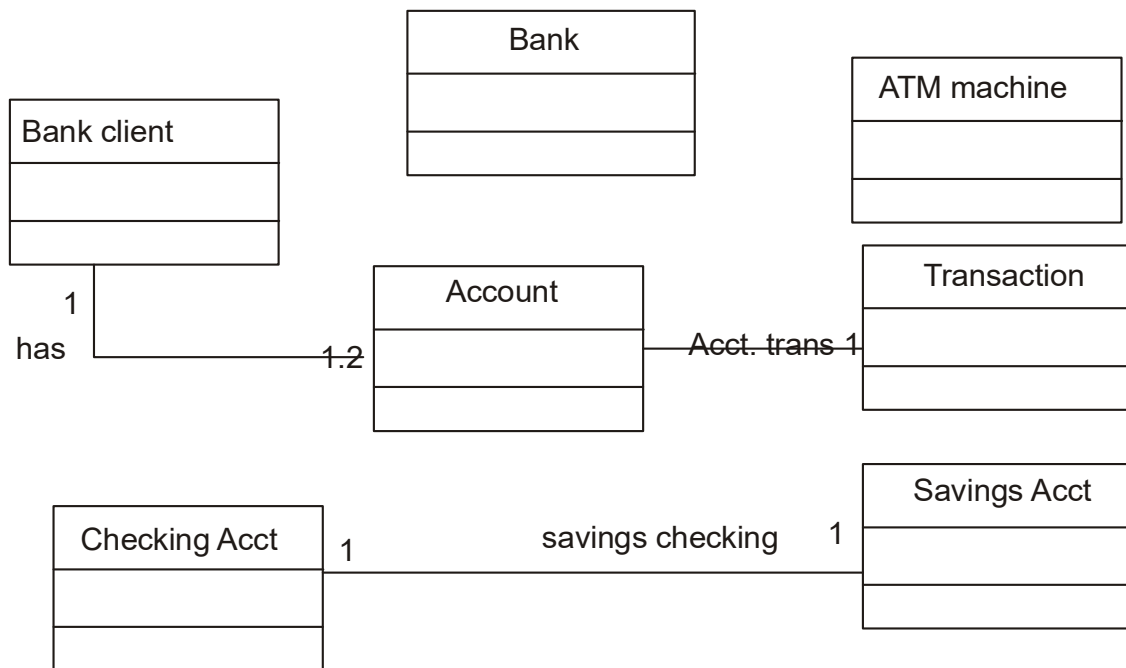


Figure 7.10

Checking account and saving account both are types of accounts. They can be defined as specifications of the account class. When implemented the account class will define attributes and services common to all kinds of accounts with checking account and saving account each defining that make them more

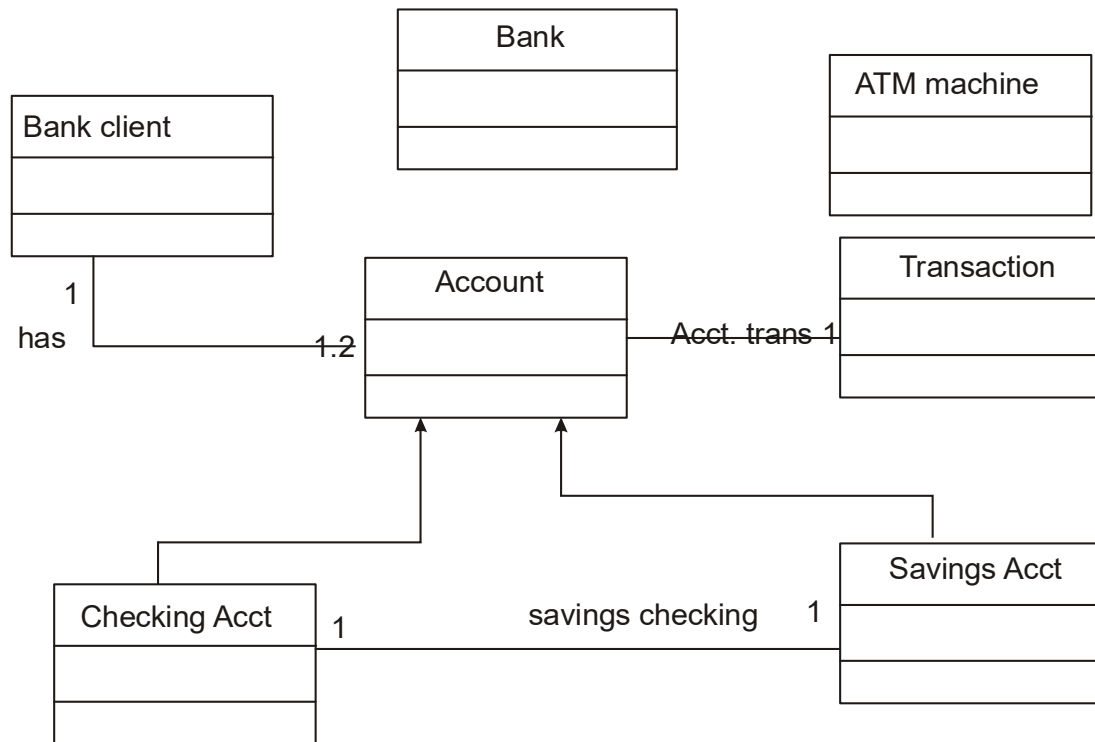


Figure 7.11

specialized. Figure 7.11 depicts the super sub relationship among accounts, checking account and saving account.

7.4.5 Identifying the aggregation/a-part-of relationship

To identify a part of structure we look for the following clues:

- **Assembly.** A physical whole is constructed from physical parts.
- **Container,** a physical whole encompasses but is not constructed from physical parts.
- **Collections-member.** A conceptual whole encompasses parts that may be physical or conceptual

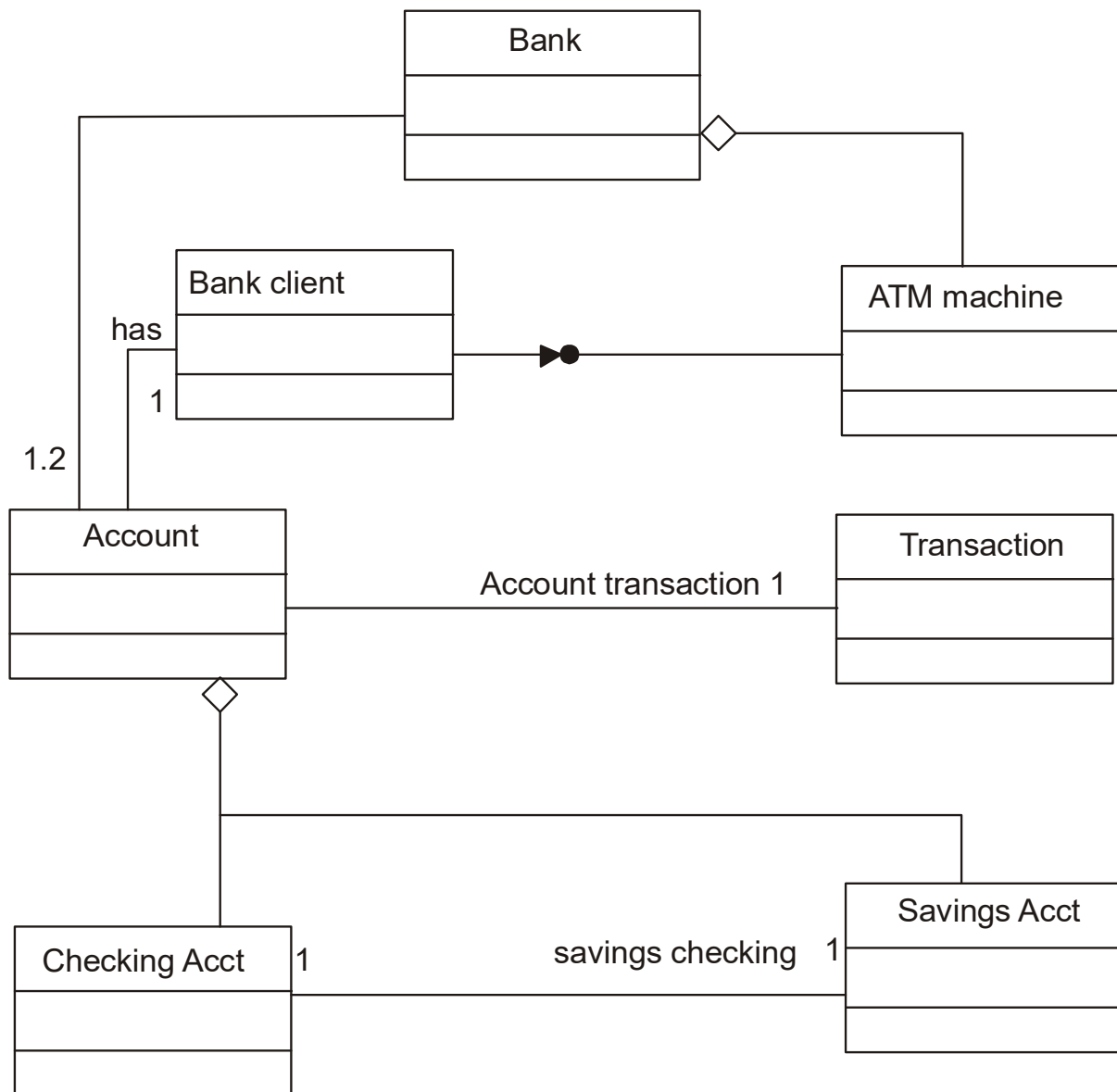


Figure 7.11

A bank consists of ATM machines, accounts, buildings, employee and so forth. However since buildings and employee are outside the domain of this application we define the bank class as an aggregation of ATM machine and account classes. Aggregation is a special type of association. Figure 7.12 depicts the association, generalization, and aggregation among the bank systems classes. If you are wondering what is the relationship between the bank client

and ATM machine it is an interface. Identifying a class interface is a design activity of object oriented system development, we look at defining the interface relationship.

7.5 Class Responsibility: identifying Attributes and Methods

Identifying attributes and methods like finding classes is still a difficult activity and an iterative process. Once again use cases and other diagrams will be our guide for identifying attributes, methods, and relationships among classes. Responsibility identifies problems to be solved. Back and Cunningham explain this point elegantly. “a responsibility serves as a handle for discussing potential solutions. The responsibilities of an object are expressed by handfuls of short verb phrases, each containing an active verb. The more that can be expressed by these phrases, the more powerful and concise the design”.

7.6 Class Responsibility: defining Attribute by Analyzing Use cases and other UML diagrams

Attributes can be derived from scenario testing; therefore by analyzing the use cases and sequences/collaboration, activity and state diagrams, you can begin to understand classes, responsibilities, and how they must interact to perform their tasks. The main goal here is to understand what the class is responsible for in a knowledge oriented environment. What kind of question would you like to ask.

- How am I going to be used?
- How am I going to collaborate with other classes?
- What do I need to know?
- What state information do I need to remember over time?
- What states can I be in?

Using previous object oriented analysis results or a pattern can be extremely useful in finding out what attributes can be reused directly and what lessons can be learned for defining attributes.

7.7.1 Guidelines for defining Attributes

Here are guidelines for identifying attributes of classes in use cases:

- Attributes usually correspond to nouns followed by prepositional phrases such as cost of the soup. Attributes also may correspond to adjectives or adverbs.
- Keep the class simple state only enough attributes to define the object state.
- Attributes are less likely to be fully described in the problem statement. You must draw on your knowledge of the application domain and the real world to find them.
- Omit derived attributes. For example do not use time elapsed since order. This can be derived from time of the order. Derived attributes should be expressed as a method.
- Do not carry discovery of attributes to excess. You can add more attributes in subsequent iteration.

7.8 Defining Attributes for via net bank objects

In this section we go through the bank system classes and define their attributes.

7.8.1 *Defining Attributes for the bank client class*

By analyzing the use cases the sequences/collaboration diagrams and the activity diagram it is apparent that for the bank client class, the problem domain and system dictate certain attributes. By looking at activity diagram we notice that the bank client must have pin number and card number. This usually is the case for defining attributes. The attributes of the bank client are

First name

Last name

Pin number

Card number

account: Account

7.8.2 Defining Attributes for the Account class

Similarly what information does the system need to know about an account? Based on the use cases in previous lessons.. bank client can interact with its account by entering the account number and then could deposit money get an account history or get the balance. Therefore we have defined the following attributes for the account class.

number

balance

7.8.3 defining Attributes for the transaction class

The transaction class for the most part must keep track of the time and amount of a transaction. Here are the attributes for the transaction class:

TransID

TransDate

Trans Time

Trans Type

Amount

PostBalance

7.8.4 Defining Attributes for the ATM machine class

The ATM machine class was identified as part of the common class pattern, which are physical objects or groups of objects that are tangible and with which the application interacts. Therefore most attributes for this class describe its physical location and its state. The ATM machine class could have the following attributes.

Address

State

7.9 Object Responsibility: Method and Messages

Objects not only describe abstract data but also must provide some services. Methods and message are the workhorses of object oriented system. In an object oriented environment,

every piece of data, or object, is surrounded by rich set of routines called methods. These methods do everything from printing the object to initializing its variables. Every class is responsible for storing certain information from the domain knowledge. Operations in the object oriented system usually correspond to queries about attributes of the object.

In other words methods are responsible for managing the value of attributes such as query, updating, reading and writing, for example an operation like `getbalance`, which can return the value of an account's balance.

7.9.1 Defining Methods by analyzing UML Diagram and use cases

An event is considered to be an action that transmits information. In other words these actions are operations that the objects must perform and as in the attributes, method also can be derived from scenario testing. For example to define methods for the account class we look at sequences diagram for the following use cases

- Deposit checking
- Deposit saving
- Withdraw checking
- Withdraw more from checking
- Withdraw saving
- Checking transaction history

7.10 Defining methods for via net bank objects

Operation in the object oriented system usually correspond to events or actions that transmit information in the sequences diagram or queries about attributes of the objects. In other words methods are responsible for managing the value of attributes such as query updating, reading, and writing

7.10.1 defining Account class operations

deposit and withdrawal operations are available to the client through the bank application but they are provided as services by the account class since the account objects must be able to manipulate their internal attributes here are the methods that we need to define for the account class.

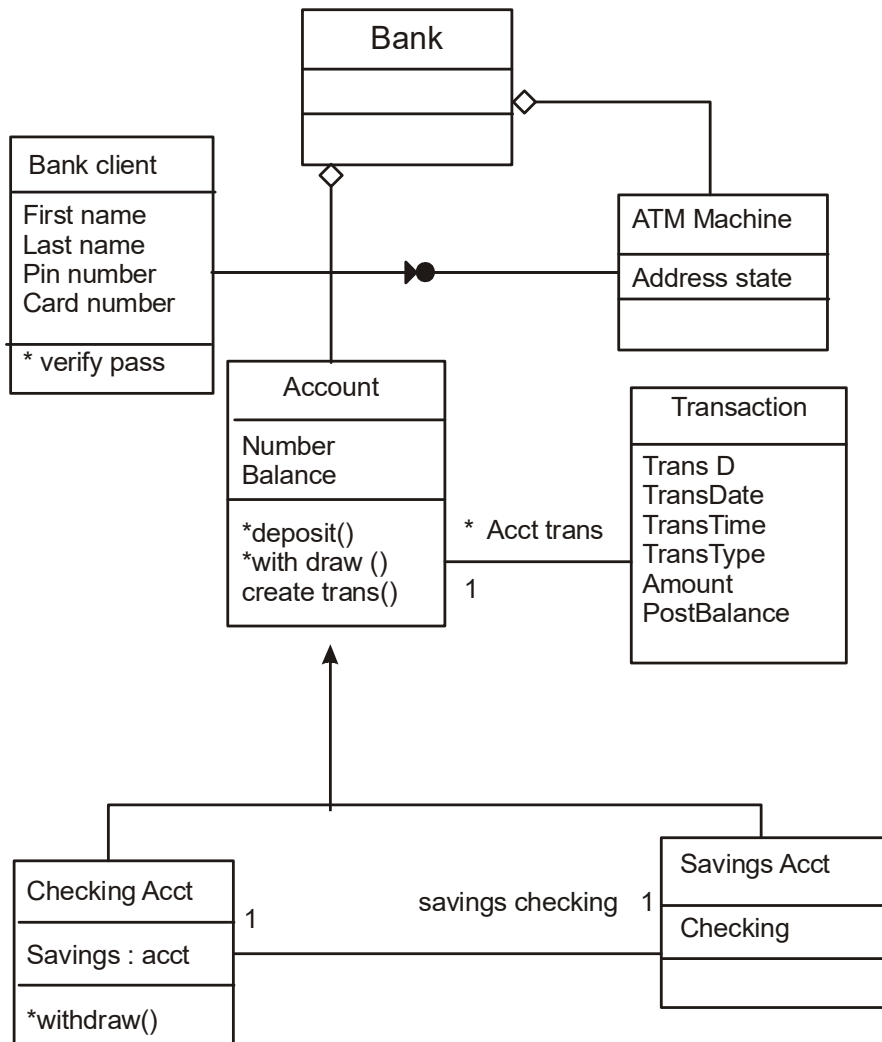


Figure 7.13

Deposit

Withdraw

Create transaction

7.10.2 Defining bank client class operation

Analyzing the sequences diagram in figure 6.4 it is apparent that the bank client funds to cover a withdrawal it must try to withdraw the insufficient amount from its related savings account. The withdrawal service appears in the checking class symbol see figure 7.13.

Summary

- Look at three types of relationship :association, super sub structure, and aggregation or a part of relations.
- Association is a relationship among classes.
- The hierarchical relation allows the sharing of properties or inheritance.
- To identify association begin by analyzing the interactions of each class and responsibilities for dependencies.
- To identify super sub relationship in the application, look for noun phrases composed of various adjectives in the class name in top down analysis
- The a part of relationship sometimes called aggregation, represents a situation where a class comprises several component classes.
- Identifying attributes and methods is like finding classes a difficult activity and an iterative process.
- Methods and methodologies are the workhorse of object oriented system.
- Methods are responsible for managing the value of attributes such as query, updating, reading, and writing

Question

1. What is association?
2. What is generalization?
3. Why is identifying class hierarchy important in object oriented analysis?
4. What are some common associations?
5. What are unnecessary association? How would you know?
6. How would you identify attributes?
7. How would you identify method?
8. What are unnecessary attributes?
9. What does repeating attributes indicate?

LESSON - 8

OBJECT ORIENTED AND DESIGN AXIOMS

Objectives

- The object oriented design process
- Object oriented design axioms corollaries
- Design pattern

Introduction

The objects discovered during analysis can serve as a framework for design. The class attributes method and association identified during analysis must be designed for implementation as a data type expressed in the implementation as a data type expressed in the implementation language. New classes must be introduced to store intermediate results during program execution. Emphasis shifts from the application domain to implementation and computer concepts such as user interfaces or view layer and access layer.

During the analysis, we look at the physical entities or business objects in the system that is who the players are and how they cooperate to do the work of the application. These objects represent tangible element of the business. This is where we begin thinking about how to actually implement the problem in a program. The goal here is to design the classes that we need to implement the system. Fortunately the design model does not look terribly different from the analysis model. The difference is that at this level, we focus on the view and access classes, such as how to maintain information or the best way to interact with user or present information.

It also is useful at this stage to have a good understanding of the classes in a development environment that we are using to enforce reusability. In software development, it is tempting not to be concerned with design. After all you are so involved with the system that it might be difficult to stop and think about the consequences of each design choice. However, the time spent on design has a great impact on the overall success of the software development project.

A large payoff is associated with creating a good design “up front”, before writing a single line of code. While this is true of all programming classes and objects underscore the approach even more. Good design usually simplifies the implementation and maintenance of a project.

In this lesson we look at the object oriented design process and axioms. The basic goal the axiomatic approach is to formalize the design process and assist in establishing a scientific foundation for the object oriented design process, to provide a fundamental basis for the creation of systems.

8.1 Design for object oriented systems

We introduced the concepts of the design pyramid for conventional so. Four design layers-data, architectural, interface, and proceduralwere each defined and discussed. For object oriented system we can also define a design pyramid, but the layer are a bit different. As shown in figure 8.1 the four layer of the object oriented design pyramid are:

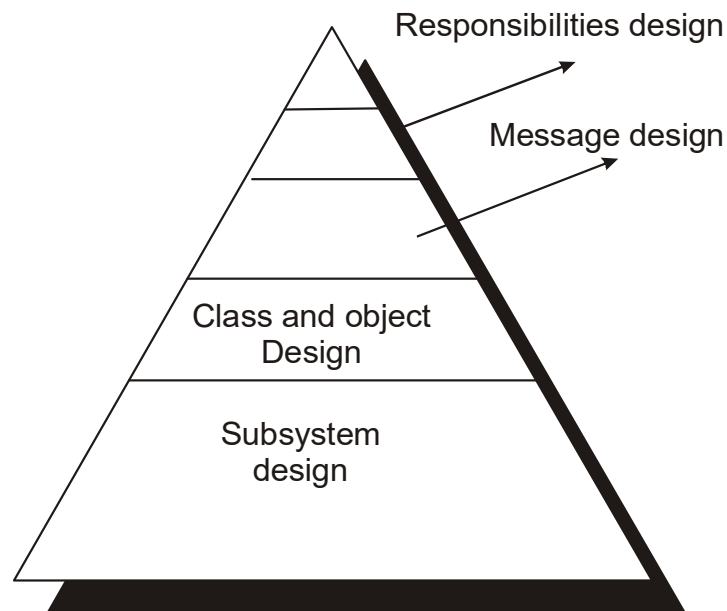


Figure 8.1

- The subsystem layer. Contains a representation of each of the subsystem that enable the software to achieve its customer defined requirements and to implement the technical infrastructure that support customer requirements.
- The class and object layer. Contains the class hierarchies that enable the system to be created using generalization and increasingly more targeted specifications. This layer also contains design representation of each object.

- The methodologies layer. Contains the details that enable each object to communicate with its collaborators. This layer establishes the external and internal interface for the system.
- The responsibilities layer. Contains the data structure and algorithmic design for all attributes and operations for each object.

8.2 The object oriented design Process

During the design phase the classes identified must be revisited with a shift in focus their implementation. New classes or attributes and methods must be added for implementation purpose and user interfaces. The object oriented design process consists of the following activities see figure 8.2.

1. Apply design axioms to design classes their attributes, method, association, structures and protocols.
 - 1.1 Refine and complete the static UML class diagram by adding detail to the UML class diagram. This step consists of the following activities.
 - 1.1.1 Refine attributes.
 - 1.1.2 Design methods and protocols by utilizing a UML activity diagram to represent the method's algorithm.
 - 1.1.3 Refine association between classes.
 - 1.1.4 Refine class hierarchy and design with inheritance.
 - 1.2 Iterate and refine again.
2. Design the access layer
 - 2.1 Create mirror classes. for every business class identified and created, create one access class. For example if there are three business classes (class 1, class2, class3), create three layer classes (class 1DB, class2DB, class3DB, class4DB).
 - 2.2 Identify access layer class relationships.
 - 2.3 Simplify classes and their relationship. The main goal here is to eliminate redundant classes and structure.
 - 2.3.1 Redundant classes: do not keep two classes that perform similar translate request and translate activities. Simply select one and eliminate the other.

- 2.3.2 Method classes: revisit the classes that consists of only one or two methods to see if they can be eliminated or combined with existing classes.
- 2.4 Iterate and refine again.
- 3. design the view layer classes
 - 3.1 design the macro level user interface, identifying view layer objects
 - 3.2 design the micro level user interface, which include these activities.
 - 3.2.1 design the view layer objects by applying the design axioms and corollaries.
 - 3.2.2 Build a prototype of the view layer interface
 - 3.3 test usability and user satisfaction
 - 3.4 Iterate and refine,
- 4. Iterate and refine the whole design. Reapply the design axioms and if needed repeat the preceding steps.

Utilizing an incremental approach such as the UA all stages of software development can be performed incrementally. Therefore all right decisions need not be made up front. From the UML class diagram you can begin to extrapolate which classes you will have build and which existing classes you can reuse. As you do this also begin thinking about the inheritance structure.

If you have several classes that seem related but have specific differences, you probably will want to make them common subclasses of an existing class or one that you define. As long as you think in terms of class of object learn what already is there and are willing to experiment you soon will feel comfortable with the process.

Design also must be traceable across requirements, analysis, design, code, and testing. There must be a clear step by step approach to the design from the requirements model. All the designed components must directly trace back to the user requirements. Usage scenarios can serve as test cases to be used during ay testing see figure 7.2

OO Design

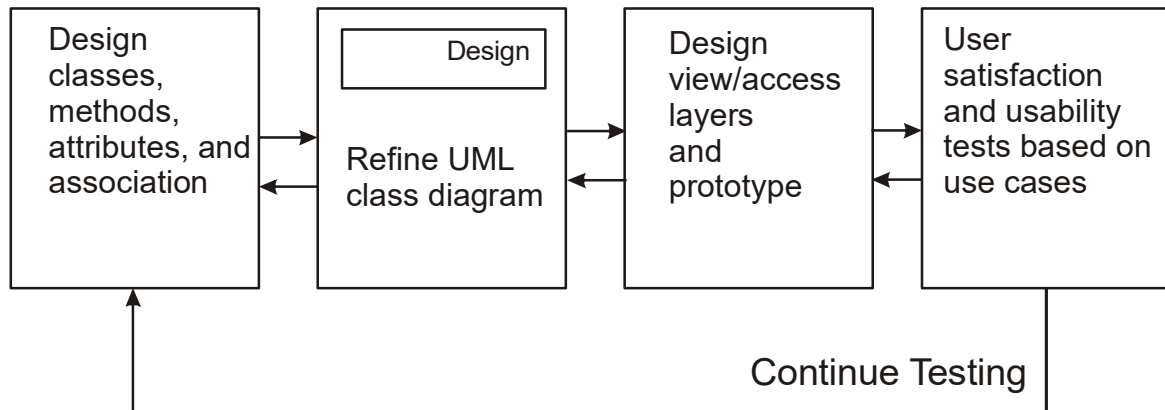


Figure 8.2

8.3 Object oriented design axioms

By definition an axiom is a fundamental truth that always is observed to be valid and for which there is no counter example or exception. Such explain that axioms may be hypothesized from a large number of observations by noting the common phenomena shared by all cases; they cannot be proven or derived, but they can be invalidated by counter example or exception.

A theorem is proposition that may not be self evident but can be proven from accepted axioms. If therefore is equivalent to a law or principle. Consequently, a theorem is valid if its referred axioms and deductive steps are valid. A corollary is a proposition that follow from an axiom or another proposition that has been proven. Again, a corollary is shown to be valid or not valid in the same manner as a theorem.

The author has applied suh's design axioms to object oriented design. Axiom 1 deals with relationship between system component and axiom 2 deals with the complexity of design.

- Axiom 1. The independence axiom. Maintain the independence of components.
- Axiom2. The information axiom. Minimize the information content of the design.

Axiom1 states that during the design process as we go from requirements and use cases to a system component each component must satisfy that requirements without affecting other requirements. To make this point clear let's take a look at an example offered by suh. You been

asked to design a refrigerator door, and there are two requirements. The door should provide access to food and the energy lost should be minimal when door is opened and closed. In other words opening the door should be independent of losing energy. Is the vertically hung door a good design? We see that vertically hung door violates axiom1

Axiom2 is concerned with simplicity. Scientific theoreticians often rely on a general rule known as occam's razor, after William of occam, a 14 th century scholastic philosopher. Briefly put occam's razor says that "the best theory explains the known facts with a minimum amount of complexity and maximum simplicity and straightforwardness". Occum's razor has a very useful implication in approaching the design of an object oriented application. Let use cases restate occam's razor rule of simplicity in object oriented terms.

8.4 corollaries

From the two design axioms many corollaries may be derived as a direct consequence of the axioms. These corollaries may be more useful in making specific design decisions, since they can be applied to actual situation more easily than the original axioms. They even may be called design rules, and all are derived from the two basic axioms see figure 8.3.

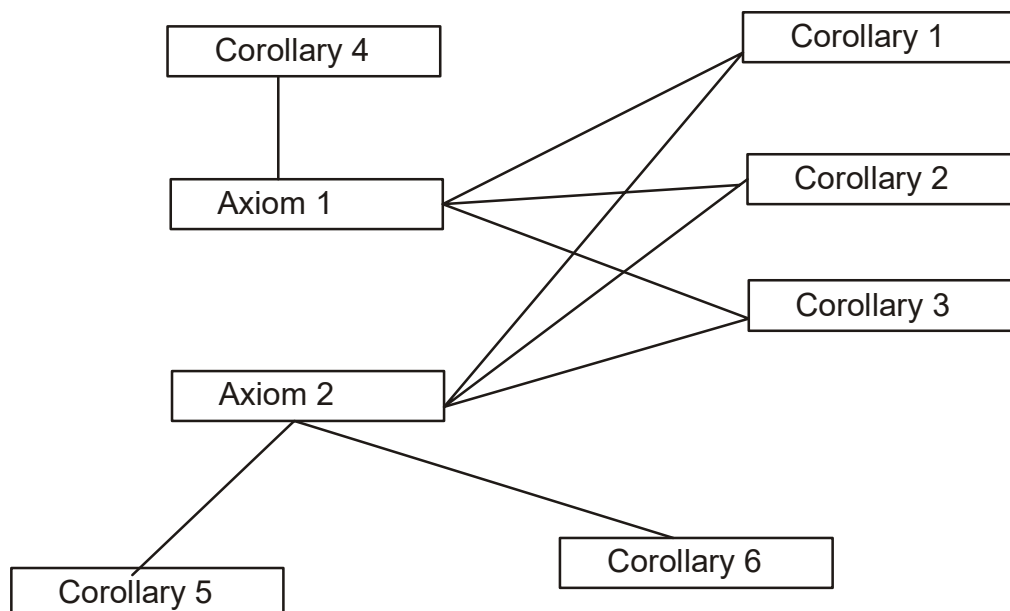


Figure 8.3

- Corollary 1 uncoupled design with less information content Highly cohesive objects can improve coupling because only a minimal amount of essential information need be passes between objects
- Corollary 2. single purpose. Each class must have a single, clearly defined purpose. When you document, you should be able to easily describe the purpose of a class in a few sentences.
- Corollary 3. large number of simple classes. Keeping the classes simple allows reusability.
- Corollary 4. strong mapping. There must be a strong association between the physical system and logical design.
- Corollary 5. standardization. Promote standardization by designing inter changeable components and reusing existing classes or components.
- Corollary 6. design with inheritance. Common behavior must be moved to super classes. The super classsubclass structure must make logical sense.

8.4.1 corollary 1. uncoupled design with less information content

The main goal here is to maximize objects cohesiveness among objects and software components in order to improve coupling because only a minimal amount of essential information need be passed between components.

8.4.1.1 coupling. Coupling is a measure of the strength of association established by a connection from one object or software components to another. Coupling is a binary relationship. A is coupled with B coupling is important when evaluating a design because it helps usa focus on an important issue in design. For example a change to one components of a system should have a minimal impact on other components. Strong coupling among objects complicates a system, since the class is harder to understand or highly interrelated with other classes. The degree of coupling is a function of

1. How complicated the connections is.
2. Whether the connection refers to the object itself or something inside it.
3. What is being sent or received.

The degree or strength of coupling between two components is measured by the amount and complexity of information transmitted between them. Cop increases with increasing complexity or obscurity of the interface. Coupling decreases when the connection is to the components interface rather than to an internal components. Coupling also is lower for data connections than for control connections. Object oriented design has two types is coupling: interaction coupling and inheritance coupling.

Interaction coupling involves the amount and complexity of message between components. It is desirable to have little interaction. Coupling also applies to the complexity of the message. The general guidelines is to keep the message as

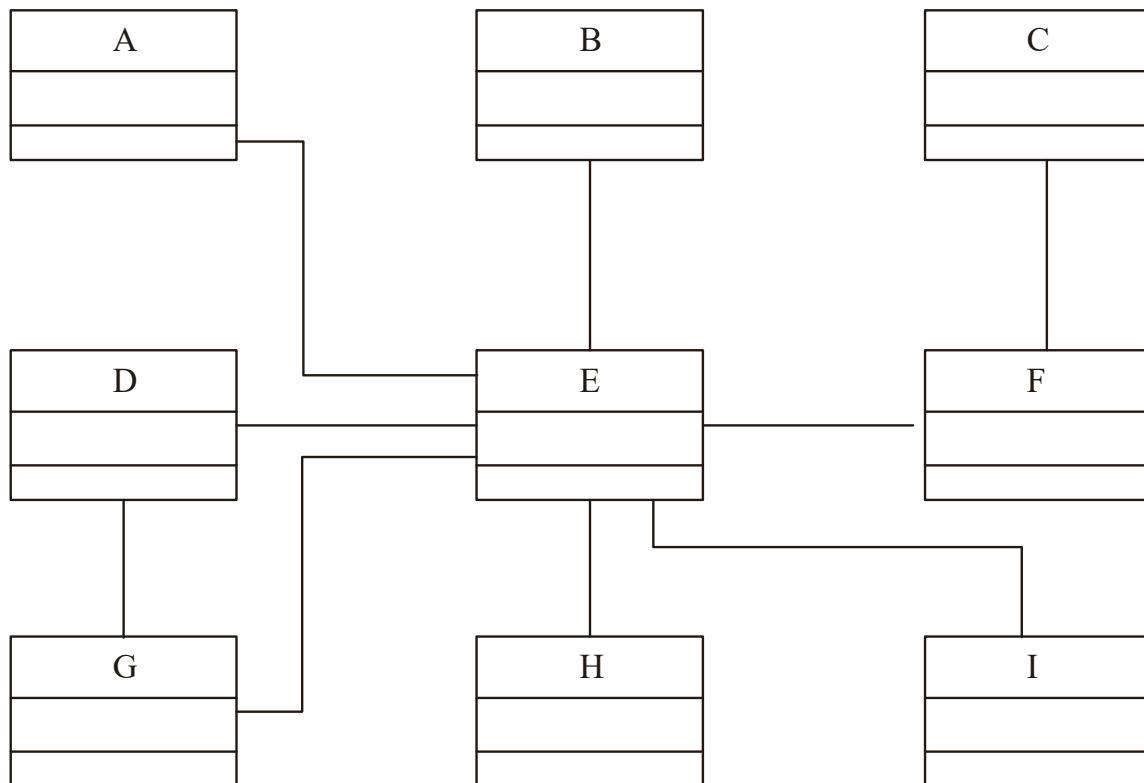


Figure 8.4

Simple and infrequent as possible. In general if a message connections involves more than three parameters the(X,Y,Z)examine it to see if it can be simplified. It has been documented that object connected to many very complex message are tightly coupling meaning any change to one invariability leads to a ripple effect of changes in other see figure 8.4

In addition to minimizing the complexity of message connections also reduce the number of message sent and received by an object. Table 8.1 contains different types of interaction couplings Inheritance is a form of coupling between super class and sun class. a sub class is coupled to its super class in terms of attributes and method. Unlike interaction coupling high inheritance coupling is desirable. However to achieve high inheritance coupling in a system, each specification class should not inherit lots of unrelated and unneeded methods and attributes. For example if the sub class is overwriting is low and the designer should look for an alternative generalization specification structure.

Table 8.1

Degree of coupling	Name	Description
Very high	Content coupling	The connection involves direct reference to attributes or method of another object
High	Common coupling	The connections involves two object accessing a “global data space” for both to read and write
Medium	Control coupling	The connections involves explicit control of the processing logic or one object by another
Low	Stamp coupling	The connections involves passing an aggregate data structure to another object, which uses only a portion of the components of the data structure
Very low	Data coupling	The connections involves either simple data items or aggregate structures all of whose elements are used by the receiving object. This should be the goal of an architectural design.

8.4.1.2 Cohesion

Coupling deals with interactions between objects or software components. We also need consider interaction within a single object or software components called cohesion. Cohesion

reflects the “single-purpose ness” of an object. Highly cohesive components can lower coupling because only a minimum of essential information need be passed between components. Cohesion also helps in designing classes that have very specific goals and clearly defined purpose.

Method cohesion like function cohesion means that a method should carry only one function. A method that carries multiple functions is undesirable. Class cohesion means that all the class’s methods and attributes must be highly cohesive meaning to be used by internal method or derived classes method. inheritance cohesion is concerned with following questions.

- How interrelated are the classes?
- Does specification really portray specification or is it just something arbitrary?

8.4.2 Corollary 2. Single purpose

Each class must have a purpose as was explained in lesson 6, every class should be clearly defined and necessary in the context of achieving the system goals. When you documents a class you should be able to easily explain its purpose in a sentence or two. If you cannot then rethink the class and try to subdivide it into more independent pieces. In summary keep it simple to be more precise each method must provide only one service. Each method should be of moderate size no more than page; half a page is better.

8.4.3 Corollary 3. Large Number of Simpler Classes, Reusability

A great benefit results from having a large number of simpler classes you cannot possibly foresee all the future scenarios in which the classes you create will be reused. The less specialized the classes are the more likely future problem can be solved by a recombination of existing classes adding a minimal number of sub classes.

A class that easily can be understood and reused contributes to the overall system, while a complex poorly designed class is just so much dead weight and usually cannot be reused. Keep the following guidelines in mind, object oriented design offers a path for producing libraries of reusable parts. The emphasis object oriented design places on encapsulation, modularization, and polymorphism suggests reuse rather than building anew. Cox’s description of a so IC library implies a similarity between object oriented development and building hardware from a standard set of chips. The software IC library is realized with introduction of design pattern discussed later in this lesson.

Coad and yourdon argue that software reusability rarely is practice effectively. But the organization that will survive in the 21s century will be those that have achieved high levels of reusabilityanywhere from 70-80 percent or more. These assets consistently are used and maintained to obtain high levels of reuse thereby optimizing the organization ability to produce high quality software products rapidly and effectively. Coad and yourdon describe four reasons why people are not utilizing this concept.

- Software engineering textbooks teach new practitioners to build system from “first principles”, reusability is not promoted or even discussed.
- The “not invented here” syndrome and the intellectual challenge of solving an interesting software problem in ones own unique way mitigates against reusing someone else’s software component.
- Unsuccessful experiences with software reusability in the past have convinced many practitioners and development managers that the concept is not practical.
- Most organization provide no reward for reusability, some times productivity is measured in terms of new lines of code written plus a discounted credit for reused lines of code.
- The primary benefit of software reusability is higher productivity. An other form of reusability is using a design pattern which will be explained in the next section.

8.4.4 corollary 4. strong Mapping

Object oriented analysis and object oriented design are based on the same model. As the model progresses from analysis to implementation more detail is added, but it remains essentially the same. For example during analysis we might identify a class employee. During the design phase, we need to design this class design its methods its association with other objects and its view and access classes. A strong mapping links classes identified during analysis and classes designed during the design phase. Asrtin and odell describe this important issue very elegantly. with OO techniques, the same paradigm is used for analysis, design, and implementation.

The analyst identifies objects types and inheritance and thinks about events that change the state of objects. The designer adds detail to this model perhaps designing screens, user interaction and client server interaction. The thought process flows so naturally from analyst to design that it may be difficult to tell where analysis ends and design begins.

8.4.5 corollary 5. Standardization

To reuse classes you must have a good understanding of the classes in the object oriented programming environment you are using. Most object oriented systems, such as smalltalk, java, or powerbulider come with several built in class libraries. Similarly object oriented system are like organic system meaning that they grow as you create new application. The knowledge of existing classes will help you determine what new classes are needed to accomplish the tasks and where you might inherit useful behavior rather than reinvent the wheel. of aqua

However class libraries are not always well documented or worse yet they are documented but not up to date. Furthermore class libraries must be easily searched based on users criteria. For example users should be able to search the class repository might provide a way to capture the design knowledge document it and store it in a repository that can be shared and reused in different applications.

8.4.6 Corollary 6. Designing with inheritance

When you implement a class you have to determine its ancestor what attributes it will have and what message it will understand. Then you have to construct its methods and protocols. Ideally you will choose inheritance to minimize the amount of program instructions. Satisfying these constraints sometimes means that a class inherits from a super class that may not be obvious at first glance.

For example say you are developing an application for the government that manages the licensing procedure for a variety of regulated entities. To simplify the example focus on just two types of entities motor vehicles and restaurants. Therefore identifying classes is straightforward. All goes well as you begin to model these two portions of class hierarchy. Assuming that the system has no existing classes similar to restaurant or a motor vehicle, you develop two classes, motor vehicle and restaurant.

Sub classes of the motor vehicle class are private vehicle and commercial vehicle. These are further sub divided into whatever level of specificity seems appropriate see figure 8.5. sub classes of restaurant are designed to reflect their own licensing procedures. This is a simple easy to understand design, although some what limited in the reusability of the classes. For example if in another project you must build a system that models a vehicle assembly plant the classes from the licensing application are not appropriate, since these classes have instructions and data that deal with the legal requirements of motor vehicle license acquisition and renewal.

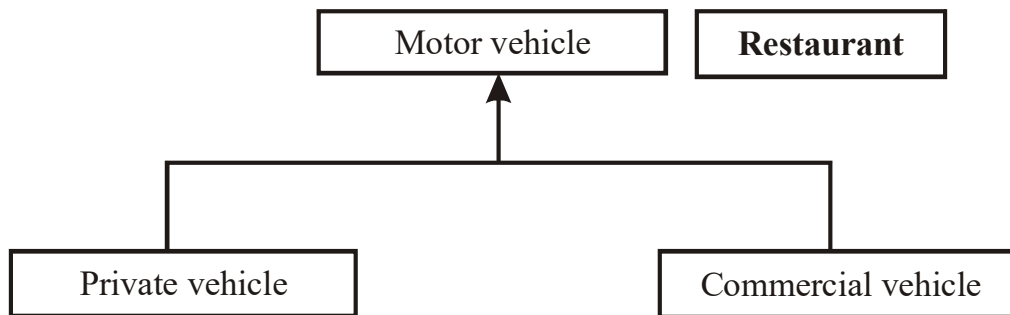


Figure 8.5

In any case the design is approved implementation is accomplished and the system goes into production. Now here comes the event that every designer both knows well and dreads when the nature of the real world problem exceeds the bounds of the system so far an elegant design. Say six month later while discussing some enhancements to the system with the right people, one of them says, “what about coffee wagons food trucks, and ice cream vendors? We’re presents planning on licensing them as both restaurants and motor vehicles.”

You know you need to redesign the application but redesign how? The answer depends greatly on the inheritance mechanism supported by the system’s target language. If the language support single inheritance exclusively the choices are somewhat limited. You can choose to define a formal super class to both motor vehicle and restaurant, license, and move common methods and attributes from both classes into this license class see figure 8.6.

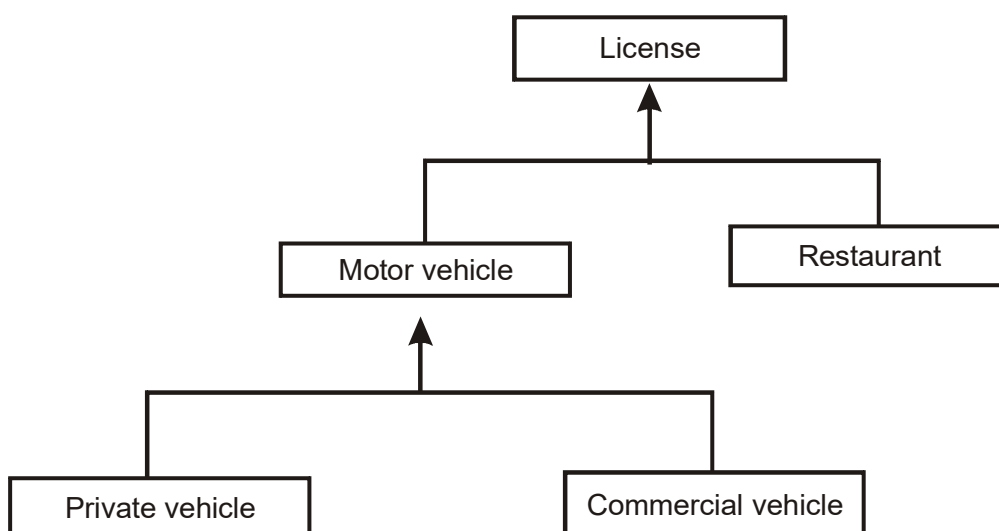


Figure 8.6

This necessitates a very weak formal class or numerous blocking behavior in both motor vehicle and restaurant. This particular decision results in the least reusable classes and potentially extra code in several locations. So let use cases try another approach. Alternatively you could preserve the original formal classes, motor vehicle and restaurant. Next define a food truck class to descend from commercial vehicle and copy enough behavior into it from the restaurant class to support the app requirements see figure 8.7.

You can give food truck copies of data and instructions from restaurant class that allow it to report on food type, health code categories number of chefs and support staff and like.

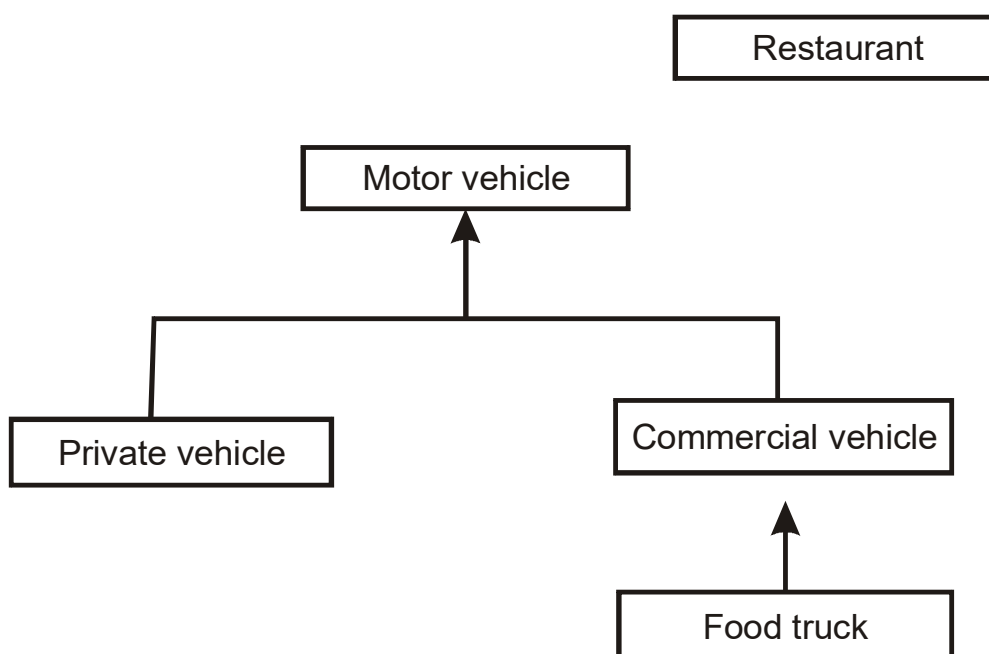


Figure 8.7

The class is not very reusable, but at least its extra code is localized, allowing simpler debugging and enhancement. Coad and yourdon describe cut and paste type reusability as follows. If on the other hand the intended language supports multiple inheritance another route can be taken, one that more closely models the real world situation. In this case you design a specialized class. Food truck and specify dual ancestry.

Our new class alternative seems to preserve the integrity and code bulk of both ancestors and does nothing that appears to affect their reusability. In actuality since we never anticipated

this problem in the original design there probably are instance variables and methods in both ancestors that share the same names. Most language that support multiple inheritance handle these “hits” by giving precedence to the first ancestor defined. Using this mechanism reworking will be required in the food truck descendant and quite possibly in both ancestors see figure 8.8. it easily can become difficult to determine which method in which class, affected an erroneously updated variable in an instance of a new descendant. The difficulties in maintaining such a design increase geometrically with the number of ancestors assigned to given class.

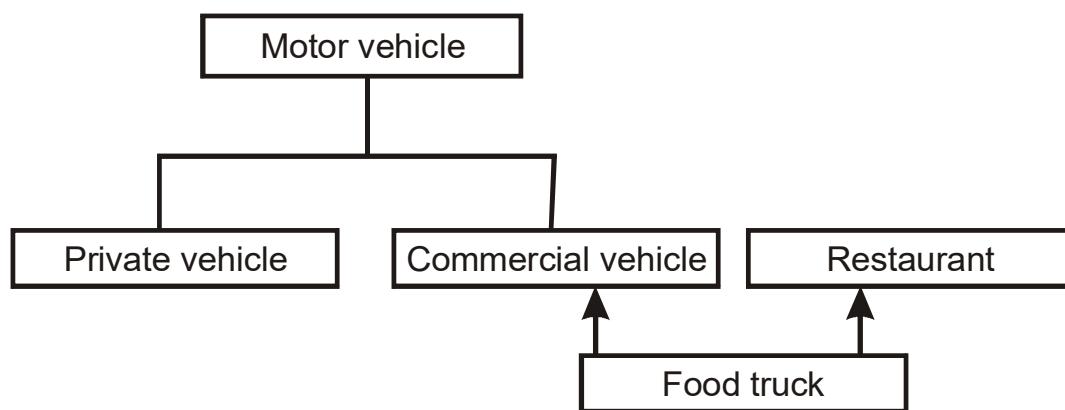


Figure 8.8

8.4.6.1 Achieving Multiple Inheritance in a Single Inheritance System

Single inheritance means that each class has only a single super class. This technique is used in smalltalk and several other object oriented systems. One result of using a single inheritance hierarchy is the absence of ambiguity as to how an object will responds to given method, you simply trace up the class tree beginning with the object’s class, looking for a method of the same name.

However language like Lisp have a multiple inheritance scheme whereby objects can inherit behavior from unrelated areas of the class tree. This could be desirable when you want a new class to behave similar to more than one existing class. however multiple behavior to get from which class particularly when several ancestors define the same method.

It also is more difficult to understand programs written in multiple inheritance system. one way of achieving the benefits of multiple inheritance in a language with single inheritance is to inherit from the most appropriate class and add an object of another class as an attributes or aggregation. Therefore as class designer you have two ways to borrow existing functionality in

a class. One is to inherit it and the other is to use the instance of the class as an attribute. This approach is described in the next section.

8.4.6.2 Avoiding Inheritance Inappropriate Behavior

Beginners in an object oriented system frequently error by designing sub classes that inherit from inappropriate super classes. Before a class inherits ask the following questions.

- Is the sub class fundamentally similar to its super class?
- l Is it an entirely new thing that simply wants to borrow some expertise from its superclass?

A better approach would be to inherit only from commercial vehicle and have an attributes of the type restaurant of restaurant class. In other words restaurant class becomes a part of food track class see figure 8.9

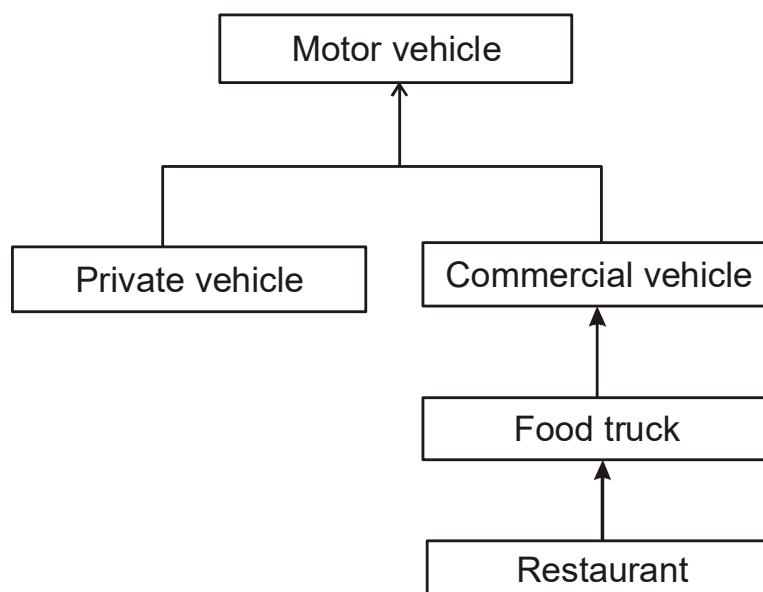


Figure 8.9

8.5 Design Pattern

A design pattern provides a scheme for refining the sub systems or components of a software system or the relationship among them. In other words design patterns are devices that allow system to share knowledge about their design by describing commonly recurring structures of communicating components that solve a general design problem within a particular

context. Essays usually are written by following a fairly well defined form and so is documenting design patterns. Let use cases take a look at a design pattern example created by kurotsuchi.

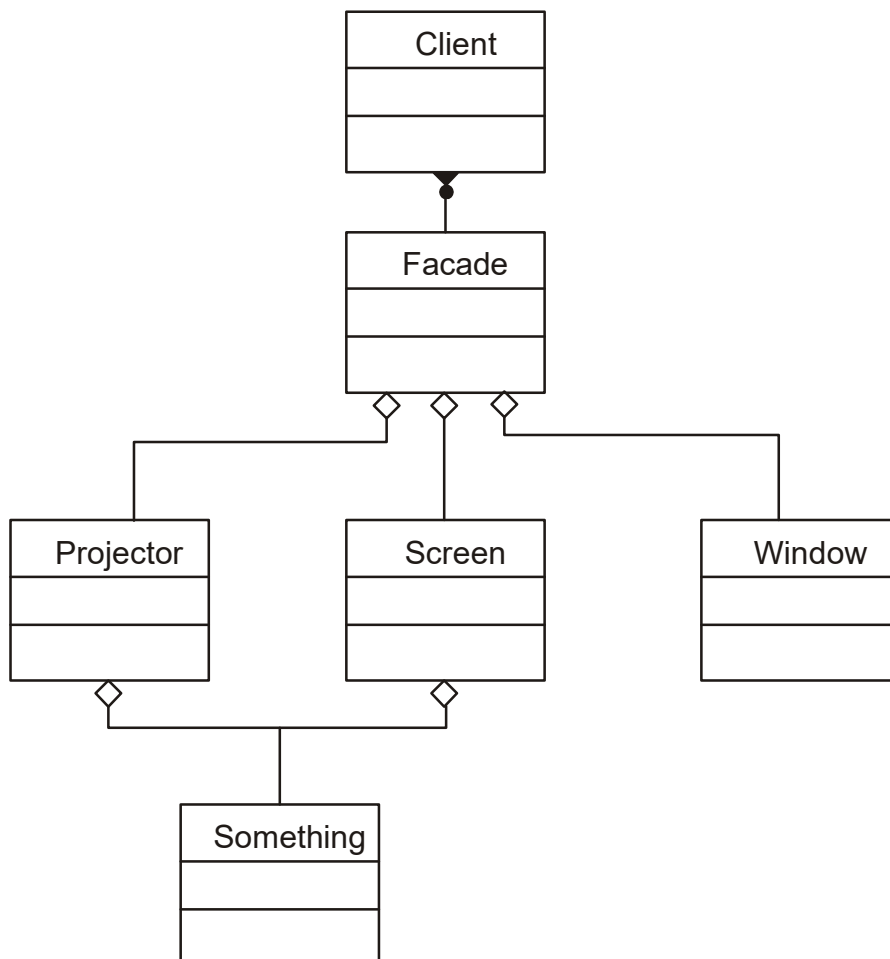


Figure 8.10

- Pattern name. Façade
- Rationale and motivation, the façade pattern can make the task of accessing a large number of modules much simpler by providing an additional interface layer.
- Classes there can be any number of classes involved in this “façade” system but at least four or more classes are required. one client the façade and the classes underneath the façade.

- **Advantages /Disadvantages.** As stated before the primary advantages to using the façade is to make the interfacing between many modules or classes more manageable. One possible disadvantage to this pattern is that you may lose some functionality contained in the lower level of classes, but this depends on how the façade was designed.
- **Example.** Imagine that you need to write a program that needs represent a building in the rooms that can be manipulatedmanipulated as in interacting with objects in the room to change their state. The client that ordered this program has determined that there will be a need for only a finite number of objects.
- A sample action for a room is to “prepare it for presentation.” you have decided that this will be part of your façade interface since it deals with a large number of classes but does not really need to bother the programmer with interacting with each of them when a room needs to be prepared. Here is how that façade might be organized see figure 8.10. consider the sheer simplicity from the client’s side of the problem. A less thought out design have looked like this making lots of interaction by the client necessary see figure 8.11.

Summary

- We looked at the object oriented design process and design axioms.
- Integrating design axioms and corollaries with incremental and evolutionary style of software development will provide you a powerful way for designing systems.
- The object oriented design process consists of
 1. designing classes and applying design axioms. If needed, this step is repeated
 - 2 Designing the access layer.
 3. designing the user interface.
 4. Testing user satisfaction and usability based on the usage and use cases.

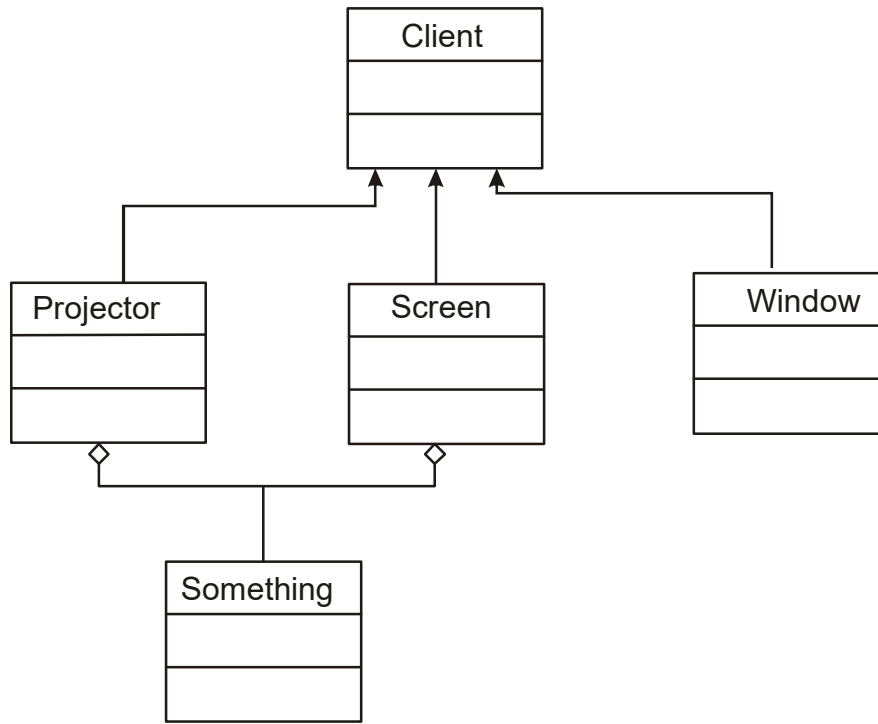


Figure 8.11

5. Iterating and refining the design

1. The two design axioms are

1. Axiom 1. The independence axiom. Maintain the independence of components.
2. Axiom 2. The information axiom. Minimize the information content of the design.

2. Six design corollaries are

1. Corollary 1 uncoupled design with less information content. Highly cohesive objects can improve coupling because only a minimal amount of essential information is passed between objects
2. Corollary 2. single purpose. Each class must have a single, clearly defined purpose. When you document, you should be able to easily describe the purpose of a class in a few sentences.
3. Corollary 3. large number of simple classes. Keeping the classes simple allows reusability.

4. Corollary 4. strong mapping. There must be a strong association between the physical system and logical design.
5. Corollary 5. standardization. Promote standardization by designing interchangeable components and reusing existing classes or components.
6. Corollary 6. design with inheritance. Common behavior must be moved to super classes. The super classsubclass structure must make logical sense.

Questions:

1. What is the significance of Occam's razor?
2. How does occam's differentiate good design from bad design?
3. What is the basic activity in designing an application?
4. What is the significance of being able to describe in a few sentences what a class does?
5. How can an object oriented system be thought of as an organic system?
6. Why are people not utilizing reusability? List some reasons?
7. What are the challenges in designing with inheritance?
8. Describe single and multiple inheritance.
9. What are the risks of a cut and paste type of reusability?
10. How can you achieve multiple inheritance in a single inheritance system?
11. How can you avoid a sub class inheriting inappropriate behavior?
12. List the object oriented design axioms and corollaries.

LESSON - 9

DESIGNING CLASSES

Objectives:

- Designing classes.
- The UML protocol and class visibility.
- The UML object constraint language (OCL)
- Designing methods.

Introduction

Object oriented design requires taking the objects identified during object oriented analysis and designing classes to represent them. As a class designer you have to know the specifics of the class you are determining and be aware of how that class interacts with other classes.

Once you have identified your classes and their interactions, you are ready to design classes. Underlying the functionality of any application is the quality of its design. In this lesson, we look at guidelines and approaches to use in designing classes and their methods.

Although the design concepts to be discussed in this lesson are general we will concentrate on designing the business classes. The access and view layer classes will be described in the subsequent chapters. However the same concepts will apply to designing access and view layer classes.

9.1 The Object Oriented Design Philosophy

Object oriented development requires that you think in terms of classes. A great benefit of the object oriented approach is that classes organize related properties into units that stand on their own. We go through a similar process as we learn about the world around use cases. As new facts are acquired, we relate them to existing structure in our environment.

After enough new facts are acquired about a certain area, we create new structures to accommodate the greater level of detail in our knowledge. The single most important activity in designing an application is coming up with set of classes that work together to provide the functionality you desire. A given problem always has many solutions. However at this stage you

must translate the attributes and operations into system implementation. You need to decide where in the class tree your new classes will go.

Many object oriented programming language and development environment such as smalltalk or PB, come with several built in class libraries. Your goal in using these system should be to reuse rather than create anew. Similarly if you design your classes with reusability in mind, you will gain a lot in productivity and reduce the time for developing new application. The first step in building an application, therefore should be to design a set of classes each of which has a specific expertise and all of which can work together in useful ways. Think of an object oriented system as an organic system one that evolves as you create each new application.

Applying design axioms and carefully designed classes can have a synergistic effect not only on the current system but on its future evolution. If you exercise some discipline as you processed you will begin to see some extraordinary gains in your productivity compared to a conventional approach.

9.2 UML Object Constraints Language

We learned that the UML is a graphical language with set of rules and semantics. The rules and semantics of the UML are expressed in English, in a form known as object constraints language. **Object Constraints Language (OCL)** is a specification language that uses simple logic for specifying the properties of a system. Many UML modeling construct require expression.

For example there are expressions for types, Boolean values and numbers. Expressions are stated as strings in object constraints language. The syntax for some common navigational expressions is shown here. These forms can be chained together. The leftmost element must be an expression for an object or a set of objects. The expressions are meant to work on sets of values when applicable.

- Item. selector, the selector is the name of an attribute in the item. The result is the value of the attribute. For example, Kumar. age.
- Item selector[qualifier-value]. The selector indicates a qualified association that qualifies the item. The result is the related object selected by the qualifier, for example array indexing as a form of qualification; for example Kumar phone. assuming Kumar has several phones.
- Set->select. The Boolean expression is written in terms of object within set. The

result is the subset of objects in the set for which the Boolean expression is true; for example company employee-> salary>30000. this represents employee with salaries over \$30,000.

Other expressions will be covered as we study their appropriate UML notation.

9.3 Designing Classes: The Process

we looked at the object oriented design process. In this section we concentrate on step 1 of process which consists of the following activities:

1. Apply design axioms to design classes, their attributes, methods, association, structure, and protocols.
 - 1.1 Refine and complete the static UML class diagram by adding details to that diagram.
 - 1.1.1 Refine attributes.
 - 1.1.2 Design methods and the protocols by utilizing a UML activity diagram to represent the method's algorithm.
 - 1.1.3 Refine the association between classes.
 - 1.1.4 Refine the class hierarchy and design with inheritance.
 - 1.2 Iterate and refine.

Object oriented design is an iterative process. After all design is as much about discovery as construction. Do not be afraid to change your class diagram as you gain experience and do not be afraid to change it a second, third or fourth time.

9.4 Class visibility: Designing well Defined Public, Private and Protected Protocol.

In designing method or attributes for classes you are confronted with two problems. One is the protocols or interface to the class operations and its visibility and the other is how it is implemented. Often the two have very little to do with each other.

For example you might have a class Bag for collecting various object that counts multiple occurrences of its elements. One implementation decision might be that the Bag class uses another class, say, dictionary to actually hold its elements. The class's protocols or the message

that a class understands, on other hand can be hidden from other objects or made available to other objects. Public protocols define the implementation of an object see figure 9.1. it is important object oriented design the public protocols between the associated classes in the application.

This is a set of message that a class of a certain type must understand although the interpretation and implementation of each message is up to the individual class. A class also might have a set of method that it uses only internally, message to itself. This the private protocol of the class include message that normally should not be sent from other objects; it is accessible only to operation of that class. In private protocol only the class itself can use the method.

The public protocol define the stated behavior of the class as a citizen in a population and important information for users as well as future descendants so it is accessible to all classes. If the method or attributes can be used by the class itself or its subclasses, a protected protocols can be used. In a protected protocol sub classes the can the model in addition to the class itself. Lack of a well designed protocols can manifest itself as encapsulation leakage. The problem of encapsulation leakage occurs when details about a class internal implementation are disclosed through the interface.

As more internal details become visible the flexibility to make changes in the future decreases. If an implementation is completely open almost no flexibility is retained for future changes. It is fine revel implementation when that is intentional, necessary, and carefully controlled. However do not make such a decision lightly because Othat could impact the flexibility and therefore the quality of the design.

For example public or protected method that can access private attributes can reveal an important aspect of your implementation. If anyone uses these functions and you change their location the type of attributes or the protocols of the method this could make the client application inoperable. Design the interface between a super class and its sub classes just as carefully as the class's interface to clients this is the contract between the super and sub classes.

If this interface is not designed properly it can lead to violating the encapsulation of the sub classes. This feature is helpful but cannot express the totality of the relationship between a class and its sub classes. other important factors include which functions might or might not be overridden and how they must behave. It also is crucial to consider the relationship among method. Some methods might need to overridden in groups to preserve the class's semantics.

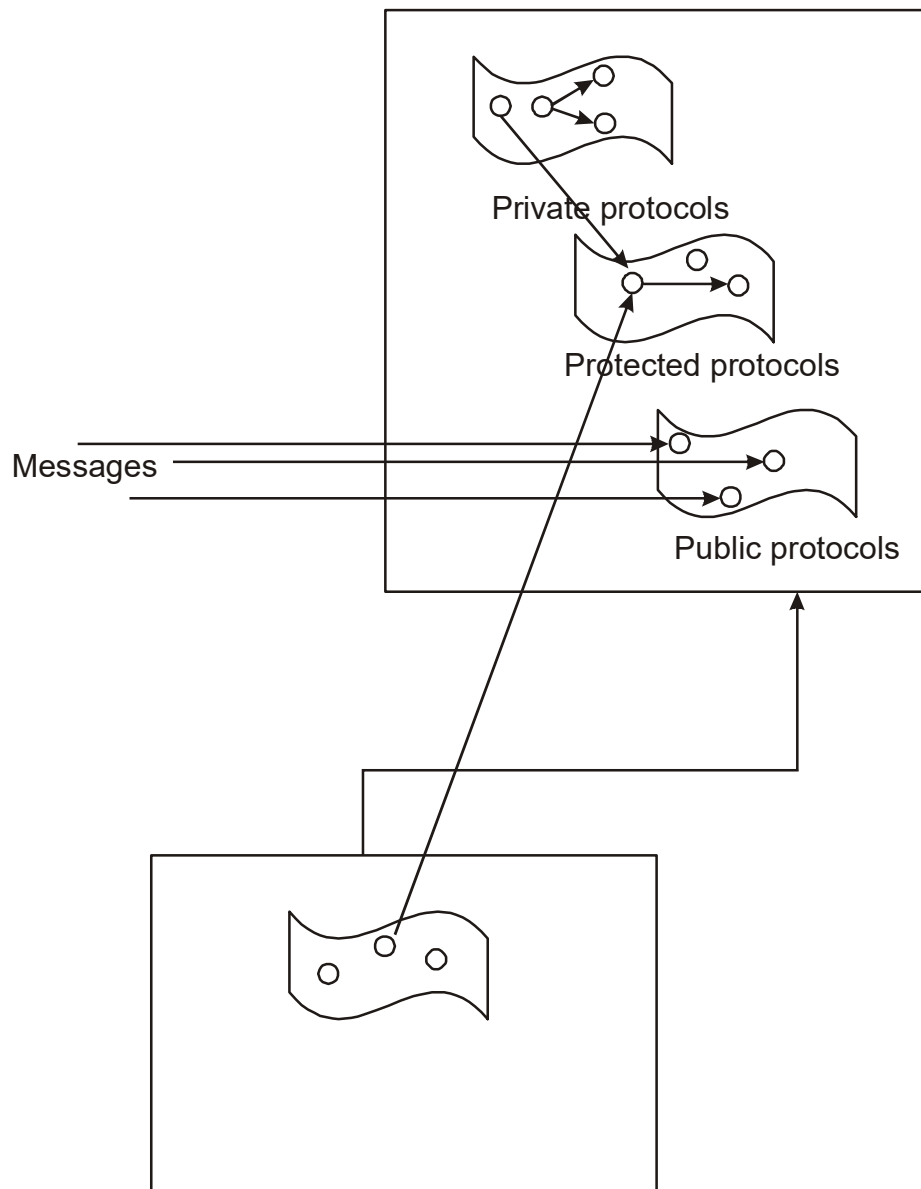


Figure 9.1

The bottom line is this: design your interface to sub classes so that a sub class that uses every supported aspect of that interface does not compromise the integrity of the public interface.

9.4.1 Private and protected Protocol layers :: Internal

Items in these layers define the implementation of the object. Apply the design axioms and corollaries, especially corollary 1 to decide what should be private: what attributes ?what methods? remember, highly cohesive objects can improve coupling because only a minimal amount of essential information need be passed between objects.

9.4.2 public Protocols Layer: External

Items in this layer define the functionality of the object. Here are some things to keep in mind when designing class protocols.

- Good design allow for polymorphism
- Not all protocols should be public, again apply design axioms and corollaries

The following key questions must be answered.

- What are the class interface and protocols ?
- What public protocol will be used or what external message must the system understand?
- What private or protected protocol will be used or what internal message or methodologies from a sub classes must the system understand?

9.5 Designing Classes: Refining Attributes

Attributes identified in object oriented analysis must be refined with an eye on implementation during this phase. In the analysis phase the name of the attributes was sufficient. However in the design phase, detailed information must be added to the model. The main goal of this activity is to refine existing attributes or add attributes that can elevate the system into implementation.

9.5.1 Attributes types

The three basic types of attributes are

1. single value attributes.
2. multiplicity or multivalued attributes.
3. reference to another object, or instance connection.

Attributes represent the state of an object. When the state of the object changes, these changes are reflected in the value of attributes. The single value attributes is the most common attribute type. It has only one value or state. for example attributes such as name, address or salary are of the single value type. The multiplicity or multivalued attribute is the opposite of the single value attributes since as its name implies, it can have a collection of many values at any point in time. For example if we want to keep track of the names of people who have called a customer support line for help we must use the multivalued attributes.

Instance connection attributes are required provide the mapping needed by an object to fulfill its responsibility in other words instance connection model association. For example a person might have one or more bank accounts. A person has zero to many instance connection to Account. Similarly, an account can be assigned to one or more persons. There fore an account also has zero to many instance connections to persons.

9.5.2 UML Attributes Presentation

OCLE can be used during the design phase to define class attributes. The following is the attributes presentation suggested by UML.

Visibility: type-expression = initial-value.

Where visibility is one of the following

+ public visibility

protected visibility

- provide visibility

Type expression is language dependent expression for the initial value of a newly type of an attributes. Initial value is a language dependent expression for the initial value of a newly created object. The initial value is optional. for example_size: length = 100. The UML style guidelines recommend beginning attributes names with a lower case letter.

In the absence of a multiplicity indicator an attribute holds exactly one value. Multiplicity may be indicated by placing a multiplicity indicator in brackets after attributes name; for example,

Names[10]: string

Point[2..*] point

The multiplicity of 0..1 provide the possibility of null values the absence of a value as opposed to a particular value from the range. For example the following declaration permits a distinction between the null value and an empty string names[0..1] string.

9.6 Refining Attributes for The Via Net bank Objects

In this section we go through the via net bank ATM system classes and refine attributes identified during object oriented analysis

9.6.1 Refining Attributes for the Bank client Class

During object oriented analysis we identified the following attributes.

Firstname

Lastname

Pinnumber

Cardnumber

At this stage we need to add more information to these attributes such as visibility and implementation type. Furthermore additional attributes can be identified during this phase to enable implementation of the class

#Firstname: string

#Lastname: string

#Pinnumber: string

#Cardnumber: string

#account: Account

we identified an association between the bank client and the account classes. To design this association we need to add an account attributes of type account, since the bank client needs to know about his or her account and this attributes can provide such information for the bank client class. This is an example of instance connection, where it represent the association between the bank client and the account and the account objects. All the attributes have been given protected visibility.

9.6.2 Refining Attributes for the Account Class

Here is refined list of attributes for the account class:

```
#number: string
#balance: float
#transaction: transaction
#bank client bank client.
```

At this point we must make the account class very general so that it can be reused by the checking and saving account.

9.6.3 Refining Attributes for the Transaction Class

The Attributes for the transaction class are these:

```
#transID :String
#transDate :date
#trans Time time
#trans Type:String
#amount:float
#postBalance: float
```

9.6.4 Refining Attributes for the ATM Machine Class

The ATM Machine class could have the following attributes

```
#address: String
#state: String
```

9.6.5 Refining Attributes for the Checking Account Class

Add the savings attributes to class. The purpose of this attributes is to implementation the association between the checking account and saving account classes.

9.6.6 Refining Attributes for the Saving Account Class

Add the checking attributes to the class. The purpose of this attributes is to implement the association between the savings account and checking account classes. Figure 9.2 shows a more complete UML class diagram for the bank system. At this stage we also need to add a very short description of each attributes or certain attributes constraints. For example

Class ATM Machine

#address:string

#state: string

9.7 Designing Methods and Protocols

The main goal of this activity is to specify the algorithm for methods identified so far. Once you have designed your methods in some formal structure such as UML activity diagram with an OCL description, they can be converted to programmer language manually or in automated fashion. A class provide several types of methods:

- constructor. Method that create instances of the class.
- Destructor. The method that destroys instances.
- Conversion method. The method that convert a value from one unit of measure to another.
- Copy method. The method that copies the content of one instance to another instance.
- Attribute set. The method that sets the values of one or more attributes.
- Attribute get. The method that returns the values of one or more attributes.
- VO methods. The method that provide or receive data to or from a device.
- Domain specific. The method specific to the application

Corollary 1 that in designing method and protocols you must minimize the complexity of message connections and keep as low as possible the number of message sent and received by an object Your goal should be maximize cohesiveness among object and software components to improve coupling because only a minimal amount of essential information should be passed between components. Abstraction leads to simplicity and straight forwardness and at the same time, increases class versatility. Seems to lead to increased utility. Here are five rules.

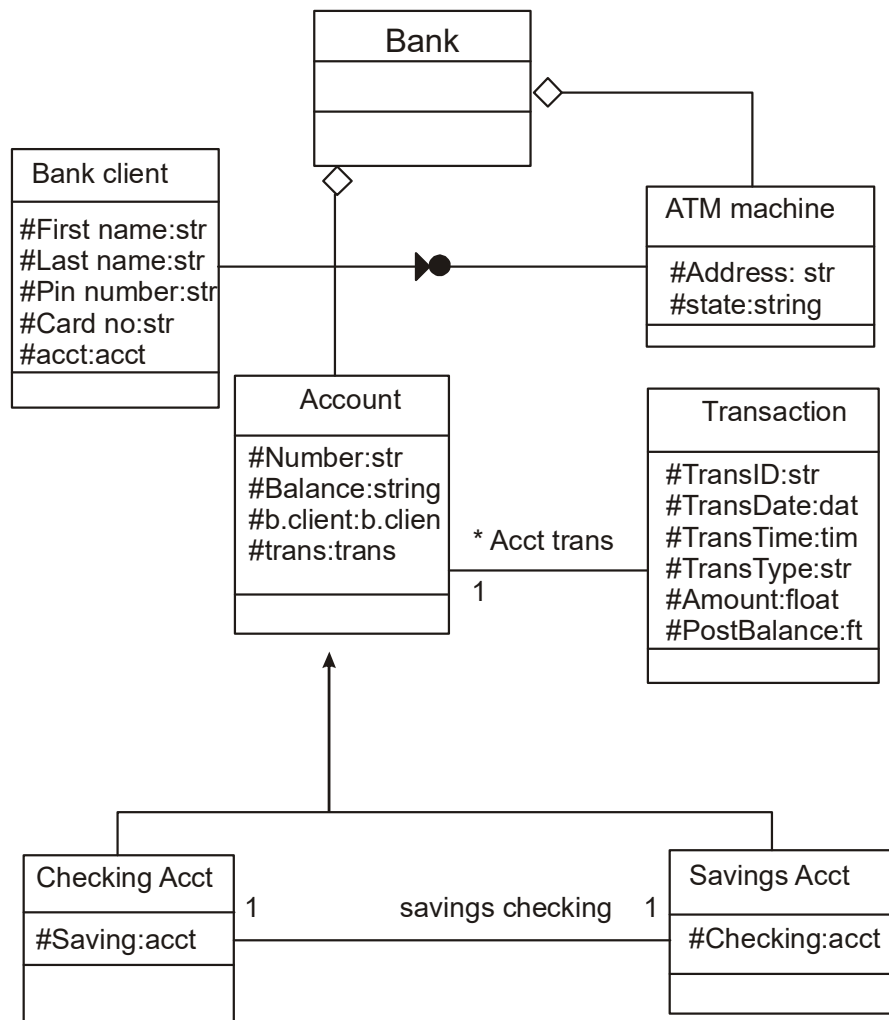


Figure 9.2

1. If it looks messy, then it's probably a bad design.
2. If it is too complex, then it's probably a bad design.
3. If it is too big, then it's probably a bad design.
4. If people don't like it, then it's probably a bad design.
5. If it is doesn't work, then it's probably a bad design.

9.7.1 Design Issues :avoiding Design Pitfalls

It is important to apply design axioms to avoid common design problems and pitfalls. For example we learned that it is much better to have a large set of simple classes than a few large complex classes. A common occurrence is that in your first attempt, your first attempt, your class might be too big and therefore more complex than it needs to be. You may find you can gather common pieces of expertise from several classes, which in itself becomes another “peer” class that the other consult or you might be able to create a super class for several classes that gather in a single place very similar code. Your goal should be maximum reuse of what you have to avoid creating new classes as much as possible. Take the time to think in this way good news, this gets easier over time. Lost object focus is another problem with class definitions. A meaningful class definition start out simple and clean but as time goes on add changes are made, becomes larger and larger, with the class but identify becoming harder to state concisely. This happens when keep making incremental changes adds a tweak to its description.

When the next problem comes up another tweak is added. Or when a new feature is requested another tweak is added and so on. These problem can be detected early on. Here are some of the warning signs that something is going amiss. There are bugs because the internal state of an object is too hard to track and solutions consists of adding patches. Patches are characterized by code that looks like this “if this is the case then force that to be true” or “do this just in case we need to” or “do this before calling function, because it expects this”

Some possible actions to sole this problem are these:

- Keep a careful eye on the class design and make sure that an object’s role remains well defined. If an object loses focus you need to modify the design. Apply corollary 2.
- Move some functions into new classes that the object would use. Apply corollary 1.
- Break up the class into two or more classes. Apply corollary 3.
- Rethink the class definition based on experience gained.

9.7.2 UML Operation Presentation

The following operation presentation has been suggested by the UML. The operation syntax is this:

Visibility name Sparameter list) return type-expression.

Where visibility is one of the following

+public visibility.

#protected visibility

private visibility.

Here name is the name of the operation.

Parameter-list is a list of parameters separated by commas each specified by name :type
expression = default value.

Return type expression: is a language dependent specification of the implementation of the value returned by the method. If return type is omitted the operation does not return a value for example

+getName():aName

+getAccountNumber(account: type) account number

the UML guidelines recommend beginning operation names with a lowercase letter.

9.8 Designing Methods for the Via Net Bank Objects

At this point the design of the bank business model is conceptually complete. You have identified the object make up your business layer as well as what services they provide. All that remains is to design methods the user interface database access and implement the methods using any object oriented programmer language. To keep the book language independent we represent the methods algorithms with UML activity diagram which very easily can be translated into any language. In essence, this phase prepares the system for the implementation. The actual coding and implementation should be relatively easy and for most part can be automated by using CASE tools. This is because we know what we want to code. It is always difficult to code when we have no clear understanding of what we want to do.

9.8.1 Bank Client Class Verify Password Method

The following describes the verify password service in greater detail. A client pin code is sent from the ATM machine object and used as an argument in the verify password method. The verify password method retrieves the client record and check the entered pin number against the client's pin number. If they match, it allows the user proceed. Otherwise a message

sent to the ATM Machine displays “incorrect pin please try again” see figure 9.3. the verify password methods performs first creates a bank client object and attempts to retrieve the client data based on the supplied card and pin numbers. At this stage we realize that we need to have another method, retrieve client. The retrieve client method tasked two arguments the card number and pin number and returns the client object or “nil” if the password is not valid.

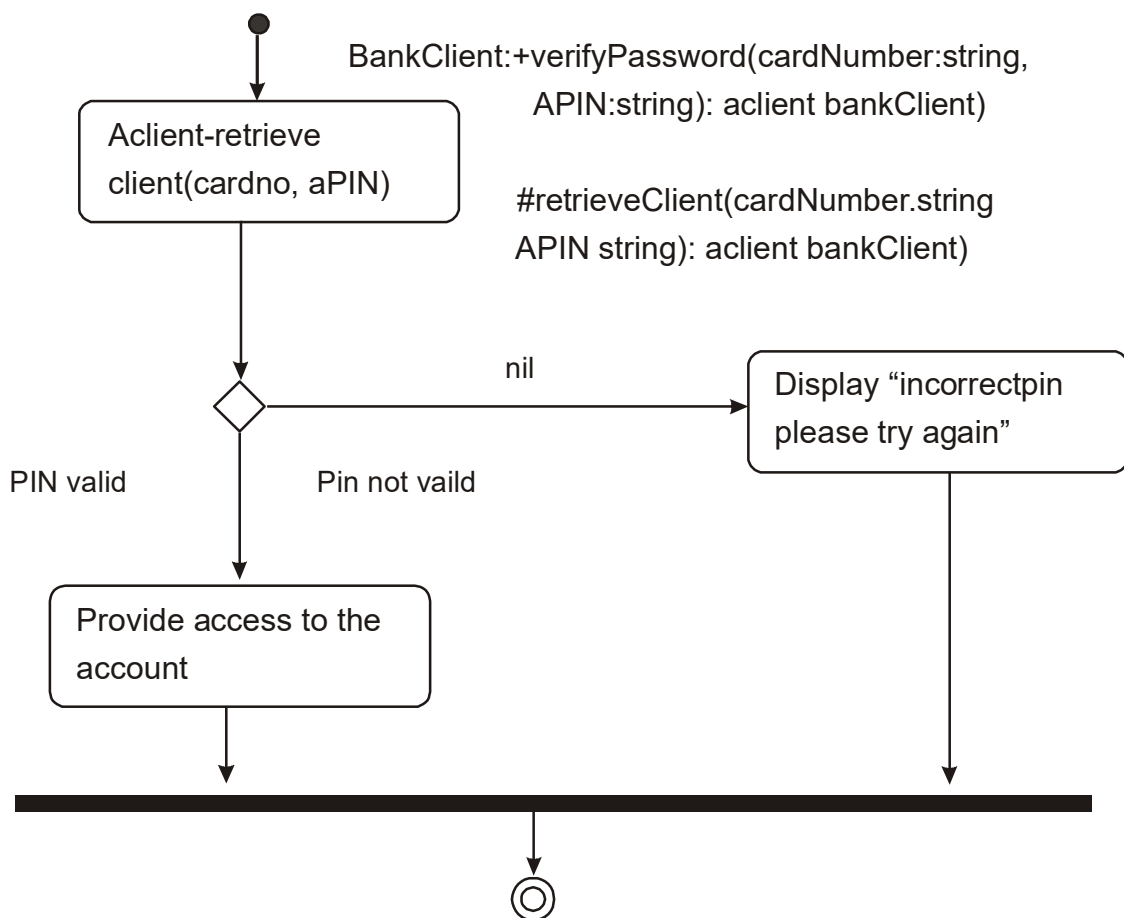


Figure 9.3

9.8.2 Account Class Deposit Method

The following describes the deposit in greater detail. An amount to be deposited is sent to an account object and used as an argument to the deposit service. The account adjusts its balance to its current balance plus the deposit amount.

The account object records the deposit by creating a transaction object containing the data and time, posted balance, and transaction type and amount see figure 9.4. once again we have discovered another method, update client. This method as the name suggests, updates client data. We postpone design of the update client method.

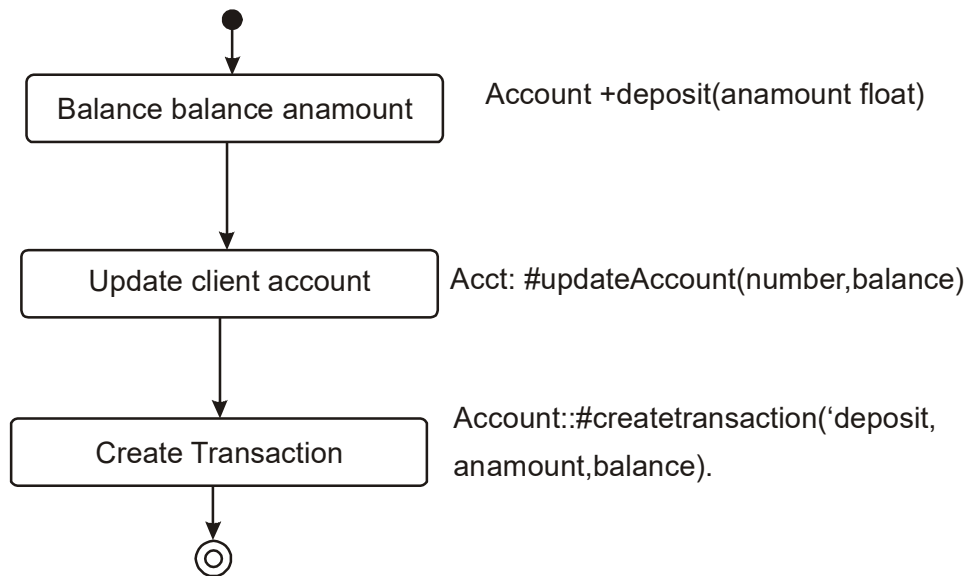


Figure 9.4

9.8.3 Account Class Withdraw Method

This is the generic withdrawal method that simply withdraws funds if they are available. It is designed to be inherited by the a checking Account and saving account classes to implement automatic funds transfer. The following describes the withdraw method. An amount to be withdraw is sent to an account object and used as the argument to the withdraw service.

The account check its balance for sufficient funds. If enough funds are available, the account makes the withdrawal and updates its balance; otherwise it return an error saying "insufficient funds".

If successful the account; records the withdrawal by creating a transaction object containing date and time, posted balance, and transaction type and amount see figure 9.5.

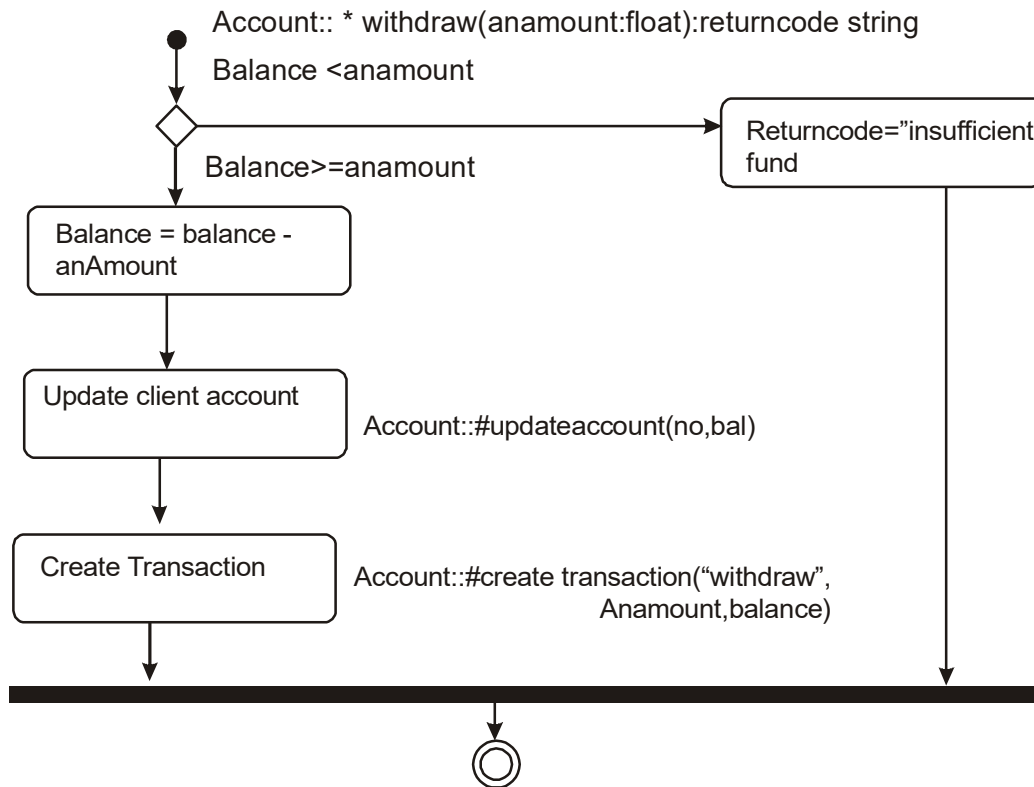


Figure 9.5

9.8.4 Account Class CreateTransaction Method

The create transaction method generates a record of each transaction performed against it. The description is as follows. Each time a successful transaction is performed against an account the amount object a transaction object to record it Arguments into this service include transaction type, the transaction amount, and the balance after transaction. The account create a new transaction object and sets its attributes to the desired information. Add this description to the create transaction's description field see figure 9.6.

9.8.5 Checking Account Class Withdraw method

This is the checking account specific version of the withdrawal service. It takes into consideration the possibility of withdrawing excess funds from a companion savings account. The description is as follows. An amount to be withdrawn is sent to a checking account and

used as the argument to the withdrawal service. If the companion account has enough funds, the excess is withdrawn from there, and the checking account balance goes to zero. If successful, the account records the withdrawal by creating a transaction object containing the date and time, posted balance and transaction type and amount see figure 9.7.

```
Account :: # createtransaction (atype: string, anamount:float, pbalance: float)
```

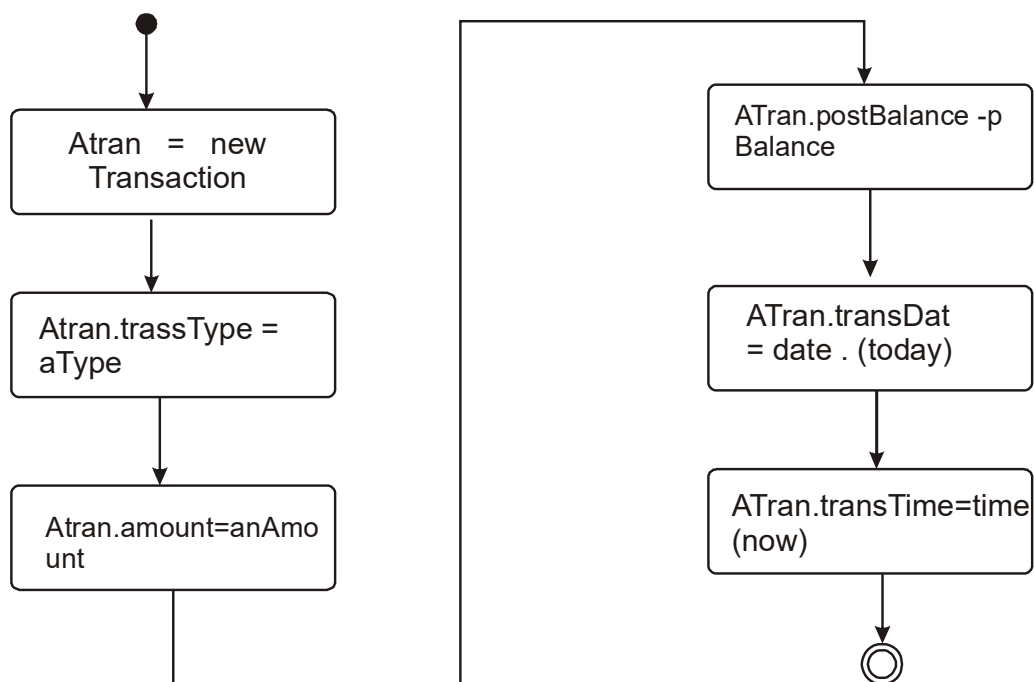


Figure 9.6

9.8.6 ATM Machine class operation

The ATM machine class provides an interface to the bank system.

9.9 Packages and Managing Classes

A package group and manage the modeling elements such as classes, their association and their structure. Package themselves may be nested within other packages. A package may contain both other package and ordinary model elements. The entire system description can be thought of as a single high level sub system package with every thing else in it.

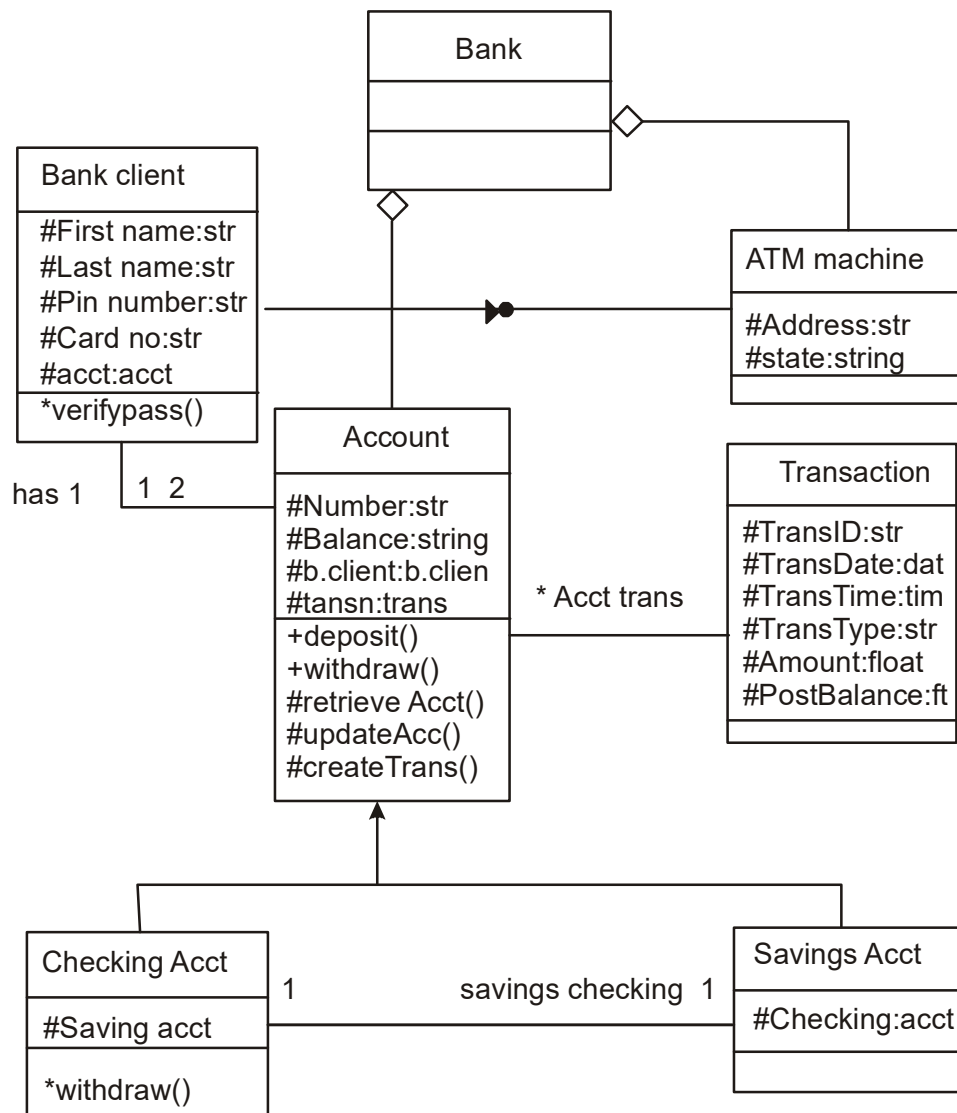


Figure 9.8

For example the bank system can be viewed as a package that contains other packages such as account package, client package and so on. Classes can be access classes and view classes see figure 9.8. furthermore since packages own model element and model fragments they can be used by CASE tool as the basic stor age and access control. Relationship may be drawn between package symbol to show relationship

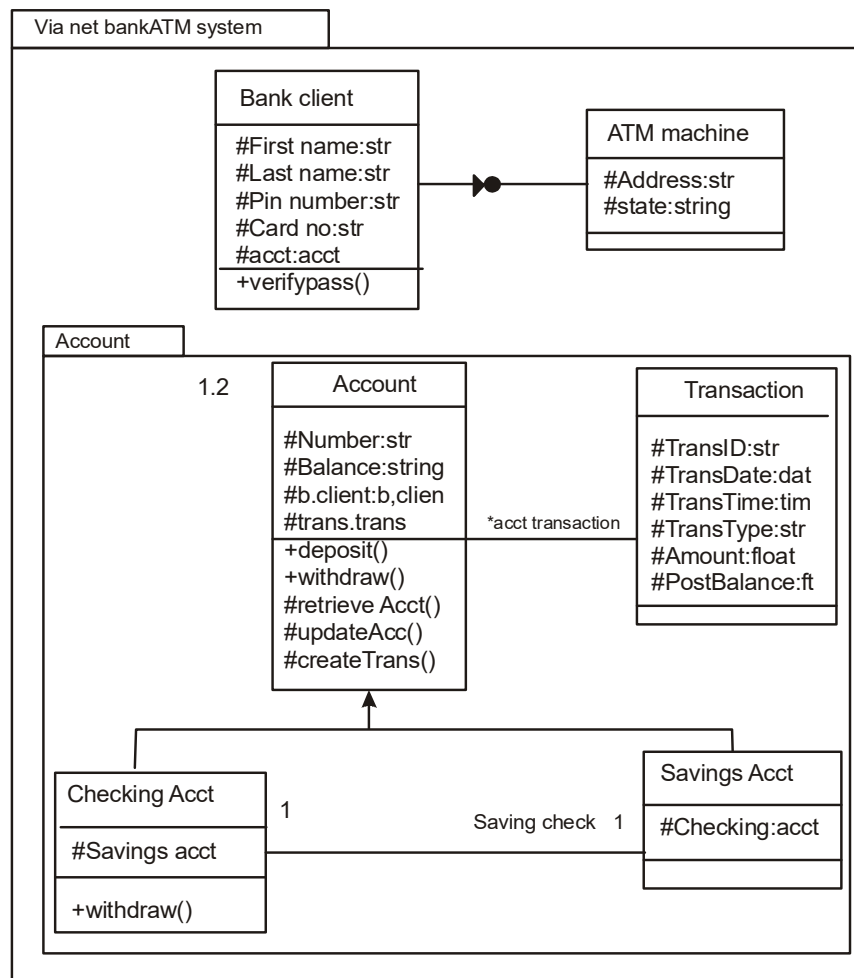


Figure 9.9

Between at least of the elements in the packages. In particular, dependency between packages implies one or more dependencies among the element. figure 9.9 depicts an even more complete class diagram the via net bank ATM system.

Summary

- The single most important activity in designing an application is coming up with a set of classes that work together to provide the needed functionality.
- The first step of the object oriented design process which consists of applying the design axioms and corollaries to design classes their attributes, method, association, structure, and protocols.

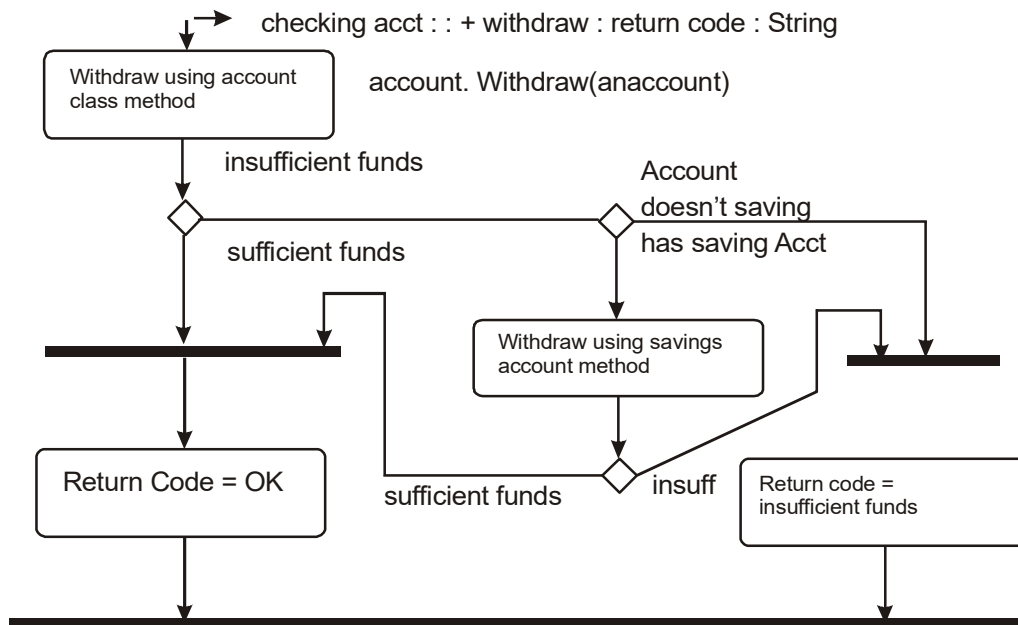


Figure 9.7

- The rule and semantics of the UML can be expressed in English, in a form known as object constraints language (OCL).
- Remember five rules to avoid bad design.
 - a. If it looks messy, then it's probably a bad design.
 - b. If it is too complex, then it's probably a bad design.
 - c. If it is too big, then it's probably a bad design.
 - d. If people don't like it, then it's probably a bad design.
 - e. If it is doesn't work, then it's probably a bad design.

Questions

1. What are public and private protocols ?
2. What is the significance of separating these two protocols ?
3. What are some characteristics of a bad design?
4. How do design axioms help avoid design pitfalls?

LESSON - 10

OBJECT STORAGE AND OBJECT INTEROPERABILITY

Objectives:

- Object storage and persistence.
- Database management systems and their technology
- Client sever computing.
- Distributed databases.
- Distributed object computing.
- Object oriented database management systems.
- Object relational systems.
- Designing access layer objects.

Introduction

A data base management system is set of programs that enables the creation and maintenance of a collection of related data. A dbms and associated program access, manipulate, protect and manage the data. The fundamental purpose of a DBMS is to provide a reliable persistent data storage. The data model incorporated into a database system defines a frame work of concepts that can be used to express an application.

Persistence refers to the ability of some objects to outlive the program that created them. Object lifetimes can be short as for local object or long, as for object stored indefinitely in a database. Most object oriented language do not support serialization or object persistence which is the process of writing or reading an object to add from a persistence storage medium such as a disk file. Even though a reliable, persistent storage facility is the most important object stores do not support query or interactive user interface facilities, as found in fully supported object oriented data base management system.

Furthermore controlling concurrent access by users providing ad-hoc query capability and allowing independent control over the physical location of data are example of features that differentiate a full database from simply a persistent store. This lesson introduces you to the

issues regarding object storage, relational and object oriented data base management systems. Object interoperability and other technologies. we then look at current trends to combine object and relational system to provide a very practical solution to object storage.

10.1 Object store and persistence :An overview

A program will create a large of data throughout its execution. Each item of data will have a different lifetime. Atkinson et analyzing. Describe six broad categories for the lifetime of data:

- Transient result to the evaluation of expressions.
- Variables involved in procedure activation.
- Global variables and variables that are dynamically allocated.
- Data that exist between the executions of a program.
- Data that exist between the versions of a program.
- Data that outlive a program

The first three categories are transient data, data that cases to exist beyond the lifetime of the creating process. The other three are non transient or persistence data. Typically programmer language provide excellent, integrated support for the first three categories of transient data. The other three categories can be supported by a DBMS or file systems. A file or database can provide a longer life for objects-longer than the duration of the process in which they were created. From a language perspective this characteristic is called persistence. Essential elements in providing a persistent store are:

- Identification of persistence object or reach ability stability.
- Properties of object and their interconnections. The store must be able to coherently manage non pointer and pointer data.
- Scale of the object store. The object store should provide a conceptually infinite store.
- The system should be able to recover from unexpected failures and return the system to a recent self consistent state. This is similar to the reliability requirements of a DBMS object oriented or not.

Having separate method of manipulating the data presents many problems. Additionally the use of these external storage mechanisms leads to a variety of technical issue, which will be examined in the following sections.

10.2 Database Management System

Database usually are large bodies of data seen as critical resources to a company. As mentioned earlier a DBMS is a set of program that enable the creation and maintenance of a collection of related data. DBMSs have a number of properties that distinguish them from the file based data management approach. In traditional file processing each application defines and implements the file it requires.

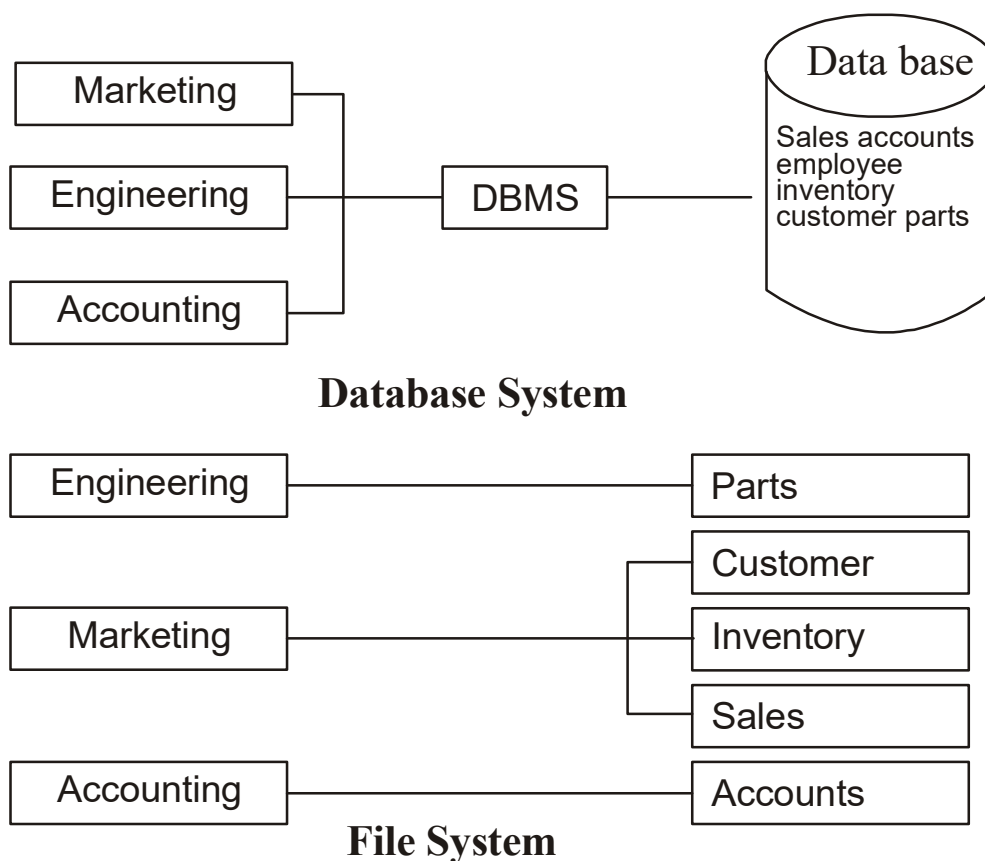


Figure 10.1

Using a database approach, a single repository of data is maintained, which can be defined once and sequentially accessed by various users (see figure 10.1). A fundamental characteristic of the database approach is that the DBMS contains not only the data but a complete definition of the data formats it manages. This description is known as the schema, or meta data, and contains a complete definition of the data formats it manages. This description is known as the schema or meta data, and contains a complete definition of the data formats such as the data structure, types and constraints.

In traditional file processing applications such meta data usually are encapsulated in the application program themselves. In DBMS the format of the meta data is independent of any particular application data structure; therefore it will provide a generic storage management mechanism.

Another advantage of the database approach is program data independence. By moving the meta data into an external DBMS a layer of insulation is created between the applications and the stored data structure. This allows any number of applications to access the data in a simplified and uniform manner.

10.2.1 Database Views

The DBMS provides the database users with a conceptual representation that is independent of the low-level details of how the data are stored. The database can provide an abstract data model that uses logical concepts such as fields, records, and tables and their interrelationships. Such a model is understood more easily by the user than the low-level storage concepts.

This abstract data model also can facilitate multiple views of the same underlying data. Many applications will use the same shared information but each will be interested in only a subset of the data.

The DBMS can provide multiple virtual views of a data that are tailored to individual applications. This allows the convenience of a private data representation with the advantage of globally managed information.

10.2.2 Database Models

A database model is a collection of logical constructs used to represent the data structure and data relationships within the database. Basically, database models may be grouped into two

categories, conceptual models and implementation models. The conceptual model focuses on the logical nature of the data presentation. Therefore the conceptual model is concerned with what is represented in the database and the implementation model is concerned with how it is represented.

10.2.2.1 Hierarchical Model

The inheritance model represents data as a single rooted tree. Each node in the tree represents a data object and the connections represent a parent child relationship.

For example a node might be a record containing about motor vehicle and its child nodes could contain a record about bus parts see figure 10.2. interestingly enough a hierarchical model resembles super sub relationship of objects.

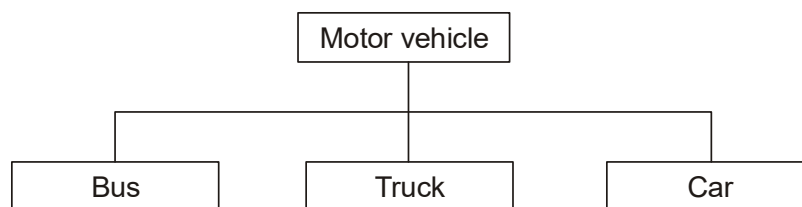


Figure 10.2

10.2.2.2 Network Model

A network database model is similar to hierarchical database with one distinction. Unlike the hierarchical model a network model's record can have more than one parent.

For example 10.3 an order contains data from the soup and customer nodes.

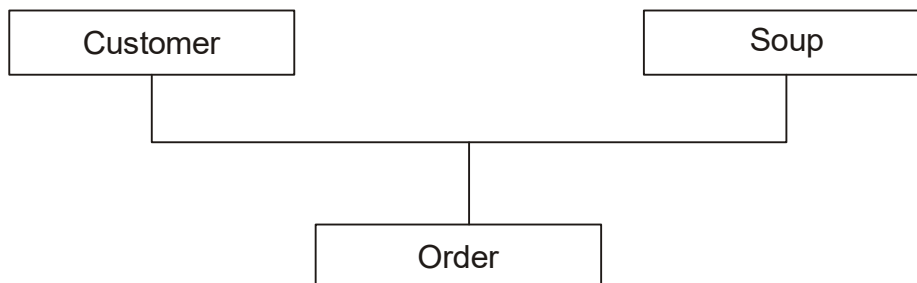


Figure 10.3

10.2.2.3 Relational Model

Of all the database models the relational model has the simplest, most uniform structure and is the most commercially widespread. The primary concept in this database model is the relation, which can be thought of as table. The columns of each table are attributes that define the data or value domain for entries in that column.

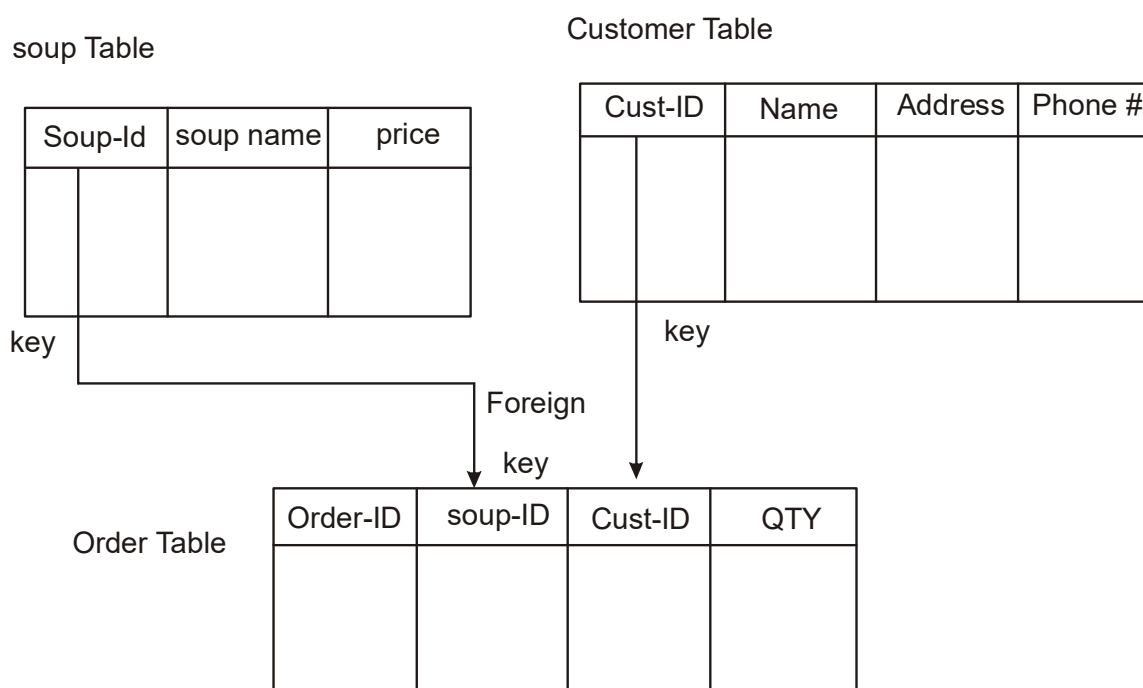


Figure 10.4

The rows of each table are tuples representing individual data objects being stored. A relational table should have only one primary key. A primary key is a combination of one or more attributes whose value and order IDENTIFYING are primary keys in soup, customer and order tables. A foreign key is a primary key of one table that is embedded in another table to link the tables. In figure 10.4, soup-ID and cust-ID are foreign keys in the order table.

10.2.3 Database Interface

The interface on a database must include a data definition language (DDL) query and data manipulation language (DML). These language must be designed to fully reflect the flexibility and constraints inherent in the data model. Database system have adopted two approaches for

interface with the system. One is to embed a database language, such as structured query language (SQL), in the host programmer language.

This approach is very popular way of defining and designing a database and its schema, especially with the popularity of language such as SQL which has become an industry standard for defining database. The two different languages. Furthermore the application programmers have to negotiate the differences in the data model and data structures allowed in both language.

Another approach is to extend the host programmer language with database need to learn only a new construct of the same language rather than a completely new language. Many of the currently operational databases and object oriented database systems have adopted this approaches, a good example is gemstone from servio logic, which has extended in the smalltalk object oriented programming

10.2.3.1 Database schema and Data Definition Language

To represent information in a database a mechanism must exist to describe or specify to the database the entities of interest. A data definition language (DDL) is the language used to describe the structure of and relationship between objects stored in a database.

This structure of information is termed the database schema. In traditional database the schema of a database is a collection of record types and set types or the collection of relationship, templates, and table records used to store information about entities of interest to the application

For example to create logical structure or schema the following SQL command can be used:

```
CREATE SCHEMA AUTHORIZATION(creator)
```

```
CREATE DATABASE(database name)
```

For example

```
CREATE TABLE INV(INV_NO CHAR(10) NOT NULL DESCRIPTION CHAR(25) NOT
NULL PRICE DECIMAL(9,2));
```

10.2.3.2 Data Manipulation Language and Query Capabilities

Any time data are collected on virtually any topic some one will want to ask questions about it. Some one will want the answers to simple question like “how many of them are there?” or more intricate questions. A query usually is expressed through a query language. A Data Manipulation Language (DML) is the language that allows users to access and manipulate data organization. The structured query language(SQL) is the standard DML for relational DBMSS.SQL is widely used for its query capabilities. The query usually specifies.

- The domain of the discovers over which to ask the query.
- The element of general interest.
- The conditions or constraints that apply.
- The ordering, sorting or grouping of element and the constraints that apply to the ordering or grouping.

Query processes generally have sophisticated “engines” that determine the best way to approaches the database and execute the query over it. They may use information in the database or knowledge of the whereabouts of particular data in the network to optimize the retrival of a query.

Traditionally, DML are either procedural or nonprocedural. A procedural dml requires users to specify what data are desired and how to get the data. A nonprocedural DML, like most databases' fourth generation programming language (4GLs), requires users to specify what data are needed but not how to get the data Object-oriented query and data manipulation languages, such as Object SQL, provide object management capabilities to the data manipulation language.

In a relational DBMS, the DML is independent of the host programming language. A host language such as Class or COBOL would be used write the body of the application. Typically, SQL statements then are embedded in Class or COBOL applications to manipulate data. Once SQL is used to request and retrieve database data, the results of the SQL retrieval must be transformed into the data structures of the programming language. A disadvantage of this approach is that programmer's ceded in two languages, SQL and the host language. Another is that the structural transformation is required in both database access directions, to and from the database.

For example, to check the table content, the SELECT command is used, followed by the desired attributes. Or, if you want to see all the attributes listed, use the (*) to indicate all the attributes: SELECT DESCRIPTION,PRICE FROM INVENTORY: where inventory is the name of the table.

10.3 Logical and Physical Database Organization and Access Control

Logical database organization refers to the conceptual view of database structure and the relationship within the database. For example object oriented system represent database composed of objects and many allow multiple database to share information by defining the same object. Physical database organization refers to how the logical components of the database are represented in a physical form by operating system constructs.

10.3.1 Share ability and Transactions

Data and object in the often need to be accessed and shared by different applications. With multiple application having access to the object concurrently, it is likely that conflicts over object access will arise. A transaction is a unit of change in which many individual modification are aggregated into a single modification that occurs in its entirety or not at all.

Thus either all changes to object within a given transaction are applied to the database or none of the changes. A transactions is said to commit if all changes can be made successfully to the database and to abort if canceled because all changes to the database cannot be made successfully. This ability of transactions ensures atomicity of changes that maintain the database in a consistent state. Many transactions system are designed primarily for short transactions. They are less suitable for long transactions lasting hours or longer. Object database typically are designed to support both short and long transactions. A concurrence control policy dictates what happens when conflicts arise between transactions that attempt access to the same object and how these conflicts are to be resolved.

10.3.2 Concurrency Policy

As you might expect when several users attempt to read and write the same object simultaneously, they create a contention for object. The concurrency control mechanism is established to mediate such conflicts by making policies that dictate how they will be handled. A basic goal the transactions is to provide each user with a consistent view of the database.

This means that transactions must occur in serial order. In other words a given user must see the database as it exists either before a given transactions occurs or after that transactions.

The most conservative way to enforce serialization is to allow a user to lock all objects or record when they are accessed to release the locks only after transactions commits. This approach traditionally known as a conservative or pessimistic policy, provide exclusive access to the object. However by distinguishing between querying the object and writing to it somewhat greater concurrency can be achieved. This policy allow many readers of an object but only one writer.

Under an optimistic policy, two conflicting transactions are compared in their entirety and then their serial ordering determined. As long as the database is able to serialize them so that all the objects viewed by each transactions are from a consistent state of the database, both can continue even though they have read and write on an object already write locked if its entirely after the conflicting transactions. In such cases the optimistic policy allows more processes to operate concurrently than the conservative policy.

10.4 Distributed Database and Client Server Computing

Many modern databases are distributed databases which implies that portions of the database reside on different nodes and disk drives in the network. Usually each portion of the database is managed by a server a process responsibility for controlling access and retrieval of data from the database portion. The sever dispenses information to client application or other servers execute. However both can reside on the same node too.

10.5.1 What is Client server Computing

Client server computing is the logical extension of modular programming. The fundamental assumption of modular programming is that separation of large piece of software into its constituent parts create the possibility for easier development and better maintainability. Client server computing extends this theory a step further by recognizing that all those modules need not be executed within the same memory space or even on the same machine.

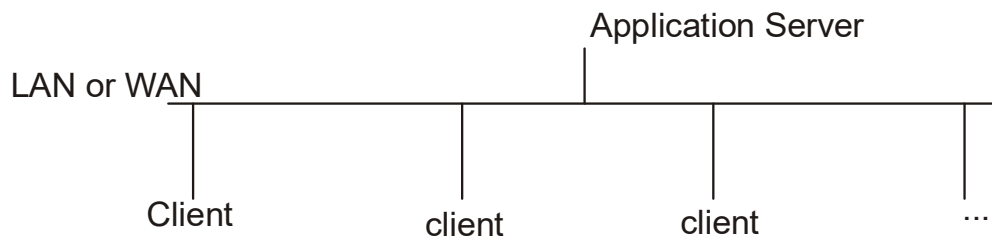


Figure 10.5

With this architecture the calling module becomes the “client” and the called module becomes a “server” that which provide the service, see figure 10.5. another important components of client server computing is connectivity, which allows application to communicate transparently with other program or processes, regardless of their location. The location key element of connectivity is the network operating system also known as middleware. The client is a process that sends a message to a server process requesting that server perform a task.

The client process contains solution specific logic and provides the interface between the user and rest of the application system. It also manages the local resources with which the user interacts, such as the monitor, keyboard, workstation, CPU, and peripherals. A key components of a client workstation is the graphical user interface, which normally is a part of the operating system. It is responsibility for detecting user actions, managing the windows on the display, and displaying the data in the windows. A server process fulfills the client request by performing the task requested. Sever program generally receive request from client programs, execute database retrieval and updates, manage data integrity, and dispatch responses to logic.

Some time server programs system execute common or complex business logic. The server based process “may” run on another machine on the network. This server could be the host operating system or network file server, the server then is provide both file system services and application services. In some cases another desktop machine provide the application services.

The server process acts as a software engine that manage shared resources such as database, printer, communication links, or high powered processors. The server process perform the back end tasks that are common to similar application. The server can take different forms. The simplest form of server is a file server. With a file server the client passes request for files or file records over a network to the file server. This form of data service requires large bandwidth

and can considerably slow down a network with many users. Traditional LAN computing allows users to share resources such as data files and peripheral devices.

More advanced forms of servers are database servers transactions servers, application servers, and more recently object servers. With database servers clients pass SQL requests a messages to the servers and the results of the query are returned over the network. Both the code that processes the SQL request and the data reside on the server, allowing it to use its own processing power to find the requested data. This is in contract to file server which requires passing all the records back to the client and then letting the client find its own data. With transactions servers clients invoke remote procedures that reside on servers, which also contain SQL database engine. The server has procedural statement to execute a group of SQL statements which either all succeed or fail as a unit. The application based on transactions servers handled by on line transactions processing tend to be mission critical applications that always require a 1-3 second response time and tight control over the security and the integrity of the database. The communication overhead in this approach is kept a minimum, since the exchange typically consists of a single request and reply.

Approaches server are not necessarily database centered but are used to serve user needs such as downloading capabilities from Dow Jones or regulating an electronic mail process. basing resources on a server allows users to share data while security and management services also based on the server, ensure data integrity and security.

The logical extension of this is to have clients and servers running on the appropriate hardware and so platforms for their functions. For example designed and configured to perform queries and files servers should run on platform with special elements for managing files. In a two tier architecture a client talks directly to a server with no intervening server.

This type of architecture typically is used in small environment with less than 50 users see figure 10.5. a common error in client server development is to prepare a prototype of an application in a small. Two tier environment then scale up by simply adding more users to the server becomes overwhelmed. To properly scale up to hundreds or thousand of users, it usually is necessary to move to three tier architecture. A three tier architecture introduces a server between the client and the server. The role of the application or web server is manifold. It can provide translation services, metering services or intelligent agent results and returning a single response to the client see figure 10.6.

Ravi kalakota describes the basic characteristics of client server architectures as follow.

- A combination of a client or front end portion that interacts with the user and server or back end portion that interface with the shared resource. The client process contains solution specific logic and provides the interface between the user and the rest of the application system. The server process acts as a so engine that manages shared resources such as database, printer, modems or high powered processors.
- The front ends task and back end task have fundamentally different requirements for computing resources such as process speeds, memory, disk speeds and capacities and input/ output devices.
- The environment is typically heterogeneous and multivendor the hardware platform and operating system of client and server are not usually the same. Client and server processes communicate through a well defined set of standard application program interfaces.
- An important characteristic of client server system is scalability. They can be scaled horizontally or vertically. horizontal scaling means adding or removing client workstation with only a slight performance impact.

Client server and distributed computing have arisen because of change in bus needs unfortunately most businesses have existing systems based on older technology, that must be incorporated into the new, integrated environment, that is mainframe with a great deal of legacy software.

Robertson Dunn answers the question “why build client server application?” by pointing out that “business demands the increased benefits”. The distinguishing characteristic of a client sever application is the high degree of interaction among various application components. these are the interaction between the client requests and server’s reactions to those requests. To understand these interactions we look at the client server application’s components. A typical client server application consists of the following components:

1. User interface. This major components of the client server application interacts with users, screens, windows, management, keyboard, and mouse handling.
2. Business processing. This part of the application uses the user interface data to perform business tasks.

3. Database processing. This part of the application code manipulates data within the application. The data are managed by a database management system, Object oriented or not. Data manipulation is done using a data manipulation language such as SQL or dialect of SQL. Ideally the DBMS processing is transparent to the business processing layer of the application.

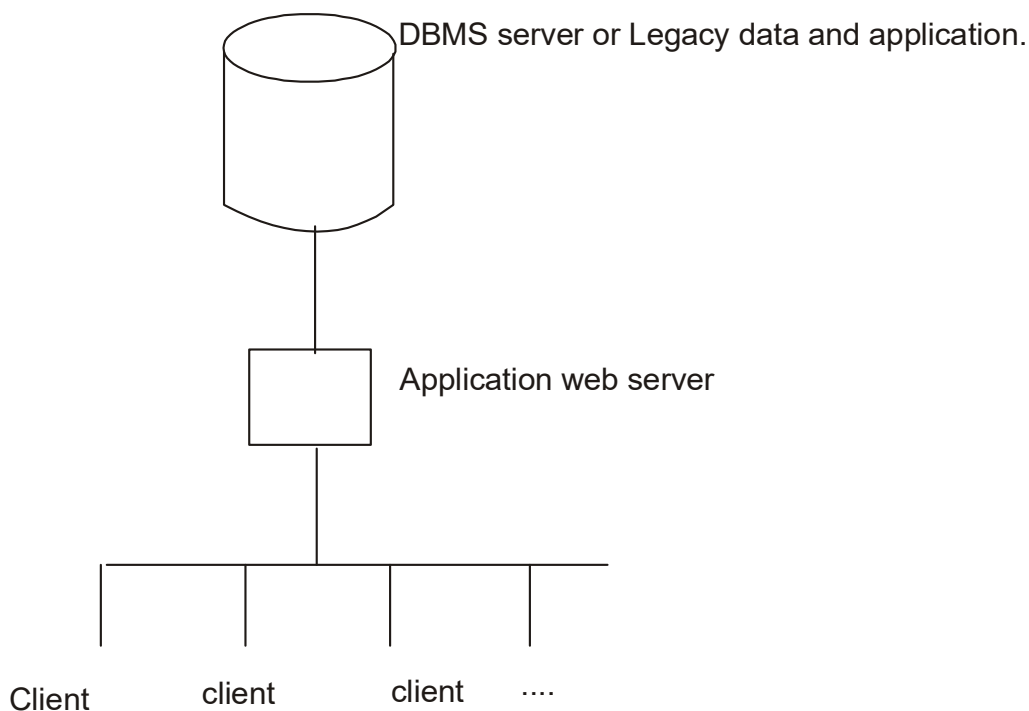


Figure 10.6

10.4.2 Distributed and Cooperative Processing

The distributed processing means distributed of application and business logic across multiple processing platforms. Distributed processing implies that processing will occur than one processor in order for a transactions to be completed. In other words, processing is distributed across two or more machines.

Where each process performs part of an application in a sequence. These processes may not run at the same time see figure 10.7. for example in processing an order from a client the client information may process at one machine and the account information then may process on a different machine. Often, the object used in distributed processing environment also is distributed across a platforms.

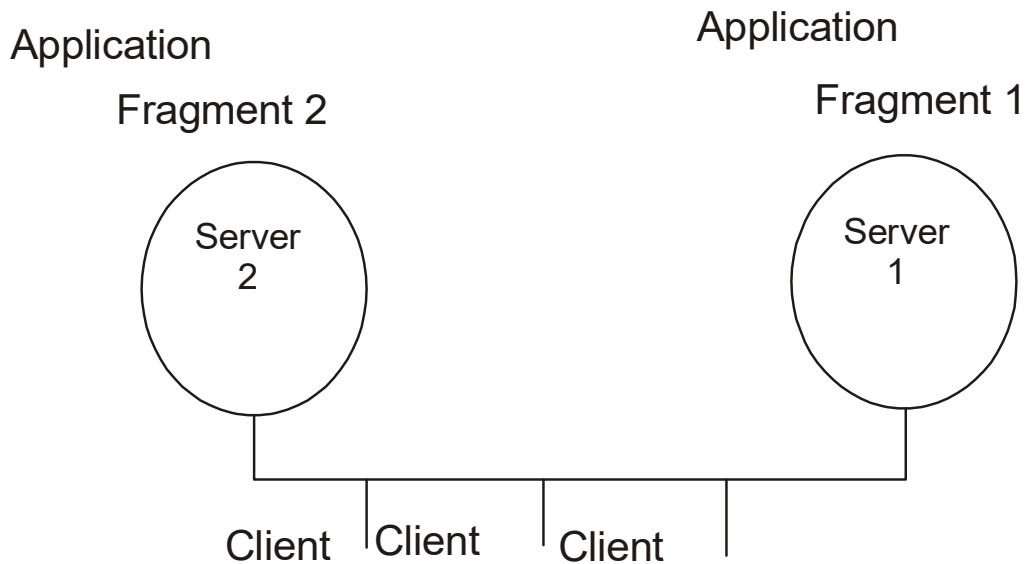


Figure 10.7

Cooperative processing is related to both distributed and client server process.. cooperative processing is a form of distributed computing in which two or more distinct processes are required to complete a signal business transaction. Usually these program interact and execute concurrently on different processors see figure 10.8. cooperative processing also can be considered to be a style of distributed processing, if comm. Between processors is performed through a message passing architecture.

10.6 Distributed Objects Computing: The next Generation of Client Server computing

In the preceding section, we looked at what is now considered the first generation of client server computing. Eventually, the sever code in your client server system will give way to collections of distributed objects. since all of them will need to talk to each other, the second generation of client server computing is based on the distributed object computing, which will be covered in the next section.

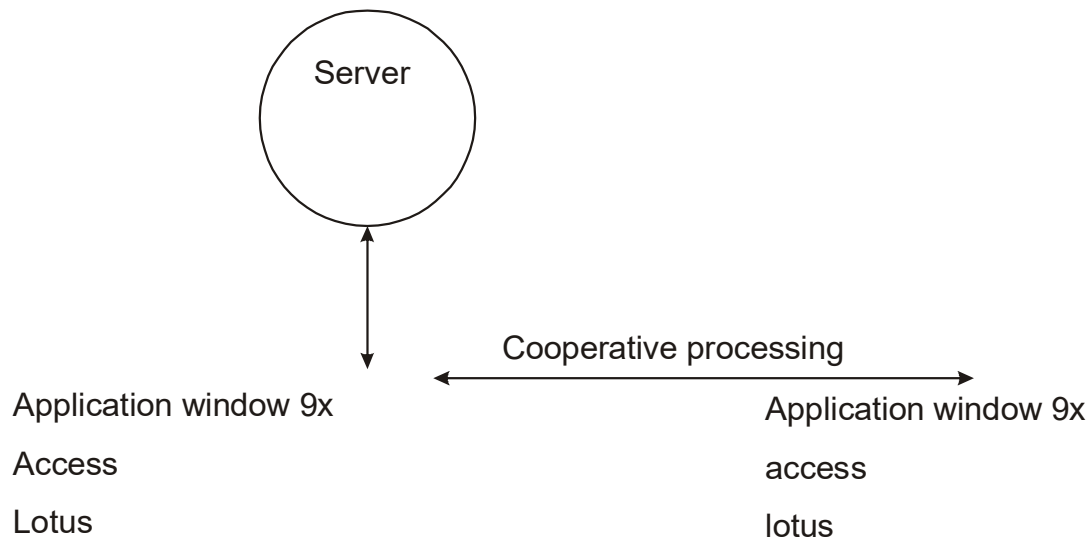


Figure 10.8

Software technology is in the midst of a major computational shift towards distributed object computing, distributed computing is poised for a second client server revolution, a transition from first generation client server era to a next generation client server era. In this new client server model, servers are plentiful instead of scarce and proximity no longer matters.

This immensely network growth and the progress in network aware multithreaded desktop operating systems. In the first generation client server which still is very much in progress SQL database, transaction processing monitors and groupware have begun to displace file servers as client application models. In the new client-server era. Distributed object technology is expected to dominate other client-server application models.

Distributed object computing promises the most flexible clientserver system because it utilizes reusable software components that can roam anywhere on network run on different platforms, communicate with legacy application by means of object wrappers and manage themselves and resources they control. Objects can help break monolithic application into more manageable components that coexist on the expanded bus. Distributed object are reusable software components that can be distributed and accessed by users across the network.

These objects can be assembled into distributed application. Distributed object computing introduces a higher level of abstraction into the world of distributed applications. Applications

no longer consists of clients and servers but users, objects and methods. The user no longer needs to know which server process performs given function. All information about the function is hidden inside the encapsulated object. A message requesting an operation is sent to the object and the appropriate method is invoked.

10.5.1 Common Object Request Broker Architecture

Many organization are now adopting the object manage-” ment group’s Common Object Request Broker Architecture (CORBA), a standard proposed as a means to integrate distributed, heterogeneous business application and data.

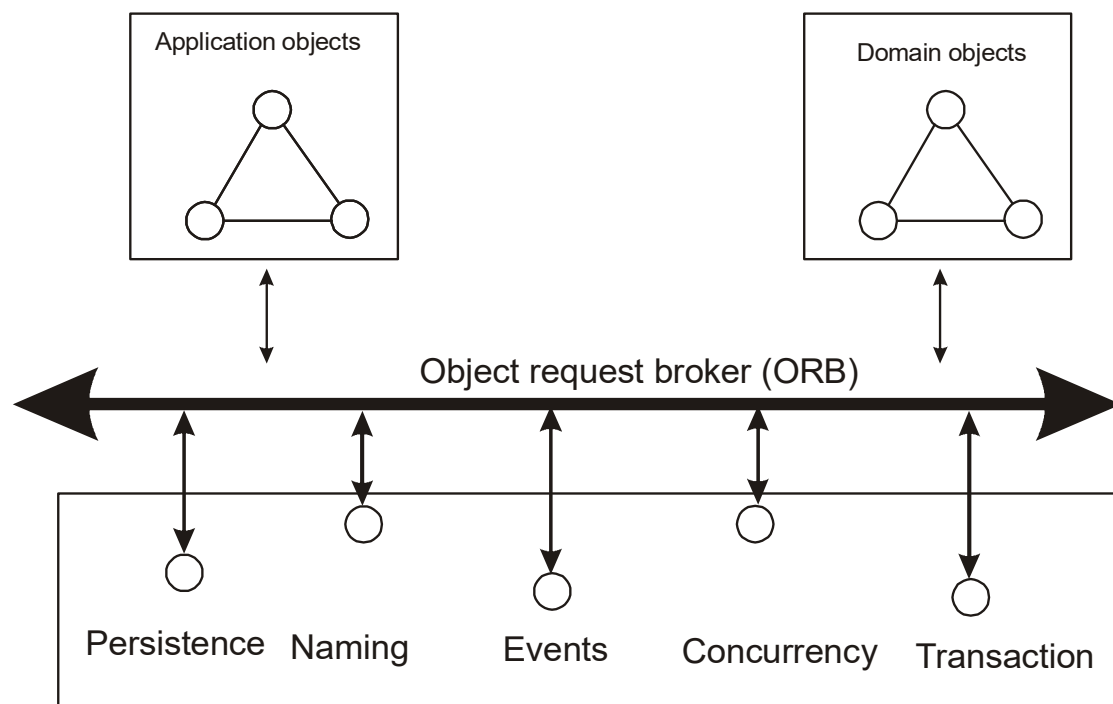


Figure 10.9

The CORBA interface definition language (IDL) allows developers to specify language neutral object oriented interfaces for application and system components. IDL definitions are stored in an interface repository a sort of phone book that offers object interface repository is central to comm. Among objects located on different systems. CORBA object request brokers (ORBs) implement a comm. Channel through which application can access object interface and request data and services see figure 10.9 the CORBA common object

environment(COE) provides system level services as life cycle management for objects accessed through CORBA, event notification between objects and transactions and concurrency control.

10.5.2 Microsoft's ActiveX/DOCM

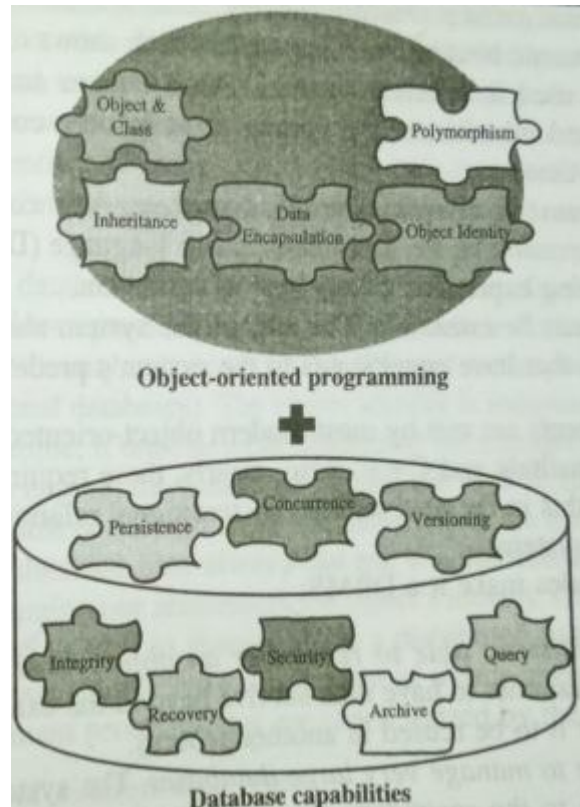
Microsoft's components object model(COMMUNICATION) and its successor the distributed components object model (DCOM) are Microsoft's alternatives to OMG's distributed object architecture CORBA. Microsoft and the OMG are competitors and few can say for sure which technology will win the challenge. Although CORBA benefits from wide industry support, DCOM is supported mostly by one enterprise, Microsoft.

However, Microsoft is no small business concern and holds firmly a huge part of the microcomputer population, so DCOM has appeared a very serious competitor to CORBA. The distributed components object model. Microsoft's alternative to OMG's, CORBA, is an internet and components strategy where ActiveX plays the role of DCOM object.

10.6 Object Oriented Management System:The Pure World

Database management system have progressed from indexed files ton network and hierarchical database system to relationship systems. The requirements of traditional business data processing applications are well met in functionality and performance be relational database systems focused on the needs of business data processing application. However as many researchers observed they are inadequate for a broader class application with unconventional and complex data type requirements.

These requirements along with the popularity of object oriented programming have resulted in great demand for an object oriented DBMS(OODBMS). Therefore the interest in OODBMS initially stemmed from the data storage requirements of design support applications. The object oriented data base management system is a marriage of object oriented programming the and database technology see figure 10.10. to provide what we now call object oriented databases. Additionally, object oriented databases allow all the benefits of an object orientation as well as the ability to have a strong equivalence with object oriented programs, an equivalence that would be lost if an alternative were chosen as with a purely relational databases. By combining object oriented programming with database technology, e have an integrated application development system, a significant characteristic of object oriented database technology.



The “object oriented database system manifesto” by Malcom Atkinson et analyzing. Described the necessary characteristics that a system must satisfy to be considered an object oriented database. These categories can be broadly divided into object oriented language properties and database requirements.

First the rules that make it an object oriented system are as

1. the system must support complex objects. A system must provide simple atomic types of objects from which complex follow objects can be built by applying constructors to atomic objects or other complex objects or both.
2. object identity must be supported. A dada object must have 2 an identity and existence independent of its value.
3. objects must be encapsulated. An object must encapsulate both a program and its data. Encapsulation embodies the separation in interface and implementation and

need for modularity.

4. the system must support types or classes. The system must support either type concept or the class concept.
5. the system must support inheritance. Classes and types can participate in a class hierarchy. The primary advantage of inheritance is that it factors out shared code and interfaces.
6. the system must avoid premature binding. This feature also is known as late binding or dynamic binding. Since classes and types support encapsulation and inheritance the system must resolve conflicts in operation names at run time.
7. the system must be computationally complete. Any computable function should be expressible in the data manipulation language of the system thereby allowing expression of any type of operation.
8. the system must be extensible. The user of the system should be able to create new types that have equal status to the system's predefined.

These requirements are met by most modern object oriented programming language such as Smalltalk and C++. Also clearly these requirements are not met directly by traditional, hierarchical or network database system. Second these rules make it a DBMS

9. it must be persistent able to remember an object state. The system must allow the programmer to have data survive beyond the execution of the creating process for it to be reused in another process.
10. it must be able to manage very large databases. The system must efficiently manage access to the secondary storage and provide performance features, such as indexing, buffering and query optimization.
11. it must accept concurrent users. The system must allow multiple concurrent users and support the notions of atomic, serializable transactions.
12. it must be able to recover from hardware and software failures. The system must be able to recover from software and hardware failures and return to coherent state.
13. data query must be simple. The system must provide some high level mechanism for ad-hoc browsing of the contents of the database. A graphical browser might fulfill this requirements sufficiently.

10.6.1 Object oriented database versus Traditional Databases

The scope of the responsibility of an OODBMS includes definition of the object structure, object manipulation, and recovery which is the ability to maintain data integrity regardless of system, network, or media failure. One obvious difference between the traditional and object oriented database is derived from the object's ability to interact with other objects and with itself. The objects are an "active" components in the an object oriented database in contrast conventional database systems, where records play a passive role. yet another distinguishing feature of object oriented database is inheritance. Relational database system do not explicitly provide inheritance of attributes and methods. object oriented database on the other hand represent relationship explicitly, supporting g both navigational and associative access to information.

Summary

- A data base management system (DBMS) is a collection of related data and associated programs that access, manipulate, protect, and manage data.
- The fundamental purpose of a DBMS is to provide a reliable persistent data storage facility and the mechanisms for efficient, convenient data access and retrieval.
- Each potion of the database is managed by a server, a process responsible for controlling access and retrieval of data from the database portion.
- Client-server computing is a logical extension of modular programming.
- The fundamental assumption of modular programming is that separation of a large piece of software into its constituent parts creates the possibility for easier development and better maintainability
- Distributed computing is poised for a second client-server
- The object oriented database technology is a marriage of object oriented programming and database technology.
- The programming and database come together to provide what we well call object oriented databases.

Question

1. What is mean by entity?
2. What are the types of attributes?
3. Consult the WWW to obtain information on ODC especially comparing CORBA with DCOM. Write a research paper based on your findings.
4. What is a DLL?
5. What a distributed database?
6. What is concurrency control?
7. What is database schema?
8. What is difference between a schema and meta data?
9. What is relational database?
10. Is a persistence object the same as a DBMS? What are the differences?

LESSON - 11

OBJECT RELATIONAL SYSTEMS AND ACCESS LAYER

Objectives

- Object relational systems.
- Designing access layer objects.
- Multidatabase system

Introduction

Persistence refers to the ability of some objects to outlive the program that created them. Object lifetimes can be short as for local object or long, as for object stored indefinitely in a database. Most object oriented language do not support serialization or object persistence which is the process of writing or reading an object to add from a persistence storage medium such as a disk file. Even though a reliable, persistent storage facility is the most important object stores do not support query or interactive user interface facilities, as found in fully supported object oriented data base management system.

Furthermore controlling concurrent access by users providing ad-hoc query capability and allowing independent control over the physical location of data are example of features that differentiate a full database from simply a persistent store. This lesson introduces you to the issues regarding object storage, relational and object oriented data base management systems. Object interoperability and other technologies. we then look at current trends to combine object and relational system to provide a very practical solution to object storage.

11.1 Object Relational systems: The Practical world

In practice, even though many application increasingly are developed in an object oriented programming technology chances are good that the data those applications need to access live in very different universe-relational database. In such an environment the introduction of object oriented development creates a fundamental mismatch between the programming model and the way in which existing data are stored.

To resolve the mismatch a mapping tool between the application objects and the relational must be established. Creating an object model from an existing relational database layout often

to as reverse engineering. Conversely, creating relational schema from an existing object model often is referred to as forward engineering. In practice, over the life cycle of application forward and reverse engineering need to be combined in an iterative process to maintain the relationship between the object relational data representations.

Tools that can be used to establish the object relational mapping processes have begun to emerge. The main process is relational and object integration is defining the relationship between the table structure in the relational database with classes in the object model. Java Blend also has mapping capabilities to define java classes from relational tables from the java classes.

11.1.1 Object Relation Mapping

If the relational database, the schema is made up of tables consisting of rows and columns, where each column has name and simple data type. In an object model, the counterpart to a table is a class which has a set of attributes. Object classes describe behavior with methods. A tuple of a table data for a single entity that correlates to an object in an object oriented system.

In addition a stored procedure in a relational database may correlate to method in an object oriented architecture. A Stored procedure is a module of precompiled SQL code maintained within the database that executes on the server to enforce rules the business has set about the data.

Therefore the mappings essential to object and relational integration are between a table and a class, between columns and attributes between a row and an object and between a stored procedure and a method. It must have at least the following mapping capabilities:

1. table-class mapping
2. table-multiple classes mapping.
3. table-inherited classes mapping.
4. table-inherited classes mapping.

Furthermore in addition to mapping column values the tool must be capable of interpretation of relational foreign keys. In the mapped object model and how referential integrity is maintained. Referential integrity means making sure that a dependent table's foreign key contains a value that refers to an existing valid tuple in another relation.

11.1.2 Table class Mapping

Table class mapping is a simple one to one mapping of a table to a class and the mapping of columns in the table to properties in a class. In this mapping, a single table is mapped to a single v, as shown in figure 11.1. in such mapping it is common to map all the columns to properties.

Car table

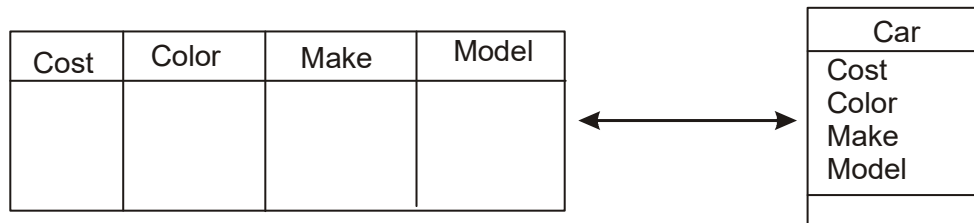


Figure 11.1

However this is not required, and it may be more efficient to map only those columns for which an object model is required by the application. With the table class approaches each row in the table represents an object instance and each column in the table corresponds to an object attribute.

This one to one mapping of the table class approaches provide a literal translation between a relational data representation and an application object. It is appealing in its simplicity but refers little flexibility.

11.1.3 Table Multiple Classes Mapping

In the table multiple classes mapping a single table maps to multiple classes non inheritance classes. two or more distinct, non inheriting classes have properties that are mapped to columns in a single table. At run time a mapped table row is accessed as an instance of one of the classes based on a column value in the table. In figure 11.2.

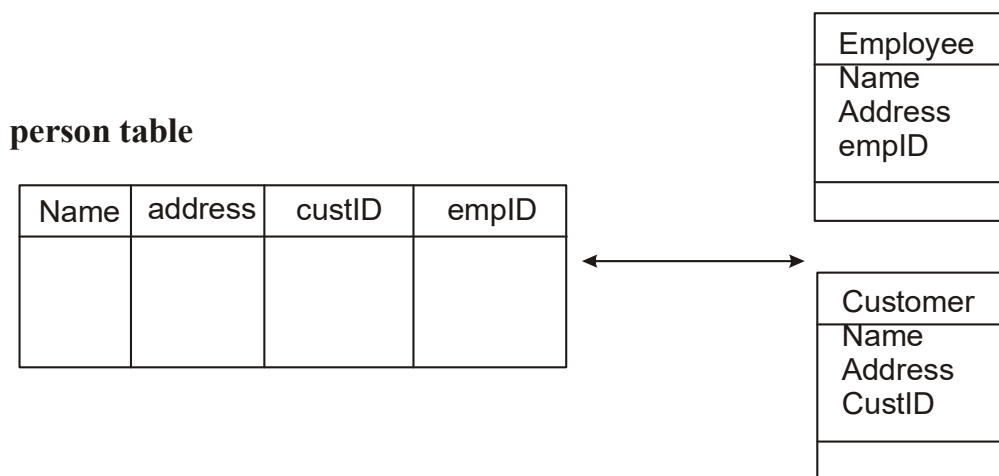


Figure 11.2

The custid column provides the discriminate. If the value for custid is null, an employee instance is created at run time otherwise a customer instance is created.

11.1.4 Table Inherited Classes Mapping

In table inherited classes mapping, a single table maps to many classes that have a common super class. This mapping allows the user to specify the columns to be shared among the related classes. The super class may be either abstract or instantiated. In figure 11.3, instances of salaried employee can be created for any row in the person table that has a non null value for the salary column. If salary is null the row is represented by an hourly employee instance.

11.1.5 Table-Inherited Classes Mapping

Another approach here is table inherited classes mapping, which allows the translation of a relationship that exists among tables in the relational schema into class inheritance relationship in the object model. In a relational database an

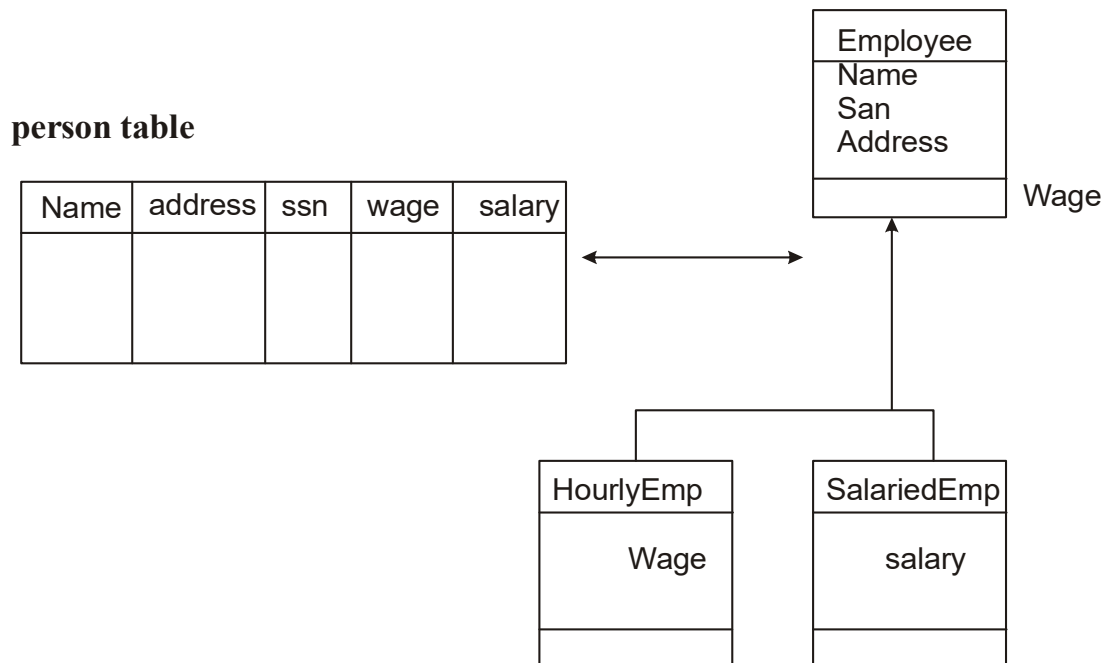


Figure 11.3

is a relationship often is modeled by a primary key that acts as a foreign key to another table. In the object model, is a is another term for an inheritance relationship figure 11.4 shows an example that maps a person table to class person and then a related employee table to class employee, which is a sub class of class person. In this example, instances of person are mapped directly from the person table. However instances of employee can be created only for the row in the employee table.

11.1.6 Keys for Instance navigation

In mapping columns to properties the simplest approaches is to translate a column's value into the corresponding class property value. There are two interpretation of this mapping: either the column is a data value or it defines a navigable relationship between instances. The mapping also should specify how to convert each data value into a property value on an instance. In addition to simple data conversion, mapping of column values defines the interpretation of relational foreign keys. The mapping describes both how the foreign key can be used to navigate among classes and instances in the mapped object model and how referential integrity is maintained. A foreign key defines a relationship between tables in a relational database. in an

object model this association is where objects can have references to other objects that enable instance to instance navigation

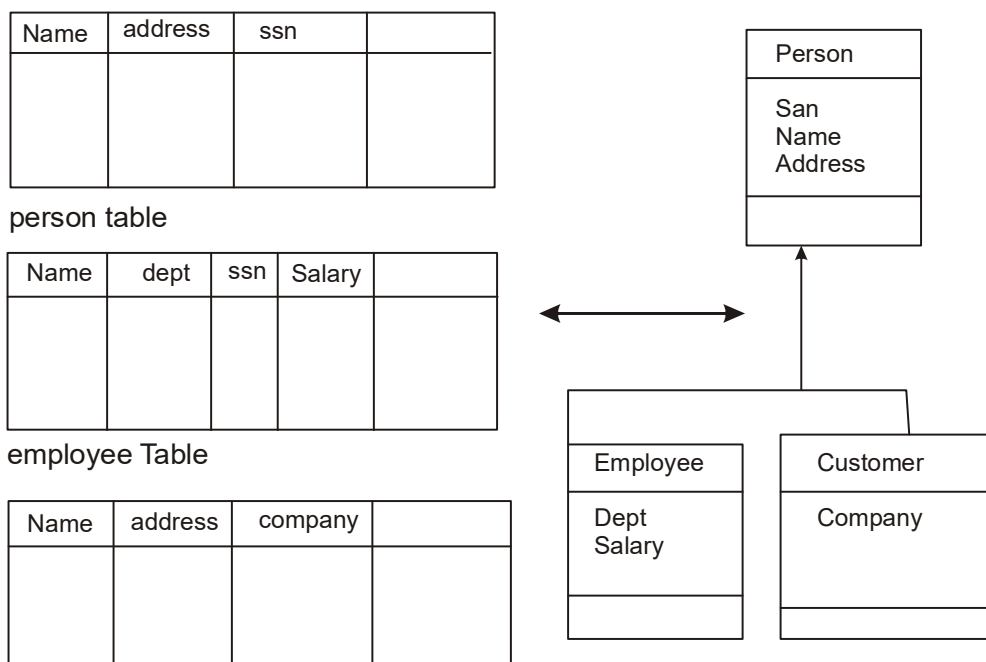


Figure 11.4

In figure 11.5 the departmentID property of employee uses the foreign key in column employee. DepartmentID. Each employee instance has a direct reference of class Department to the department object to which it belongs.

A popular mechanism in relational database is the use of stored procedures. As mentioned earlier stored procedures are modules of precompiled SQL code stored in the database that execute on the server to enforce rules the business has set about the data.

Mapping should support the use of stored procedures by allowing mapping of existing stored procedures to object methods.

11.2 Multi Database Systems

A different approach for integrating object oriented application with relational data environments is multi database systems or heterogeneous database systems which facilitate the integration of heterogeneous databases and other information sources.

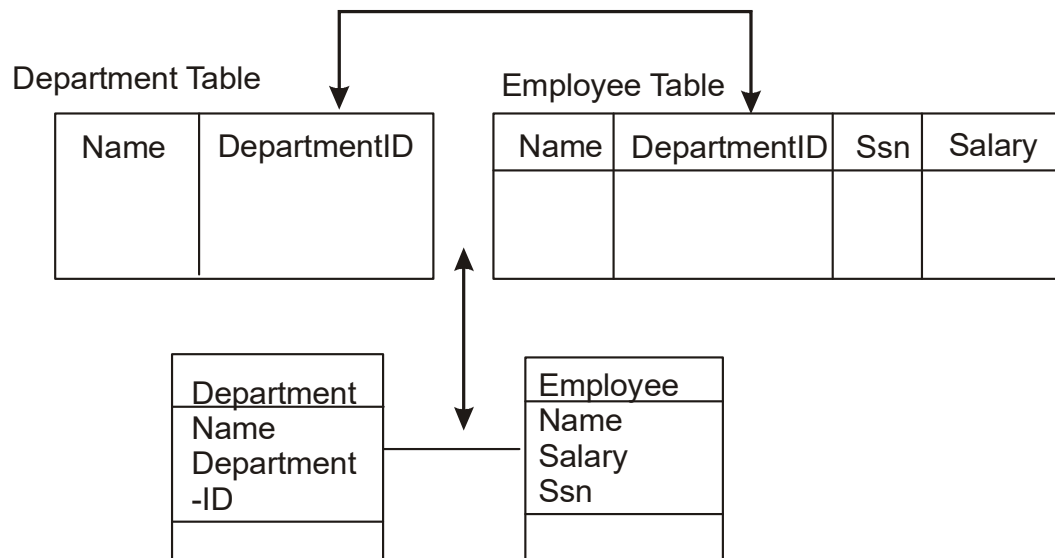


Figure 11.5

Heterogeneous information systems facilitate the integration of heterogeneous information source, where they can be structured, semi structure and sometimes even unstructured. Some heterogeneous information systems are constructed on a global schema over several database. Such heterogeneous information systems are referred to as federated multi database systems.

Federated multi database systems as a general solution to the problem of inter operating heterogeneous data systems provide uniform access to data stored in multiple databases that involve several different data models. A multi database system is a database system that resides unobtrusively on top of say existing relational and object databases and the file systems and present a single database illusion to its users see figure 11.6. The global schema is constructed by consolidating the schemata of local databases the schematic differences among them are handled by neutralization the process of consolidating the local schemata. The MDBS translates the global queries and updates for dispatch to the appropriate local database system for actual processing, merges the result from them and generates the final result for the user.

To summarize the distinctive characteristics of multi database systems.

- Automatic generation of a unified global database schema from local databases in addition to schema capturing and mapping for local databases.
- Provision of cross database functionality by using unified schemata

- Integration of heterogeneous database systems with multiple databases.
- Integration of data types other than relational data through the use of such tools as driver generators.
- Provision of a uniform but diverse set of interfaces to access and manipulate data stored in local databases.

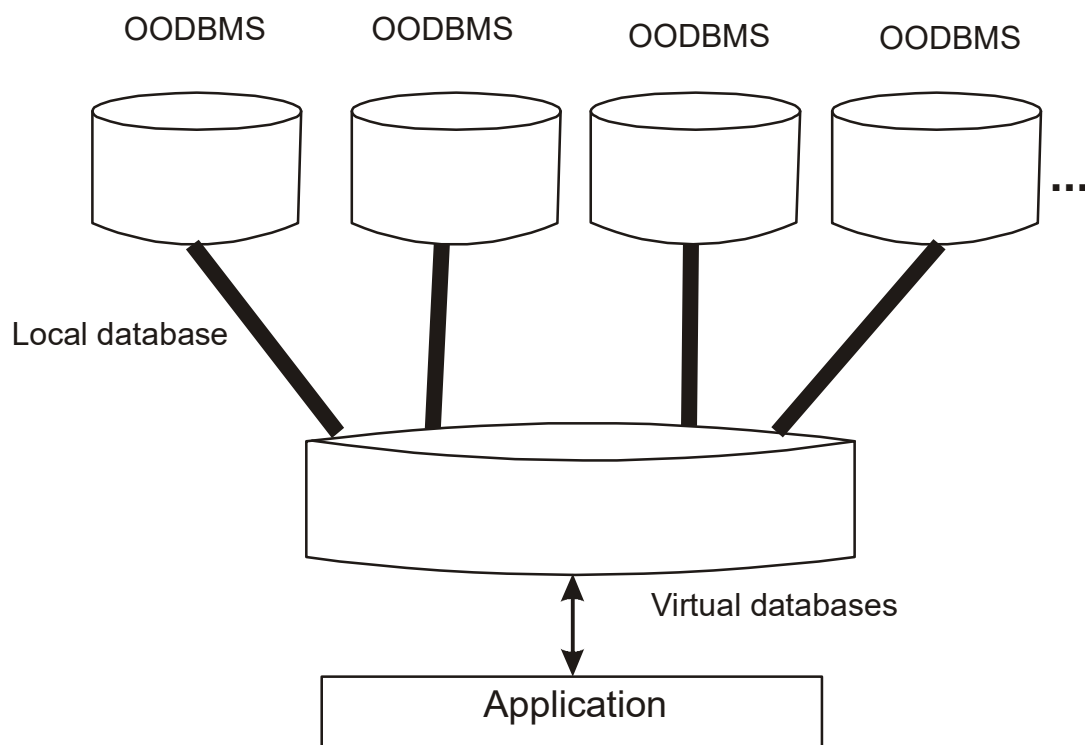


Figure 11.6

11.2.1 Open database Connectivity: Multidatabase Application Programming Interfaces

The benefits of being able to port database application by writing to an application programming interface(API) for a virtual DBMS are so appealing to so developers that the computer industry recently introduced several multidatabase APIs.

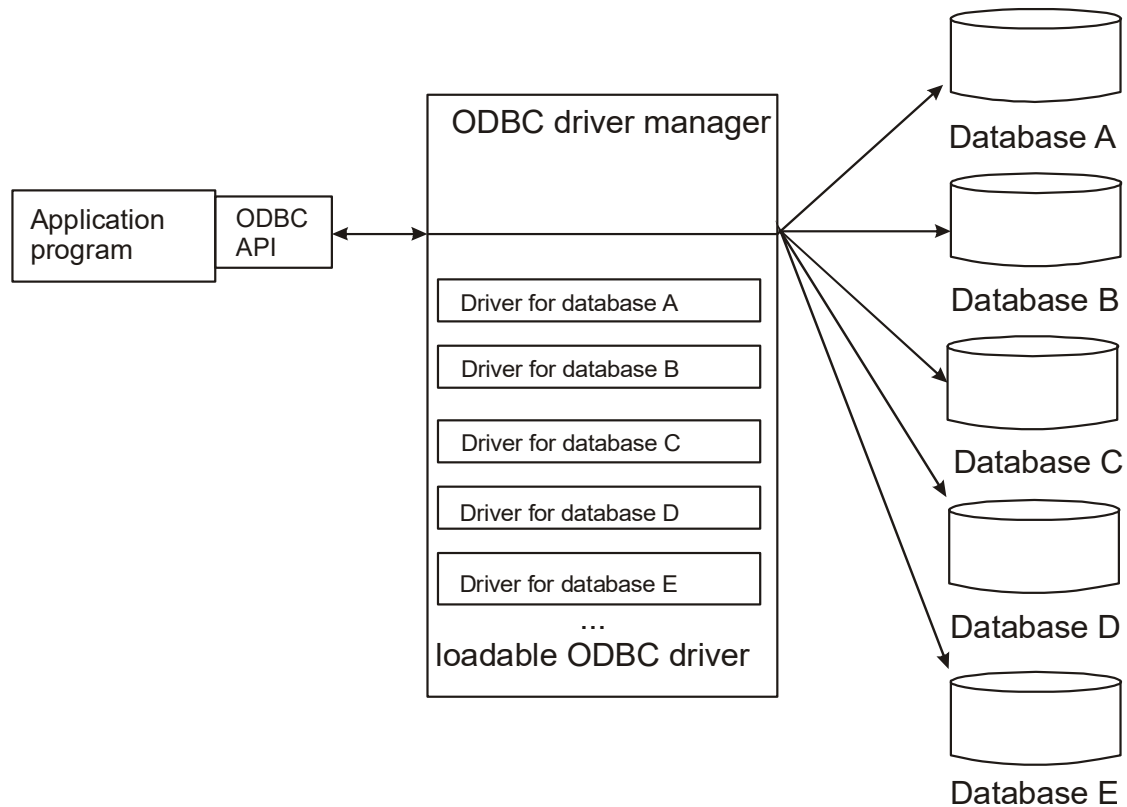


Figure 11.7

Developer use these call level interfaces for application that access multiple database using a single set of function calls, minimizing differences in application source code. Open database connectivity(ODBC) is an application programming interface that provides solutions to the multidatabase programming problem. ODBC and other APIs provide standard database access through a common client-server interface. As a standard ODBC has strong industry support.

Currently a majority of software and hardware vendors including bou crosoft and apple have endorsed ODBC as the database interoperability standard. In addition most database vendors either provide or will soon provide ODBC compliant interface.

The application interacts with the ODBC driver manager which sends the application calls to the database. The driver manager loads and unloads drivers, performs status checks and manages multiple connection between application and data sources see figure 11.7.

11.3 Designing Access Layer Classes

The main idea behind creating an access layer is to create a set of classes that know how to communicate with the place where the data actually reside. Regardless of where the data actually reside, whether it be a file relational database mainframe, internet, DCOM or via ORB, the access classes must be able to translate any data related requests from the business layer into the appropriate protocols for data access. Furthermore these classes also must be able to translate the data retrieved back into the appropriate business objects.

The access layer main responsibility is to provide a link between business or view objects and data storage. Three layer architecture in essence, is similar to three tier architecture. For example the view layer corresponds to the client tier, the business layer to the application server tier and the access layer to the database tier of three tier architecture see figure 11.8. the access layer performs two major tasks:

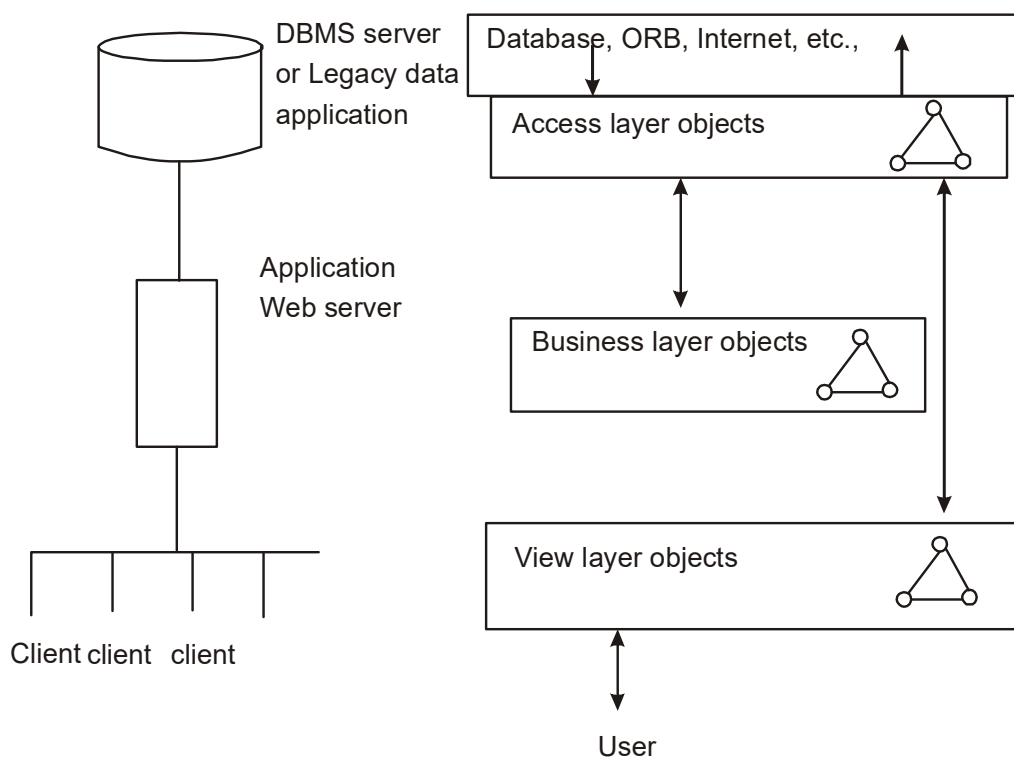


Figure 11.8

- Translate the request. The access layer must be able to translate any data related requests from the business layer into the appropriate protocols for data access.
- Translate the results. The access layer also must be able to translate the data retrieved back into the appropriate business objects and pass those objects back into business layer.

The main advantage of this approaches is that the design is not tied to any database engine or distributed object technology, such as CORBA or DCOM.

With this approaches, we very easily can switch from one database to another with no major changes to the user interface or business layer objects. All we need to change are the access classes methods. Other benefits of access layer classes are these:

- Access layer classes provide easy migration to emerging distributed object technology such as CORBA and DCOM.
- These classes should be able to address the modest needs of two tier client-server architectures as well as the difficult demands of fine grained peer to peer distributed object architecture.

Designing the access layer object is the same as for business layer objects and the same guidelines apply to access layer classes.

11.3.1 The Process

The access layer design process consists of the following activities see figure 11.9 and 11.10. If a class interacts with non human actor such as another system, database or the web, then the class automatically should become an access class. the process of creating an access class for the business classes we identified so far follows:

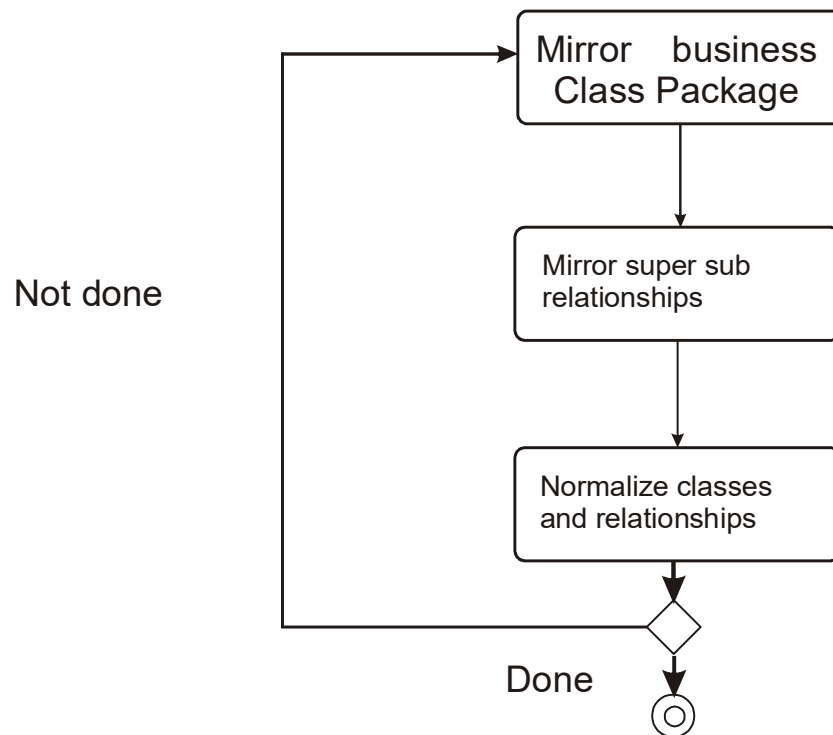
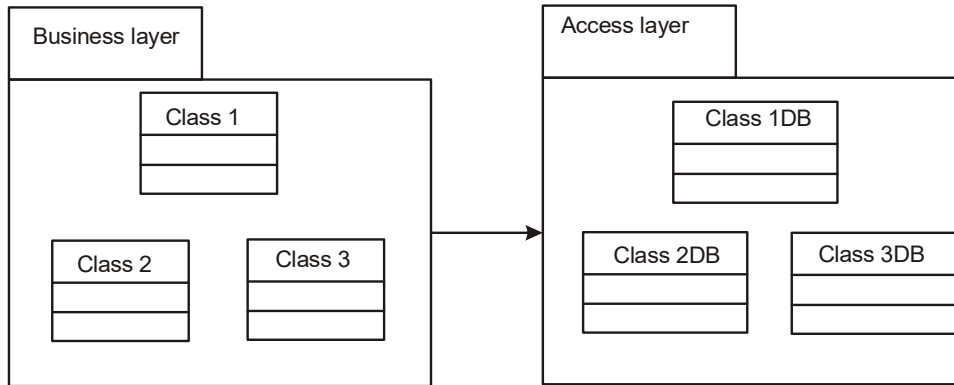


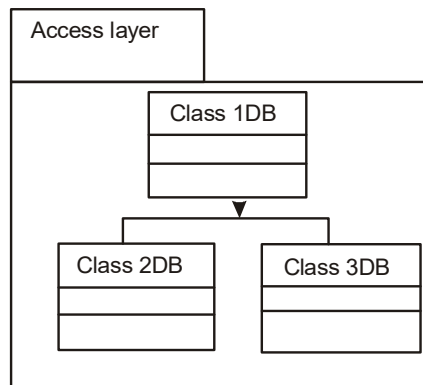
Figure 11.9

1. for every business class identified mirror the business class package. For every business class that has been identified and created, create one access class in the access layer package
2. define relationship. The same rule as Applying among business class objects also applies among access classes
3. simplify classes and relationship. The main goal here is eliminate redundant or unnecessary classes or structure.

step 1. mirror business class package



Step 2. Define relationship among access layer class



simplify classes and relationship

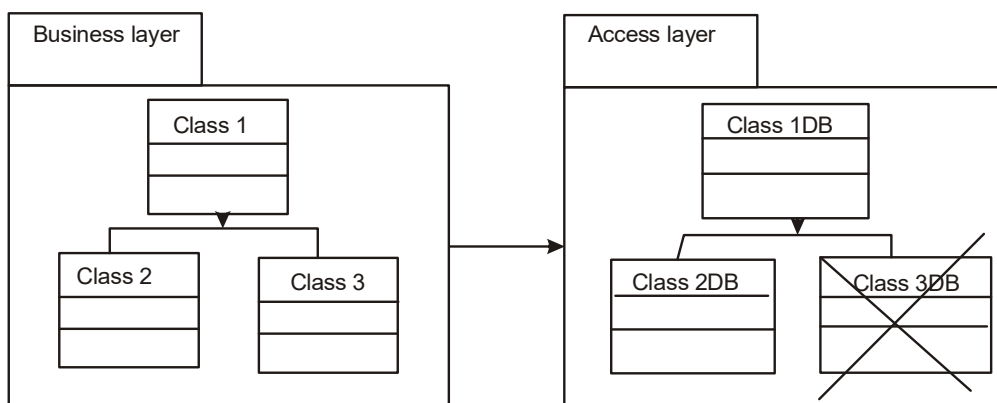


Figure 11.10

- 3.1 Redundant classes. If you have more than one class that provides similar services, simply select one and eliminate the other
- 3.2 Method classes. Revisit the classes that consists of only one or two methods to see if they can be eliminated or combined with existing classes. If you can no class from access layer package, select class from the business package and add the method as a private method to it.
4. Iterate and refine.

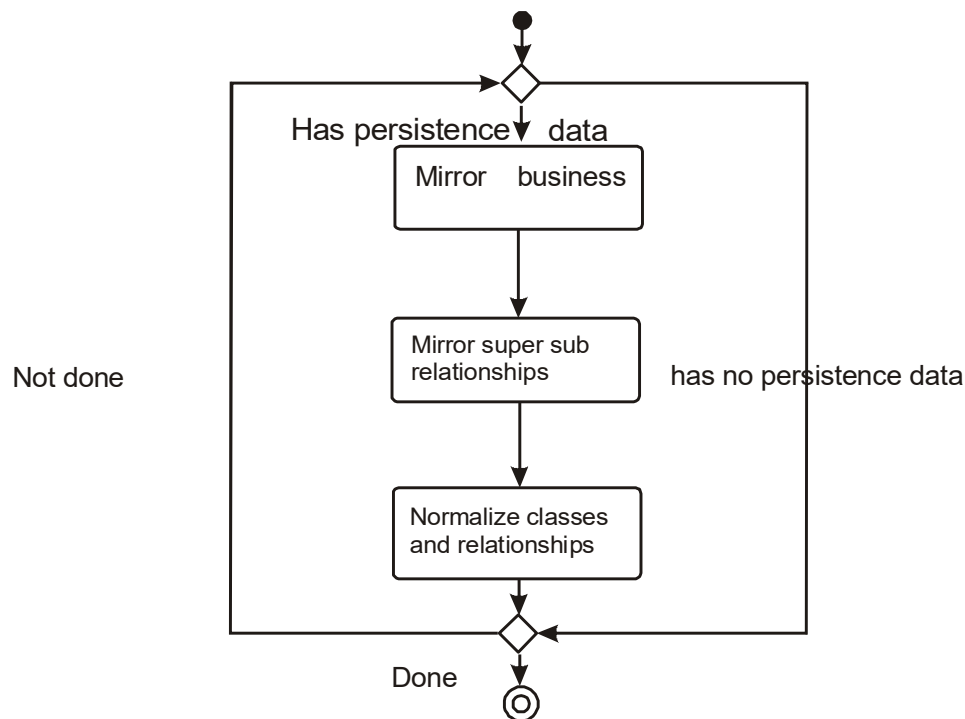


Figure 11.11

Another approaches is to let the methods be stored in program and store only the persistence attributes. Here is the modified process:

1. for every business class identified see figure 11.11,determine if the class has persistence data. An attributes can be either transient or persistence. An attributes is transient if the following condition exist. Temporary storage for an expression evaluation or its value can be dynamically allocated

2. Mirror the business class package. Every business class identified and created, create one access class in the access layer package.
3. define relationship. The same rule as applies among business class objects also applies among access classes.
4. simplify classes and relationship. The main goal here is to eliminate redundant or unnecessary classes and structures. In most cases you can combine simple access classes and simplify the super and sub class structures.
 - 4.1 redundant classes. If you have more than one class that provides similar services, simply select one and eliminate the other.
 - 4.2 method classes. Revisit the classes that consists of only one or two methods to see if they can be eliminated or combined with existing classes.
5. iterate and refine.

In either case, once an access class has been defined, all you need do is make it a part of its business class see figure 11.12.

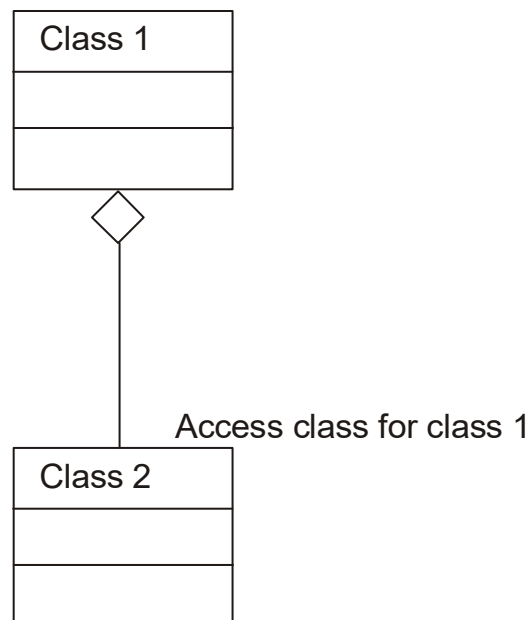


Figure 11.12

11.4 Case Study: Designing The Access Layer for The ViaNet bank ATM

We are ready to develop the access layer for the via net bank ATM. Remember that the main idea behind an access layer is to create a set of classes that know how to communicate with the data source. They are simply mediators between business or view classes and storage places or they communicate with other object over a network through the ORB/DCOM in the case of distributed objects.

11.4.1 Creating an Access Class for the Bank Client Class

Here we apply the access layer design process to identify the access classes.

Step 1. determine if a class has persistence data.

Step 2. mirror the business class package. Since the bank client has persistence attributes, we need to create an access class for it.

Step 3. define relationship.

The bank client class has the following attributes

First name

Last name

Card number

Pin number

Account

To link the bank client table to the account table we need to use card number as a primary key in both tables see figure 11.13. in other words it must update and retrieve the bank client attributes by translating any data related request from the bank client class into the appropriate protocols for data access. Notice that retrieve client method of bank client object simply sends a message to the bankDB object to get the client information.

Listing 1.

```
Bankclient::+retrieveClient(aCardnumber, APIN):BankClient
```

```
ABankDb: BankDB
```

```
ABankDB.retrieveClient(aCardNumber,aPIN)
```

In here all we need to do is to create an instance of the access class bankDB and then send a message to it to get information on the client object.

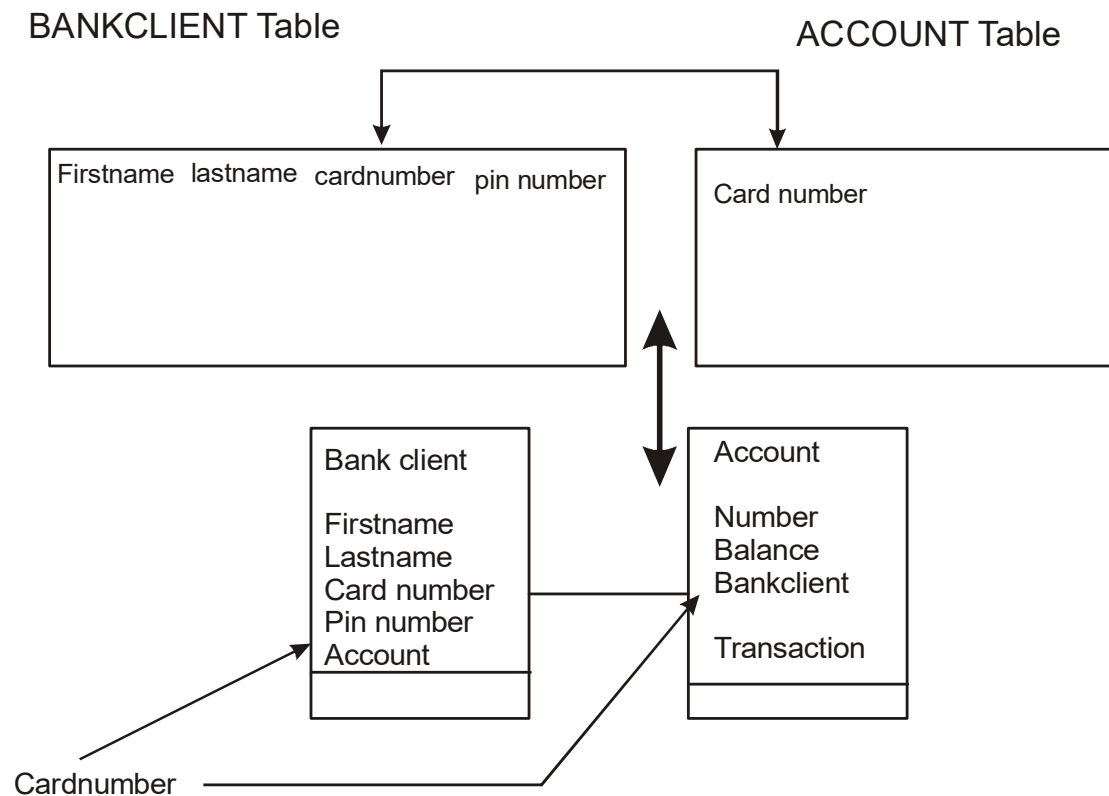


Figure 11.13

Listing 2.

```
BankDB::+retrieveClient(aCardnumber, APIN):BankClient
```

```
SELECT firstname, lastname FROM BANKCLIENT WHERE (cardNumber=aCardNumber
and pinNumber=apin)
```

The retrieve client return type is defined as bankclient to return the attributes of the bank client. Access class methods are highly language dependent the update client method updates or change attributes such as pin Number, firstname, lastname. Here again just like the retrieve client method.

Listing 3.

Bank+update Client(a Client Bank Client, a Card Number:string)

UPDATE BANKCLIENT

SET firstname=aclient.firstname

SET lastname=aClient.lastName

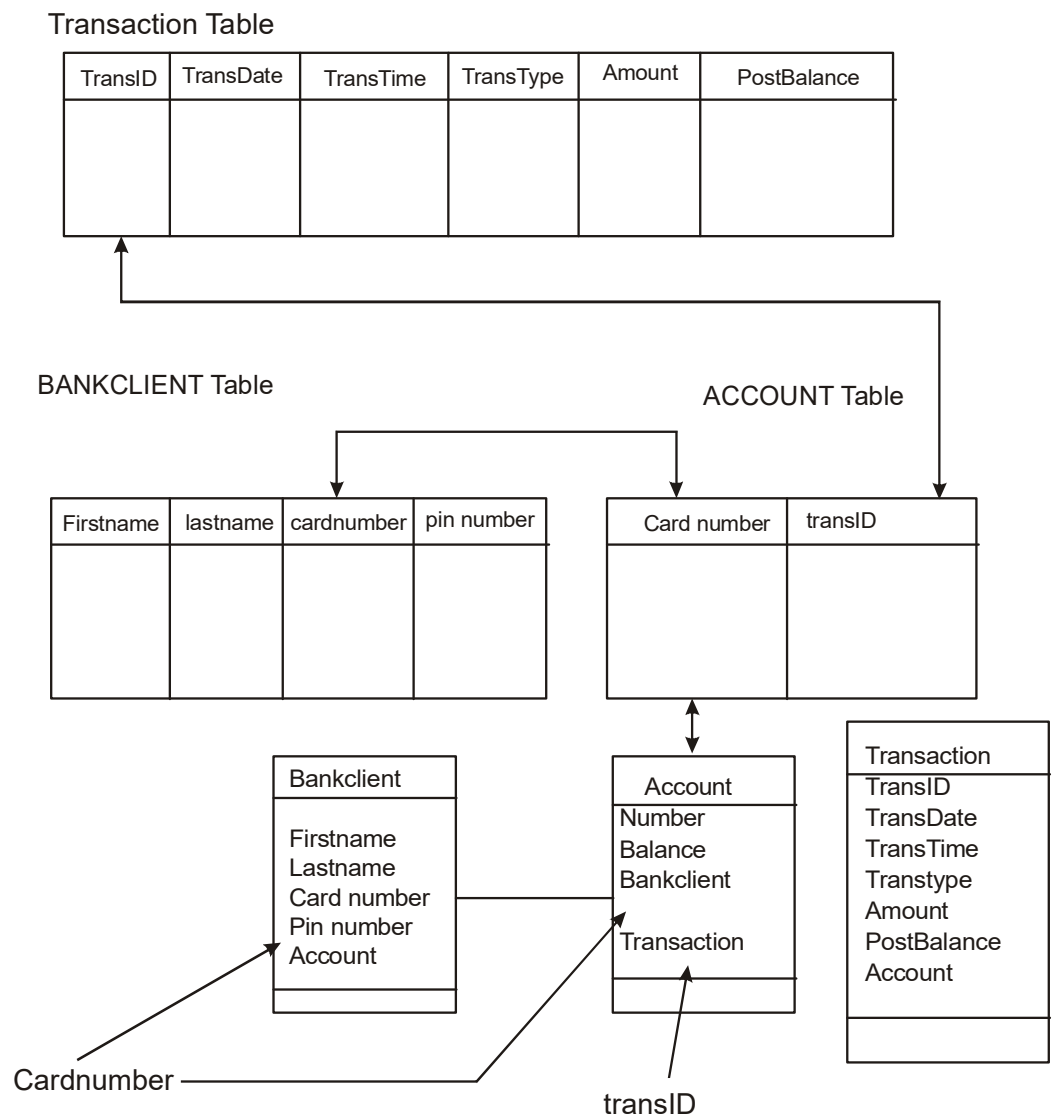


Figure 11.14

SET pinNumber Client pinNumber

WHERE cardNumber "acrad number")

The account class has following attributes

Number

Balance

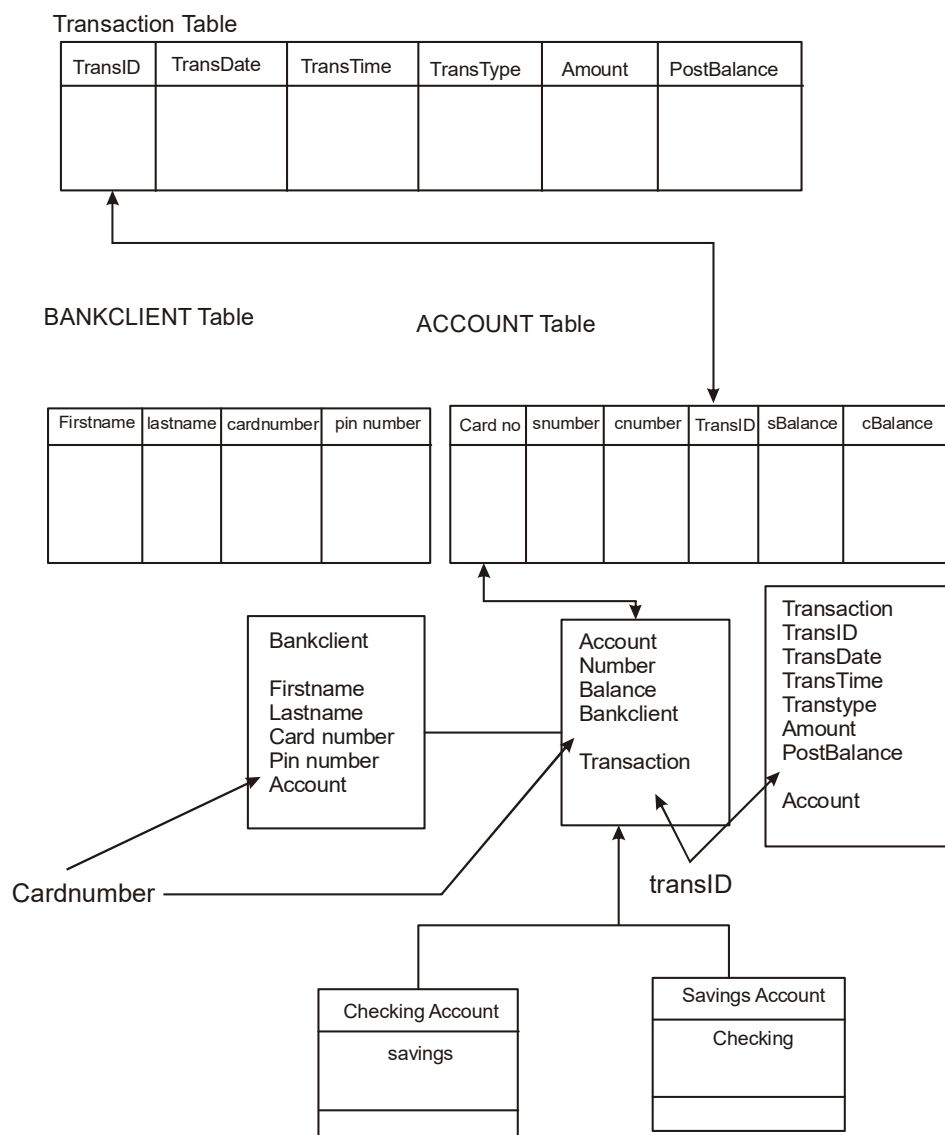


Figure 11.15

BankClient

Transaction

Attributes such as number and balance are persistence. The bank client attributes is transient and is used for implementing between the account and bank client classes. However to link the account table to the transaction table we need to add the transID as a foreign key to account table see figure 11.14. figure 11.15 shows how generalization relationship among the account checking account, and saving account classes have been represented in our relationship database. According to step 2 we need to add three more access classes. One for the account class one for the checking account class and one for the saving account class.

The account class is an abstract class has no instances since the accounts are either savings or checking account and therefore needs no additional methods. The methods are

UpdateSaving Account

retrieve Saving Account

updateCheking Account

retrieveChecking Account

However here we add the method to the BankDb class. After all an Account is associated with a bankclient class and most times account and client information need to be accessed at the same time. Here Saving Account retrieveAccount and saving Account::updateAccount method send messages to the access class BankDB::retrieveSaving Account BankDB updateSaving Account methods to perform the job. and

Listing 4.

Saving Account-retrieveAccount() Account

BankDB retrieveSavings Account(bankClient card Numbernumber)

The retrieve Account is an example of polymorphism, where it is overloading its super class method by sending a message to the access class BankDb object to retrieve the savings account information.

Listing 5.

```

BankDB:+retrieveSavings Account(aCardNumber string, savingsNumber:string): Account

SELECT sBalance, transID,

FROM ACCOUNT

WHERE cardNumber=aCardNumber and sNumber= savingsNumber)

```

Listing 6.

```

Checking Account-retrieve Account Account

BankDB retrieveChecking Account(bankClient.cardNumber number)

```

Listing 7.

```

Bank DB::+retrieve Checking Account (a Card Number string, Checking Number:string):
Account

SELECT cBalance,transID,

FROM ACCOUNT

WHERE cardNumber=aCardNumber and cNumber= checking Number)

```

Listing 8.

```

savings Account::-updateAccount(): Account

Bank DB. update savings Account (bank Client. card Number, number, balance)

```

Listing 9.

```

BankDB:: + update savings (a Cardpin Number: string, a Number: string, new Balance:
float): Account

UPDATE ACCOUNT

SET sBalance newbalance

WHERE cardNumber=aCardNumber and sNumber=aNumber)

```

Listing 10.

```
checkingAccount::updateAccount(): Account
```

```
Bank DB. up date checking Account (bank Client. card Number, number, balance)
```

Listing 11.

```
BankDB::+ update checking (a Cardpin Number: string, a Number: string, new Balance: float): Account
```

```
UPDATE ACCOUNT
```

```
SET cBalance newbalance
```

```
WHERE cardNumber=aCardNumber and cNumber aNumber)
```

Figure 11.16 depicts the relationship among the classes we have designed so far especially the relationship among the access class and other business classes.

Summary

- Many application increasingly are developed using object oriented programming technology.
- A relational data must to resolve such a mismatch, the application object and the relational data must be mapped.
- A different approaches for integrating object oriented applications with relational data environments involves multidatabase system or heterogeneous database system.
- The main idea behind an access layer is to create a set of classes that know how to communicate with a data source.
- Access layer classes provide easy migration to emerging distributed object technology, such as CORBA and DCOM.
- They should be able to address the modest needs of two tier client-server architectures as well as the difficult demands of fine grained, peer to peer distributed object architectures.

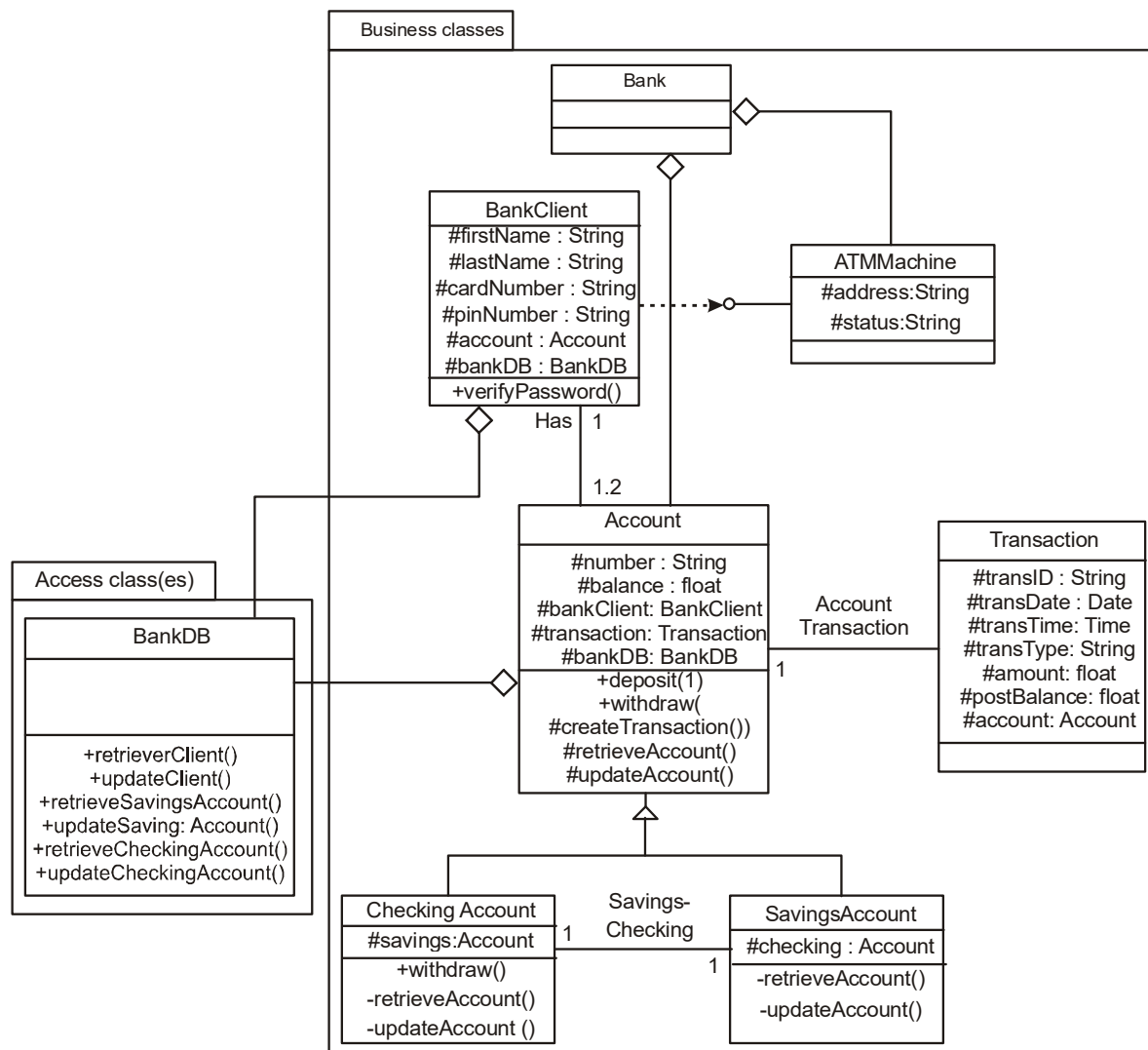


Figure 11.16

Questions

1. Describe client-server computing.
2. Describe reverse and forward engineering
3. Describe a federated multi database system
4. Describe the process of creating the access layer classes.
5. What is an OODBMS? Describe the difference between an OODBMS and object oriented programming.
6. Describe CORBA, ORB, and DCOM.
7. What is different types of servers? Briefly describe each.
8. What is query?

LESSON - 12

VIEW LAYER DESIGNING INTERFACE OBJECTS

Objectives:

After studying this lesson you should be able to

- Identifying view classes.
- Designing interface objects.
- User interface design as a creative process
- Designing view layer classes
- Macro level process

12.1 Introduction

Once the analysis is complete (and sometimes concurrently), we can start designing the user interfaces for the objects and determining how these objects are to be presented. The main goal of a user interface (UI) is to display and obtain needed information in an accessible, efficient manner.

The design of the software's interface, more than anything else, affects how a user interacts and therefore experiences an application [5]. It is important for a design to provide users the information they need and clearly tell them how to successfully complete a task. A well-designed UI has visual appeal that motivates users to use your application. In addition, it should use the limited screen space efficiently.

In this chapter, we learn how to design the view layer by mapping the UI objects to the view layer objects, we look at UI design rules based on the design corollaries, and finally, we look at the guidelines for developing a graphical user interface. A graphical user interface (GUI) uses icons to represent objects, a pointing device to select operations, and graphic imagery to represent relationships. See end of this lesson for a review of Windows and graphical user interface basics and treatments.

12.2 User Interface Design as a Creative Process

Creative thinking is not confined to a particular field or a few individuals but is possessed in varying degrees by people in many occupations: The artist sketches, the journalist promotes an idea, the teacher encourages student development, the scientist develops a theory, the manager implements a new strategy, and the programmer develops a new software system or improves an existing system to create a better one.

Creativity implies newness, but often it is concerned with the improvement of old products as much as with the creation of a new one. For example, newly created software must be useful, it should be of benefit to people, yet should not be so much of an innovation that others will not use it. A “how to make something better” attitude, tempered with good judgment, is an essential characteristic of an effective, creative process.

By bringing together, in the mind, various combinations of known objects or situations, we are using inventive imagination to develop new products, systems, or designs. It is not necessary to visualize absolutely new objects or to go beyond the

bounds of our own experience. Inventive imagination can take place simply by putting together known materials (objects) in a new way. Therefore, a developer might conceive new software by using inventive imagination to combine objects already in his or her mind to satisfy user needs and requirements. As an example of this, see the Real-World Issues on Agenda “Toward an ObjectOriented User Interface”.

Is creative ability born in an individual or can someone develop this ability? Both parts of this question can be answered in the affirmative. Certainly, some people are born with more creativity than others, just as certain people are born with better skills (athletes, artists, etc.) in some areas, than others. Just as it is possible to develop mental and physical skills through study and practice, it is possible to develop and improve one’s creative ability.

To view user interface design as a creative process, it is necessary to understand what the creative process really involves. The creative process, in part, is a combination of the following:

1. A curious and imaginative mind
2. A broad background and fundamental knowledge of existing tools and methods.

3. An enthusiastic desire to do a complete and thorough job of discovering solutions once a problem has been defined.
4. Being able to deal with uncertainty and ambiguity and to defer premature closure.

One aid to development or restoration of curiosity is to train yourself to be observant. You must be observant of any software that you are using. You must ask how or from what objects or components the user interface is made, how satisfied the users are with the UI, why it was designed using particular controls, why and how it was developed as it was, and how much it costs. These observations lead the creative thinker to take its place.

12.3 Designing view Layer Classes

An implicit benefit of three-layer architecture and separation of the view layer from the business and access layers is that, when you design the UI objects, you have to think more explicitly about distinctions between objects that are useful to users. A distinguishing characteristic of view layer objects or interface objects is that they are the only exposed objects of an application with which users can interact. After all, view layer classes or interface objects are objects that represent the set of operations in the business that users must perform to complete their tasks, ideally in a way they find natural, easy to remember, and useful. Any objects that have direct contact with the outside world are visible in interface objects, whereas business or access objects are more independent of the two major aspects of the application.

1. Input responding to user interaction. The user interface must be designed to translate an action by the user, such as clicking on a button or selecting from a menu, into an appropriate response. That response may be to open or close another interface or to send a message down into the business layer to start some business process. Remember, the business logic does not exist here, just the knowledge of which message to send to which business object.
2. Output-displaying or printing business objects. This layer must paint the best picture possible of the business objects for the user. In one interface, this may mean entry fields and list boxes to display an order and its items. In another, it may be a graph of the total price of a customer's orders.

The process of designing view layer classes is divided into four major activities:

1. The macro level UI design process-identifying view layer objects. This activity, for the most part, takes place during the analysis phase of system development. The main objective of the macro process is to identify classes that interact with human actors by analyzing the use cases developed in the analysis phase. As described in previous chapters, each use case involves actors and the task they want the system to do. These use cases should capture a complete, unambiguous, and consistent picture of the interface requirements of the system. After all, use cases concentrate on describing what the system does rather than how it does it by separating the behavior of a system from the way it is implemented, which requires viewing the system from the user's perspective rather than that of the machine. However, in this phase, we also need to address the issue of how the interface must be implemented. Sequence or collaboration diagrams can help by allowing us to zoom in on the actorsystem interaction and extrapolate interface classes that interact with human actors; thus, assisting us in identifying and gathering the requirements for the view layer objects and designing them.
2. **Micro level UI design activities:**
 - 1.1 Designing the view layer objects by applying design axioms and corollaries. In designing view layer objects, decide how to use and extend the components so they best support application-specific functions and provide the most usable interface.
 - 1.2 Prototyping the view layer interface. After defining a design model, prepare a prototype of some of the basic aspects of the design. Prototyping is particularly useful early in the design process.
3. Testing usability and user satisfaction. "We must test the application to make sure it meets the audience requirements. To ensure user satisfaction, we must measure user satisfaction and its usability along the way as the UI design takes form Usability experts agree that usability evaluation should be part of the development process rather than a postmortem or forensic activity. Despite the importance of usability and user satisfaction, many system developers still fail to pay adequate attention to usability, focusing primarily on functionality" [4, pp. 61-62]. In too many cases, usability still is not given adequate consideration. Adoption of usability in the later stages of the life cycle will not produce sufficient improvement of overall quality. We will study how to develop user satisfaction and usability in Chapter 14.

4. Refining and iterating the design

12.4 Level Process: Identifying view classes by analyzing use cases

The interface object handles all communication with the actor but processes no business rules or object storage activities. In essence, the interface object will operate as a buffer between the user and the rest of the business objects [3]. The interface object is responsible for behavior related directly to the tasks involving contact with actors. Interface objects are unlike business objects, which lie inside the business layer and involve is an example of a business object service. However, the data entry for the employee overtime is an interface object.

Jacobson, Ericsson, and Jacobson explain that an interface object can participate in several use cases. Often, the interface object has a coordinating responsibility in the process, at least responsibility for those tasks that come into direct contact with the user. As explained in earlier chapters, the first step here is to begin with the use cases, which help us to understand the users' objectives and tasks. Different users have different needs;

For example, advanced, or "power," users want efficiency whereas other users may want ease of use. Similarly, users with disabilities or in an international market have still different requirements. The challenge is to provide efficiency for advanced users without introducing complexity for less-experienced users. However, developing use cases for advanced as well as less-experienced users might lead you to solutions such as shortcuts to support more advanced users.

The view layer macro process consists of two steps:

1. For every class identified (see Figure 12-1), determine if the class interacts with a human actor. If so, perform the following, otherwise, move to the next class.
 - 1.1 Identify the view (interface) objects for the class. Zoom in on the view objects by utilizing sequence or collaboration diagrams to identify for this class.
 - 1.2 Define the relationships among the view (interface) objects. The interface objects, like access classes, for the most part, are associated with the business classes. Therefore, you can let business classes guide

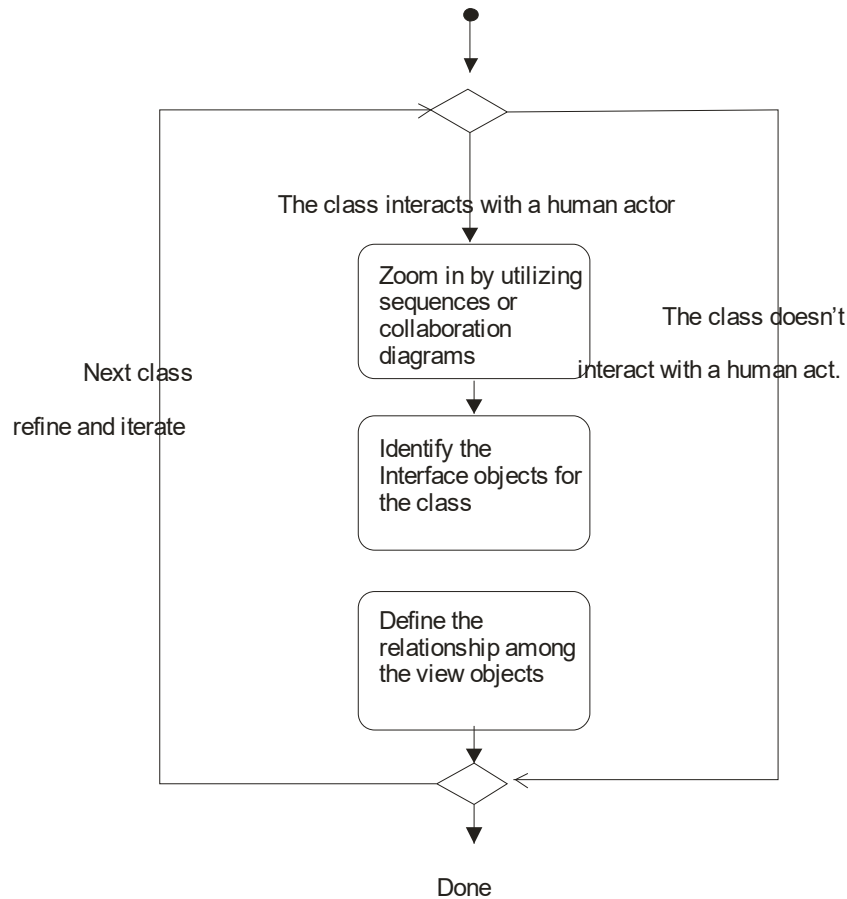


Figure 12.1

- 1.3 you in defining the relationships among the view classes. Furthermore, the same rule as applies in identifying relationships among business class objects also applies among interface objects

2. Iterate and refine

The advantage of utilizing use cases in identifying and designing view layer objects is that the focus centers on the user, and including users as part of the planning and design is the best way to ensure accommodating them. Once the interface objects have been identified, we must identify the basic components or objects used in the user tasks and the behavior and the characteristics that differentiate each kind of object, including the relationships of interface objects to each other and to the user.

Also identify the actions performed, the objects to which they apply, and the state information or attributes that each object in the task must preserve, display, and allow to be edited.

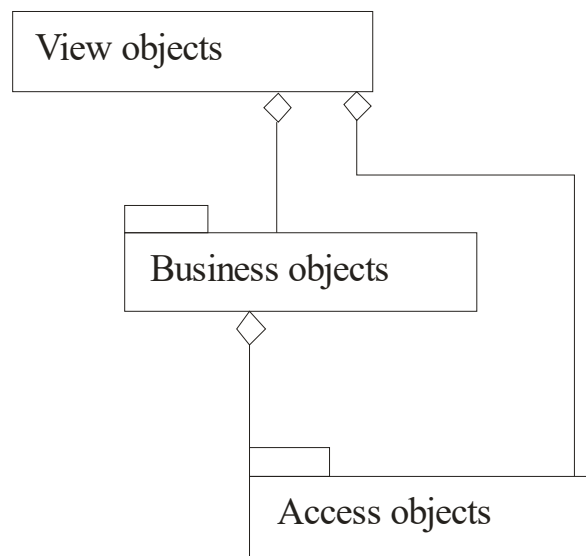


Figure 12.2

Figure 12-2 shows the relationships among business, access, and view layer objects. The relationships among view class and business class objects is opposite of that among business class and access class objects. After all the interface object handles all communication with the user but does not process any business rules; that will be done by the business objects.

Effective interface design is more than just following a set of rules. It also involves early planning of the interface and continued work through the software development process. The process of designing the user interface involves clarifying the specific needs of the application, identifying the use cases and interface objects, and then devising a design that best meets users' needs. The remainder of this chapter describes the micro-level UI design process and the issues involved.

12.5 Micro-Level Process

To be successful, the design of the view layer objects must be user driven or user centered. A user-centered interface replicates the user's view of doing things by providing the outcomes

users expect for any action. For example, if the goal of an application is to automate what was a paper process, then the tool should be simple and natural. Design your application

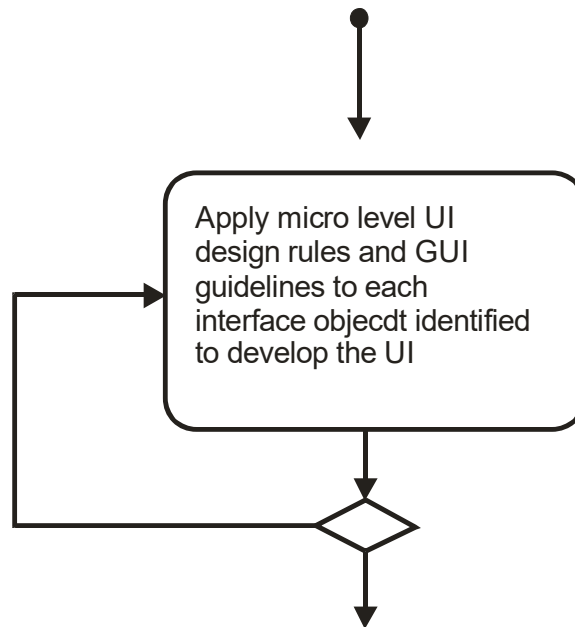


Figure 12.3

So it allows users to apply their previous real-world knowledge of the paper process to the application interface. Your design then can support this work environment and goal. After all, the main goal of view layer design is to address users' needs.

1. For every interface object identified in the macro UI design process (see Figure 12-3), apply micro-level UI design rules and corollaries to develop the UI. Apply design rules and GUI guidelines to design the UI for the interface objects identified.
2. Iterate and refine.

In the following sections, we look at the three UI design rules based on the design axioms and corollaries.

12.5.1 UI Design Rule.1 Making the Interface Simple (Application of Corollary 2)

First and foremost, your user interface should so simple that users are unaware of the tools and mechanisms that make the application work. As application become more complicated,

users must have an even simpler interface, they can learn new applications more easily. To days cars are also complex that they have onboard computers and sophisticated electronics. However, the driver interface remains simple: The driver needs only a steering wheel and the gas and brake pedals to operate a car. Drivers do not have to understand what is under the hood or even be aware of it to drive a car, because the driver interface remains simple. The UI should provide the same simplicity for users.

This rule is an application of Corollary 2 (single purpose, see Chapter 9) in UI design. Here, it means that each UI class must have a single, clearly defined purpose. Similarly, when you document, you should be able easily to describe the purpose of the UI class with a few sentences. Furthermore, we have all heard the acronym KISS (Keep It Simple, Stupid). Maria Capacitate, an expert in user interface design and standards, has a better acronym for KISS: Keep It Simple and Straightforward. She says that, once you know what fields or choices to include in your application, ask your self if they really are necessary. Labels, static text, check boxes, group boxes, and option buttons often clutter the interfaces and take up twice the room mandated by the actual data. If a user cannot sit before one of your screens and figure out what to do without asking a multitude of questions, your interface is not simple enough; ideally, in the final product, all the problems will have been solved.

A number of additional factors may affect the design of your application. For example, deadlines may require you to deliver a product to market with a minimal design process, or comparative evaluations may force you to consider additional features. Remember that additional features and shortcuts can affect the product,. There is no simple equation to determine when a design tradeoff is appropriate. So in evaluating the impact, consider the following:

- ❖ Every additional feature potentially affects the performance, complexity, stability, maintenance, and support costs of an application.
- ❖ It is harder to fix a design problem after the release of a product because users may adapt, or even become dependent on, a peculiarity in the design.
- ❖ Simplicity is different from being simplistic. Making something simple to use often requires a good deal of work and code.
- ❖ Features implemented by a small extension in the application code do not necessarily have a proportional effect in a user interface. For example, if the primary task is selecting a single object, extending it to support selection of multiple objects could

make the frequent, simple task more difficult to carry out. Designing a UI based on its purpose will be explained in the next section.

12.5.2 UI Design Rule 2. Making the Interface Transparent and Natural (Application of Corollary 4)

The user interface should be so intuitive and natural that users can anticipate what to do next by applying their previous knowledge of doing tasks without a computer. An application, therefore, should reflect a real-world model of the users goals and the tasks necessary to reach those goals.

The second UI rule is an application of Corollary 4 (strong mapping) in UI design. Here, this corollary implies that there should be strong mapping between the user's view of doing things and UI classes. A metaphor, or analog, relates two otherwise unrelated things by using one to denote the other (such as a question mark to label a Help button). For example, writers use metaphors to help readers understand a conceptual image or model of the subject.

This principle also applies to UI design. Using metaphors is a way to develop the users conceptual model of an application. Familiar metaphors can assist the users to transfer their previous knowledge from their work environment to the application interface and create a strong mapping between the users view and the UI objects. You must be careful in choosing a metaphor to make sure it meets the expectations users have because of their realworld experience. Often an application design is based on a single metaphor. For example, billing, insurance, inventory, and banking applications can represent forms that are visually equivalent to the paper forms users are accustomed to seeing.

The UI should not make users focus on the mechanics of an application. A good user interface does not bother the user with mechanics. Computers should be viewed as a tool for completing tasks, as a car is a tool for getting from one place to another. Users should not have to know how an application works to get a task done, as they should not have to objectives know how a car engine works to get from one place to another. A goal of user interface design is to make the user interaction with the computer as simple and natural as possible.

12.5.3 UI Design Rule 3. Allowing Users to Be in Control of the Software (Application of Corollary 1)

The third UI design rule states that the users always should feel in control of the software, rather than feeling controlled by the software. This concept has a number of implications. The first implication is the operational assumption that actions are started by the user rather than the computer or software, that the user plays an active rather than reactive role. Task automation and constraints still are possible, but you should implement them in a balanced way that allows the user freedom of the choice.

The second implication is that users, because of their widely varying skills and preferences, must be able to customize aspects of the interface. The system software provides user access to many of these aspects. The software should reflect user settings for different system properties such as color, fonts or other options.

The final implication is that the software should be as interactive and responsive as possible. Avoid modes whenever possible. A mode is a state that excludes general interaction or otherwise limits the user to specific interactions,

Users are in control when they are able to switch from one activity to another, change their minds easily, and stop activities they no longer want to continue. Users should be able to cancel or suspend any time-consuming activity without consuming activity without causing disastrous results. There are situations in which modes are useful; for example, selecting a file name before opening it. The dialog that gets me the file name must be modal (more on this later in the section).

This rule is a subtle but important application of corollary 1 (uncoupled design with less information content) in UI design. It implies that the UI object should represent, at most one business object perhaps just some services of that business object. The main idea here is to avoid creating a single UI class for several business objects, since it makes the UI less flexible and forces the user to perform tasks in a monolithic way. Some of the ways to put users in control are these.

- Make the interface forgiving.
- Make the interface visual
- Provide immediate feedback.

- Avoid modes.
- Make the interface consistent.

12.5.1 Make the Interface Forgiving

The users actions should be easily reversed. When users are in control, they should be able to explore without fear of causing an irreversible mistake. Users like to explore an interface and often learn by trial and error. They should be able to back up or undo previous actions. An effective interface allows for interactive discovery. Actions that are destructive and may cause the unexpected 305

loss of data should require a confirmation or, better, should be reversible or recoverable. Even within the best designed interface, users can make mistakes.

These mistakes can be both physical (accidentally pointing to the wrong command or data) and mental (making a wrong decision about which command or data to select). An effective design avoids situations that are likely to result in errors, It also accommodates potential user errors and makes it easy for the user to recover. Users feel more comfortable with a system when their mistakes do not cause serious or irreversible results.

12.5.2 Make the Interface Visual

Design the interface so users can see, rather than recall, how to proceed. Whenever possible, provide users a list of items from which to choose, instead of making them remember valid choices.

12.5.3 Provide Immediate Feedback

Users should never press a key or select an action without receiving immediate visual or audible feedback or both. When the cursor is on a choice, for example, the color, emphasis, and selection indicators show users they can select that choice. After users select a choice, the color emphasis, and selection indicators change to show users their choice is selected.

12.5.4 Avoid Modes

Users are in a mode whenever they must cancel what they are doing before they can do something else or when the same action has different results in different situations. Modes force users to focus on the way an application works, instead of on the task they want to

complete. Modes, therefore interfere with users ability to use their conceptual model of how the application should work. It is not always possible to design a modeless application; however, should work. It is not always possible to

Modal dialog. Sometimes an application needs information to continue, such as the name of a file into which users want to save something. When an error occurs, users may be required to perform an action before they continue their tasks. The visual cue for modal dialog is a color boundary for the dialog box that contains the modal dialog.

Spring-loaded modes. Users are in a spring-loaded mode when they continually must take some action to remain in that mode; for example, dragging the mouse with a mouse button pressed to highlight a portion of text, In this case, the visual cue for the other operations such as Cut Paste.

Tool-driven modes. If you are in a drawing application, you may be able to choose a tool, such as a pencil or a paintbrush, for drawing. After you select the tool, the mouse pointer shape changes to match the selected tool. You are in a mode, but you are not likely to be confused because the changed mouse pointer is a constant reminder you are in a mode.

12.5.5 Make the Interface Consistent

Consistency is one way to develop and reinforce the user's conceptual model of applications and give the users the feeling that he or she is in control, since the user can predict the behavior of the system. User interface should be consistent throughout the applications; for example, using a consistent user interface for the inventory application.

12.6 Graphical user interface objects

A window application consists of graphical objects and the instructions that give them functionality. A GUI object is a small window that has very simple input or output functions. For example, an "edit object" is a simple window that lets the user enter and edit text. Some of the GUI objects include.

- **Menus.** Menus are the principle means of user input in a window application. A menu is a list of commands that users can view and choose from.
- **Dialog boxes.** A dialog box is a temporary window displayed to let the user supply more information for a command. A dialog box contains one or more GUI objects.

- **Icons.** Icons are small bitmap graphics, generally 32* 32 or 16* 32 pixels in size used to represent minimized windows. Icons can represent an individual application or group of application.
- **Cursor.** A cursor is a visual cue or small bitmap graphic 32*32 pixels in size that represents to user the current position pf the mouse on the screen. Windows programs use customized cursors to show what type of task the user currently is performing.
- **Bitmap.** A bitmap is a binary representation of a graphic image in a program. Windows itself uses lots of bitmap graph- ics; for example, the images representing various controls on a typical window, such as scroll bar arrows.
- **Strings.** Strings contain text, such as descriptions, prompts, and error messages that is displayed as a part of the windows program.
- **Accelerators.** Accelerators are keyboard combinations that a user presses to perform a task in an application. For example, a window program can include the accelerator Ctrl + V.
- **Fonts.** Windows programs use fonts to define the typeface. size, and style of text.
- **Edit boxes.** Edit boxes are rectangular areas used to enter or modify a single field. They are most useful when used for entering data during program operation. For example multiple edit controls could be used to collect information on a data entry screen. See figure 12.4.

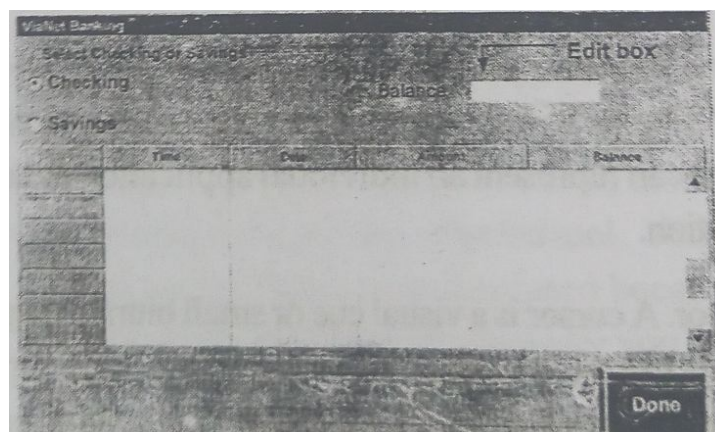


Figure 12.4

Push Buttons

A task is performed when the push button is selected or clicked on. Because they perform a task, push buttons are commonly referred to as command buttons. A default push button is displayed with a bold border to distinguish it from a standard push button see figure 12.5

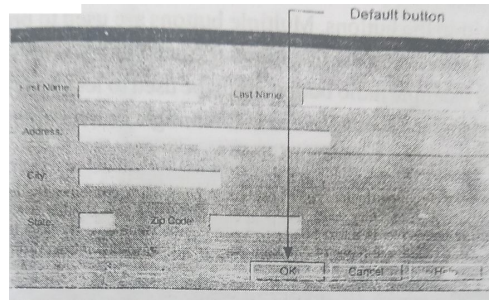


Figure 12.5

Check Boxes

Check boxes generally are used to present a two state options: basically, they let you enable or disable an option. A check box is a small square frames. For example that can be loaded into an application. You check each font you want to load those you not check will not be loaded see figure 12.6.

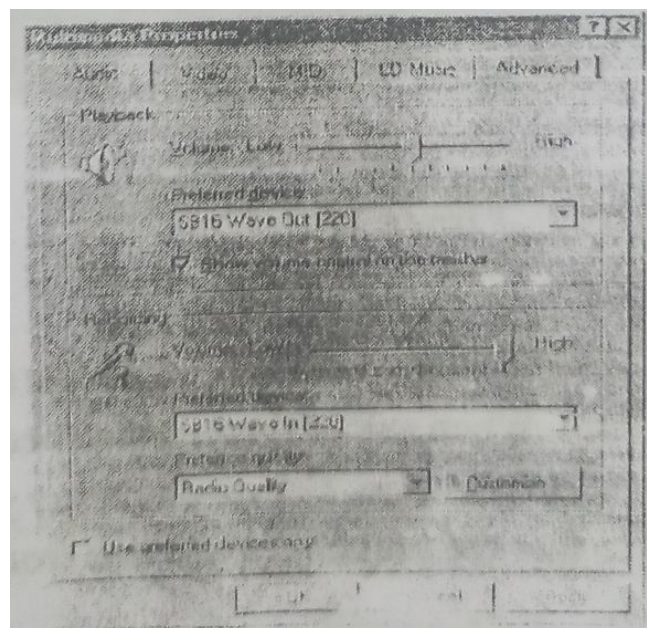


Figure 12.6

Radio Buttons

A radio button is a small round frame; when selected a dot appears within the buttons. Multiple buttons are used to present mutually exclusive options, like selecting a radio station on older car radios. For example radio button might be used to let you select a font for a character although several fonts might be available, only one could be selected see figure 12.7

The screenshot shows a 'ViaNet Banking' application window. At the top, there's a title bar 'ViaNet Banking'. Below it, a section titled 'Select Checking or Savings' contains two radio buttons: 'Checking' (which is selected, indicated by a dot) and 'Savings'. To the right of these buttons is a 'Balance' label followed by a text input field. Below this section is a table with four columns: 'Time', 'Date', 'Amount', and 'Balance'. The table has several empty rows. At the bottom right of the window is a 'Done' button.

	Time	Date	Amount	Balance

Figure 12.7

List Boxes

You can activate a list box, scroll through the list then click or double click on an item. When any of these events occurs, the list box receives an event notification. For example say you want to display file names so that the user can select one from the list to open see figure 12.8.

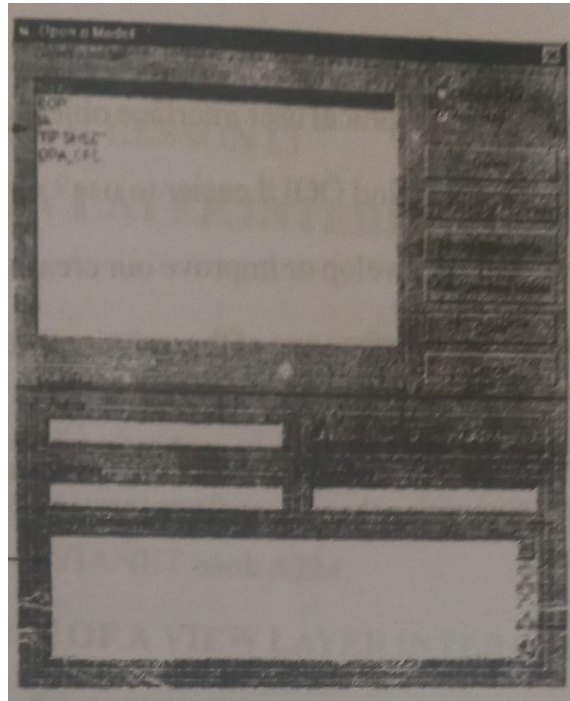


Figure 12.8

Spin boxes

Spin boxes control allows you to enter values by either typing them in the text box or by increasing or decreasing the current value using the up and down arrow buttons see figure 12.9.

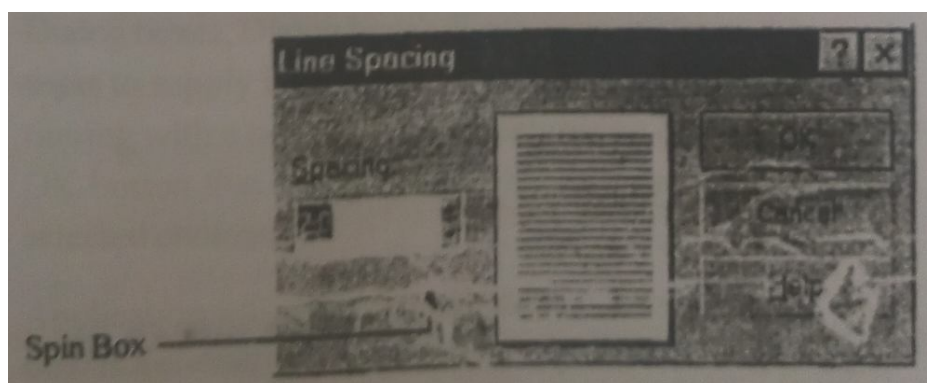


Figure 12.9

Questions

1. Explain the graphical user interface objects.
2. Why do users find OOUI easier to use?
3. How can we develop or improve our creativity?
4. Why is user interface one of the most important components of any software?
5. Perform a research on GUI and OOUI and write a short paper comparing them.

LESSON - 13

A VIEW LAYER INTERFACE

Objectives:

After studying this lesson you should be able to

- The purpose of a view layer interface.
- Prototyping the user interface.
- Case study of VIANET bank ATM.

13.1 The purpose of the view Layer Interface

Your user interface can employ one or more windows. Each window should serve a clear, specific purpose. Windows commonly are used for the following purpose;

- Forms and data entry windows. Data entry windows provide access to data that users can retrieve, display, and change in the application.
- Dialog boxes, Dialog boxes display status information or ask users to supply information or make a decision before continuing with a task. A typical feature of a dialog box is the OK button that a user clicks with a mouse to process the selected choices.
- Application window (main windows). An application window is a container of application objects or icons. In other words, it contains an entire application with which users can interact.

You should be able to explain the purpose of a window in the application in a single sentence. If a window serves multiple purpose, consider creating a separate one for each.

13.1.1 Guidelines for Designing Forms and Data Entry Windows

When designing a data entry window of forms (or web forms), identify the information you want to display or change. Consider the following issues.

In general, what kind of information will users work with and why?, For example, a user might want to change inventory information enter orders or maintain prices for stock items.

- Do users need access to all the information in a table or just some, information? When working with a apportion of the information in a table use a query that selects the rows and columns users want.
- In what order do users want rows to appear? For example users might want to change inventory information stored alphabetically, chronologically, or by inventory number. You have to provide a mechanism for the user so that the order can be modified.

Next, identify the tasks that users need to work with data on the form or data entry window, Typical data entry tasks include the following.

Navigating rows in a table, such as moving forward and backward, and going to the first and last record.

- Adding and deleting rows.
- Changing data in rows
- Saving and abandoning changes.

You can provide menus, push buttons, and speed bar buttons at users choose to initiate tasks. You can put controls anywhere a window. However, the layout you choose determines how successfully users can enter data using the form. Here are some guidelines to consider:

- You can use an existing paper form, such as a printed invoice, as the starting point for your design.
- If the printed form contains too much information to fit on a screen, consider using a main window with optional smaller windows that users can display on demand or using a window with multiple pages (see Figure 13-1). Users typically are more productive when a screen is not cluttered.

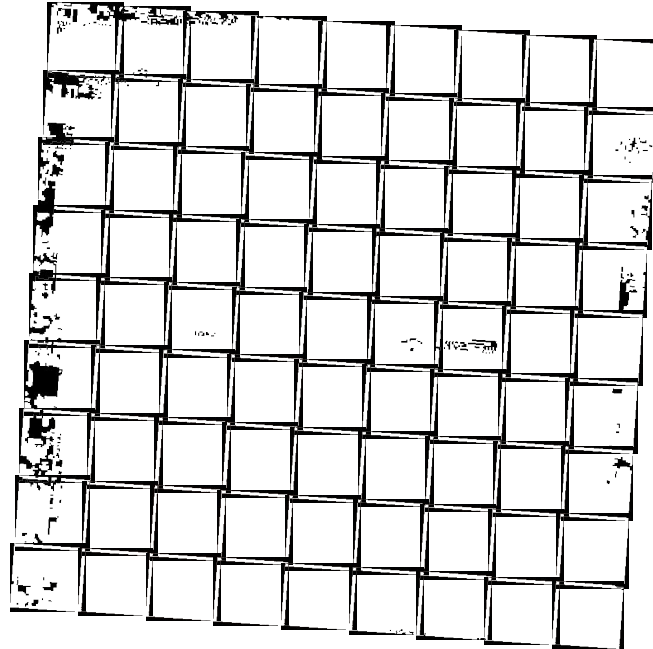


Figure 13.1

Users scan a screen in the same way they read a page of a book, from left to right and top to bottom. In general, put required or frequently entered information toward the top and left side of the form, entering optional or seldom-entered information toward the bottom and right side. For example, on a window for entering inventory data the inventory number and item name might best be placed in the upper-left corner, while the signature could appear lower and to the right (see Figure 13-2)



Figure 13.2

When information is positioned vertically, align fields at their left edges (in Western countries). This usually makes it easier for the user to scan the information.

Text labels usually are left aligned and placed above or to the left of the areas to which they apply. When placing text labels to the left of text box controls, align the height of the text with text displayed in the text box (see Figure 13-3)

When entering data, users expect to type information from left to right and top to bottom, as if they were using a typewriter (usually the Tab key moves the focus from one control to another). Arrange controls in the sequence users expect to enter data. However you may want the users to be able to jump from one group of controls to the beginning of another group, skipping over individual controls. For example when entering address information, users expect to enter the Address, City, State, and Zip Code (see Figure 13-4)

Label locations for text Labels

Label

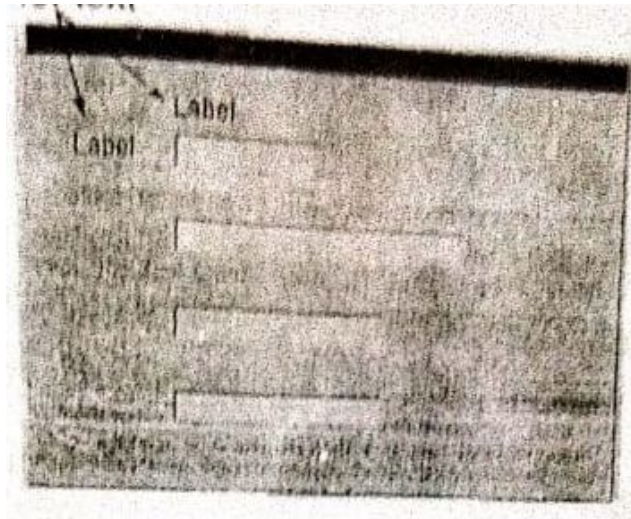


Figure 13.3

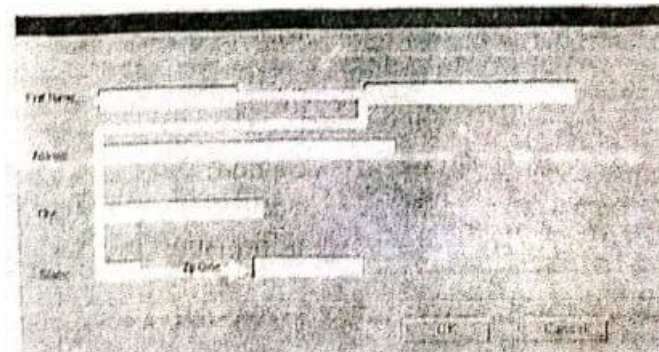


Figure 13.4

Put similar or related information together, and use visual effects to emphasize the grouping. For example, you might want to put a company billing and shipping address information in separate groups. To emphasize a group, you can enclose its controls in a distinct visual area using a rectangle lines, alignment, or colors (see Figure 13-1)

13.1.2 Guidelines for Designing Dialog Boxes and Error Messages

A dialog box provides an exchange of information or a dialog between the user and the application. Because dialog boxes generally appear after a particular menu item (including pop-up or cascading menu items) or a command button, define the title text to be the name of the associated command from the menu item or command button. However, do not include ellipses and avoid including the commands menu title unless necessary to compose a reasonable title for the dialog box. For example, for a Print command on the File Menu, define the dialog box windows title text as Print, Not Print..... or File Print

If the dialog box is for an error message, use the following guidelines:

- Your error message should be positive. For example instead of displaying “you have typed an illegal date format,” display the message “Enter date format mm/dd/yyyy”.
- Your error message should be constructive. For example avoid messages such as “You should know better! Use the OK button”, instead display “Press the Undo button and try again”. The users should feel as if they are controlling the system rather than the software is controlling them.

- Your error message should be brief and meaningful. For example, “ERROR: tvne check Offending Command
- Although this message might be useful for the programmes during the testing and debugging phase, it is not a useful message for the user for your system.
- Orient the controls in the dialog box, in the direction people read. This usually means left to right and top to bottom. Locate the primary field with which the user interacts as close to the upper-left corner as possible. Follow similar guidelines for orienting controls within a group in the dialog box.

13.1.3 Guidelines for the Command Buttons Layout

Lay out the major command buttons either stacked along the upper-eight border of the dialog box (see Figure 13.5). Positioning buttons on the left border is very popular in Web interfaces. Position the most important button, typically the default command, as the first button in the set. If you see the OK and Cancel buttons, group them together. If you include a Help command button, make it the last button in the set.

You can use other arrangements if there is a compelling reason, such as a natural mapping relationship. For example it makes sense to place buttons labeled North, South, East, and West in a compass like layout. Similarly, a command button that modifies or provides direct support to be in the conventional location. Once again, let consistency guide you through the design.

For easy readability, make buttons a consistent length. Consistent visual and operational styles will allow users to transfer their knowledge and skills more easily. However, if maintaining this consistency greatly expands the space required by a set of buttons, it may be reasonable to have one button larger than the rest. Placement of command buttons (or other controls) within a tabbed page implies the application of only the transactions on that page. If command buttons are placed within the window but not on the tabbed page, they apply to the entire window (see Figure 13-1).

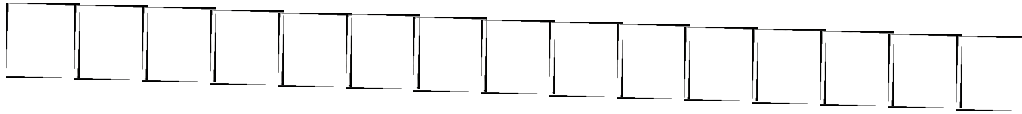
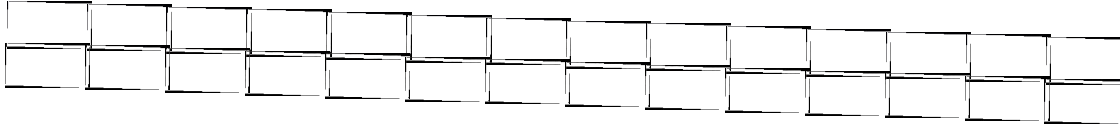
13.1.4 Guidelines for Designing Application Windows

A typical application window consists of a frame (or border) that defines its extent and a title bar that identifies what is being viewed in the window. If the viewable content of the window exceeds the current size of the window, scroll bars are used. The window also can include other components like menu bars, toolbars, and status bars.

An application window usually contains common drop-down menus. While a command drop-down menu is not required for all applications, apply these guidelines when including such menus in your software interface.

The File menu. The File menu provides an interface for the primary operations that apply to a file. Your application should include commands such as Open, Save, Save As, and Print. Place the Exit command at the bottom of the File menu preceded by a menu separator. When the user chooses the Exit command, close any open windows and files and stop any further processing. If the object remains active even when its window is closed, such as a folder or printer, then include the Close command instead of Exit.

- The Edit menu. Include general purpose editing commands on the Edit menu. These commands include the Cut, Copy, and Paste commands. Depending on your application, you might include the Undo, find, and Delete commands.
- The View menu and other command menus. Commands on the View menu should change the users view of data in the window. On this menu, include commands that affect the view and not the data itself. for example, Zoom or Outline. Also include commands for controlling the display of particular interface elements in the view for example, Show Ruler. These commands should be placed on the pro-up menu of the window.
- The window menu. Use the Window menu in multiple document, interface-style applications for managing the windows within the main workspace.
- The Help menu. The Help menu contains commands that provide access to Help information. Include a Help Topics command. This command provides access to the Help Topics browser, which displays topics included in the applications of the Help Topics browser, such as Contents, Index, and Find Topic, Also include other user assistance commands or wizards that can guide the users and show them how to use the system. It is conventional to provide access to copy right and version information for the application, which should be included in the About Application name command on this menu. Other command menus can be added, depending on your applications needs.

*Figure 13.6**Figure 13.7*

Tool bars and status bars. Like menu bars, toolbars and status bars are special interface constructs for managing sets of controls. A toolbar is a panel that contains a set of controls, as shown in Figure 13. 6 designed to provide quick access to specific commands or options. Some specialized toolbars are called ribbons, toolboxes, and palettes. A status bar, shown in Figure 13.7 is a special area within a window, typically at the bottom, that displays information such as the current state of what is being viewed in the window or any other contextual information, such as keyboard state. You also can use the status bar to provide descriptive messages about a selected menu or toolbar button, and it provides excellent feedback to the users. Like a toolbar, a status bar can contain controls however, typically, it includes read-only or non interactive information.

13.1.5 Guidelines for Using Colors

For all objects on a window, you can use colors to add visual appeal to the form. However, consider the hardware. Your Windows-based application may end up being run on just about any sort of monitor. Do not choose colors exclusive to a particular configuration, unless you know your application will be run on that specific hardware. In fact, do not dismiss the possibility that a user will run your application with no color support at all.

Figure out a color scheme. If you use multiple colors, do not mix them indiscriminately. Nothing looks worse than a circus interface [1]. Do you have a good color sense? If you cannot make everyday color decisions, ask an artist or a designer to review your color scheme. Use color as a highlight to get attention. If there is one field you want the user to fill first, color it in such a way that it will stand out from the other fields.

How long will users be sitting in front of your application? If it is eight hours a day, this is not the place for screaming red text on a sunny yellow background. Use common sense and consideration. Go for soothing, cool, and neutral colors such as blues or other neutral colors.

Text must be readable at all times; black is the standard color, but blue and dark gray also can work.

Associate meanings to the colors of your interface. For example, use blue for all the editable fields, green to indicate fields that will update dynamically, and red to indicate error conditions. If you choose to do this, ensure color consistency from screen to screen and make sure the users know what these various colors indicate. Do not use light gray for any text except to indicate an unavailable condition.

Remember that a certain percentage of the population is color blind. Do not let color be your only visual cue. Use an animated button, a sound package, or a message box. Finally, color will not hide poor functionality.

The following guidelines can help you use colors in the most effective manner:

- You can use identical or similar colors to indicate related information. For example, in an entry fields might appear in one color. Use different or contrasting colors to distinguish groups of information from each other. For example checking and savings accounts could appear in different colors.
- For an object background, use a contrasting but complementary color. For example in an entry field, make sure that the background color contrasts with the data color so that the user can easily read data in the field.
- You can use bright colors to call attention to certain elements on the screen and you can use dim colors to make other elements less noticeable. For example you might want to display the required field in a brighter color than optional fields.
- Use colors consistently within each window and among all windows in your application. For example the color for push buttons should be the same throughout.
- Using too many colors can be visually distracting and will make your application less interesting.
- Allow the user to modify the color configuration of your application

13.1.6 Guidelines for Using Fonts

Consistency is the key to an effective use of fonts and color in your interface. Most commercial applications use 12 point system font for menus and 10-point System is font in dialog boxes. These are fairly safe choices for most purposes. If System is too boring for you,

any other sans serif font is easy to read (such as Arial or Helvetica). The most practical serif font is Times New Roman.

Avoid Courier unless you deliberately want something to look like it came from a type writer. Other fonts may be appropriate for word processing or desktop publishing purposes but do not really belong on Windows-based application screens. Avoid using all uppercase text in labels or any other text on your screens: It is harder to read and feels like you are shouting at a users. The only exception is the OK command button. Also avoid mixing more than two fonts point sizes or styles so your screens have a cohesive look. The following guidelines can help you use fonts to best convey information:

- Use commonly installed fonts, not specialized fonts that users might not have on their machines.
- Use bold for control labels so they will remain legible when the object is dimmed.
- Use fonts consistently within each form and among all forms in your application. For example, the fonts for check box controls should be the same throughout. Consistency is reassuring to users, and psychologically makes users feel in control.
- Using too many font styles, size, and colors can be visually distracting and should be avoided. Too many font styles are confusing and make users feel less in control.
- To emphasize text, increase its font size relative to other words on the form or use a contrasting color. Avoid underlines; they can be confusing and difficult to read on the screen.

13.2 Prototyping the User Interface

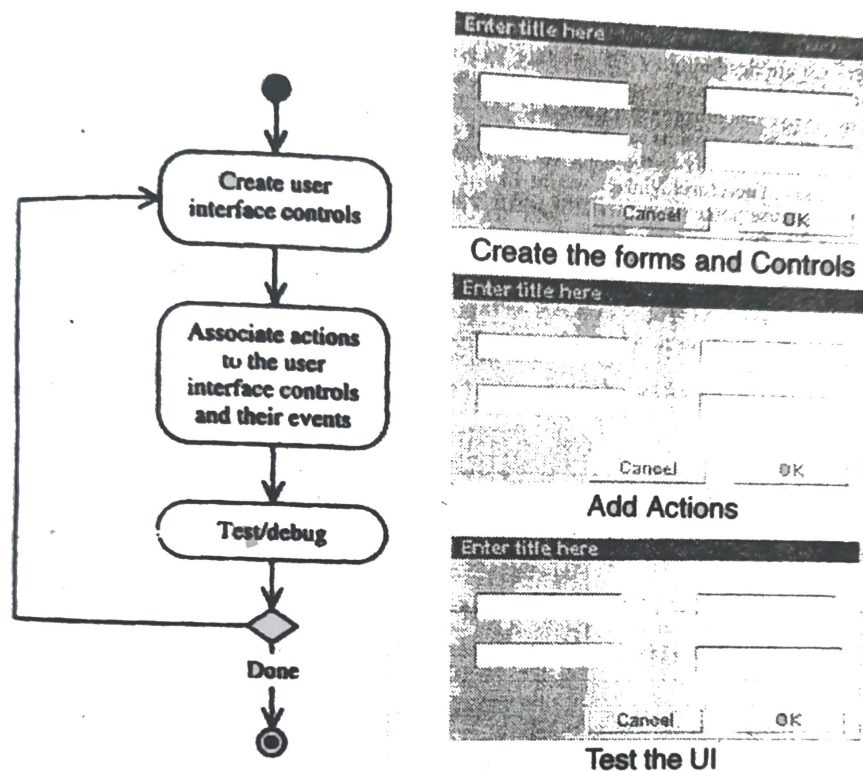
Rapid prototyping encourages the incremental development approach, “grow, don’t build”. Prototyping involves a number of iterations. Through each iteration, we add a little more to the application, and as we understand the problem a little better, we can make more improvements. This in turn makes the debugging task easier.

It is highly desirable to prepare a prototype of the user interface during the analysis to better understand the system requirements. This can be done with most CASE tools. operational software using visual prototyping, or normal development tools. Visual and rapid prototyping is a valuable asset in many ways. First it provides an effective tool for communicating the design. Second, it can help you define task flow and better visualize the design. Finally, it provides a

low-cost vehicle for getting user input on a design. This is particularly useful early in the design process.

Creating a user interface generally consists of three steps (see Figure 13-8):

1. Create the user interface objects (such as buttons, data entry fields)
2. Link or assign the appropriate behaviors or actions to these user interface objects and their events.
3. Test, debug, then and more by going back to step 1



When you complete the first prototype, meet with the user for an exchange of ideas about how the system would go together. When approaching the user, describe your prototype briefly. The main purpose should be to spark ideas. The user should not feel that you are imposing or even suggesting this design. You should be very positive about the user's system and wishes. Instead of using leading phrases like "we could do this..." or "It would be easier if

we ...” choose phrases that give the user the feeling that he or she is in charge. Some example phrases are [5]:

“Do you think that if we did... it would make it easier for the users?”

“Do users ever complain about...? We could add...to make it easier”.

This cooperative approach usually results in more of your ideas being used in the end.

13.3 CASE STUDY: DESIGNING USER INTERFACE FOR THE VIANET BANK ATM]

Here we are designing a GUI interface for the ViaNet bank ATM for two reasons. First the ViaNet bank wants to deploy touchscreen kiosks instead of conventional ATM machines (see Figure 13-9). The second reason is that in the near future, the ViaNet wants to create “on-line banking”, where customers can be connected electronically to the bank via the Internet and conduct most of their banking needs. Therefore ViaNet would like to experiment with GUI interface and perhaps reuses some of the UI design concept for the on line banking project (see Figure 13-10).

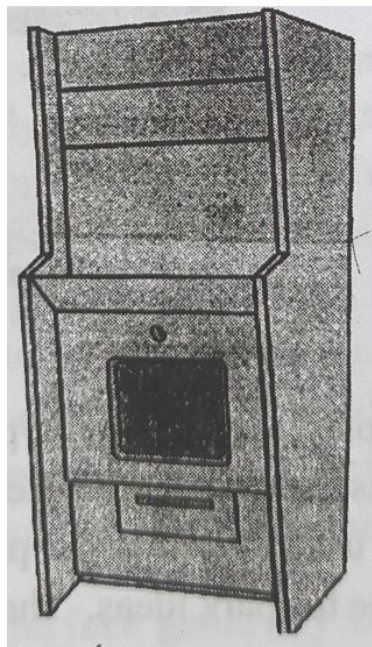


Figure 13.9

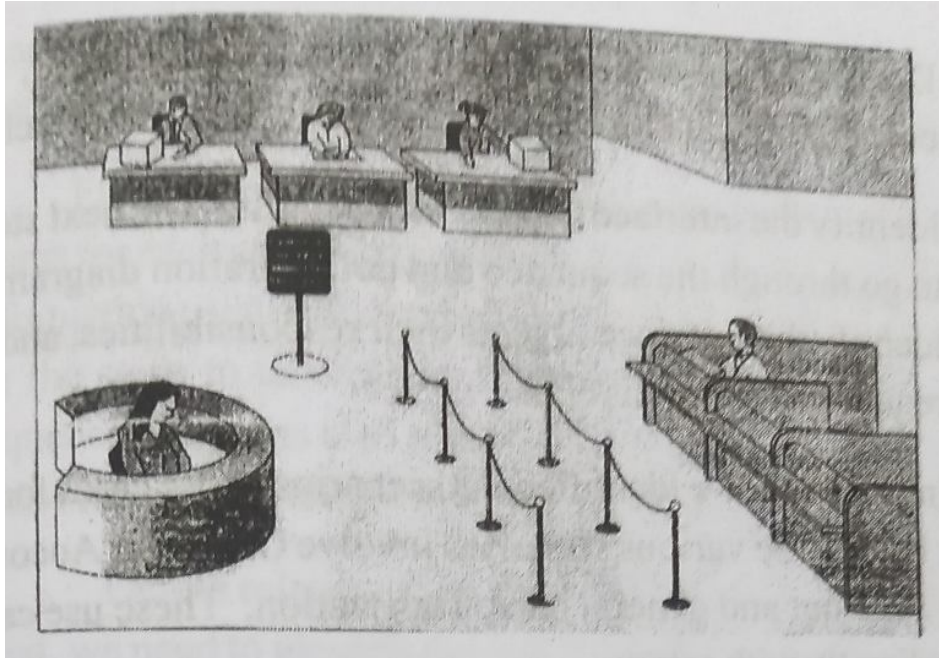


Figure 13.10

13.3.1 The View Layer Macro Process

The first step here is to identify the interface objects, their requirements, and their responsibilities by applying the macro process to identify the view classes. When creating user interfaces for a business mode, it is important to remember the role that view objects play in an application. The interface should be designed to give the user access to the business process modeled in the business layer. It is not designed to perform the business processing itself.

For every class identified (so far we have identified the following classes: Account, ATM Machine Bank, BankDB, Checking Account, Savings Account, and Transaction).

- Determine if the class interacts with a human actor, The only class that interacts with a human actor is ATM Machine.
- Identify the interface objects for the class. The next step is to go through the sequence and collaboration diagrams to identify the interface objects their responsibilities, and the requirements for this class.

In Chapter 6, we identified the scenarios or use cases for the ViaNet bank. The various scenarios involve Checking Account, Saving Account and general bank Transaction. These use cases interact directly with actors:

1. Bank transaction
2. Checking transaction history
3. Deposit checking
4. Deposit savings
5. Savings transaction history
6. Withdraw checking
7. Withdraw savings
8. Valid/invalid PIN

Based on these use cases, we have identified eight view or interface objects. The sequence and collaboration diagrams can be very useful here to help us better understand the functionalities of the interface objects, we need to look at the sequence and collaboration diagrams and study the events that these interface objects must process or generate. Such events will tell us the makeup of these objects. For example the PIN validation user interface must be able to get user PIN number and check whether it is valid.

Furthermore by walking through the steps of sequence diagrams for each scenario (such as withdraw, deposit, or an account information), you can determine what view objects are necessary for the steps to take place. Therefore, the process of creating sequence diagrams also assists in identifying view layer classes and even understanding their relationships.

Define relationships among the view (interface) object: Next, we need to identify the relationships among these view objects and their associated business classes.

So far, we have identified eight view classes:

Account Transaction UI (for a bank transaction)

Checking Transaction History UI

Savings Transaction History UI

Bank Client Access UI (for validating a PIN code)

Deposit Checking UI

the Deposit Savings UI Withdraw Checking UI

Withdraw Savings UI

The three transaction view objects Account Transaction UI, Checking Transaction History UI, and Savings Transaction History UI-basically do the same thing, display the transaction history on either a checking or savings account. (To refresh your memory, see how we implemented this for object storage and the access class). Therefore we need only one view class for displaying transaction history, and let us call it Account Transaction UI.

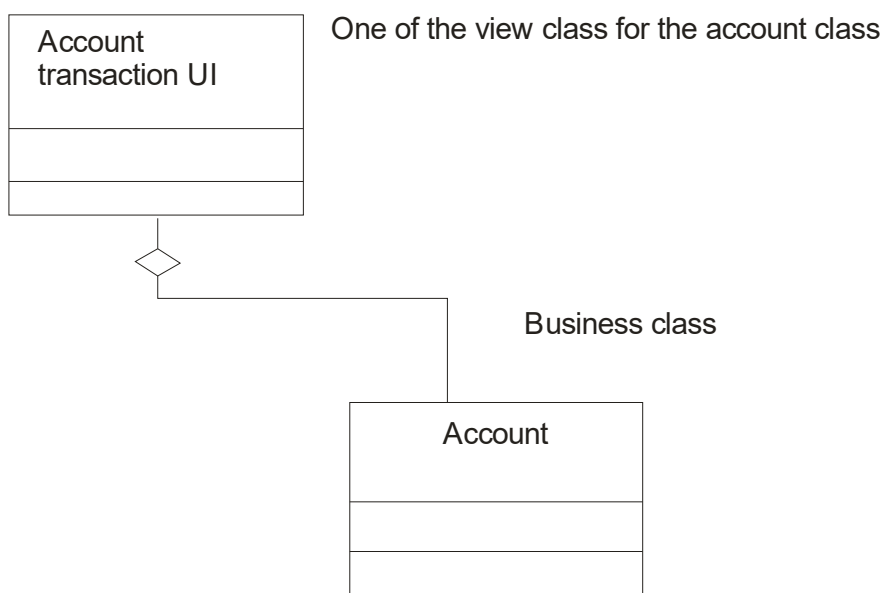


Figure 13.11

The Account Transaction UI view class is the account transaction interface that displays the transaction history for both savings and checking accounts. Figure 13-11 depicts the relation among the Account Transaction object is opposite of that between business class and access class. As said earlier, the interface object handles all communication with the user but processes no business rules and lets that work be done by the business objects themselves. In this case, the account class provides the information to Account Transaction UI for display to the users.

The Bank Client Access UI view class provides access control and PIN code validation for a bank client (see Figure 13.12).

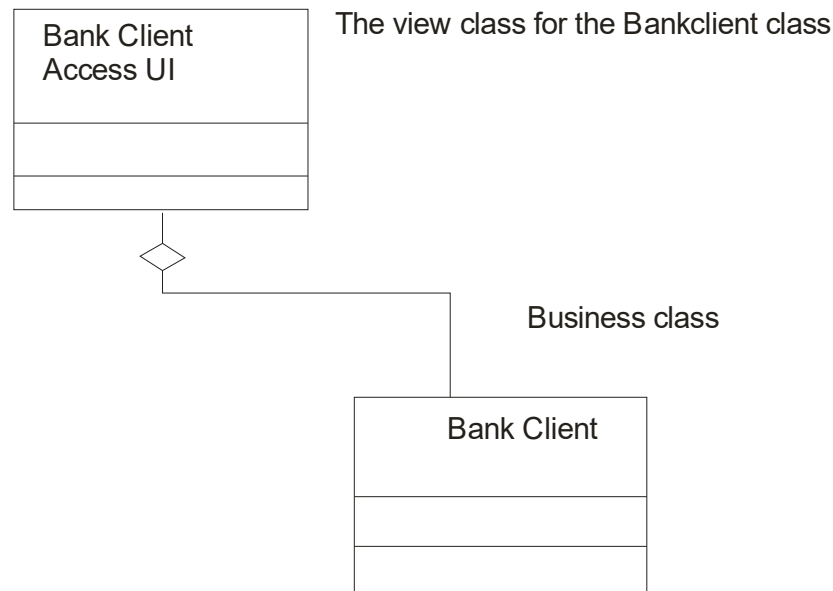


Figure 13.12

The four remaining view objects are the Deposit Checking UI view class (interface for deposit to checking accounts), Withdraw Savings UI view (interface for withdrawal from savings accounts and Withdraw Checking UI view class (interface for withdrawal from savings accounts).

- Iterate and refine. This is the final step. Through the iteration and refinement process, we notice that the four classes Deposit Checking UI, Deposit Saving UI, Withdraw Savings UI, and Withdraw checking UI basically provide a single service, which is getting the amount of the transaction (whether user wants to withdraw or deposit) and sending appropriate messages to Savings Account or Checking Account business classes. Therefore they are good candidates to be combined into two view classes one for Checking Account UI and Savings Account UI allow users to deposit money to or withdraw money from checking and savings accounts.

The Checking Account UI view class provides the interface for a checking account deposit or withdrawal (see Figure 13-13)

The Savings Account UI view class provides the interface for a savings account deposit or withdrawal (see Figure 13-13)

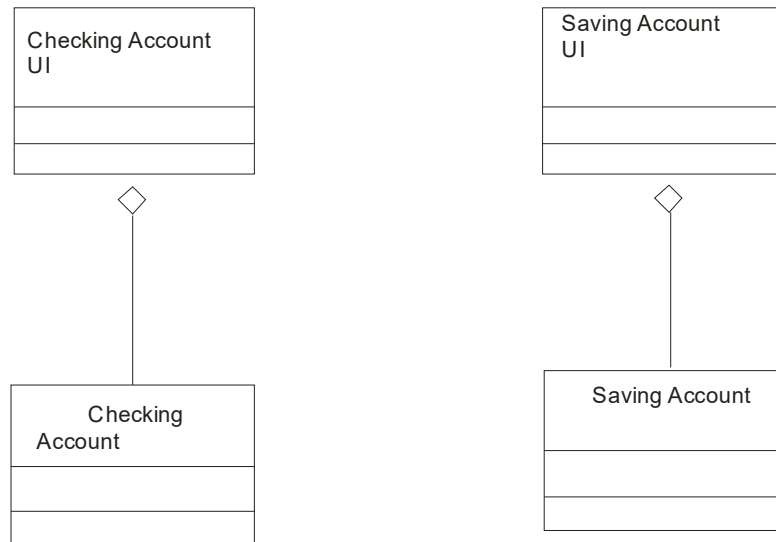


Figure 13.13

Finally, we need to create one more view class that provides the main control or the main UI to the ViaNet bank system. The Main UI view class provides the main control interface for the ViaNet bank system.

13.3.2 The View Layer Micro Process

Based on the outcome of the macro process, we have the following view classes:

Bank Client Access UI

Main UI

Account Transaction UI

Checking Account UI

Saving Account UI

For every interface object identified in the macro UI design process,

- Apply micro-level UI design rules and corollaries to level the UI. We need to go through each identified interface object and apply design rules (such as making the UI simple transparent and controlled by the user) and GUI guideline to design them
- Iterate and refine.

13.3.3 The Bank Client Access UI Interface Object

The Bank Client Access UI provides clients access to the system by allowing them to enter their PIN code for validation. The Bank Client Access UI is designed to work with a card reader device, where the user can insert the card and the card number should be displayed automatically in the card number field. In situation where there is no card reader, Such as on-line banking (e.g., user wants to log onto the system from home), the user must enter his or her card number (see Figure 13-14).

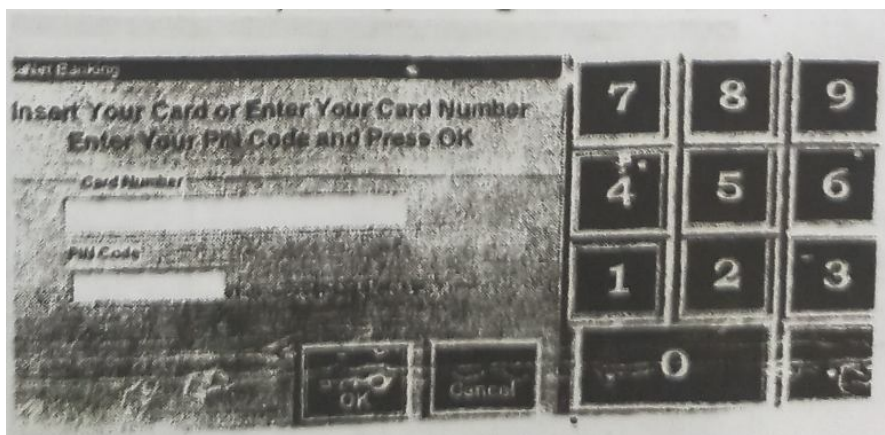


Figure 13.14

13.3.4 The Main UI Interface Object

The Main UI provides the main control to the ATM services. Users can select to deposit money to savings or checking, withdraw money from savings or checking inquire as to a balance or transaction history or quit the session (see Figure 13-15)

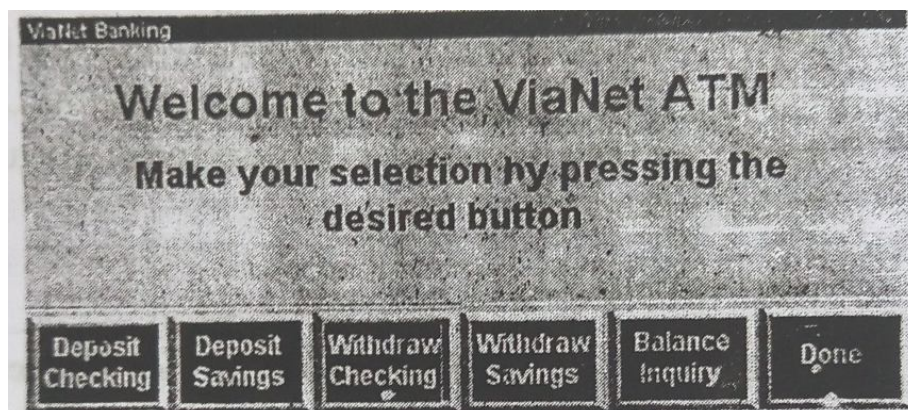


Figure 13.15

13.3.5 The Account Transaction UI Interface Object

The Account Transaction UI interface Object will display the transaction history of either a savings or checking account. The user must select the account type by pressing the radio buttons. Figure 13-16 displays the account balance inquiry and transaction history interface.

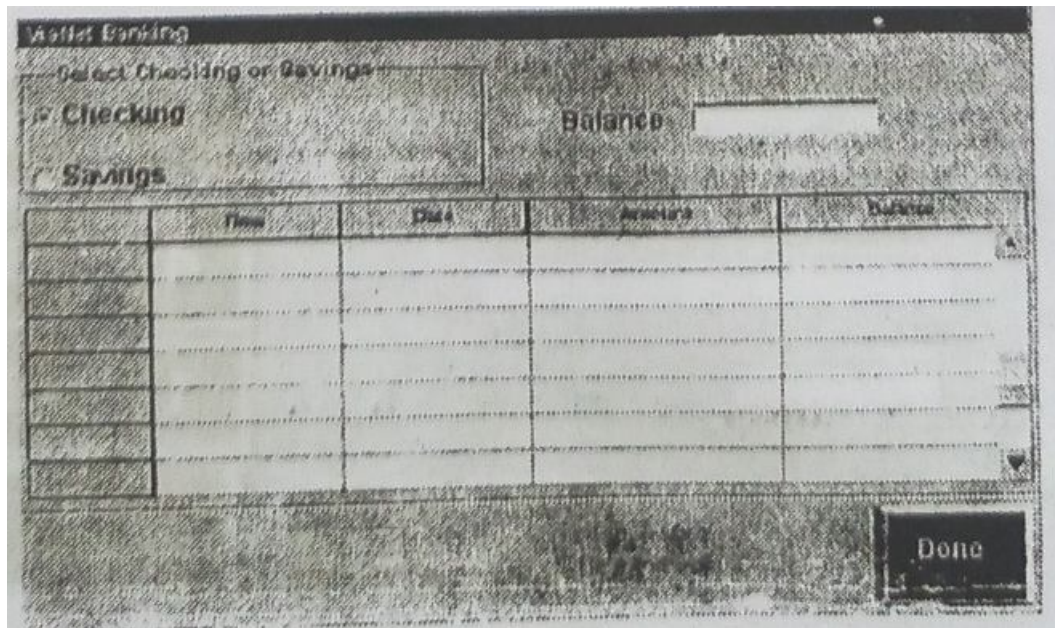


Figure 13.16

13.3.6 The Checking Account UI and Savings Account UI

The Checking Account UI and Savings Account UI interface objects allow users to deposit to and withdraw from checking and savings accounts. These two interface are designed with two tabs, one for deposit and one for withdrawal. When users press one of the Main UIs buttons, say Deposit Savings the Savings Account UI will be activated and the system should go automatically to the Deposit Accounts UI interfaces.

See problem 8 for an alternative design for Savings Account UI and Checking Account UI classes. It always is a good idea to create an alternative design and select the one that best satisfies the requirements.

13.3.7 Defining the Interface Behavior

The role of a view layer object is to allow the users to manipulate the business model. The actions a user takes on a screen (for example pressing the Done button) should be translated into a request to the business object for some kind of processing. When the processing is completed, the interface can update itself by displaying new information opening a new window, or the like.

Defining behavior for an interface consists of identifying the events to which you want the system to respond and the actions to be taken when the event occurs. Both GUI and business objects can generate events when something happens to these events, you define actions to take. An action is a combination of an object and a message sent to it.

13.3.7.1 *Identifying Events and Actions for the Bank Client Access UI Interface Object*

When the user inserts his or her card, types in a PIN code, and presses the OK button, the interface should send the message Bank Client :verify pass word (see Chapter 10) to the object to identify the client. If the password is found correct, the Main UI should be displayed and provide users with the ATM services; otherwise an error message should be displayed. Figure 13-17 is the UML activity diagram of Bank Client Access UI events and actions.

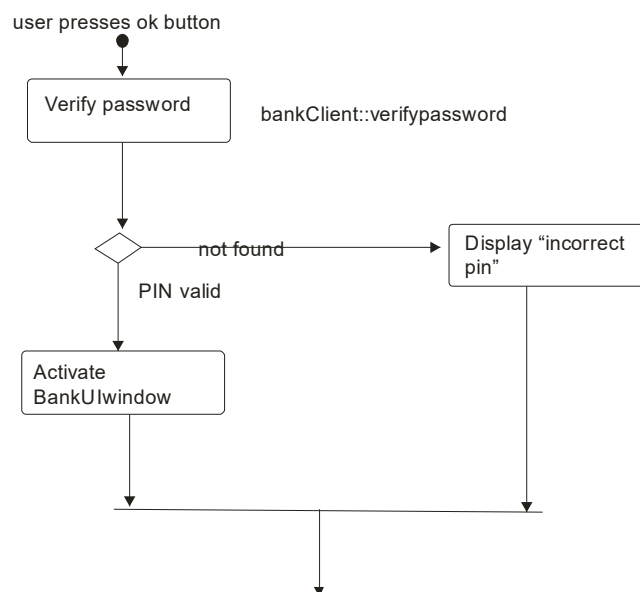


Figure 13.17

13.3.7.2 Identifying Events and Actions for the Main UI Interface Object

From this interface, the user should be able to do the following:

- Deposit into the checking account by pressing the Deposit Checking button.
- Deposit into the savings account by pressing the Deposit Savings button.
- Withdraw from the savings account by pressing the Withdraw Savings button.
- Withdraw from the Checking account by pressing the Withdraw Checking button.
- View balance and transaction history by pressing the Balance Inquiry button.
- Exit the ATM by pressing Done.

Figure 13.18 is the UML activity diagram of Main UI events and actions.

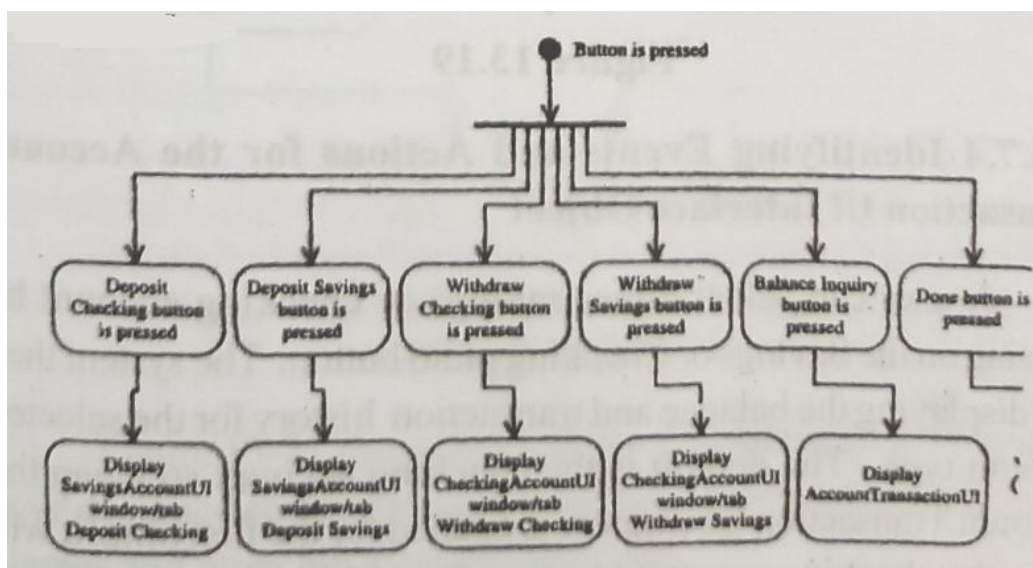


Figure 13.18

13.3.7.3 Identifying Events and Actions for the Savings Account UI Interface Object

The Savings Account UI has two tabs. First, the Savings Account UI opens the appropriate tab. For example if the user selects the Deposit Savings from the Main UI, the Savings Account UI will display the Deposit Savings tab. Figure 13-19 shows the activity diagram for the Deposit Savings. A withdrawal is similar to Deposit Savings and has been left as an exercise;

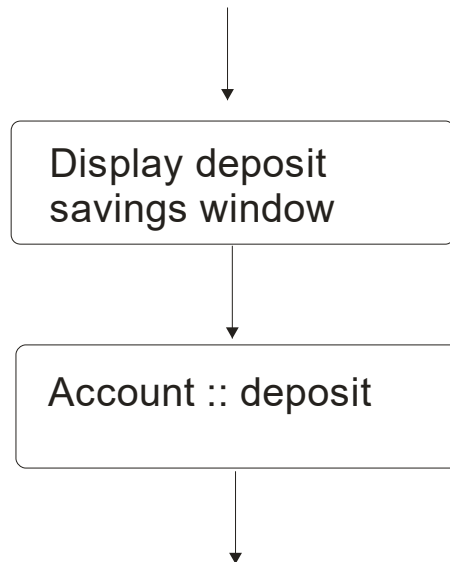


Figure 13.19

13.3.7.4 Identifying Events and Actions for the Account Transaction UI Interface Object

A user can select either savings or checking account by pressing on the Savings or Checking radio button. The system then will displaying the balance and transaction history for the selected account type. The default is the checking account so when the Account Transaction UI window is opened for the first time, it will show the checking account history. Pressing on the savings account radio button will cause it to display the savings account balance and history. To close the display and get back to Main UI, the user presses the Done button (see Figure 13-20). Notice that here we assume that the account has a method called display Trans, which takes a string parameter for type of account (Savings or Checking) and retrieves the appropriate transaction. Since we did not identify of design it, we need to develop it here. This occurs quite often during software development, which is why the process is interactive.

13.4 Summary

The main goal of a user interface is to display and obtain information needed in an accessible, efficient manner. The design of a software interface more than anything else, affects how the user interacts and therefore, experiences the application. It is important for the design to provide users the information they need and clearly tell them how to complete a task successfully. A well designed UI has visual appeal that motivates users to use the application. In addition, it should use the limited screen space efficiently.

In this chapter, we learned that the process of designing view layer classes consists of the following steps.

1. Macro-level UI design process: identify view layer objects.
2. 'Micro-level UI design activities
 - 1.1 Design the view layer objects by applying the design axioms and corollaries.
 - 1.2 Prepare a prototype of the view layer interface
2. Test usability and user satisfaction
3. Refine and iterate.

The first step of the process concerns identifying the view classes and their responsibilities by utilizing the view layer macrolevel processes. The second step is to design these classes by utilizing view layer micro-level processes. User satisfaction and usability testing will be studied in the next chapter. Furthermore, we looked at UI design rules, which are based on the design corollaries and finally we studied the guidelines for developing a graphical user interface (GUI).

The guidelines are not a substitution for effective evaluation and iterative refinement within a design. However, they can provide helpful advice during the design process. Guidelines emphasize the need to understand the intended audience and the tasks to be carried out, the need to adopt an iterative design process and identify use cases, and the need to consider carefully how the guidelines can be applied in specific situations. Nevertheless, the benefits gained from following design guideline should not be underestimated. They provide valuable reference material to help with difficult decisions that crop up during the design process, and they are a springboard for ideas and a checklist for omissions. Used with the proper respect

and in context, they are a valuable adjunct to relying on designer intuition alone to solve interface problems.

Questions

1. Describe the UI design rules.
2. What is KISS?
3. How can use cases help us design the view layer objects?
4. How can you make your UI forgiving?
5. How would you achieve consistency in your user interface?
6. How can metaphors be used in the design of a user interface?

LESSON - 14

SOFTWARE QUALITY ASSURANCE

Objectives

After studying this lesson you should be able to

- You Testing strategies
- The impact of an object orientation on testing
- How to develop test cases.
- How to develop test plans.

INTRODUCTION

To develop and deliver robust systems we need a high level of confidence that [2]

- Each component will behave correctly
- Collective behavior is correct.
- No incorrect collective behavior will be produced.

Not only do we need to test components or the individual objects we also must examine collective behaviors to ensure maximum operational reliability. Verifying components in isolation is necessary but not sufficient to meet this end [2]

In the early history of computers live bugs could be a problem (see Bugs and Debugging). Moths and other forms of natural insect life no longer trouble digital computers, However, bugs and the need to debug programs remain.

In a 1966 article in Scientific American, computer scientist Christopher Strachey wrote

Although programming techniques have improved immensely since the early years the process of finding and correcting errors in programming-"debugging" still remains a most difficult, confused and unsatisfactory operation. The chief impact of this state of affairs is psychological. Although we are happy to pay lip service to the adage that to err is human, most of us like to make a small private reservation about our own performance on special occasions when we really try. It is somewhat deflating to be shown publicly and incontrovertibly by a machine that

even when we do try we in fact make just as many mistakes as other people. If your oride cannot recover from this blow, you will never make a programmer.

Although three decades have elapsed since these lines were written they still capture the essence and mystique of debugging.

Precisely speaking, the elimination of the syntactical bug is the process of debugging, whereas the detection and elimination of the logical bug is the process of testing. Gruenberger writes,

The logical bugs can be extremely subtle and may need a great deal of effort to eliminate them. It is commonly accepted that will large software systems (operating or application) have bugs remaining in them. The number of possible paths through a large computer program is enormous and it is physically impossible to explore all of them. The single path containing a bug may not be followed in actual production runs for a long time (if ever) after the program has been certified as correct by its author or others [10]

In this chapter we look at the testing strategies, the impact of an object orientation on software quality, and some guidelines for developing comprehensive test cases and plans that can detect and identify potential problems before delivering the software to its users. After all defects can affect how well the software satisfies the users requirements. Addresses usability and user satisfaction tests.

14.1 Quality Assurance Tests

One reason why quality assurance is needed is because computers are infamous for doing what you tell them to do. To close this gap, the code must be free of errors or bugs that cause unexpected results, a process called debugging. Debugging is the process of finding out where something went wrong and correcting the code to eliminate the errors or bugs that cause unexpected results. For example, if an incorrect result was produced at the end of a long series of computations, perhaps you forgot to assign the correct value to a variable, chose the wrong operator, or used an incorrect formula.

Testing and searching for bugs is a balance of science, art, and luck. Sometimes, the error is obvious: The bug shows its hideous head the first time you run the application value or until you take a closer look at the output and find out that the results are off by a factor of a certain fraction or the middle initials in a list of names are wrong. There are no magic tricks to

debugging: however, by selecting appropriate testing strategies and a sound test plan, you can locate the errors in your system and fix them using readily available debugging tools. A software debugging system can provide tools for finding errors in programs and correcting them. Let us take a look at the kinds of errors you might encounter when you run our program:

- Language (syntax) errors results from incorrectly constructed code, such as an incorrectly typed keyword or some necessary punctuations omitted. These are the easiest types of errors to detect; for the most part you need no debugging tools to detect them, The very first time you run your program the system will report the existence of these errors.
- Run-time errors occur and are detected as the program is running. when a statement attempts an operation that is impossible to carry out. For example if the program tries to access a nonexistent object (say a file) a run-time error will Occur.
- Logic errors occur when code does not perform the way you intended. The code might be syntactically valid and run without performing any invalid operations and yet produce incorrect results. Only by testing the code and analyzing the results can you verify that the code performs as intended. Logic errors also can produce run time errors.

The elimination of the syntactical bug is the process of debugging whereas the detection and elimination of the logical bug is the process of testing. As you might have experienced by now, logical errors are the hardest type of error to find.

Quality assurance testing can be divided into two major categories; error based testing and scenario based testing Error-based testing techniques search a given class method for particular clues of interests, then describe how these clues should be tested. For example say we want to test the payroll computation method we must try different values for hours (say, 40,0, 100, and -10) to see if the program can handle them (this also is know as testing the boundary conditions). The method should be able to handle any value if not the error must be recorded and reported. Similarly, the technique can be used to perform integration testing by testing the objects that processes a message and not the objects that sends the message.

Scenario-based testing also called usage-based testing concentrates on what the user does not what the product does. This means capturing use cases and the tasks users perform, then performing them and their variants as tests. These scenarios also can identify interaction

bugs. They often are more complex and realistic than error-based tests. Scenario-based tests tend to exercise multiple subsystems in a single test, because that is what users do. The tests will not find everything, but they will cover at least the higher visibility system interaction bugs [12]

14.2 Testing Strategies

The extent of testing a system is controlled by many factors, such as the risks involved, limitations on resources, and deadlines. In light of these issues, we must deploy a testing strategy that does the best job of finding defects in a product within the given constraints. There are many testing strategies, but most testing uses a combination of these: black box testing, white box testing, top down testing, and bottom-up testing. However, no strategy or combination of strategies truly can prove the correctness of a system: it can establish only its 'acceptability'.

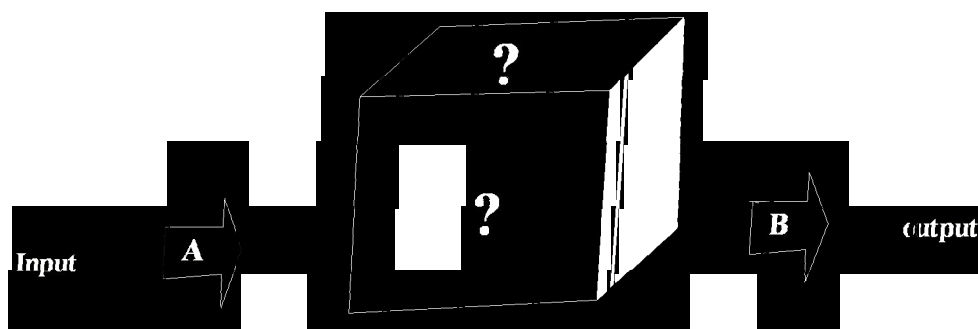


Figure 14.1

14.2.1 Black Box Testing

The concept of the black box is used to represent a system whose inside workings are not available for inspection [16]. In a black box, the test item is treated as “black”; since its logic is unknown all that is known is what goes in and what comes, out or the input and output (see Figure 14-1). Weinberg describes writing a user manual as an example of a black box approach to requirements. The user manual does not show the internal logic, because the users of the system do not care about what is inside the system.

In black box testing you try various inputs and examine that resulting output. you can learn what the box does but nothing about how this conversion is implemented [15]. Black box testing works very nicely in testing objects in and object-oriented environment. The black box testing technique also can be used for scenario-based tests, where the systems inside may not

be available for inspection but the input and output are defined through use cases or other analysis information.

14.2.2 White BOX Testing

White box testing assumes that the specific logic is important and must be tested to guarantee the systems proper functioning.

The main use of the white box is in error based testing, when you already have tested all objects of an application and all external or public methods of an object that you believe to be of greater importance (see Figure 14-2). In white box testing, you are looking for bugs that have a low probability of execution, have been carelessly implemented, or were overlooked previously[3].

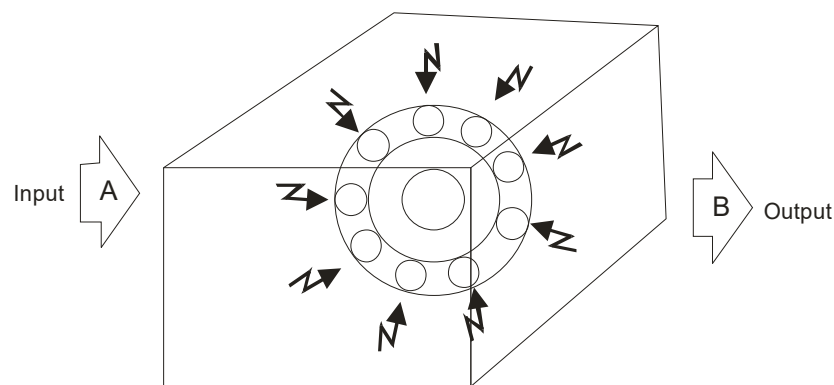


Figure 14.2

One form of white box testing called path testing makes certain that each path in an object's method is executed at least once during testing. Two types of path testing are statement testing coverage and branch testing coverage [3]

Statement testing coverage. The main idea of statement testing coverage is to test every statement in the object's method by executing it at least once. Murray states, "Testing less than this for new software is unconscionable and should be criminalized" [quoted in 2]. However realistically, it is impossible to test a program on every single input, so you never can be sure that a program will not fail on some input.

Branch testing coverage. The main idea behind branch testing coverage is to perform enough tests to ensure that every branch alternative has been executed at least once under

some test [3]. As in statement testing coverage, it is unfeasible to fully test any program of considerable size.

Most debugging tools are excellent in statement and branch testing coverage. White box testing is useful for error-based testing.

14.2.3 Top-Down Testing

Top-down testing assumes that the main logic or object interactions and systems message of the application need more testing than an individual objects methods or supporting logic. A topdown strategy can detect the serious design flaws early in the implementation.

In theory, top-down testing should find critical design errors early in the testing process and significantly improve the quality of the delivered software because of the iterative nature of the test [6]. A top-down strategy supports testing the user interface and event driven systems.

Testing the user interface using a top-down approach means testing interface navigation. This serves tow purpose according to Conger, First the top-down approach can test the navigation through screens and verify that it matches the requirements. Second, users can see, at an early stage, how the final application will look and feel [6]. This approach also is useful for scenario based testing. Top-down testing is useful to test subsystem and system integration.

14.2.4 Bottom-Up Testing

Bottom-up testing you starts with the details of the system and proceeds to higher levels by a progressive aggregation of details until they collectively fit the requirements for the system. This approach is more appropriate for testing the individual objects in a system. Here, you test each objects, then combine them and test their interaction and the messages passed among objects by utilizing the top-down approach.

In bottom-up testing, you start with the methods and classes that call rely on no others. You test them thoroughly. Then you progress to the next level up those methods and classes that use only the bottom level ones already tested. Next, you test combinations of the bottom two layers. Proceed until you are testing the entire program. This strategy makes sense because you are checking the behavior of a piece of code before it is used by another. Bottom up testing leads to integration testing, which leads to systems testings.

14.3 Impact of Object Orientation on Testing

The impact of an object orientation on testing is summarized by the following [1].

- Some types of errors could become less plausible (not worth testing for).
- Some types of errors could become more plausible (worth testing for now).
- Some new types of errors might appear.

For example when you invoke a method, it may be hard to tell exactly which method gets executed. The method may belong to one of many classes. It can be hard to tell the exact class of the method. When the code accesses it, it may get an unexpected value. In a non-object-oriented system, when you looked at

```
x = computer Payroll ().
```

you had to think about the behaviors of a single function. In an object-oriented environment, you may have to think about the behaviors of `base::compute_payroll()` or `derived::compute_payroll()`, and so on. For a single message, you need to explore (or at least think about) the union of all distinct behaviors. The problem can be complicated if you have multiple inheritance. However by applying the design axioms and corollaries of object-oriented design (OOD; see Chapter 9) you can limit the difference in behavior between base and derived classes.

The testing approach is essentially the same in both environments. The problem of testing messages in an object orientation is the same as testing code that takes a function as a parameter and then invokes it. Marick argues that the process of testing variable uses in OOD essentially does not change, but you have to look in more places to decide what needs testing. Has the plausibility of faults changed? Are some types of fault now more plausible or less plausible? Since object-oriented methods generally are smaller, these are easier to test. At the same time, there are more opportunities for integration faults. They become more likely, more plausible.

14.3.1 Impact of Inheritance in Testing

Suppose you have this situation [12]: The base class contains methods `inherited()` and `redefined()` and the derived class redefines the `redefined()` method.

The `derived::redefined` has to be tested afresh since it is a new code. Does `derived::inherited()` have to be retested? If it uses `redefined()` and the `redefined()` has changed, the

derived :: inherited may mishandle the new behavior. So it needs new tests even though the derived :: inherited() itself has not changed.

If the base:: inherited has been changed, the derived inherited() may not have to be completely tested. Whether it does depends on the base methods otherwise, guidelines especially if you don't test incrementally, you will end up with objects that are extremely hard to debug and maintain.

14.3.2 Reusability of Tests

If based :: redefined() and derived :: redefined() are two different methods with different protocols and implementations, each needs a different set of test requirements derived from the use cases, analysis, design or implementation. But the methods are likely to be similar. Their sets of test requirements will overlap. The better the OOD, the greater is the overlap. You need to write new tests only for those derived :: redefined requirements not satisfied by the base :: redefined tests. If you have to apply the base :: redefined tests to objects of the class "derived," the test inputs may be appropriate for both classes but the expected results might differ in the derived class [12].

Marick argues that the simpler is a test, the more likely it is to be reusable in subclasses. But simple tests tend to find only the faults you specifically target, complex tests are better at both finding those faults you stumbling across others by sheerluck. There is a trade-off here, one of many between simple and complex tests.

The models developed for analysis and design should be used for testing as well. For example the class diagram describes relationships between objects; that is an object of one class may use or contain an object of another class, which is useful information for testing. Furthermore, the use-case diagram and the highest level class diagrams can benefit the scenario-based testing. Since a class diagram shows the inheritance structure, which is important information for error based testing, it can be used not only during analysis but also during testing.

14.4 Test Cases

To have a comprehensive testing scheme, the test must cover all methods or a good majority of them. All the services of your system must be checked by at least one test. To test a system you must construct some test input cases, then describe how the output will look.

Next, perform the tests and compare the outcome with the expected output. The good news is that the use cases developed during analysis can be used to describe the usage test cases. After all, tests always should be designed from specifications and not by looking at the product. Myers describes the objective of testing as follows [13]

Testing is the process of executing a program with the intent of finding errors

A good test case is the one that has a high probability of detecting an as-yet undiscovered error.

A successful test case is the detects an as-yet undiscovered error.

14.4.1 Guidelines for Developing Quality Assurance Test Cases

Gause and Weinberg provide a wonderful example to highlight the essence of a test case. Say we want to test out new and improved “Superchalk”

Writing a geometry lesson on a blackboard is clearly normal use for Superchalk. Drawing on clothing is not normal. but is quite reasonable to expect. Eating Superchalk may be unreasonable, but the design will have to deal with this issue in some way, in order to prevent lawsuits. No single failure of requirements work leads to more lawsuits than the confident declaration. [8 p.252]

Basically, a test case is a set of what-if questions. Freedman [Weinberg][7] and Thomas [14] have developed guidelines that have been adapted for the UA:

- Describe which feature or service (external or internal) your test attempts to cover.
- If the test case is based on a use case (i.e this is a usage test) it is a good idea to refer to the use case name. Remember that the use cases are the source of test cases. In theory, the software is supposed to match the cases, not the reverse. As soon as you have enough of use cases, go ahead and write the test plan for that piece.
- Specify what you are testing and which particular feature (methods). Then, specify what you are going to do to test the feature and what you expect to happen.
- Test the normal use of the objects methods.
- Test the abnormal but reasonable use of the objects methods.
- Test the abnormal and unreasonable use of the objects methods.

- Test the boundary conditions, For example if an edit control accepts 32 characters try 32 then try 40. Also specify when you expect error dialog boxes, when functionality still is being defined.
- Test objects interactions and the messages sent among them, If you have developed sequence diagrams, they can assist you in this process.
- When the revisions have been made, document the cases so they become the starting basis for the follow-up test.
- Attempting to reach agreement on answers generally will raise other what if questions. Add these to the list and answer them, repeat the process until the list is stabilized, then you need not add any more questions.
- The internal quality of the software such as its reusability and extendability should be assessed as well. Although the reusability and extendability are more difficult to test, nevertheless they are extremely important. Software reusability rarely is practiced effectively. The organizations that will survive in the 21st century will be those that have achieved high levels of reusability anywhere from 70-80 percent or more [5]. Griss [9] argues that, although reuse is widely desired and often the benefit of utilizing object technology, may object oriented reuse efforts fail because of too narrow a focus on technology rather than the policies set forth by an organization. He recommends an institutionalized approach to software development in which software assets intentionally are created or acquired to be reusable. These assets then are consistently used and maintained to obtain high levels of reuse thereby optimizing the organizations ability to produce high quality software products rapidly and effectively. Your test case may measure what percentage of the system has been reused say measured in terms of reused lines of code as opposed to new lines of code written.

Specifying results is crucial in developing test cases. You should be test case that are supposed to fail. During such test, it is a good idea to alert to the person running them that failure is expected, Say we are testing a File Open feature. We need to specify the result as follows.

1. Drop down the File menu and select Open.
2. Try opening the following types of files,

A file that is there (should work)

A file that is not there (should get an Error message)

A file name with international characters (should work).

A file type that the program does not open (should get a message or conversion dialog box).

14.5 Test Plan

On paper, it may seem that everything will fall into place with no preparation and a bug-free product will be shipped. However in the real world, it may be a good idea to use a test plan to find bugs and remove them. A dreaded and frequently overlooked activity in software development is writing the test plan. A test plan is developed to detect and identify potential problems before delivering the software to its users. Your users might demand a test plan as one item delivered with the program. In other cases, no test plan is required, but that does not mean you should not have one. A test plan offers a road map for testing activities, whether usability user satisfaction or quality assurance tests. It should state the test objectives and how to meet them.

The test plan need not be very large: in fact, devoting too much time to the plan can be counterproductive. The following steps are needed to create a test plan:

1. Objectives of the test. Create the objectives and describe how to achieve them. For example, if the objective is usability of the system, that must be stated and also how to realize it.
2. Development of a test case. Develop test data, both input and expected output, based on the domain of the data and the expected behaviors that must be tested (more on this in the next section).
3. Test analysis. This step involves the examination of the test output and the documentation of the test results. If bugs are detected, then this is reported and the activity centers on debugging. After debugging, steps 1 through 3 must be repeated until no bugs can be detected.
4. All passed tests should be repeated with the revised program, called regression testing, which can discover errors introduced during the debugging process. When

sufficient testing is believed to have been conducted, this fact should be reported, and testing for this specific product is complete[3].

5. According to Tamara Thomas [14], the test planner at Microsoft, a good test plan is one of the strongest tools you might have. It gives you the chance to be clear with other groups or departments about what will be tested, how it will be tested, and the intended schedule. Thomas explains that, with a good, clear test plan, you can assign testing features to other people in an efficient manner. You then can use the plan to track what has been tested, who did the testing, and how the testing was done. You also can use your plan as a checklist, to make sure that you do not forget to test any features.
6. Who should do the testing! For a small application, the designer or the design team usually will develop the test plan and test cases and, in some situations, actually will perform the tests. However, many organizations have a separate, team, such as a quality assurance group, that works closely with the design team and is responsible for these activities (such as developing the test plans and actually performing the tests). Most software companies also use beta testing, a popular in expensive, and effective way to test software on a select group of the actual users of the system. This is in contrast to alpha testing, where testing is done by in house testers, such as programmers, software engineers, and internal users. If you are going to perform beta testing, make sure to include it in your plan, since it needs to be communicated to your users well in advance of the availability of your application in a beta version.

14.5.1 Guidelines for Developing Test plans

As software gains complexity and interaction among programs is more tightly coupled, planning becomes essential. A good test plan not only prevents overlooking a feature (or features), it also helps divide the work load among other people, explains Thomas.

The following guidelines have been developed by Thomas for writing test plans [14]:

You may have requirements that dictate a specific appearance or format for your test plan. These requirements may be generated by users. Whatever the appearance of your test plan, try to include as much detail as possible about the tests.

The test plan should contain a schedule and a list of required resources. List how many people will be needed, when the testing will be done, and what equipment will be required.

After you have determined what types of testing are necessary (such as black box, white box, top-down, or bottom-up testing), you need to document specifically what you are going to do.

Document every type of test you plan to complete. The level of detail in your plan may be driven by several factors, such as following: How much test time do you have? Will you use the test plan as a training tool for team members?

A configuration control system provides a way of tracking the changes to the code. At a minimum, every time the code changes, a record should be kept that tracks which module has been changed, who changed it, and when it was altered, with a comment about why the change was made. Without configuration control, you may have difficulty keeping the testing in line with the changes, since frequent changes may occur without being communicated to the testers.

Seat A well-thought-out design tends to produce better code and result in more complete testing, so it is a good idea to try to keep the plan up to date. Generally, the older a plan gets, the less useful it becomes. If a test plan is so old that it has become part of the fossil record, it is not terribly useful. As you do not take the end of a project, you will have less time to create plans. If you do not take the time to document the work that needs to be done, you risk forgetting something in the mad dash to the finish line. Try to develop a habit of routinely bringing the test plan in sync with the product or product specification.

At the end of each month or as you reach each milestone, take time to complete routine up dates. This will help you avoid being overwhelmed by being so out of-date that you need to rewrite the whole plan. Keep configuration information on your plan, too. Notes about who made which updates and when can be very helpful down the road.

14.6 Continuous Testing

Software is tested to determine whether it conforms to the specifications of requirements. Software is maintained when errors are found and corrected, and software is extended when new functionality is added to an already existing program. There can be different reasons for testing, Such as to test for potential problem in a proposed design or to compare two or determine which is better, given a specific task or set of tasks.

A common practice among developers is to turn over applications to a quality assurance (QA) group for testing only after development is completed. Since it is not involved in the initial plan, the testing team usually has no clear picture of the system and therefore cannot develop an effective test plan. Furthermore, testing the whole system and detecting bugs is more difficult than testing smaller pieces of the application as it being developed. The practice of waiting until after the development to test an application for bugs and performance could waste thousands of dollars and hours of time.

Testing often uncovers design weaknesses or, at least, provides information you will use. Then, you can repeat the entire process, taking what you have learned and reworking your design, or move onto reprototyping and retesting. Testing must take place on a continuous basis, and this refining cycle must continue throughout the development process until you are satisfied with the results. During this iterative process, your prototype will be transformed incrementally into the actual application. The use cases and usage scenarios can become test scenarios and, therefore, will drive the test plans. Here are the steps to successful testing:

- Understand and communicate the business case for improved testing.
- Develop an internal infrastructure to support continuous testing.
- Look for leaders who will commit to and own the process.
- Measure and document your findings in a defect recording system.
- Publicize improvements as they are made and let people know what they are doing better.

14.7 Myer's Debugging Principles

I conclude this discussion with the Myers's bug location and debugging principles [13]:
both

1. Bug Locating principles

- Think
- If you reach an impasse, sleep on it
- If the impasse remains, describe the problem to someone else.
- Use debugging tools (this is slightly different from Myers suggestion).

- Experimentation should be done as a last resort (this is slightly different from Myers suggestion)

2. Debugging Principles.

- where there is one bug, there is likely to be another.
- Fix the error not just the symptom of it.
- The probability of the solution being correct drops as the size of the program increase.
- Beware of the possibility that an error correction will create a new error (this is less of a problem in an object oriented environment)

14.8 Case Study: Developing Test cases for the valAnet Bank ATM System

In Chapter 6, we identified the scenarios or use cases for the ViaNet bank ATM system. The ViaNET bank. Transaction (see Figures 6-9, 6-10 and 6-11). Here again is a list of the use cases that drive any object oriented activities, including the usability testing:

- Bank Transaction
- Checking Transaction History
- Deposit Checking
- Deposit Savings
- Savings Transaction History
- withdraw checking
- withdraw Savings.
- valid/Invalid PIN.

The activity diagrams and sequence/collaboration diagrams created for these use cases are used to develop the usability test cases. For example, you can draw activity and sequence diagrams to model each scenario that exists when a bank client withdraws, or needs information on an account. walking through the steps can assist you in developing a usage test case.

Let us develop a test for the activities involved in the ATM transaction based on the use cases identified so far. A test case general format is his: If it receives certain input, it produces certain output.

1. If a bank client inserts his or her card, the system will respond by requesting the password.
2. If the password is incorrect the system must display the bad password message, eject the card, and request the client to take the card.
3. After the transaction is completed, the system should show the main screen.
4. If a bank client selected an incorrect command when entering the transaction, the system will respond immediately by indicating that some error has been made. The system should allow the customer to correct the mistake and not force him or her to make a withdrawal if the intention was to make a deposit.
5. In an unauthorized customer attempts access to someone else's account, the system should record the time, date, and if possible the identity of the person (not based on the original use cases).
6. If the cash is low, the system will notify the bank for additional money (not based on the original use cases).
7. If an act of vandalism is committed, the system will sound an alarm and call security (not based on the original use cases).
8. If the customer enters a wrong PIN number, the system will give the customer three more chances to correct it; otherwise, it will abort the operation.

And so on.

This is an iterative process; at every iteration, a new issue will be exposed that will help you refine the system. A positive side effect of a test plan is that it might raise some new questions. As the test cases are performed, new test cases will come up, and you should repeat the test. Gause and Weinberg argue that, even if you never attempt to answer the test plans, just developing them might lead you to other, overlooked issues.

14.9 Summary

Testing may be conducted for different reasons. Quality assurance testing looks for potential problems in a proposed design. In this chapter, we looked at guidelines and the basic concept of test plans and saw that, for the most part, use cases can be used to describe the usage test cases. Also, some of the techniques, strategies, and approaches for quality assurance testing and the impact of object orientation on testing are discussed.

Testing is a balance of art, science, and luck. It may seem that everything will fall into place without any preparation and a bug-free product will be shipped. However, in the real world, we must develop a test plan for locating and removing bugs. A test plan offers a road map for testing activity; it should state test objectives and how to meet them. The plan need not be very large; in fact, devoting too much time to the plan can be counterproductive.

There are no magic tricks to debugging; however, by selecting appropriate testing strategies and a sound test plan, you can locate the errors in your system and fix them by utilizing debugging tools.

Once you have created fully tested and debugged classes of objects, you can put them into a library for use or reuse. The essence of an object-oriented system is that you can take for granted that these fully tested objects will perform their desired functions and seal them off in your mind like black boxes.

Testing must take place on a continuous basis, and this refining cycle must continue throughout the development process until you are satisfied with the results.

Questions

1. What is a successful test when you are looking for bugs?
2. What is white box testing?
3. What is path testing?
4. What is statement testing coverage?
5. What is branch testing coverage?
6. What is regression testing?
7. What is black box testing?
8. When would you use white box testing?
9. What is the importance of developing a test case?
10. Why are debugging tools important?
11. What basic activities are performed in using debugging tools?

LESSON - 15

SYSTEM USABILITY AND MEASURING USER SATISFACTION

Objectives:

After studying this lesson you should be able to

- Usability testing.
- User satisfaction testing.
- The user satisfaction test template
- Case study of VIANET bank ATM system.

Introduction

Quality refers to the ability of products to meet the users needs and expectations. The task of satisfying user requirements is the basic motivation for quality. Quality also means striving to do the things right the first time, while always looking to improve how things are being done. Sometimes this even means spending more time in the initial phases of a project-such as analysis and design -making sure that you are doing the right things. Having to correct fewer problems means significantly less wasted time and capital. When all the losses caused by poor quality are considered, high quality usually costs less than poor quality.

Two main issues in software quality are validation or user satisfaction and verification or quality assurance. There are different reasons for test can focus on comparing two or more designs to determine which is better, given a specific task or set of tasks. Usability testing is different from quality assurance testing in that, rather than finding programming defects, you assess how well the interface or the software fits the use cases, which are the reflections of users needs and expectations. To ensure user satisfaction, we must measure it throughout the system development with user satisfaction test. Furthermore, these tests can be used as a communication vehicle between designers and users [3]. In the next section, we look at user satisfaction tests that can be invaluable in developing high quality software. We learned that the process of designing view layer classes consists of the following steps.

1. Macro-level UI Design Process-Identifying view layer objects.
2. Macro-level UI Design Activities.
3. Testing usability and user satisfaction.
4. Refining and iterating the design

In spite of the great effort involved in developing user interfaces and the potential costs of bad ones, many user interface are not evaluated for their usability or acceptability to users, risking failure in the field. Bad user interfaces can contribute to human error, possibly resulting in personal and financial damages [2]. Usability should be a subset of software quality characteristics. This means that usability must be placed at the same level as other characteristics, such as reliability correctness, and maintainability [1]. Usability is an important characteristic in determining the quality of the product. In this chapter, we look at usability testing and issues regarding user satisfaction and its measurement. We study how to develop user satisfaction and usability tests that are based on the use cases. The use cases identified during the analysis phase can be used in testing the design. Once the design is complete, you can walk users through the steps of the scenarios to determine if the design enables the scenarios to occur as planned.

15.1 Usability Testing

The International Organization for Standardization (ISO) defines usability as the effectiveness, efficiency and satisfaction with which a specified set of users can achieve a specified set of tasks in particular environments. The ISO definition requires.

- Defining tasks. What are the tasks?
- Defining users. Who are the users?
- A means for measuring effectiveness, efficiency, and satisfaction. How do we measure usability?

The phrase two sides of the same coin is helpful for describing the relationship between the usability and functionality of a system. Both are essential for the development of high quality software [4]. Usability testing measures the ease of use as well as the degree of comfort and satisfaction users have with the software products with poor usability can be difficult to learn, complicated to operate, and misused or not used at all. Therefore, low product usability leads to high costs for users and a bad reputation for the developers.

Usability is one of the most crucial factors in the design and development of a product, especially the user interface. Therefore, usability testing must be a key part of the UI design process. Usability testing should begin in the early stages of product development for example it can be used to gather information about how users do their work and find out their tasks, which can complements use cases. You can incorporate your findings into the usability test plan and test cases. As the design progresses usability testing continues to provide valuable input for analyzing initial design concepts and in the later stages of product development, can be used to test specific product tasks. especially the UI.

Usability test cases begin with the identification of use cases that can specify the target audience, tasks and test goals. When designing a test focus on use cases or tasks not features. Even if your goal is testing specific features, remember that your users will use them within the context of particular tasks. It also is a good idea to run a pilot test to work the equipment work smoothly.

Test cases must include all use cases identified so far. Recall from Chapter 4 that the use case can be used through most activities of software development. Furare traceable across requirements, analysis design implementation, and testing. The main advantage is that all design traces directly back to the user requirements. Use cases and usage scenarios can become test scenarios and therefore the use case will drive the usability user satisfaction, and quality assurance test cases.

15.1.1 Guidelines for Developing Usability Testing

Many techniques can be used gather usability information. In addition to use cases, focus groups can be helpful for generating initial ideas or trying out new ideas. A focus group requires a moderator who directs the discussion about aspects of a task or design but allows participants to freely express their options.

Usability tests can be conducted in a one-on one fashion, as demonstration, or as a “walk through”, in which you take the users through a set of sample scenarios and ask about their impressions along the way. In a technique called the wizard of oz, a testing specialist simulates the interaction of an interface. Although these latter techniques can be valuable, they often require a trained, experienced test coordinator [5]. Let us take a look at some guidelines for developing usability testing:

- The usability testing should include all of a software's components.
- Usability testing need not be very expensive or elaborate, such as including trained specialists working in a soundproof lab with one-way mirrors and sophisticated recording equipment. Even the small investment of tape recorder, stopwatch, and notepad in an office or conference room can produce excellent results.
- Similarly, all tests need not involve many subjects. More typically, quick, iterative tests with a small, well-targeted sample of 6 to 10 participants can identify 80-90 percent of most design problems.
- Consider the user's experience as part of your software usability. You can study 80-90 percent of most design problems with as few as three or four users if you target only a single skill level of users, such as novices or intermediate level users.
- Apply usability testing early and often.

15.1.2 Recording the Usability Test

When conducting the usability test, provide an environment comparable to the target setting; usually a quiet location, free from distractions is best. Make participants feel comfortable. It often helps to emphasize that you are testing the software, not the participants. If the participants become confused or frustrated, it is no reflection on them. Unless you have participated yourself, you may be surprised by the pressure many test participants feel. You have participated yourself, you may be surprised by the pressure many test participants feel. You can alleviate some of the pressure by explaining the testing process and equipment.

Tandy Trower, director of the Advanced User Interface group at Microsoft, explains that the users must have reasonable time to try to work through any difficult situation they encounter. Although it generally is best not to interrupt participants during a test, they may get stuck or end up in situations that require intervention. This need not disqualify the test data, as long as the test coordinator carefully guides or hints around a problem. Begin with general hints before moving to specific advice. For more difficult situations, you may need to stop the test and make adjustments. Keep in mind that less intervention usually yields better results. Always record the techniques and search patterns users employ when attempting to work through a difficulty and the number and type of hints you have to provide them.

Ask subjects to think aloud as they work, so you can hear what assumptions and interface they are making. As the participants work, record the time they take to perform a task as well as any problems they encounter. You may want to follow up the session with the user satisfaction test (more on this in the next section) and a questionnaire that asks the participants to evaluate the product or tasks they performed.

Record the test results using a portable tape recorder or, better, a video camera. Since even the best observer can miss details, reviewing the data later will prove invaluable. Recorded data also allows more direct comparison among multiple participants. It usually is risky to base conclusions on observing a single subject. Recorded data allows the design team to review and evaluate the results.

Whenever possible, involve all members of the design team in observing the test and reviewing the results. This ensures a common reference point and better design solutions as team members apply their own insights to what they observe. If direct observation is not possible, make the recorded results available to the entire team.

To ensure user satisfaction and therefore high-quality software, measure user at the user satisfaction test, which can be an invaluable tool in developing high-quality software.

15.2 Satisfaction Test

User satisfaction testing is the process of quantifying the usability test with some measurable attributes of the test, such as functionality, cost, or ease of use. Usability can be assessed by defining measurable goals such as.

- 95 percent of users should be able to find how to withdraw money from the ATM machine without error and with no formal training.
- 70 percent of all users should experience the new function as “a clear improvement over the previous one.”
- 90 percent of consumers should be able to operate the VCR within 30 minutes.

Furthermore, if the product is being built incrementally, the best measure of user satisfaction is the product itself since you can observe how users are using it or avoiding it. A positive side effect of testing with a prototype is that you can observe how people actually use the software. In addition to prototyping and usability testing another tool that can assist us in developing high

quality software is measuring and monitoring user satisfaction during software development, especially during the design and development of the user interface. Gause and Weinberg have developed a user satisfaction test that can be used along with usability testing. Here are the principal objectives of the user satisfaction test.

- As a communication vehicle between designers as well as between users and designers.
- To detect and evaluate changes during the design process.
- To provide a periodic indication of dissatisfaction for remedy.
- To provide a clear understanding of just how the completed design is to be evaluated.

Even if the results are never summarized and no one fills out a questionnaire, the process of creating the test itself will provide useful information. Additionally, the test is inexpensive easy to use and it is educational to both those who administer it and those who take it.

15.2.1 guidelines for developing a user satisfaction test

The format of every user satisfaction test basically the same but its content is different for each project. Once again the use cases can provide you with an excellent source of information throughout this process. Furthermore you must work with the user or clients to find out what attributes should in the test. Ask the user to select a limited number of attributes by which the final product can be evaluated. For example the user might select the following attributes for a customer tracking system ease of use functionality cost, intuitiveness of user interface and reliability.

A test based on those attributes is shown in 15.1. once these attributes have been identified they can play a crucial role in the evaluation of the final product. Keep these attributes in the foreground rather than make assumptions about how the design will be evaluated. The user must use his or her judgment to answer each questions by selecting a number between 1 and 10 with 10 as the most favorable and 1 as the least. Comments often are the most significant part of the test.

How do you rate the customer tracking project at this time?

	10	9	8	7	6	5	4	3	2	1	
ease of use very easy											very hard
functionally very functional											not function
Cost very inexpensive											very expensive
intuitive UI very intuitive											very hardto follow
Reliability very reliable											not reliable
Comments:											
☐ I have more to say: I would like to see you											

Figure 15-1

Gause and Weinberg raise the following important point in conducting a user satisfaction test.

15.3 A tool for analyzing user satisfaction: the user satisfaction test temple

Commercial off the shelf (COTS) software tools are already written and a few are available for analyzing and conducting user satisfaction tests. However here I have selected an electronic spreadsheet to demonstrate how it can be used to record and analyze the user satisfaction test.

The user satisfaction test spreadsheet automates many bookkeeping tasks and can assist in analyzing the user satisfaction test result. Furthermore it offers a quick start for creating a

user satisfaction test for a particular project. Recall from the previous section that the tests need not involve many subjects.

More typically quick iterative tests with a small well targeted sample of 6 to 10 participants can identify 80-90 percent of most design problems. The spreadsheet should be designed to record responses from up to 10 users. However if there are inputs from more than 10 users, it must allow for that see figure 15-2 and 15-3.

Measuring User Satisfaction
Project Name: Customer Tracking System

User	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ease of Use	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Functionality	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Reliability	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Comments																														

Figure 15-2

Measuring User Satisfaction
Project Name: Customer Tracking System

Test Period	Period 1	Period 2	Period 3	Period 4	Period 5	Period 6
Ease of Use	1	2	3	4	5	6
Functionality	1	2	3	4	5	6
Reliability	1	2	3	4	5	6
Comments						

Figure 15-3

One use a tool like this if that it shows patterns in user satisfaction level. For example a shift in the users satisfaction rating

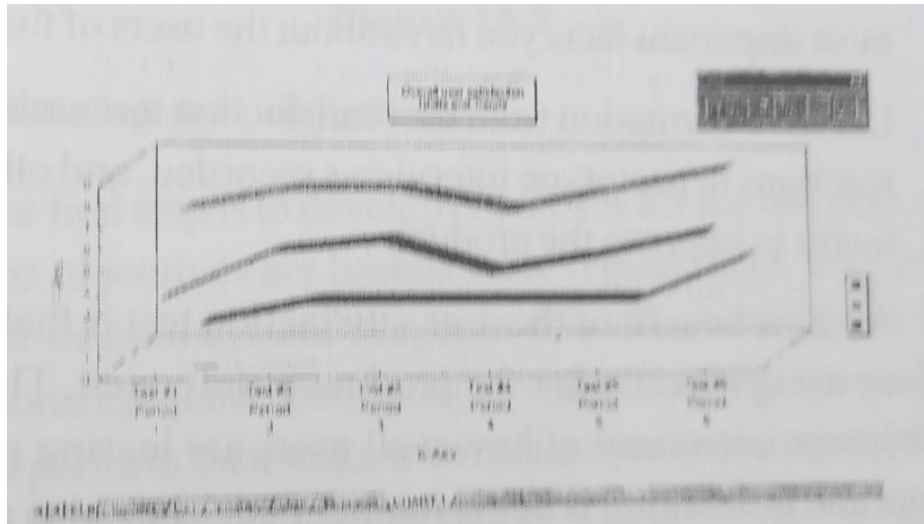


Figure 15-4

Gause and Weinberg explain that this shift is sufficient cause to follow up with an interview. The user satisfaction test can be tool for finding out what attributes are important or unimportant. An interesting side effect of developing a user satisfaction test is that you benefit from it even if the test is never administered to anyone; it still provides useful information. However performing the test regularly helps to keep the user involved in the system. It also helps you focus on user wishes. Here is the user satisfaction cycle that as bon suggested by gause and Weinberg:

- Creating a user satisfaction test for your own project. Creating a custom form that fits the projects needs and the culture of your organization. Use cases are a great source of information however make sure to involve the user in creation of the test.
- Conduct the test regularly and frequently.
- Read the comments very carefully especially if they express a strong feeling. Never forget that feelings are facts, the most important facts you have about the users of the system.
- Use the information from user satisfaction test usability test, reactions to prototype interviews recorded, and other comments to improve the product.

Another benefit of the user satisfaction test is that you can continue using it even after the product is delivered. The results then become a measure of how well users are learning to use the product and how well it is being maintained. They also provide a starting point for initiating follow up projects.

15.4 case study: developing usability test plans and test cases for the VIANET bank ATM system

We learned that test plans need not be very large in fact devoting too much time to the plans can be counterproductive. Having this in mind let us develop a usability test plan for the ViaNet ATM kiosk by going through the followings steps.

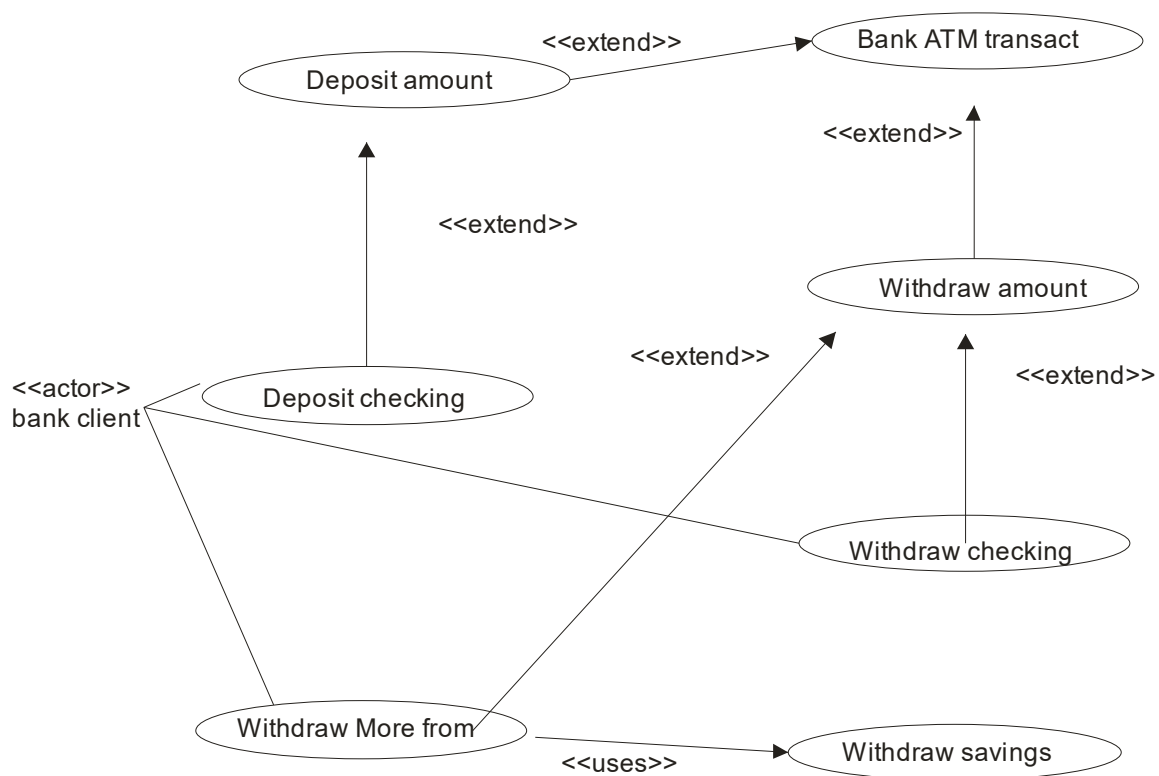


Figure 15.5

15.4.1 Develop test objectives

The first step is to develop objectives for the test plan. Generally, test objectives are based on the requirement, use case, or current or desired system usage. In this case, ease of use is the most important requirement, since the ViaNet bank customers should be able to perform their tasks with basically no training and are not expected to read a user manual before withdrawing money from their checking accounts. Here are the objectives to test the usability of the ViaNet bank ATM kiosk and its user interface:

- 95 percent of users should be able to find out how to withdraw money from the ATM machine without error or any formal training.
- 90 percent of consumer should be able to operate the ATM within 90 seconds.

15.4.2 Develop test cases

Test cases for usability testing are slightly different from test cases for quality assurance. Basically here we are not testing the input and expected output but how users interact with the system once again the use cases created during analysis can be used to develop scenarios for the usability test. The usability test scenarios are based on the following use cases:

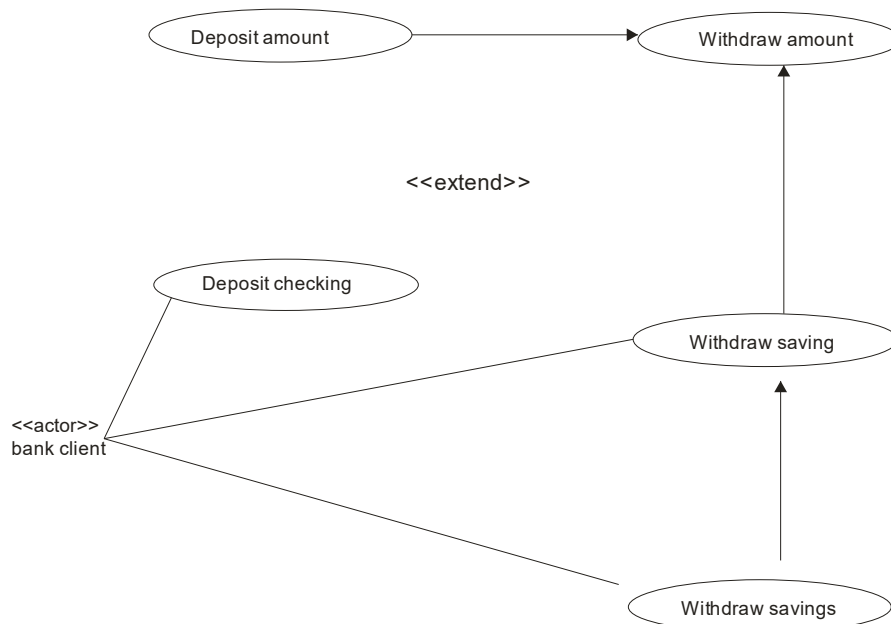


Figure 15.6

Deposit checking see figure 15.5

Withdraw checking see figure 15.5.

Deposit savings see figure 15.6

Withdraw savings see figure 15.6.

Savings transaction history see figure 15.7

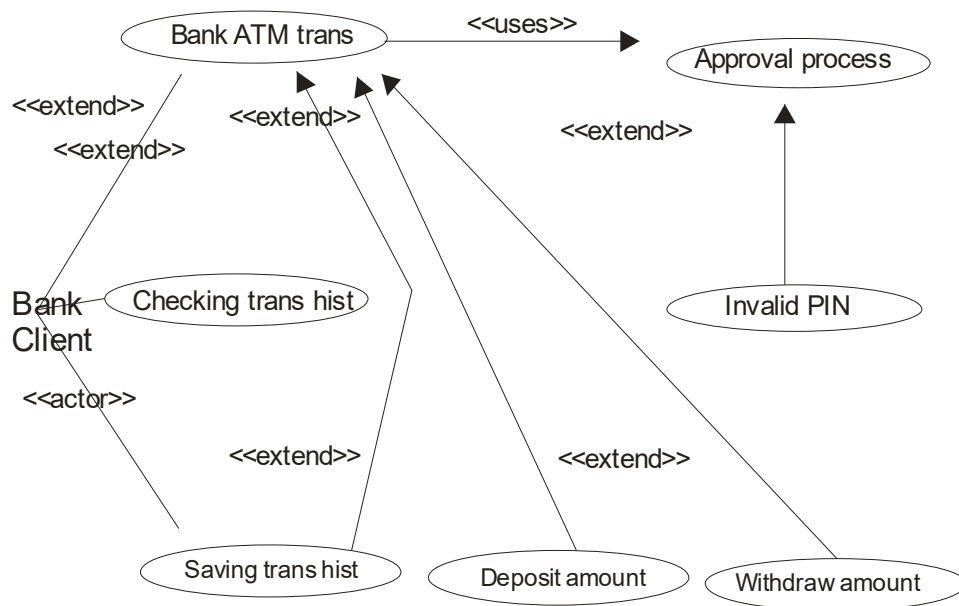


Figure 15.7

Checking transaction history see figure 15.7

Next we need to select a small number of test participants 6 to 10 who have never before used the kiosk and ask them to perform the following scenarios based on the case:

1. Deposit \$1000.00 to your checking account.
2. Withdraw \$20 from your checking account.
3. Deposit \$400 to your savings account.
4. Withdraw \$25 from saving account.
5. Get your savings account transaction history.
6. Get your checking account transaction history.

Start by explaining the testing process and equipment to the participants to ease the pressure. Remember to make participants feel comfortable by emphasizing that you are testing the software not them.

If they become confused or frustrated, it is no reflection on them but the poor usability of the system. Make sure to ask them to think aloud as they work so you can hear what assumptions and inferences they are making. After all if they cannot perform these tasks with ease then the system is not useful.

As the participants work record the time they take to perform a task as well as any problems they encounter. In this case we used the kiosk video camera to record the test results along with a tape recorder. This allowed the design team to review and evaluate how the participants interacted with the user interface, like those developed.

For example look for things such as whether they are finding the appropriate buttons easily and buttons are the right size. Once the test subjects complete their tasks conduct a user satisfaction test to measure their level of satisfaction with the kiosk. The format of the user satisfaction test is basically the same as the one we studied earlier in this lesson see figure 15.1, but its content is different for the Vianet bank. The users, use cases, and test objects should provide the attributes to be included in the test. Here the following attributes have been selected since the ease of use is the main issue of the user interface.

- Is easy to operate.
- Buttons are the right size and easily located.
- Is efficient to use.
- Is fun to use.
- Is visually pleasing.
- Provides easy recovery from errors.
- Based on these attributes the test shown in figure 15.8 can be performed. Remember, as explained earlier, these attributes can play a crucial role in the evaluation of the final product.

15.4.3 Analyze the test

The final step is to analyze the tests and document the test results. Here we need to answer questions such as these. What percentage were able to operate the ATM within 90 seconds or without error? Were the participants able to find out how to withdraw money from the ATM machine with no help? The results of the analysis must be examined.

How do you rate the Vianet bank ATM kiosk interface?		
	10 9 8 7 6 5 4 3 2 1	
it ease to operate very easy	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	very hard
	10 9 8 7 6 5 4 3 2 1	
buttons very appropriate	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	not appropriate
	10 9 8 7 6 5 4 3 2 1	
is efficient very efficient	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	very inefficient
	10 9 8 7 6 5 4 3 2 1	
is fun fun	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	no fun
	10 9 8 7 6 5 4 3 2 1	
is visually very pleasing	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	not pleasing
	10 9 8 7 6 5 4 3 2 1	
provides easy	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	not at all
Comments : ☐ I have more to say: I would like to see you		

Figure 15-8

We also need to analyze the results of user satisfaction tests. The USTS described earlier or a tool similar to it can be used to record graph the results of user satisfaction tests. As we learned earlier a shift in user satisfaction pattern indicates that something is happening and a follow up interview is needed to find out the reasons for the changes.

The user satisfaction test can be used as a tool for finding out what attributes are important or unimportant. For example based on the user satisfaction test, we might find that the users do not agree that the system “is efficient to use” and it got a low score. After the follow of interview, it became apparent that participants wanted in addition to entering the amount for withdrawal, to be able to select from a list with predefined values. This would speed up the process at the ATM kiosk. Based on the result of the test, the UI was modified to reflect the wishes of the users see figure 15.9

How do you rate the Vianet bank ATM kiosk interface?											
	10	9	8	7	6	5	4	3	2	1	
Is easy to operate:	Very easy to use ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ Very hard to use										
Buttons are right size and easily can be located:	Very appropriate ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ Not appropriate										
Is efficient to use :	Very efficient ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ Very inefficient										
Is fun to use:	Very ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ Not fun at all										
Is visually pleasing :	Very pleasing ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ Not pleasing										
Provides easy recovery from errors:	Very easy recovery ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ Not at all										
Comments:											
☐ I have more to say: I would like to see you											

Figure 15-9

Summary

- The main point here is that you must focus on the user's perception. Many system that are adequate technically have failed because of poor user perception.
- This can be prevented or at least minimized by utilizing usability and user satisfaction tests as part of your UI design process.
- Usability testing begins with defining the target audience and test goals.

- When designing a test, focus on tasks, not features.
- Even if your goal is testing specific features, remember that your customer will use them within the context of particular tasks.
- The use cases identified during analysis can be used in testing your design.
- Once the design is complete you can walk users through the steps of the scenarios to determine if the design enables the scenarios to occur as planned.
- Performing the test regularly helps keep the user actively involved in the system development.

Questions

1. What is validation?
2. What is verification?
3. What is quality?
4. Why should usability testing include all components of the software?
5. What is a user satisfaction test?
6. Why do we need to measure user satisfaction?
7. How do you develop a custom form for a user satisfaction test?

LESSON - 16

TESTING STRATEGIES

Objectives:

- **Verification and validation**
- **Strategic issues**
- **Unit testing**
- **Integration testing**
- **Validation TESTING**
- **System testing**
- **Art of debugging**

Introduction

A strategy for software testing integrates software test case design methods into a well planned series of steps that result in the successful construction of software. A important a software testing strategy provides a road ma,, for the software developer, the quality assurance organization, and the customer a road map that describes the steps to be conducted as part of testing, when these steps are planned and then undertaken, and how much effort, time, and resources will be required.

Therefore any testing strategy must incorporate test planning test case design, test execution and resultant data collection and evaluation. The strategy must be rigid enough to promote reasonable planning and management tracking as the project greases. These approaches and philosophies are what we shall call strategy.

16.1 A strategic approach to software testing

Testing is a set of activities that can be planned in advance and conducted systematically. For this reason a template for software testinga set of steps into which we can place specific test case design methods should be defined for the software engineering process.

A number of software testing strategies have been proposed in the literature. All provide the software developer with a template for testing and all have the following generic characteristics:

- Testing begins at the module level and works “outward” toward the integration of the entire computer based systems.
- Different testing techniques are appropriate at different points in time.
- Testing is conducted by the developer of the software and an independent test group.
- Testing and debugging are different activities, but debugging must be accommodated in any testing strategy.

16.1.1 Verification and validation

Software testing is one element of a broader topic that is often referred to as verification and validation. Verification refers to the set of activities that ensure that software correctly implements a specific function. Validation refers to a different set of activities that ensure that the software that has been built is traceable to customer requirements.

Verification: “Are we building the product right?”

Validation: “Are we building the right product?”

The definition of V & V encompasses many of the activities that we have referred to as software quality assurance. The activities required to achieve it may be viewed as set of components depicted in figure 16.1. software engineering provide the foundation from which quality is built. Analysis, design and construction methods act to enhance quality by providing uniform techniques and predictable results.

Testing provides the last bastion from which quality can be assessed and more pragmatically, error can be uncovered. But testing should not be viewed as a safety net. As they say “ you can’t test in quality. If it’s not there before you begin testing, it won’t be there when you’re presents finished testing”. It is important to note that verification and validation encompass a wide array of SQA activities that include formal technical reviews, quality and configuration audits, performance monitoring, simulation, feasibility study, documentation review, database review, algorithm analysis, development testing, qualification testing, and installation testing.

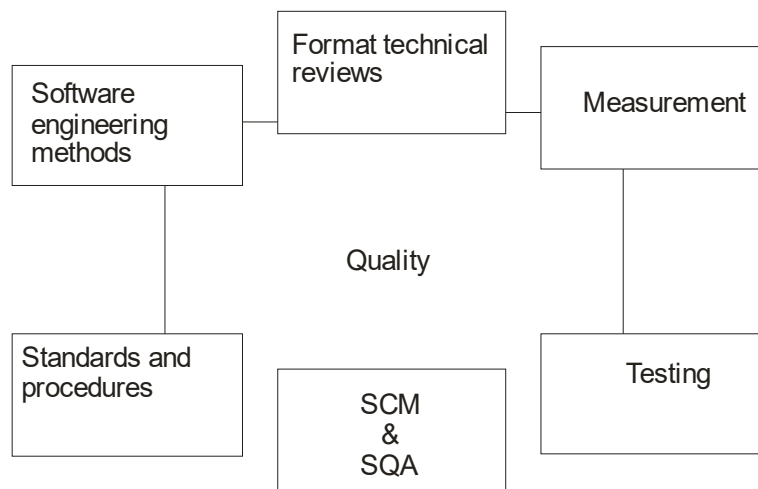


Figure 16.1

16.1.2 Organizing for software testing

For every software project, there is an inherent conflict of interest that occurs as testing begins. The people who have built the software are now asked to test the software. From a psychological point of view software analysis and design are constructive tasks the software engineer creates a computer program its documentation and related data structures. When testing commences, there is a subtle, yet definite, attempt to “break” the thing that the software engineer has built. From the point of view of the builder testing can be considered to be destructive.

There are often a number of misconceptions that can be erroneously inferred from the above discussion:

1. that the developer of software should do no testing at all.
2. that the software should be “tossed over the wall” to strangers who will test it mercilessly.
3. that testers get involved with the project only when the testing steps are about to begin. Each of these statements is incorrect.

The software developer is always responsible for testing the individual units of the program, ensuring that each performs the function for which it was designed. In many cases, the developer also conducts integration testing. The role of an independent test group (ITG) is to remove the inherent problems associated with letting the builder test the thing that has been built.

The developer and the ITG work closely throughout a software project to ensure that thorough tests will be conducted. While testing is conducted, the developer must be available to correct errors that are uncovered.

16.1.3 A software testing strategy

The software engineering process may be viewed as a spiral illustrated in figure 16-2, initially system engineering defines the role of software and leads to software requirements analysis where the information domain, function, behavior, performance, constraints and validation criteria for software are established. Moving inward along the spiral, we come to design and finally to coding.

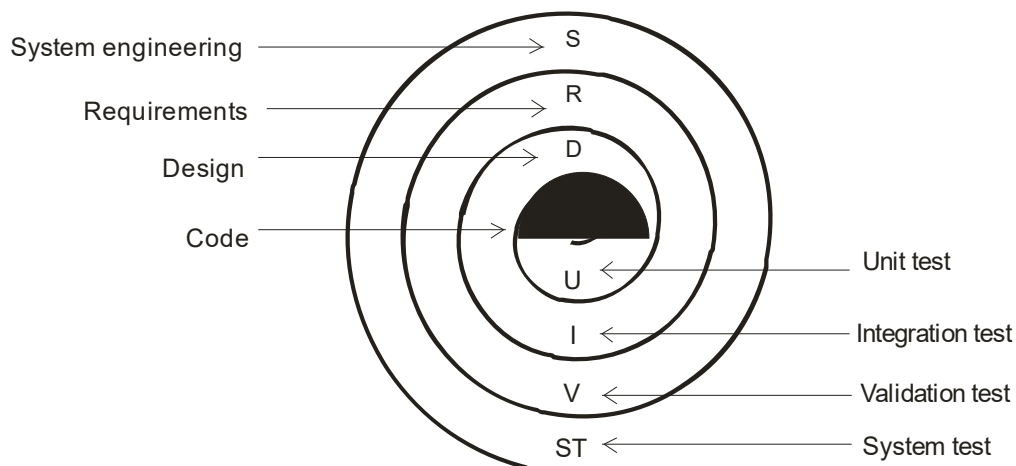


Figure 16.2

To develop computer, we spiral in along streamlines that decrease the level of abstraction on each turn. A strategy for software testing may also be viewed in the context of the spiral. Unit testing begins at the vortex of the spiral and concentrates on each unit of the software as implemented in source code. Testing progresses by moving outward along the spiral to integration testing, where the focus is on design and the construction of the software validation testing.

We arrive at system testing where the software and other system elements are tested as a whole. Considering the process from a procedural point of view testing within the context of software engineering is actually a series of four steps that are implemented sequentially. The steps are shown in figure 16-3. initially tests focus on each module individually, assuring that it functions properly as a unit. Hence, the name unit testing. Unit testing makes heavy use of

white box testing techniques, exercising specific paths in a module's control structure to ensure complete coverage and maximum error detection.

Black box test case design techniques are the most prevalent during integration although a limited amount of white box testing may be used to ensure coverage of major control paths. After the software has been integrated, a set of high order tests are conducted. Validation criteria must be tested. Validation testing provides final assurance that software meets all functional, behavioral and performance requirements. Black box testing techniques are used exclusively during validation.

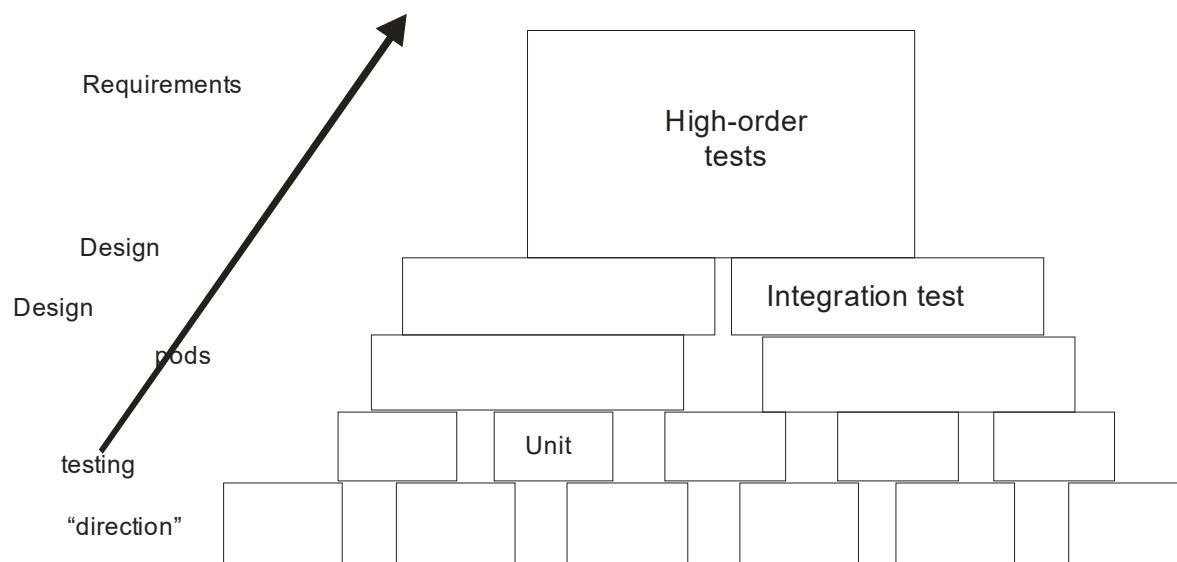


Figure 16.3

16.1.4 Criteria for completion of testing

A classic question arises every time software testing is discussed: "when are we done testing how do we know that we've tested enough?" one response to the above question is this: "you are never done testing the burden simply shifts from you to your customer." Every time the customer/user executes a computer program, the program is being tested on a new data. Using statistical modeling and software reliability theory, models of software failures as a function of execution time can be developed. A version of the failure model, called a logarithmic Poisson execution time model, takes the form:

$$F(D) - (1/p)f_n(lopt+1)$$

Where $f(1)$ = cumulative number of failures that are expected to occur once the software has been tested for a certain amount of execution t .

L_0 =- the initial software failure intensity at the beginning of

P = the exponential reduction in failure intensity as error are uncovered and repairs are made.

The instantaneous failure intensity, $1(t)$ can be derived by taking the derivative of $f(1)$

$$L(0) = l_0/(lopt + 1)$$

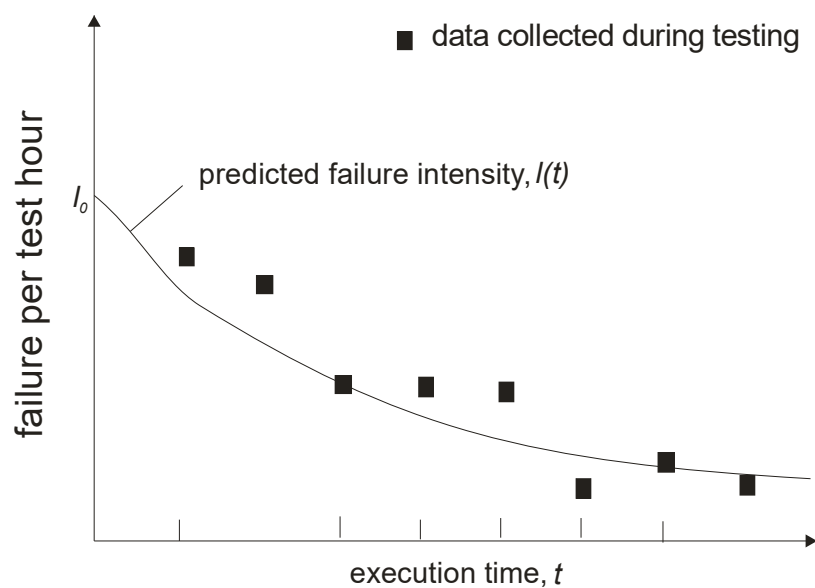


Figure 16.4

Using the relationship noted in equation testers can predict the drop off of errors as testing progresses. The actual error intensity can be plotted against the predicted curve figure 16-4. if the actual data gathered during testing and the logarithmic Poisson execution time model are reasonable close to one another over a number of data points, the model can be used to predict total testing time required to achieve an acceptably low failure intensity.

16.2 Strategic issues

Tom Gilb argues that the following issues must be addressed if a successful software testing strategy is to be implemented:

- **specify product requirements in quantifiable manner long before testing commences.** Although the overriding objectives of testing is to find errors, a good testing strategy also assesses other quality characteristics such as portability, maintainability and usability.
- **State testing objectives explicitly.** The specific objectives of testing should be stated in measurable terms.
- **Understand the users of the software and develop a profile for each user category.** Use cases which describe the interaction scenario for each class of user can reduce overall testing effort by focusing testing on actual use of the product.
- **Develop a testing plan that emphasizes “rapid cycle testing”.** The feedback generated from these rapid cycle tests can be used to control quality levels and the corresponding test strategies.
- **Build” robust “software that is designed to test itself.** Software should be designed in a manner that uses anti bugging techniques.
- **Use effective formal technical reviews as a filter prior to testing.** Formal technical reviews can be as effective as testing in uncovering errors.
- **Develop a continuous improvement approach for the testing process.** The test strategy should be measured. The metrics collected during testing should be used as part of a statistical process control approach for software testing.

16.3 Unit testing

Unit testing focuses verification effort on the smallest unit of software design the module. Using the procedural design description as a guide, important control paths are tested to uncover errors within the boundary of the models. The relative complexity of test and uncovered errors is limited by the constrained scope established for unit testing. The unit test is normally white box oriented and the step can be conducted in parallel for multiple modules.

16.3.1 Unit test consideration

The test that occur as part of unit testing are illustrated schematically in figure 16-5. the module interface is tested to ensure that information properly flows into and out of the program unit under test. Boundary conditions are tested to ensure that the module operates properly at boundaries established to limit or restrict processing. Test of data flow across a module interface are required before any other test is initiated. If data do not enter and exit properly, all other tests are moot. In his text on software testing.

1. number of input parameters equal to number of arguments?

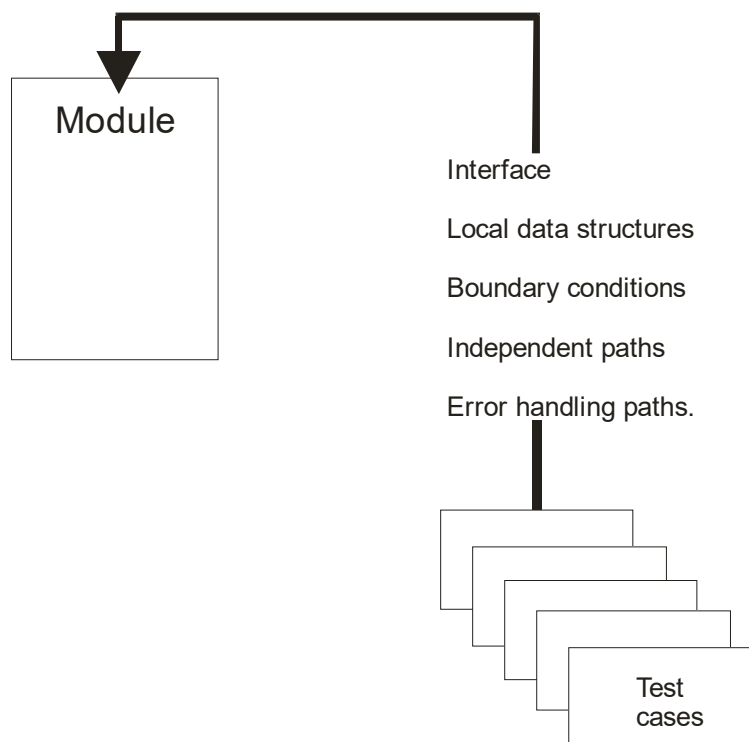


Figure 16.5

2. Parameter and argument attributes match?
3. parameter and argument units systems match?
4. constraints passed as arguments?
5. Global variable definitions consistent across modules?

The local data structure for a module is a common source of Test cases should be designed to uncover errors in the following categories:

1. improper or inconsistent typing.
2. erroneous initialization or default values.
3. Incorrect variable names
4. inconsistent data types.
5. underflow, overflow, and addressing exceptions.

Among the more common error in computing are

1. misunderstood or incorrect arithmetic precedence.
2. mixed mode operations.
3. incorrect initialization.
4. precision inaccuracy.
5. incorrect symbolic representation of an expression.

Good design dictates that error conditions be anticipated and error handling paths set up to reroute or cleanly terminate processing when an error does occur. Yourdon calls this approach anti bugging. Unfortunately, there is a tendency to incorporate error handling into software and then never test it. A real life anecdote may serve to illustrate.

Among the potential error that should be tested when error handling is evaluated are:

1. error description is unintelligible.
2. error noted does not correspond to error encountered.
3. Error condition causes system intervention prior to error handling.
4. Error description does not provide enough information to assist in the location of the cause of the error.
5. Exception condition processing is incorrect.

16.3.2 Unit test procedures

Unit testing is normally considered as an adjunct to the coding step. After source level code has been developed, reviewed, and verified for correct syntax unit test case design begins. A review of design information provides guidance for establishing test cases that are likely to uncover errors in each of the categories discussed above. Because a module is not a standalone program driver and/or stub software must be developed for each unit test. The unit test environment is illustrated in figure 16-6. In most applications a driver is nothing more than a “main program” that accepts test case data, passes such data to the module and prints relevant results. Stubs serve to replace modules that are subordinate to the module to be tested. A stub or “dummy subprogram” uses the subordinate module’s interface.

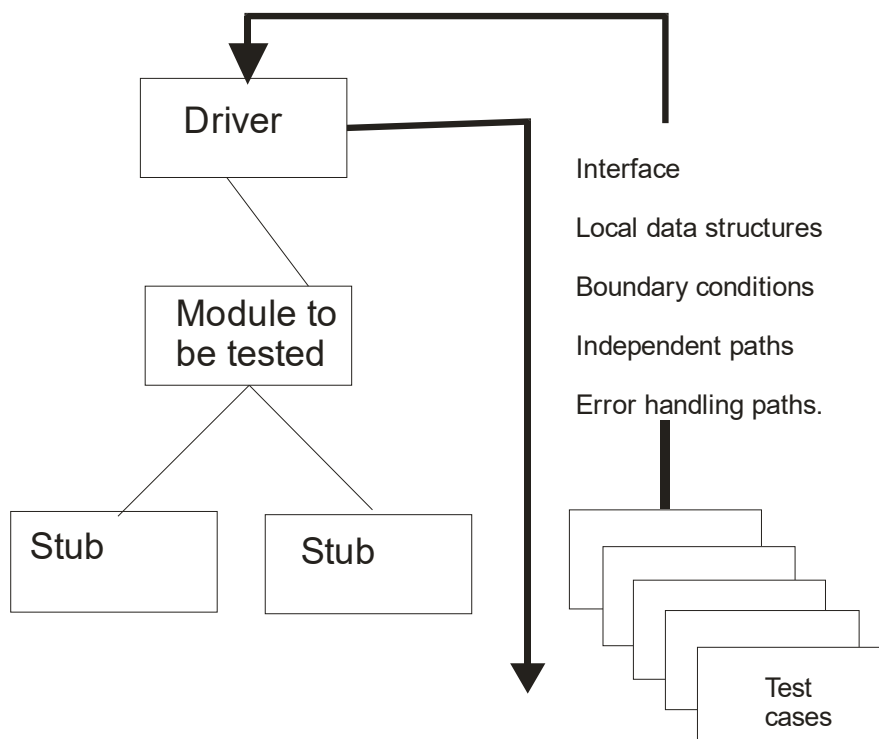


Figure 16.6

16.4 Integration testing

A neophyte in the software world might ask a seemingly legitimate question once all modules have been unit tested: “if they all work individually, why do you doubt that they’ll work

when we put them together” integration testing is a systematic technique for constructing the program structure while conducting tests to uncover errors associated with interfacing. The objective is to take unit tested modules and build a program structure that has been dictated by design.

There is often a tendency to attempt non incremental integration; that is to construct the program using a “big bang” approach. All modules are combined in advance. The entire program is tested as a whole. And chaos usually results! A set of errors are encountered. Incremental integration is the antithesis of the big bang approach. The program is constructed and tested in small segments, where errors are easier to isolate and correct; interfaces are more likely to be tested completely and a systematic test approach may be applied. In the sections that follow, a number of different incremental integration strategies are discussed.’

16.4.1 Top down integration

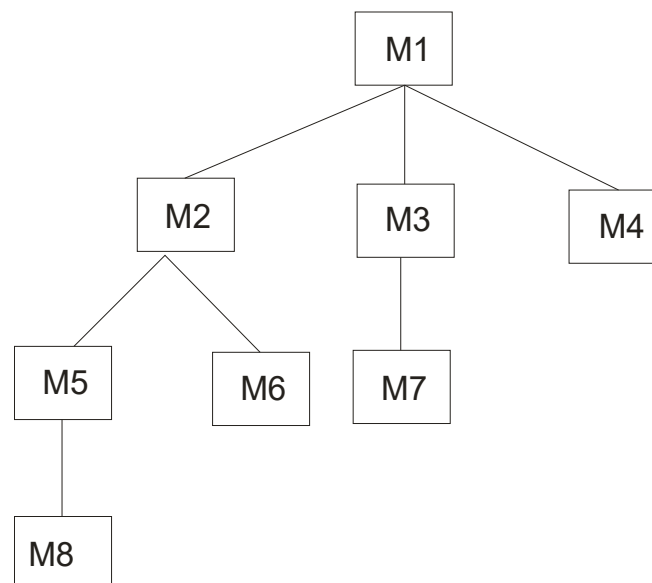


Figure 16.7

Top down integration is an incremental approach to construction of program structure. Modules are integrated by moving downward through the control hierarchy beginning with the main control module. Modules subordinate to the main control module are incorporated into the structure in either a depth first or breadth first manner. As shown in Figure 16.7 depth first integration would integrate all modules on a major control path of the structure.

Selection of a major path is some what arbitrary and depends on applicaiton specific characteristics. For example selected the left hand path modules method 1. method 2 and method 5 would be integrated first,... next method 8 or method 6 would be integrated. The next control level, method5, method 6 and so on follow. The integration process is performed in a series of five steps.

1. The main control module is used as a test driver and stubs are substituted for all modules directly subordinate to the main control module.
2. Depending on the integration approach selected subordinate stubs replaced one at a time with actual modules.
3. Tests are conducted as each module is integrated.

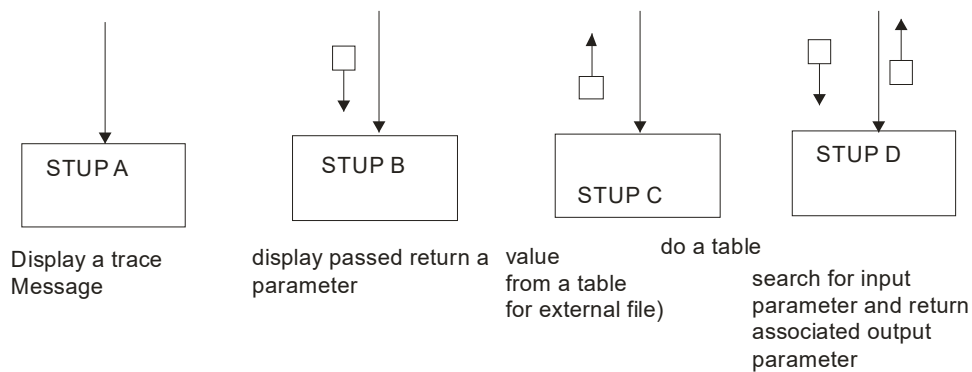


Figure 168

4. On completion of each set of tests another stub is replaced with the real module.
5. Regression testing may be conducted to ensure that new errors have not been introduced.

The process continues from step 2 until the entire program structure is built. The top down integration strategy verifies major control or decision point early in test process.

Stubs replace low level modules at the beginning of top down testing; therefore no significant data can flow upward in the program structure. The tester is left with three choices:

- Delay many tests until stubs are replaced with actual modules.
- Develop stubs that perform limited functions that simulate the actual module

- Integrate the software from the bottom of the hierarchy upward. Figure 16-8 illustrates typical classes of stubs, ranging: from the simplest to most complex.

16.4.2 Bottom-up integration

Bottom up integration as its name implies begins construction and testing with atomic modules. Because modules are integrated from the bottom up processing required for modules subordinate to a given level is always available and the need for stubs is eliminated. A bottom up integration strategy may be implemented with the following steps.

1. Low level modules are combined into clusters that perform a specific software sub function.
2. A driver is written to coordinate test case input and output.
3. The cluster is tested.
4. Drivers are removed and clusters are combined moving upward in the program structure.

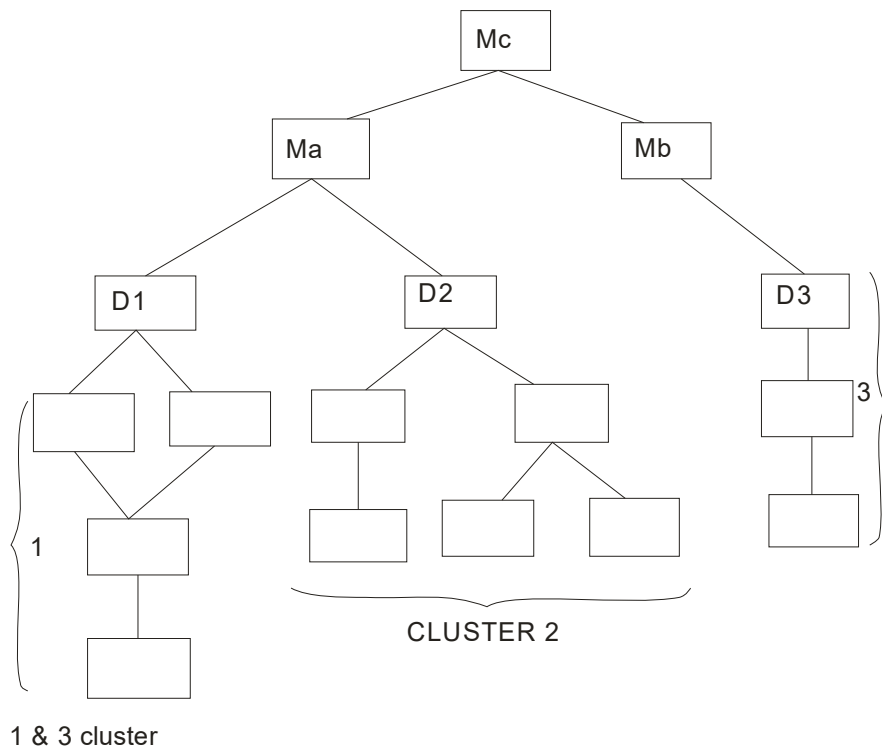


Figure 16.9

Integration follows the pattern illustrated in figure 16-9. modules are combined too form clusters 1,2 and 3. each of the clusters is tested using a driver. Modules in clusters 1 and 2 are subordinate to Madrivers d1 and d2 removed, and the clusters are interfaced directly to M. Both Maand Mb will ultimately integrated with module Mc, and so forth. Different categories of drivers are illustered in figure 16.0.

16.4.3 Regression testing

Each time a new module is added as part of integration testing, the software changes. New data flow paths are established new I/o may occur and new control logic is invoked. These changes may cause problems with functions that previously worked flawlessly. In the context of an integration test strategy regression testing is the execution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects.

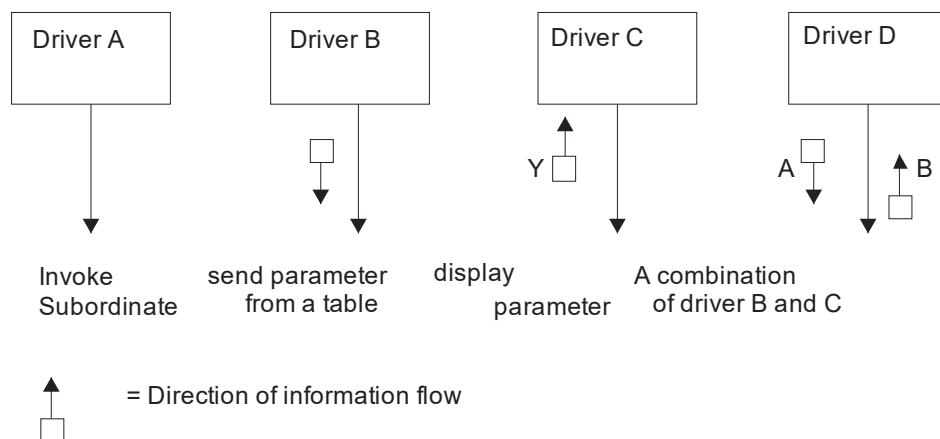


Figure 16.10

In a broader context successful tests result in the discovery of error and errors must be corrected. Whenever software is corrected, some aspect of the software configuration is changed. Regression testing is the activity that helps to ensure that changes do not introduce unintended behavior or additional errors.

The regression test suite contains three different classes of test cases.

- A representative sample of tests that will exercise all software functions.
- Additional tests that focus on software functions that are likely to be affected by the change.

- Tests that focus on the software components that have been changed.

16.5 Validation Testing

At the culmination of integration testing software is completely assembled as a package: interfacing errors have been uncovered and corrected, and a final series of software tests-validation testing may begin. Validation can be defined in many ways but a simple definition is that validation succeeds when software functions in a manner that can be reasonably expected by the customer

16.5.1 Validation test criteria

Software validation is achieved through a series of back book tests that demonstrate conformity with requirements. A test plan outlines the classes of tests to be conducted and test procedure defines specific test cases that will be used in an attempt to uncover errors in conformity with requirements. Both the plan and procedure are designed to ensure that all functional requirements are designed to ensure that all functional requirements are satisfied! all performed requirements are achieved; documentation is complete and human engineered and other requirements are met.

After each validation test case has been conducted one of two possible conditions exist:

- The function or performance characteristics conform to specification and are accepted.
- A deviation from specification is uncovered and a deficiency list is created. Deviation or error discovered at this stage in a project can rarely be corrected prior to scheduled completion.

16.5.2 Configuration Review

An important element of the validation process is a configuration review. The intent of the review is to ensure that all elements of the software configuration have been properly developed are catalogued and have the necessary detail support the maintenance phase of the software life cycle. The configuration review sometimes called an audit.

16.5.3 Alpha and Beta Testing

It is virtually impossible for a software developer to foresee how the customer will really use a program. Instructions for use may be misinterpreted; strange combinations of data may be regularly used; and output that seemed clear to the tester may be unintelligible to a user in the field. When custom software is built for one customer a series of acceptance tests are conducted to enable the customer to validate all requirements.

Conducted by the end user rather than the system developer, an acceptance test can range from an informal “test drive” to a planned and systematically executed series of tests. In fact acceptance testing can be conducted over a period of weeks or month thereby uncovering cumulative errors that might degrade the system over time. If software is developed as a product to be used by many customers, it is impractical to perform formal acceptance tests with each one. Most software product builders use a process called alpha and beta testing to uncover errors that only the end user seems able to find.

The alpha test is conducted at the developers site by a customer. The software is used in a natural setting with the developer “looking over the shoulder” of the user and recording errors and usage problems. Alpha tests are conducted in a controlled environment. The beta test is conducted at one or more customer sites by the end users of the software.

16.6 System Testing

A classic system testing problem is “finger pointing”. This occurs when an error is uncovered and each system element developer blames the other for the problem. Rather than indulging in such nonsense, the software engineer should anticipate potential interfacing problems

1. Design error handling paths that test all information coming from other elements of the system.
2. Conduct a series of tests that simulate bad data or other potential errors at the software interface.
3. Record the results of tests to use as “evidence” if finger pointing does occur.
4. Participate in planning and design of system tests to ensure that software is adequately tested.

System testing is actually a series of different tests whose primary purpose is to fully exercise the computer based system. Although each test has a different purpose all work to verify that all system elements have been properly integrated and perform allocated functions.

16.6.1 Recovery Testing

Many computer based system must recover from faults and resume processing within a pre specified time. In some cases a system must be fault tolerant that is processing faults use not cause overall system function to cease. In other cases a system failure must be corrected within a specified period of time or severe economic damage will occur.

Recovery testing is a system test forces the software to fail in a variety of ways and verifies that recovery is properly performed. If recovery is automatic initialization, check pointing mechanisms, data recovery, and restart are each evaluated for correctness.

16.6.2 Security testing

Any computer based system that manage sensitive information or causes actions that can improperly harm individuals is a target for improper or illegal penetration. Penetration spans a broad range of activities: hackers who attempt to penetrate system for sport. Security testing attempts to verify that protection mechanisms built into a system will in fact protect it from improper penetration. To quote beizer” the system’s security must of course, be tested for invulnerability from frontal attack but must also be tested for invulnerability from flank or rear attack”.

The tester may attempt to acquire passwords through external clerical means may attack the system with custom software designed to break down any defenses that have been constructed, may overwhelm the system, thereby denying service to others may purposely cause system errors, hoping to penetrate service to others. The role of the system designer is to make penetration cost greater than the value of the information that will be obtained.

16.6.3 Stress Testing

During earlier software testing steps white box and black box techniques resulted in through evaluation of normal program functions and performance. Stress tests are designed to confront programs with abnormal situations. In essence the tester who performs stress testing asks: “how high can we crank this up before it fails”. Stress testing executes a system in manner that

demands resources that demands resources in abnormal quantity, frequency or volume. For example:

1. Special tests may be designed that generate 10 inter: apts persistence second when one or two is the average rate.
2. Input data rates may be increased by an order magnitude to determine how input functions will respond.
3. Test cases that requires maximum memory or other resources may be executed.
4. Test cases that may cause thrashing in a virtual operating system may be designed.
5. Test case that may cause excessive hunting for disk resident data may be created.

A variation of stress testing is a technique called sensitivity testing in some situation a very small range of data contained within the bounds of valid data for a program may cause extreme and even erroneous processing or profound performance degradation. This situation is analogous to singularity in a mathematical function.

16.6.4 Performance Testing

For real time and embedded systems software that provide required function but does not conform to performance requirements is unacceptable. Performance testing is designed to test run time performance of software within the context of an integrated system. Performance testing occurs throughout all steps in the testing process. Even at the unit level, the performance of an individual module may be assessed as white box tests are conducted. However it is not until all system element are fully integrated that the true performance of a system can be ascertained.

Performance tests are often coupled with stress testing and often require both hardware and software instrumentation. That is is often necessary to measure resource utilization in an exacting fashion. External instrumentation can monitor execution intervals log events as they occur and sample machine states on a regular basis. By instrument a system, the tester can uncover situation that lead to degradation and possible system failure.

Summary

- Software testing accounts for the largest percentage of technical effort in the software process. Yet we are only beginning to understand the subtleties of systematic test planning, execution and control.
- The objective of software testing is to uncover errors,
- To fulfill this objective a series of test steps-unit, integration, validation, and system tests are planned and executed.
- Unit and integration tests concentrate on functional verification of a module and incorporation of modules into a program structure.
- Validation testing demonstrates trace ability to software requirements and system testing validates software once it has been incorporated into a larger system.
- Unlike testing, debugging must be viewed as an art. Beginning with a symptomatic indication of a problem, the debugging activity tracks down the cause of an error.

Questions:

1. What is validation and verification?
2. Explain system testing?
3. What are the criteria of validation test?
4. Explain alpha and beta testing.
5. Explain integration testing

MODEL QUESTION PAPER
Master of Computer Applications
First Year – Second Semester
Paper – VII
OBJECT ORIENTED ANALYSIS AND DESIGN

Time: 3 Hrs

Maximum: 80 marks

Section - A (10 x 2 = 20 Marks)

Answer any 10 Questions

1. What is an Object?
2. What is inheritance?
3. What is process?
4. What is RAD?
5. What is usecase?
6. How do you identify attributes?
7. Define a class
8. What is Concurrency control?
9. What is a DLN?
10. What is Query?
11. Describe CORBA and DCOM.
12. Who are the actors?

Section - B (6 x 5 = 30 Marks)**Answer any five Questions**

13. How is software verification differs from validation?
14. What are the advantages and disadvantages of prototyping?
15. How can you achieve multiple inheritance in a single inheritance system?
16. Why documentation is important for the analysis?
17. Explain in detail about distributed database.
18. Describe the process of creating access layer class.
19. Explain the graphical user interface objects.

Section - C (3 x 10 = 30 Marks)**Answer any three Questions**

20. What is association? And explain its types in detail.
21. What is the significance of separating private and protected protocols?
22. What are the basic activities performed in using the debugging tools?
23. What are the different types of servers? Briefly describe each.
24. How can metaphors be used in the design of a user interface?